

**Kontextsensitive, lokale KI-Assistenz zur automatisierten Barrierefreiheitsanalyse
von Rich-Client-Webanwendungen mittels Tool-Reports,
Playwright-Interaktionen und Codeanalyse**

Bachelorarbeit

vorgelegt am 11.05.2026

Fakultät Wirtschaft und Gesundheit

Studiengang Wirtschaftsinformatik

Kurs WWI2023

von

JOEL YERAI MARTINEZ CAMPO

Betreuung in der Ausbildungsstätte:

BITBW

Ilko Hoffman

Betreuer der Bachelorarbeit

Unterschrift

DHBW Stuttgart:

Sascha Alexander Ströbel

Abstract

Web-Barrierefreiheit ist trotz gesetzlicher Verpflichtung durch den *European Accessibility Act* (EAA) und das Barrierefreiheitsstärkungsgesetz (BFSG) in der Praxis nach wie vor unzureichend umgesetzt. 94,8 % der meistbesuchten Webseiten weisen nachweisbare Web Content Accessibility Guidelines (WCAG)-Verstöße auf. Bestehende automatisierte Prüfwerkzeuge wie axe, WAVE und Lighthouse erfassen lediglich syntaktische Verstöße und übersehen semantische sowie zustandsabhängige Barrieren in dynamischen Rich-Client-Anwendungen systembedingt. Der Transfer sensibler Anwendungsdaten an cloudbasierte Large Language Model (LLM)-Application Programming Interface (API)s scheidet für öffentliche Verwaltungen aus datenschutzrechtlichen Gründen weitgehend aus.

Diese Bachelorarbeit konzipiert und implementiert nach dem Design Science Research (DSR)-Paradigma das lokal ausführbare Assistenzsystem *Accessibility Context Engine (ACE)*. Es kombiniert axe-core-Violation-Reports, Playwright-Interaktionsdaten und statische Quellcodeanalysen in einer zweistufigen Datenpipeline: Zunächst prüft ein regelbasierter LLM-Detektor die erfassten Daten auf Barrierefreiheitsverstöße, anschließend analysiert ein zweiter LLM-Schritt den zugehörigen Quellcode und generiert priorisierte, datei- und zeilenspezifische Handlungsempfehlungen. Alle Modelle werden lokal über Ollama betrieben. Als Untersuchungsobjekte dienen eine React-basierte Single-Page Application (SPA) der IT Baden-Württemberg (BITBW) sowie der BW-Empfangsclient (BWEC).

Die Evaluation auf drei kontrollierten Benchmarksuiten (test-12: 12 Violations, test-50: 50 Violations, test-100: 100 Violations) mit drei lokalen Modellen (`qwen2.5-coder:7b`, `qwen2.5-coder:14b`, `qwen3:32b`) sowie eine laufende Untersuchung am realen BWEC zeigen, dass ACE den Recall gegenüber reiner Tool-Detektion konsequent steigert: auf test-12 von 66,7 % auf 100 % und auf test-50 ebenfalls auf 100 % (jeweils `qwen3:32b`); auf test-100 steigt der Recall von 34 % auf 63 % – trotz token-bedingter Skalierungsgrenzen bei kleineren Modellen. Auf test-12 werden für 11 von 12 Violations datei- und zeilenspezifische Handlungsempfehlungen auf Fix-Qualitätsstufe 2 erzielt. Das Local-First-Prinzip wird vollständig eingehalten: Kein Quellcode und kein Document Object Model (DOM)-Inhalt verlässt das lokale System.

Schlüsselwörter: Web-Barrierefreiheit, WCAG, Large Language Models, Prompt-Engineering, axe-core, Playwright, React, Rich-Client-Anwendungen, Lokale KI, Design Science Research, Ollama, Accessibility-Automatisierung, Barrierefreie-Informationstechnik-Verordnung (BITV), EAA.

Inhaltsverzeichnis

Abkürzungsverzeichnis	V
Abbildungsverzeichnis	VII
Tabellenverzeichnis	IX
1 Einleitung	1
1.1 Motivation und Kontext	1
1.2 Problemstellung	2
1.3 Zielsetzung und Forschungsfragen	3
1.4 Aufbau der Arbeit	5
2 Theoretische Grundlagen	6
2.1 Barrierefreiheit von Webanwendungen	6
2.1.1 Begriffsdefinition und Relevanz	6
2.1.2 WCAG: Aufbau, Prinzipien und Konformitätsstufen	8
2.1.3 Rechtlicher Rahmen	9
2.2 Automatisierte Barrierefreiheitsanalyse	10
2.2.1 Klassische Tool-Ansätze	10
2.2.2 Grenzen bestehender Tools	11
2.2.3 Taxonomie der Verstöße: syntaktisch, semantisch, Layout	12
2.3 Rich-Client-Webanwendungen und Testautomatisierung	14
2.3.1 Charakteristika und Abgrenzung	14
2.3.2 Herausforderungen für die Barrierefreiheitsprüfung	15
2.3.3 Playwright als Testautomatisierungswerkzeug	15
2.4 Künstliche Intelligenz (KI)-Assistenzsysteme für Softwareanalyse und Handlungsempfehlungen	16
2.4.1 Grundlagen: Large Language Models	16
2.4.2 Prompt-Engineering-Strategien	18
2.4.3 RAG und Fine-Tuning als Erweiterungsansätze	19
2.4.4 Datenschutz bei KI-gestützten Accessibility-Services	20
3 Stand der Forschung	22
3.1 Vorgehen der Literaturrecherche	22
3.2 Von frühen KI-Ansätzen zur LLM-basierten Erkennung	22
3.3 LLM-basierte Korrektur und Prompt-Engineering	23
3.4 Hybride Pipelines und Ensemble-Testing	24
3.5 Forschungslücken und Einordnung der vorliegenden Arbeit	24
4 Methodik	26
4.1 Wissenschaftliche Methode	26
4.2 Untersuchungsgegenstand und Scope-Abgrenzung	27
4.3 Evaluationsdesign und Kriterien	27
4.3.1 Evaluationslogik	28
4.3.2 Bewertungskriterien	28
4.3.3 Limitationen des Evaluationsdesigns	29

5	Analyse und Anforderungen	30
5.1	Problemkontext und Handlungsbedarf	30
5.2	Funktionale Anforderungen (FA)	31
5.3	Nicht-funktionale Anforderungen (NFA)	32
5.4	Lokale Inferenzinfrastruktur: Ollama	33
5.4.1	Ollama als Laufzeitumgebung	33
5.4.2	Anforderungen an das Sprachmodell	34
5.4.3	Ausgewählte Modellkandidaten für die Evaluation	34
5.4.4	Zusammenfassung der Anforderungen	35
6	Konzeption des Assistenzsystems	36
6.1	Zielarchitektur und Datenpipeline	36
6.1.1	Analysequellen	36
6.1.2	Unified Finding Schema (UFS)	38
6.2	Prompt-Strategie	39
6.2.1	Zweistufige Prompt-Strategie	39
6.2.2	Wahl der Prompt-Engineering-Strategien	40
6.2.3	Token-Budget-Steuerung	41
6.3	Ausgabeformat: entwicklernahe To-Do-Liste	41
7	Implementierung des Proof of Concept	43
7.1	Technischer Rahmen und Projektstruktur	43
7.2	axe-core-Modul	44
7.3	Playwright-Modul	45
7.4	Code-Modul	45
7.4.1	Anreicherung bestehender Findings	46
7.4.2	Pattern-Findings	46
7.5	LLM-Detektor-Modul	46
7.5.1	Abgrenzung zu grep und Nutzen der Kombination	47
7.5.2	Beispielhafter Ablauf eines LLM-Findings	48
7.6	Prompt-Builder und System-Prompt	48
7.6.1	System-Prompt	48
7.6.2	Token-Budget-Steuerung	49
7.6.3	Serialisierung	49
7.7	Ollama-Client und Output-Formatter	50
7.8	Pipeline-Orchestrierung	50
7.9	Beispielabläufe und Ergebnisse am Untersuchungsobjekt	51
8	Evaluation & Diskussion	52
8.1	Versuchsdesign	52
8.1.1	Testobjekte und Ground Truth	52
8.1.2	Modelle, Konfiguration und Runs	53
8.1.3	Metriken und Messverfahren	53
8.2	Ergebnisse: Detektionsqualität (Recall)	54
8.2.1	Baseline: Tool-Pipeline ohne LLM-Detektor	54
8.2.2	Recall nach LLM-Detektor-Augmentierung	55
8.2.3	Erkennungsleistung nach Verstößkategorie	55
8.3	Ergebnisse: Priorisierungsqualität	56
8.3.1	Modellvergleich Must-have und Nice-to-have	56
8.3.2	Einfluss des Token-Limits auf die Kategorisierung	56
8.4	Ergebnisse: Fix-Qualität (qualitativ)	57

8.4.1	Bewertungsmaßstab	57
8.4.2	Fallbeispiele nach Modell	57
8.5	Ergebnisse: Effizienz und Robustheit	58
8.5.1	Laufzeit und NFA-02-Konformität	58
8.5.2	Konsistenz und Parsing-Stabilität	59
8.5.3	Token-Nutzung	59
8.6	Limitationen und Validität	60
8.6.1	Externe Validität	60
8.6.2	Interne Validität und Token-Limit-Effekt	60
8.6.3	Weitere Einschränkungen	60
8.7	Belastungsprüfung: test-50 und test-100	61
8.7.1	Evaluation an test-50	62
8.7.2	Evaluation an test-100	63
8.7.3	Evaluation am BWEC-Untersuchungsobjekt	65
8.8	Beantwortung der Forschungsfragen	67
8.8.1	Übergeordnete Forschungsfrage	67
8.8.2	TF1: Erkennungsleistung nach Verstoßtyp	67
8.8.3	TF2: Mehrwert der Kontextkombination	68
8.9	Modellwahl und Einsatzempfehlungen	68
9	Fazit	70
9.1	Zusammenfassung der Ergebnisse	70
9.1.1	Beitrag zur übergeordneten Forschungsfrage	70
9.1.2	Erfüllung der funktionalen und nicht-funktionalen Anforderungen	70
9.2	Kritische Reflexion	71
9.2.1	Grenzen der externen Validität	71
9.2.2	Methodische Einschränkungen	71
9.3	Ausblick	72
9.3.1	Kurzfristige Erweiterungen	72
9.3.2	Mittelfristige Weiterentwicklungen	73
9.3.3	Langfristige Forschungsperspektiven	73
	Anhang	75
	Literaturverzeichnis	175

Abkürzungsverzeichnis

ACE	<i>Accessibility Context Engine</i>
API	Application Programming Interface
AST	Abstract Syntax Tree
BFSG	Barrierefreiheitsstärkungsgesetz
BITBW	IT Baden-Württemberg
BITV	Barrierefreie-Informationstechnik-Verordnung
BWEC	BW-Empfangsclient
CI	Continuous Integration
CI/CD	Continuous Integration/Continuous Delivery
CLI	Command Line Interface
CPU	Central Processing Unit
CSS	Cascading Style Sheets
DSR	Design Science Research
DOM	Document Object Model
DSGVO	Datenschutz-Grundverordnung
EAA	<i>European Accessibility Act</i>
FA	Funktionale Anforderungen
GPU	Graphics Processing Unit
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
IQR	Interquartile Range
JSON	JavaScript Object Notation
JSX	JavaScript XML
KI	Künstliche Intelligenz
KPI	Key Performance Indicator
LLM	Large Language Model
NFA	Nicht-funktionale Anforderungen
PoC	Proof of Concept
PW	Playwright
RAG	Retrieval-Augmented Generation

RAM	Random-Access Memory
SPA	Single-Page Application
TF	Teilfragen
UFS	Unified Finding Schema
UI	User Interface
URL	Uniform Resource Locator
W3C	World Wide Web Consortium
WAI	Web Accessibility Initiative
WCAG	Web Content Accessibility Guidelines
WHO	World Health Organization

Abbildungsverzeichnis

1	Prozessüberblick des geplanten Assistenzsystems	4
2	WCAG-Schichtenmodell	9
3	Taxonomie der Barrierefreiheitsverstöße	13
4	Framework Popularity über die letzten Jahre	14
5	Zero-Shot und Few-Shot Prompting im Vergleich	19
6	Datenpipeline des ACE-Assistenzsystems	36
7	Komplexität von Home Pages (6 Jahre)	78
8	Schematische Übersicht der ACE-Pipeline	81
9	Aufteilung des Token-Budgets	86
10	Screenshot Anmeldefenster der test-12-Suite	107
11	Screenshot Kontaktformular der test-12-Suite	108
12	Screenshot Dashboard-Übersicht der test-50-Suite	128
13	Screenshot Profil-Tab (Profil bearbeiten) der test-50-Suite	129
14	Screenshot Serverstatus der test-50-Suite	130
15	Screenshot Benachrichtigungen der test-50-Suite	130
16	Pareto nicht erkannter Violations	159
17	Boxplot Laufzeiten test-100	160
18	Pipeline der Findings-Verarbeitung	161
19	Screenshot Chat-Modul der test-100-Suite	162
20	Screenshot Suche-Modul der test-100-Suite	162
21	Screenshot Einstellungen-Modul der test-100-Suite	163
22	Screenshot Kalender-Modul der test-100-Suite	163
23	Screenshot Hilfe-Center-Modul der test-100-Suite	164
24	Screenshot Datei-Upload-Modul der test-100-Suite	164
25	Recall-Degradation im Suite-Vergleich	165
26	Gesamtlaufzeit-Skalierung im Suite-Vergleich	166
27	LLM-Konsistenz σ im Suite-Vergleich	167
28	Recall und Überschuss im Suite-Vergleich	168
29	Truncation-Rate im Suite-Vergleich	169
30	Heatmap der Recall-Werte	170
31	Must/Nice-Verteilung je Modell und Suite	171

Tabellenverzeichnis

1	Erkennungsleistung von Accessibility-Tools	12
2	Konzeptmatrix (Hauptmatrix, hoch priorisierte Studien)	25
3	Zuordnung der DSRM-Phasen zu den Kapiteln der Arbeit.	26
4	Anforderungsübersicht: Funktionale und nicht-funktionale Anforderungen mit Ursprung und Priorität.	35
5	Zuordnung der Analysequellen zu adressierten Problemen, Verstoßtypen und theoretischer Herleitung	38
6	Wesentliche Abhängigkeiten des PoC	43
7	Implementierte Playwright-Checks mit WCAG-Zuordnung und Kategorie	45
8	Implementierte grep-Patterns mit WCAG-Zuordnung	46
9	Konfigurationsparameter der Benchmarkläufe (final optimiert)	53
10	Durchgeführte Benchmark-Runs je Modell und Testsuite	54
11	Recall-Vergleich je Bedingung und Modell	55
12	Recall nach Verstoßkategorie und Modell	55
13	Priorisierungsverhalten der drei Modelle (Mittelwerte über 10 Runs)	56
14	Mittlere Laufzeiten je Pipeline-Phase ($n = 10$ je Modell)	58
15	Effizienzvergleich: LLM-Findings je Sekunde Gesamtlaufzeit	59
16	Priorisierungsverhalten auf der test-50-Suite (Mittelwerte über 10 Runs)	63
17	Modellwahl nach Anwendungsfall	68
18	Übersicht: generell vs. suite-spezifisch	77
19	Konzeptmatrix (vollständige Übersicht)	80
20	Modellempfehlungen nach Anwendungsszenario	87
21	Zentrale Metriken im Suite-Vergleich	88
22	Bewertungsskala der Empfehlungsqualität	89
23	Erfüllungsgrad FA und NFA	90
24	Umgebungs- und Konfigurationsparameter	92
25	Threats to Validity	94
26	Navigationsübersicht test-12-Suite	96
27	Accessibility-Verstöße (test-12)	97
28	Erkennungsrate der Violations (test-12)	98
29	Recall-Matrix (test-12)	99
30	Rohdaten Benchmark-Runs (test-12)	100
31	Modell-Leistungsvergleich (test-12)	101
32	Token-Nutzung pro Modell (test-12)	102
33	Effizienzvergleich der Modelle (test-12)	102
34	Über-Detektion (test-12)	103
35	Detektor-Konsistenz (test-12)	103
36	Empfehlungsqualität (test-12)	104
37	CSV-Spaltenschema	105
38	Beispielzeilen aus dem CSV-Export (test-12)	105
39	Quantilanalyse der LLM-Findings (test-12, alle 30 Runs)	106
40	Navigationsübersicht test-50-Suite	109
41	Accessibility-Verstöße (test-50)	110
42	Erkennungsrate der Violations (test-50)	113
43	Recall-Matrix (test-50)	116
44	Rohdaten Benchmark-Runs (test-50)	119

45	Modell-Leistungsvergleich (test-50)	120
46	Token-Nutzung pro Modell (test-50)	121
47	Effizienzvergleich der Modelle (test-50)	121
48	Über-Detektion (test-50)	122
49	Detektor-Konsistenz (test-50)	122
50	Empfehlungsqualität (test-50)	123
51	Beispielzeilen aus dem CSV-Export (test-50)	126
52	Quantilanalyse der LLM-Findings (test-50, alle 30 Runs)	127
53	Navigationsübersicht test-100-Suite	131
54	Accessibility-Verstöße (test-100)	132
55	Erkennungsrate der Violations (test-100)	136
56	Recall-Matrix (test-100)	141
57	Rohdaten Benchmark-Runs (test-100)	146
58	Modell-Leistungsvergleich (test-100)	147
59	Token-Nutzung pro Modell (test-100)	148
60	Effizienzvergleich der Modelle (test-100)	148
61	Über-Detektion (test-100)	149
62	Stichproben-Plausibilisierung des finalen Reports	150
63	Detektor-Konsistenz (test-100)	151
64	Empfehlungsqualität (test-100)	152
65	Beispielzeilen aus dem CSV-Export (test-100)	158
66	Quantilanalyse der LLM-Findings (test-100, alle Modelle)	158

1 Einleitung

Web-Barrierefreiheit ist eine zentrale Voraussetzung dafür, dass alle Menschen - unabhängig von individuellen Einschränkungen - gleichberechtigten Zugang zu digitalen Informationen und Dienstleistungen erhalten. In der Praxis zeigt sich jedoch eine erhebliche Diskrepanz zwischen den normativen Anforderungen und der tatsächlichen Umsetzung: Barrierefreiheit wird häufig spät berücksichtigt, bestehende Prüfwerkzeuge erfassen nur einen Bruchteil der tatsächlichen Barrieren und manuelle Korrekturen sind zeitaufwändig.¹ Diese Arbeit untersucht, wie ein LLM-basierter Ansatz diese Lücke schließen kann: durch die automatisierte Erkennung von Barrierefreiheitsproblemen in Webanwendungen und die Ableitung entwicklernaher Handlungsempfehlungen - lokal ausführbar und ohne den Transfer sensibler Daten an externe Dienste.²

1.1 Motivation und Kontext

Rund 1,3 Milliarden Menschen (16% der Weltbevölkerung) leben mit einer Form von Behinderung.³ Für diese Personengruppe ist der Zugang zu digitalen Angeboten keine bloße Komfortfrage, sondern eine grundlegende Voraussetzung für gesellschaftliche Teilhabe: ohne barrierefreie Webanwendungen bleiben ihnen wesentliche Lebensbereiche verschlossen - von der Kommunikation mit Behörden über den Zugang zu Bildung und Gesundheitsinformationen bis hin zur Teilnahme am wirtschaftlichen und gesellschaftlichen Leben. Das Recht auf barrierefreien Zugang zu Informations- und Kommunikationstechnologien ist völkerrechtlich in der UN-Behindertenrechtskonvention verankert und hat sich in den vergangenen Jahren zunehmend auch in verbindlichem nationalen und europäischen Recht niedergeschlagen.⁴

Auf europäischer Ebene verpflichtet der EAA die Mitgliedstaaten, Barrierefreiheitsanforderungen für digitale Produkte und Dienstleistungen durchzusetzen.⁵ In Deutschland wurde diese Richtlinie durch das BFSG in nationales Recht überführt, das seit dem 28. Juni 2025 auch für weite Teile der Privatwirtschaft gilt.⁶ Ergänzt wird dieser Rahmen durch die BITV, die für Behörden und öffentliche Stellen bereits seit Jahren verbindliche Vorgaben formuliert.⁷

Trotz dieser eindeutigen Rechtslage ist die praktische Umsetzung barrierefreier Webanwendungen nach wie vor unzureichend. Die jährlich durchgeführte WebAIM-Million-Studie, die die Startseiten der meistbesuchten Webseiten weltweit analysiert, stellte 2025 fest, dass 94,8% der untersuchten Seiten nachweisbare Verstöße gegen die WCAG aufweisen.⁸ Im Vergleich mit den letzten

¹ Vgl. Souza et al. 2024; Vgl. Abuaddous, Jali, Basir 2016, S. 175

² Vgl. Bassi et al. 2025, S. 297

³ Vgl. World Health Organization 2026

⁴ Vgl. United Nations 2006

⁵ Vgl. European Commission 2026

⁶ Vgl. Deutscher Bundestag 2026

⁷ Vgl. Bundesministerium für Digitales und Staatsmodernisierung 2025; Vgl. Heinemann 2024, S. 281

⁸ Vgl. WebAIM 2025

Jahren ist seit 2019 (97,8%) lediglich eine Verbesserung um 3 Prozentpunkte zu verzeichnen.⁹ Für die öffentliche Verwaltung ist diese Situation besonders brisant: während privatwirtschaftliche Anbieter erst seit Juni 2025 durch das BFSG verpflichtet wurden, galten für Behörden und öffentliche Stellen bereits seit Jahren verbindliche Barrierefreiheitsanforderungen nach der BITV - dennoch weisen auch zahlreiche Webangebote der öffentlichen Hand weiterhin erhebliche Mängel auf.¹⁰ Damit bleibt Barrierefreiheit in der Entwicklungspraxis trotz gesetzlicher Verpflichtung systematisch unterrepräsentiert.

Die BITBW nimmt als zentrale IT-Dienstleisterin der Landesverwaltung Baden-Württemberg in diesem Kontext eine besondere Stellung ein.¹¹ Als Entwicklerin und Betreiberin öffentlicher Webanwendungen für Ministerien, Polizeibehörden und Kommunen ist sie gesetzlich zur Einhaltung der BITV und des BFSG verpflichtet.¹² Drei Faktoren machen die BITBW zu einem besonders geeigneten Praxiskontext für die vorliegende Arbeit: Erstens unterliegen ihre Anwendungen strengeren Barrierefreiheitsanforderungen als privatwirtschaftliche Produkte. Zweitens schließen die Datenschutzvorgaben der öffentlichen Verwaltung die Nutzung cloudbasierter LLM-APIs weitgehend aus. Drittens erschweren gewachsene Systemlandschaften mit Legacy-Anteilen und begrenzten Entwicklungsressourcen die manuelle Barrierefreiheitsprüfung im erforderlichen Umfang. Sie bildet damit den praktischen Rahmen der vorliegenden Arbeit.

1.2 Problemstellung

Die mangelhafte Umsetzung von Barrierefreiheit in Softwareprojekten lässt sich nicht allein auf fehlende Motivation oder Ressourcen zurückführen. Ein wesentlicher Grund sind die Schwächen der verfügbaren Tools: automatisierte Barrierefreiheitsanalyse-Tools wie axe¹³, WAVE¹⁴ oder Lighthouse¹⁵ erkennen zwar syntaktische Verstöße, stoßen jedoch bei semantischen Problemen an ihre Grenzen. Ob ein vorhandener Alternativtext den Bildinhalt tatsächlich angemessen beschreibt, oder ob eine Schaltflächenbeschriftung für Screenreader-Nutzerinnen und -Nutzer verständlich ist, entzieht sich der regelbasierten Prüflöge dieser Tools.

Empirisch belegt wird diese Schwäche durch das Mutation-Testing-Framework **Mally**, das gezielt Barrierefreiheitsfehler in reale Webanwendungen injiziert und anschließend prüft, ob bestehende Accessibility-Tools diese künstlich eingebrachten Defekte erkennen.¹⁶ In einer systematischen Evaluation mit 25 gezielt eingebrachten Barrierefreiheitsproblemen erkannte kein einziges der sechs untersuchten Tools mehr als 50% der injizierten Fehler. Somit blieben im Durchschnitt nahezu die Hälfte aller Defekte unentdeckt.¹⁷

⁹Vgl. WebAIM 2019

¹⁰Vgl. Heinemann 2024, S. 281

¹¹Vgl. BITBW 2021

¹²Vgl. Landesrecht BW 2015

¹³Vgl. Deque Systems 2026

¹⁴Vgl. WebAIM 2026a

¹⁵Vgl. Google 2025

¹⁶Vgl. Tafreshipour et al. 2024, S. 97

¹⁷Vgl. Tafreshipour et al. 2024, S. 100

Eine zweite strukturelle Schwäche betrifft die Charakteristik moderner Webanwendungen. Rich-Client-Anwendungen auf Basis von Frameworks wie React erzeugen ihren Inhalt größtenteils clientseitig und verändern das DOM dynamisch in Abhängigkeit von Nutzerinteraktionen. Barrieren entstehen häufig erst in bestimmten Zuständen: wenn ein Modalfenster geöffnet wird, eine Fehlermeldung erscheint oder ein Dropdown-Menü aktiviert ist. Rein statische Analysen - oder solche, die ausschließlich den initialen Seitenladezustand prüfen - erfassen diese zustandsabhängigen Barrieren nicht.¹⁸

Jüngere Forschungsarbeiten zeigen, dass LLMs geeignet sind, die zuvor beschriebenen Grenzen regelbasierter Accessibility-Tools zu überwinden. Systeme wie **AccessGuru**¹⁹ und **GenA11y**²⁰ setzen das cloudbasierte LLM GPT-4o ein und erzielen bei der Erkennung und Korrektur von Barrierefreiheitsverstößen vielversprechende Ergebnisse. Beide Ansätze beschränken sich jedoch auf die Analyse statischen Hypertext Markup Language (HTML)s und berücksichtigen weder dynamische Zustandswechsel noch den Quellcode der geprüften Anwendung. Der Schritt von der Fehlererkennung zu einer entwicklernahen, auf den Quellcode bezogenen Handlungsempfehlung wird in der Literatur als offenes Problem benannt,²¹ bleibt aber bislang ungelöst. Eine vertiefte Analyse dieser und weiterer Arbeiten erfolgt in Kapitel 3.

Die vorliegende Forschungsliteratur lässt somit drei zentrale Defizite erkennen: es existiert bislang kein softwaregestütztes Tool, das Entwicklerinnen und Entwickler bei der Barrierefreiheitsanalyse unterstützt und dabei (a) die dynamischen Zustände einer Rich-Client-Anwendung durch automatisierte Browser-Interaktionen erfasst, (b) die Ergebnisse automatisierter Tool-Reports mit dem relevanten Quellcode verknüpft und (c) auf dieser Grundlage priorisierte, entwicklernahe Handlungsempfehlungen erzeugt - lokal ausführbar und ohne den Transfer sensibler Anwendungsdaten an externe Dienste.

1.3 Zielsetzung und Forschungsfragen

Ziel dieser Bachelorarbeit ist die Konzeption und prototypische Umsetzung einer lokal ausführbaren, kontextsensitiven Assistenzlösung zur Barrierefreiheitsanalyse von Rich-Client-Webanwendungen. Als Untersuchungsobjekt dient eine React-basierte Webanwendung aus dem Umfeld der BITBW. Der Proof of Concept soll drei Datenquellen in einer Datenpipeline zusammenführen: (1) strukturierte Violation-Reports aus axe-core, (2) Interaktionsdaten aus Playwright-gesteuerten Browser-Tests sowie (3) statische Quellcodeanalysen der geprüften React-Komponenten, etwa hinsichtlich fehlender `aria`-Attribute oder nicht-interaktiver Elemente mit `onClick`-Handlern. Auf dieser Grundlage werden priorisierte, entwicklernahe Handlungsempfehlungen abgeleitet und strukturiert aufbereitet. Abbildung 1 veranschaulicht diesen Prozess im Überblick.

¹⁸Vgl. Bassi et al. 2025, S. 303

¹⁹Vgl. Fathallah, Hernández, Staab 2025

²⁰Vgl. He, Huq, Malek 2025

²¹Vgl. He, Huq, Malek 2025, FSE101:15–17

²²Eigene Abbildung

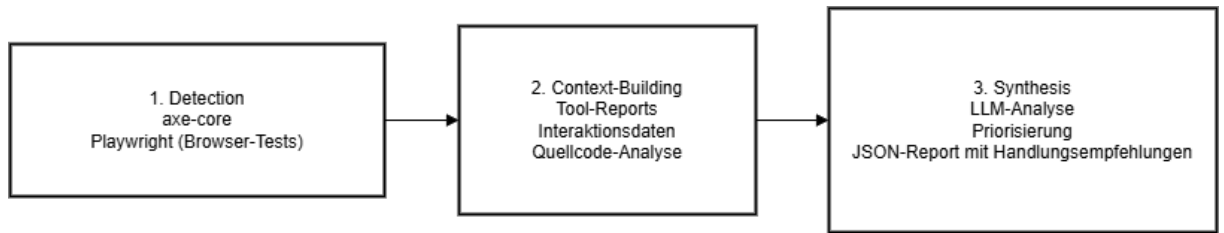


Abb. 1: Prozessüberblick: von den drei Datenquellen zur entwicklernahen Handlungsempfehlung²²

Diese Arbeit liefert drei konkrete Beiträge:

1. Einen strukturierten Überblick darüber, wie unterschiedliche Datenquellen der Barrierefreiheitsanalyse systematisch kombiniert und interpretiert werden können.
2. Einen Proof of Concept, der das Vorgehen exemplarisch demonstriert.
3. Eine qualitative Bewertung der Untersuchungsergebnisse anhand definierter Kriterien, die aus den Anforderungen und der einschlägigen Fachliteratur abgeleitet werden.

Die übergeordnete Forschungsfrage dieser Arbeit lautet:

Inwieweit kann ein kontextsensitives KI-Assistenzsystem, das Tool-Reports, Playwright-Interaktionsdaten und Quellcode kombiniert, Barrierefreiheitsverstöße in React-basierten Webanwendungen automatisiert erkennen und entwicklernahe Handlungsempfehlungen generieren?

Diese übergeordnete Frage wird durch zwei konkretisierende Teilfragen (TF) operationalisiert:

- TF1: Welche Typen von Barrierefreiheitsverstößen - syntaktisch, semantisch und layout-bezogen - lassen sich durch den gewählten Ansatz zuverlässig erkennen und wo liegen die methodischen Grenzen?
- TF2: Inwiefern verbessert die Kombination von Tool-Reports, Playwright-Interaktionsdaten und Quellcode die Qualität der generierten Handlungsempfehlungen gegenüber einer Analyse, die ausschließlich auf Tool-Reports und gerenderten DOM-Ausschnitten basiert?

TF1 adressiert dabei das Erkennungsproblem und erlaubt eine differenzierte Aussage über die Leistungsfähigkeit und Grenzen des Systems entlang der Verstoß-Taxonomie von Fathallah et al.²³ TF2 greift den in der Literatur identifizierten Mehrwert der Kontextualisierung durch Tool-Reports, Playwright-Interaktionsdaten und Quellcode auf und überprüft ihn im Rahmen eines qualitativen Ablationsvergleichs am gewählten Untersuchungsobjekt.

²³Vgl. Fathallah, Hernández, Staab 2025, S. 114–115

1.4 Aufbau der Arbeit

Die vorliegende Arbeit gliedert sich in neun Kapitel. Im Anschluss an die Einleitung werden in Kapitel 2 die theoretischen Grundlagen erarbeitet: Barrierefreiheit im Web, die WCAG-Standards und der rechtliche Rahmen, die Charakteristika von Rich-Client-Webanwendungen sowie Grundlagen zu Large Language Models und Prompt-Engineering-Strategien.

Kapitel 3 gibt einen systematischen Überblick über den Stand der Forschung. Die relevanten Arbeiten werden thematisch gegliedert - nach LLM-basierter Erkennung, Code-Korrektur, hybriden Pipelines und Evaluation - und in einer Konzeptmatrix nach Webster und Watson zusammengefasst.²⁴ Das Kapitel schließt mit einer expliziten Einordnung des Beitrags der vorliegenden Arbeit.

Kapitel 4 legt das methodische Vorgehen dar. Es beschreibt die Wahl des Design-Science-Research-Paradigmas als wissenschaftliche Methode, definiert den Untersuchungsgegenstand und entwirft das Evaluationsdesign.²⁵ Kapitel 5 analysiert den Problemkontext und leitet daraus funktionale und nicht-funktionale Anforderungen ab, einschließlich der Begründung der Modellauswahl.

Auf Basis dieser Anforderungen wird in Kapitel 6 das Assistenzsystem konzipiert: Systemarchitektur, Datenpipeline, Prompt-Strategie und Ausgabeformat werden begründet entworfen. Kapitel 7 beschreibt die prototypische Implementierung des Proof of Concept und illustriert die Systemfunktionalität anhand von Beispielabläufen für syntaktische, semantische und dynamisch erzeugte Barrierefreiheitsprobleme.

Kapitel 8 präsentiert und diskutiert die Evaluationsergebnisse, beantwortet die Forschungsfragen und reflektiert die Limitationen des Ansatzes. Das abschließende Kapitel 9 fasst die Kernaussagen zusammen, ordnet den Beitrag der Arbeit in die Forschungslandschaft ein und formuliert Empfehlungen für weiterführende Untersuchungen.

²⁴Vgl. Webster, Watson 2002

²⁵Vgl. Hevner et al. 2004, S. 75

2 Theoretische Grundlagen

Das folgende Kapitel legt die theoretischen Grundlagen für die Einführung des Proof of Concepts dar. Es werden Begriffe und Konzepte erläutert, die für das Verständnis der Problemstellung und des entwickelten Ansatzes erforderlich sind. Zunächst wird Barrierefreiheit im Web als Untersuchungsgegenstand definiert (Abschnitt 2.1), anschließend der Stand automatisierter Barrierefreiheitstools sowie ihre Möglichkeiten und Grenzen (Abschnitt 2.2). Darauf aufbauend werden dann die besonderen Herausforderungen von Rich-Client-Anwendungen und das Werkzeug Playwright eingeführt (Abschnitt 2.3). Das Kapitel schließt mit einer Einführung in LLM, Modelle und relevante Prompting Strategien sowie auch Datenschutzbetrachtungen (Abschnitt 2.4).

2.1 Barrierefreiheit von Webanwendungen

2.1.1 Begriffsdefinition und Relevanz

Der Begriff *Web Accessibility*, im Deutschen Web-Barrierefreiheit, bezeichnet nach der Definition der Web Accessibility Initiative (WAI) die Eigenschaft von Webangeboten, von allen Menschen unabhängig von körperlichen, motorischen oder sensorischen Einschränkungen sowie individuellen Fähigkeiten wahrgenommen werden zu können.²⁶ Web Accessibility geht jedoch über die reine Wahrnehmbarkeit hinaus und umfasst auch die Bedienbarkeit sowie Nutzbarkeit von Webangeboten.²⁷ Laut Angaben der World Health Organization (WHO) wird geschätzt, dass rund 16% der Weltbevölkerung mit einer Form von Behinderung leben.²⁸ Diese Zahl umfasst ein breites Spektrum an Einschränkungen, von denen nicht alle unmittelbar die Nutzung digitaler Angebote betreffen - eine Person im Rollstuhl ohne weitere sensorische oder motorische Einschränkungen der oberen Extremitäten ist beispielsweise bei der Webnutzung nicht zwingend eingeschränkt. Entscheidend für die Web-Barrierefreiheit sind vor allem visuelle, motorische und kognitive Beeinträchtigungen, die die Wahrnehmung oder Bedienung digitaler Oberflächen erschweren. Darüber hinaus profitieren auch Menschen ohne dauerhafte Behinderung von barrierefreier Gestaltung: temporäre Einschränkungen wie ein gebrochener Arm oder situationsbedingte Faktoren wie gleißendes Sonnenlicht auf dem Bildschirm können die Nutzung digitaler Angebote vorübergehend erheblich erschweren.²⁹ Damit stellt Barrierefreiheit im Web nicht nur eine technische Herausforderung dar, sondern auch eine zentrale gesellschaftliche Aufgabe zur Gewährleistung digitaler Teilhabe. Das Recht auf barrierefreien Zugang zu Informations- und Kommunikationstechnologien ist seit 2006 in Art. 9 der UN-Behindertenrechtskonvention völkerrechtlich verankert.³⁰

²⁶Vgl. Abuaddous, Jali, Basir 2016, S. 172

²⁷Vgl. World Wide Web Consortium (W3C) 2026b

²⁸Vgl. World Health Organization 2026

²⁹Vgl. World Health Organization 2011

³⁰Vgl. United Nations 2006, S. 9

In der Forschungsliteratur werden vier wesentliche Behinderungsformen unterschieden, die jeweils spezifische Anforderungen an die Gestaltung barrierefreier Webangebote stellen.³¹ Für die vorliegende Arbeit sind insbesondere zwei Kategorien relevant - visuelle Beeinträchtigungen und motorische Einschränkungen -, da sie den Großteil der automatisiert prüfbaren WCAG-Verstöße betreffen:

Visuelle Beeinträchtigungen umfassen vollständige oder partielle Blindheit sowie Farbfehlsichtigkeit.³² Betroffene Personen nutzen häufig Screenreader-Software (z. B. JAWS, NVDA, VoiceOver), die auf korrekte semantische HTML-Auszeichnung angewiesen ist.³³ Fehlende Alternativtexte, unzureichende Farbkontraste und fehlende **aria**-Labels bilden die häufigsten Verstöße in dieser Kategorie³⁴ und zugleich den primären Erkennungsbereich des in dieser Arbeit entwickelten Systems.

Motorische Einschränkungen betreffen Personen, die konventionelle Eingabegeräte wie Maus oder Touchscreen nicht präzise bedienen können.³⁵ Sie sind auf Tastaturnavigation, Sprachsteuerung oder spezielle assistive Eingabegeräte angewiesen.³⁶ Eine vollständige Bedienbarkeit über die Tastatur ist daher eine zentrale Voraussetzung für barrierefreie Nutzung.³⁷ Für das vorliegende System ist diese Kategorie besonders relevant, da Tastaturfallen und fehlende Fokussteuerung in (React-)Anwendungen häufig erst durch interaktionsbasierte Tests - etwa mittels Playwright - aufgedeckt werden können.³⁸

Daneben adressiert die WCAG auch **auditive Beeinträchtigungen** (Untertitel, Transkriptionen)³⁹ sowie **kognitive und neurologische Beeinträchtigungen** (klare Sprache, konsistente Strukturen, reduzierte visuelle Komplexität).⁴⁰ Beide Kategorien erfordern jedoch überwiegend menschliches Urteilsvermögen und entziehen sich weitgehend der automatisierten Prüfung.⁴¹ Sie liegen daher außerhalb des Scopes des vorliegenden Proof of Concept (PoC).

Assistive Technologien - darunter Screenreader, Bildschirmvergrößerungssoftware und alternative Eingabegeräte - bilden die technische Voraussetzung für die Nutzung digitaler Angebote durch die genannten Personengruppen.⁴² Ihre Wirksamkeit hängt jedoch unmittelbar davon ab, dass Webanwendungen klar definierte strukturelle und semantische Anforderungen erfüllen.⁴³ Die Nichterfüllung dieser Anforderungen führt nicht nur zu einer Beeinträchtigung der betroffenen Personen, sondern kann auch wirtschaftliche und rechtliche Konsequenzen nach sich ziehen.⁴⁴

³¹Vgl. Lazar, Feng, Hochheiser 2017, S. 493–494

³²Vgl. Gubbi Mohanbabu, Pavel 2024

³³Vgl. Morris et al. 2016, S. 5508

³⁴Vgl. World Wide Web Consortium (W3C) 2023, Erfolgskriterium 1.1.1

³⁵Vgl. Pérez et al. 2020, S. 110381–110382

³⁶Vgl. Kouroupetroglou 2013, S. 208

³⁷Vgl. WebAIM 2026b

³⁸Vgl. Chiou, Alotaibi, Halfond 2021, S. 855

³⁹Vgl. Harrenstien 2009

⁴⁰Vgl. World Wide Web Consortium (W3C) 2023

⁴¹Vgl. Abou-Zahra, Brewer, Cooper 2018

⁴²Vgl. World Wide Web Consortium (W3C) 2026b; Abou-Zahra, Brewer, Cooper 2018

⁴³Vgl. Morrison et al. 2017, S. 82

⁴⁴Vgl. Krok 2024, S. 864–865

2.1.2 WCAG: Aufbau, Prinzipien und Konformitätsstufen

Die WCAG sind der internationale Standard für barrierefreie Webinhalte und werden vom World Wide Web Consortium (W3C) im Rahmen der WAI herausgegeben. Die derzeit maßgebliche Referenzversion ist WCAG 2.2, die im Oktober 2023 WCAG 2.1 (2018) als aktuelle Referenzversion ablöste.⁴⁵ WCAG 2.2 führt neun zusätzliche Erfolgskriterien ein und berücksichtigt insbesondere kognitive Einschränkungen und mobile Nutzung.⁴⁶ Zu diesen Kriterien gehören konsistentere Fokusindikatoren (2.4.11–2.4.13) und die Vermeidung von Drag-and-Drop-Pflichtinteraktionen (WCAG 2.5.7).⁴⁷

Die WCAG ist in vier Schichten gegliedert: Prinzipien, Richtlinien, Erfolgskriterien und unterstützende Techniken. Auf der obersten Ebene stehen vier grundlegende Prinzipien, zusammengefasst im Akronym **POUR**.⁴⁸

- **Perceivable (Wahrnehmbar):** Alle Informationen und Benutzerschnittstellen müssen für alle Nutzer und Nutzerinnen wahrnehmbar sein.⁴⁹ Typische Anforderungen sind Alternativtexte für Bilder (Erfolgskriterium 1.1.1) und ausreichende Farbkontraste (Erfolgskriterium 1.4.3).⁵⁰
- **Operable (Bedienbar):** Alle Funktionen der Benutzeroberfläche müssen über die Tastatur erreichbar sein.⁵¹ Zeitlich beschränkte Inhalte, die nach einer vordefinierten Zeit verschwinden, müssen auch verhindert werden. Auch Inhalte, die Anfälle auslösen können, müssen vermieden werden.⁵²
- **Understandable (Verständlichkeit):** Inhalte und Bedienelemente müssen so verständlich gestaltet sein, dass die verwendete Sprache angegeben ist und Fehlermeldungen aussagekräftig sowie beschreibend formuliert sind.⁵³
- **Robust (Robust):** Inhalte müssen so implementiert sein, dass sie von einer breiten Palette von aktuellen und zukünftigen assistiver Technologien interpretiert und ausgeführt werden können.⁵⁴

Die 13 Richtlinien unterhalb der Prinzipien werden durch insgesamt 87 testbare Erfolgskriterien operationalisiert (WCAG 2.2; gegenüber 78 in WCAG 2.1 kommen neun neue Kriterien hinzu, während SC 4.1.1 als obsolet markiert wurde).⁵⁵ Jedes Erfolgskriterium ist einer von drei

⁴⁵Vgl. World Wide Web Consortium (W3C) 2018; World Wide Web Consortium (W3C) 2023

⁴⁶Vgl. Purwandari et al. 2025, S. 118; Vgl. Souza et al. 2024

⁴⁷Vgl. World Wide Web Consortium (W3C) 2023

⁴⁸Vgl. Souza et al. 2024

⁴⁹Vgl. World Wide Web Consortium (W3C) 2023, Abschnitt 1

⁵⁰Vgl. Souza et al. 2024

⁵¹Vgl. World Wide Web Consortium (W3C) 2023, Abschnitt 2

⁵²Vgl. Souza et al. 2024

⁵³Vgl. World Wide Web Consortium (W3C) 2023, Abschnitt 3

⁵⁴Vgl. World Wide Web Consortium (W3C) 2023, Abschnitt 4

⁵⁵Vgl. World Wide Web Consortium (W3C) 2023

Konformitätsstufen zugeordnet.⁵⁶ Stufe A umfasst grundlegende Anforderungen, deren Nichteinhaltung bestimmte Nutzergruppen vollständig ausschließt. Stufe AA ist der in Europa gesetzlich geforderte Mindeststandard für öffentliche Stellen und weitere Teile der Privatwirtschaft.⁵⁷ Stufe AAA beschreibt Anforderungen, die nicht für alle Inhalte generell erfüllbar sind. Die vorliegende Arbeit fokussiert sich entsprechend dem gesetzlichen Mindeststandard auf Stufe A und AA.

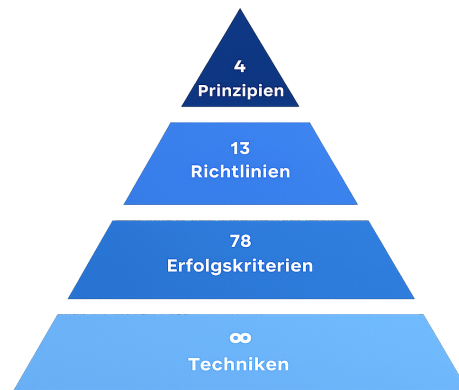


Abb. 2: WCAG-Schichtenmodell⁵⁸

Die praktische Umsetzung dieser Standards ist trotz ihrer langen Existenz nach wie vor unzureichend.⁵⁹ Die WebAIM-Million-Studie 2025 identifizierte im Durchschnitt 56,8 WCAG-Fehler pro Seite. Die häufigsten Verstößkategorien betrafen fehlende Alternativtexte (54,5%), unzureichende Farbkontraste (80,6%), fehlende Labels (47,4%), leere Links (44,6%) und fehlende Sprachangaben im HTML (17,1%).⁶⁰

2.1.3 Rechtlicher Rahmen

Auf europäischer Ebene bildet der *European Accessibility Act* (EAA) den übergeordneten rechtlichen Rahmen.⁶¹ Er verpflichtet Mitgliedstaaten, einheitliche und barrierefreiheitskonforme Inhalte und Webangebote für die digitalen Produkte und Dienstleistungen durchzusetzen.

In Deutschland wurde der EAA durch das Barrierefreiheitsstärkungsgesetz (BFSG) in nationales Recht überführt.⁶² Das BFSG ist seit dem 28. Juni 2025 für privatwirtschaftliche Anbieter verbindlich. Unternehmen mit weniger als zehn Beschäftigten und einem Jahresumsatz unter zwei Millionen Euro sind von bestimmten Pflichten ausgenommen.⁶³

Für öffentliche Stellen des Bundes gilt in Deutschland bereits seit 2011 die Barrierefreie-Informationstechnik-Verordnung (BITV) 2.0, die Behörden verpflichtet, ihre Webangebote nach WCAG

⁵⁶Vgl. Fathallah, Hernández, Staab 2025

⁵⁷Vgl. Kerkmann, Lewandowski 2015, S. 40, 195 ff.

⁵⁸Eigene Abbildung

⁵⁹Vgl. Bassi et al. 2025, S. 297

⁶⁰Vgl. WebAIM 2025

⁶¹Vgl. European Commission 2026

⁶²Vgl. Deutscher Bundestag 2026

⁶³Vgl. Bundesamt der Justiz und für Verbraucherschutz 2026, §1 (Anwendungsbereich) und §3 (Barrierefreiheit)

2.1 auf mindestens Konformitätsstufe AA zu gestalten und zu dokumentieren.⁶⁴ Die BITV 2.0 verweist in ihrer Anlage 1 unmittelbar auf die WCAG-Erfolgskriterien und ist somit dynamisch mit deren Weiterentwicklung verknüpft.

Diese rechtlichen Anforderungen sind für alle betroffenen Organisationen verbindlich - öffentliche Stellen ebenso wie privatwirtschaftliche Anbieter im Geltungsbereich des BFGG. Die IT Baden-Württemberg (BITBW) als zentrale IT-Dienstleisterin der Landesverwaltung Baden-Württembergs ist davon in besonderem Maße betroffen, da sämtliche von ihr entwickelten oder betriebenen Anwendungen den Anforderungen entsprechen müssen.⁶⁵ Dies begründet das praktische Interesse an effizienten, automatisierten Methoden der Barrierefreiheitsüberprüfung, wie sie in dieser Arbeit untersucht werden.

2.2 Automatisierte Barrierefreiheitsanalyse

Trotz der klar definierten Richtlinien der WCAG zeigt die Praxis, dass Barrierefreiheit in der Webentwicklung häufig unzureichend umgesetzt wird. Ein zentraler Grund dafür liegt in den Entwicklungsprozessen moderner Webanwendungen. In vielen Softwareprojekten wird Barrierefreiheit nicht von Beginn an als integraler Bestandteil der Entwicklung berücksichtigt, sondern erst in späten Testphasen oder gar nicht adressiert. Hinzu kommt, dass viele Entwicklerinnen und Entwickler nur begrenztes Wissen über die WCAG-Anforderungen besitzen und Accessibility-Aspekte nicht automatisch berücksichtigt werden.⁶⁶ Auch der zunehmende Einsatz komplexer Architekturen und Frameworks erschwert die korrekte Umsetzung barrierefreier Interaktionen. In Kombination mit Zeitdruck und fehlenden automatisierten Prüfverfahren führt dies dazu, dass Accessibility-Probleme häufig erst spät erkannt oder vollständig übersehen werden.⁶⁷

2.2.1 Klassische Tool-Ansätze

Für die automatisierte Überprüfung von Webanwendungen auf Barrierefreiheit gibt es eine stetig wachsende Anzahl an Werkzeugen. Die WAI führt in ihrer aktuellen Evaluation-Tool-Liste über 160 Werkzeuge⁶⁸, von denen 84 explizit die WCAG 2.1 adressieren.⁶⁹ Alle gängigen Werkzeuge arbeiten regelbasiert, d. h. sie prüfen den DOM einer geladenen Seite anhand eines vordefinierten Regelkatalogs gegen bekannte Verstoßmuster.⁷⁰ Das Document Object Model (DOM) ist eine vom Browser erzeugte, baumartige Objektstruktur, die den HTML-Code einer Webseite als hierarchische Struktur aus Elementknoten repräsentiert und über Programmierschnittstellen von Skripten und Analysewerkzeugen zugänglich macht.⁷¹

⁶⁴Vgl. Bundesministerium für Digitales und Staatsmodernisierung 2025

⁶⁵Vgl. Bundesministerium für Digitales und Staatsmodernisierung 2025

⁶⁶Vgl. Bassi et al. 2025, S. 298

⁶⁷Vgl. Mowar 2024, S. 123

⁶⁸Vgl. World Wide Web Consortium (W3C) 2026a

⁶⁹Vgl. Delnevo, Andruccioli, Mirri 2024

⁷⁰Vgl. Kodithuwakku, Wickramaarachchi 2025; Manca et al. 2023, 3:9

⁷¹Vgl. Robie 2026

Zu den meistgenutzten automatisierten Barrierefreiheitstools gehören:

- **axe-core**, entwickelt von Deque Systems, hat sich als Standardlösung für die automatische Integration von Accessibility-Tests in Entwicklungspipelines etabliert.⁷²
- **WAVE** von WebAIM bietet visuelle Browser-Extensions, die Fehler direkt in der gerenderten Seitenansicht annotiert.⁷³
- **Lighthouse** ist in die Chrome-DevTools integriert und kombiniert Accessibility-Testing mit SEO-Metriken und Performanceanalysen.⁷⁴
- **IBM Equal Access Checker** und **ARC Toolkit** (TPGi) runden das Spektrum der verbreiteten Werkzeuge ab.

Der typische Output solcher Werkzeuge ist ein strukturierter Violation Report, bei dem jedem erkannten Verstoß eine Bezeichnung, Beschreibung und Schweregrad zugewiesen wird (Beispielsweise bei axe-core: minor, moderate, serious, critical), sowie falls möglich auch ein HTML-Ausschnitt des betroffenen Elements ausgegeben.⁷⁵ Dieser standardisierte Output bildet in der vorliegenden Arbeit die Grundlage für die erste Stufe der Datenpipeline.

2.2.2 Grenzen bestehender Tools

Trotz ihrer weiten Verbreitung weisen regelbasierte Accessibility-Tools fundamentale Grenzen auf. Diese lassen sich in drei Kategorien einteilen: begrenzte Regelabdeckung, semantische Blindheit und fehlende Dynamik.

Die Regelabdeckung der Werkzeuge ist erheblich geringer als oftmals angenommen. Kodithuwakku und Wickramarachchi zeigen, dass selbst die leistungsfähigsten untersuchten Tools lediglich 21% aller WCAG-Tests automatisiert abdecken können (vgl. Tabelle 1).⁷⁶ Für Tastaturzugänglichkeit und auditive Barrieren lag die Erkennungsrate in ihrer Studie bei null Prozent. Ein zusätzliches Ergebnis der Studie war die hohe Rate an *False Positives*, die eine manuelle Nachprüfung erfordern.⁷⁷ Bassi et al. kommen zu dem Ergebnis, dass das beste verfügbare Einzeltool (**ARC Toolkit**) nur 26% der WCAG-Tests abdeckt.⁷⁸ Die Kombination mehrerer Tools in einem Ensemble verbessert die Abdeckung, beseitigt das strukturelle Problem jedoch nicht vollständig. Pool zeigt, dass selbst ein Ensemble aus acht gleichzeitig eingesetzten Tools, darunter auch **axe-core**, **WAVE**, **Qualweb** usw., erhebliche Lücken hinterlässt.⁷⁹ Insgesamt stellte sich in der Studie heraus, dass jedes einzelne Tool mindestens sieben Violations exklusiv erkennt, die von keinem anderen Tool gefunden werden. Problematisch ist dabei auch, dass die meisten Tools

⁷²Vgl. Deque Systems 2026

⁷³Vgl. WebAIM 2026a

⁷⁴Vgl. Google 2025

⁷⁵Vgl. Kodithuwakku, Wickramarachchi 2025; Deque Systems 2026

⁷⁶Vgl. Kodithuwakku, Wickramarachchi 2025

⁷⁷Vgl. Kodithuwakku, Wickramarachchi 2025

⁷⁸Vgl. Bassi et al. 2025, S. 298

⁷⁹Vgl. Pool 2023

ihre eigene Abdeckung nicht kommunizieren: Manca et al. stellen fest, dass von elf untersuchten Werkzeugen lediglich drei explizit auf ihre Erkennungslücken hinweisen.⁸⁰

Tool	True Positives	False Positives	CER (%)	ICR (%)
Lighthouse	17	16	52.94	15.92
WAVE	16	12	35.29	14.65
SiteImprove	23	18	47.06	19.75
axe	15	10	52.94	15.29
Accessibility Insights	24	19	47.06	15.29
A11y	6	8	23.53	3.82
Includia Accessibility Checker	33	31	41.18	21.02

Tab. 1: Vergleich der Erkennungsleistung verschiedener Accessibility-Prüfwerkzeuge.⁸¹ CER beschreibt den Anteil korrekt erkannter Verstöße im Verhältnis zu allen gemeldeten Verstößen, während ICR den Anteil tatsächlich vorhandener Accessibility-Probleme misst, die vom jeweiligen Tool erkannt wurden.

Das gravierendste Defizit liegt in der **semantischen Blindheit** der Werkzeuge.⁸² Regelbasierte Tools können lediglich prüfen, ob ein Accessibility-Attribut vorhanden ist, nicht aber, ob der Inhalt dem richtigen Zweck dient. Ein Bild mit dem Alternativtext *image* oder *IMG_0042.jpg* wird von allen Tools als regelkonform eingestuft, obwohl dieser Text für Screenreadernutzer und Nutzerinnen keinen Mehrwert schafft. **Ma11y**, ein Mutation-Testing-Framework für Accessibility, hat dieses Problem empirisch quantifiziert: bei der systematischen Injektion von 25 gezielten Defekten in reale Webanwendungen erkannten die sechs ausgewählten Tools semantische Verstöße nur zu 26%.⁸³ Die Quote der erkannten syntaktischen Verstöße dagegen lag bei 54%.⁸⁴ Im Durchschnitt blieben also nahezu die Hälfte aller injizierten Defekte unentdeckt.⁸⁵

Ein weiteres strukturelles Problem ist die **fehlende Dynamik** der Analyse. Viele Tools analysieren ausschließlich den initial geladenen DOM-Zustand einer Seite.⁸⁶ Barrieren, die erst durch Nutzerinteraktionen entstehen, wie ein Hilfs- oder Modalfenster, das geöffnet wird; ein Dropdown-Menü, um eine Kategorie auszuwählen; oder das Erscheinen einer Fehlermeldung, bleiben dabei unsichtbar. Hinzu kommt, dass alle Violation Reports generischer Natur sind. Sie beschreiben das Problem auf der Ebene des gerenderten DOM, ohne Bezug zur Quellcodestruktur der geprüften Anwendung herzustellen.⁸⁷

2.2.3 Taxonomie der Verstöße: syntaktisch, semantisch, Layout

Um Barrierefreiheitsverstöße systematisch analysieren und gezielt adressieren zu können, wird in dieser Arbeit eine dreiteilige Taxonomie zugrunde gelegt, die auf Fathallah et al. zurückgeht

⁸⁰Vgl. Manca et al. 2023, S. 4

⁸²Vgl. Fathallah, Hernández, Staab 2025

⁸³Vgl. Tafreshipour et al. 2024, S. 108

⁸⁴Vgl. Tafreshipour et al. 2024, S. 108

⁸⁵Vgl. Tafreshipour et al. 2024, S. 108

⁸⁶Vgl. Fathallah, Hernández, Staab 2025

⁸⁷Vgl. Delnevo, Andruccioli, Mirri 2024

(vgl. Abbildung 3).⁸⁸

- **Syntaktische Verstöße** bezeichnen Fälle, in denen ein erforderliches Accessibility-Element vollständig fehlt oder formal falsch gebildet ist.⁸⁹ Typische Beispiele sind ein fehlendes `alt`-Attribut an einem `<img>`-Element, ein fehlender `aria-label` an einer Schaltfläche ohne sichtbaren Text oder `<table>` ohne `<th>`-Elemente. Diese Kategorie ist für automatisierte regelbasierte Tools am besten prüfbar, da die Verletzung rein struktureller Natur ist.⁹⁰
- **Semantische Verstöße** liegen vor, wenn Accessibility-Attribute zwar vorhanden sind, ihren Zweck aber inhaltlich verfehlen.⁹¹ Ein Bild mit dem Alternativtext *Bild*, ein Formularelement mit der Beschriftung *Eingabe* oder ein Link mit dem Text *hier klicken* sind typische Beispiele. Da die Bewertung solcher Verstöße Sprachverständnis und Kontextwissen erfordert, sind diese viel schwieriger von Tools zu entdecken und bilden damit das primäre Anwendungsfeld für LLM-basierte Analysen.⁹²
- **Layout-Verstöße** betreffen visuelle Barrieren. Zu dieser Kategorie zählen insbesondere unzureichende Farbkontraste⁹³, fehlende oder schwer erkennbare Fokusindikatoren⁹⁴ sowie eine eingeschränkte Zoomfähigkeit. Einige dieser Verstöße können algorithmisch geprüft werden; andere erfordern ein kontextuelles visuelles Urteil.

Taxonomie der Barrierefreiheitsverstöße

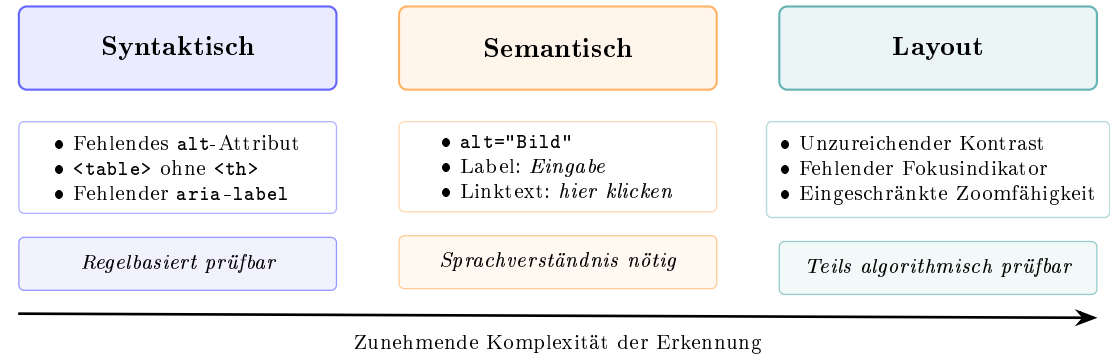


Abb. 3: Taxonomie der Barrierefreiheitsverstöße nach Fathallah et al. mit Beispielen und Erkennungscharakteristik je Kategorie⁹⁵

⁸⁸Vgl. Fathallah, Hernández, Staab 2025

⁸⁹Vgl. Tafreshipour et al. 2024, S. 102

⁹⁰Vgl. Flanagan 2020, S. 569 ff.

⁹¹Vgl. World Wide Web Consortium (W3C) 2023

⁹²Vgl. He, Huq, Malek 2025, FSE101:2

⁹³Vgl. World Wide Web Consortium (W3C) 2023, Erfolgskriterium 1.4.3

⁹⁴Vgl. World Wide Web Consortium (W3C) 2023, Erfolgskriterium 2.4.7–2.4.13

⁹⁵Eigene Abbildung, inspiriert aus Fathallah, Hernández, Staab 2025

2.3 Rich-Client-Webanwendungen und Testautomatisierung

2.3.1 Charakteristika und Abgrenzung

Der Begriff **Rich-Client-Webanwendung** bezeichnet Webanwendungen, bei denen ein Großteil der Anwendungslogik und Darstellungssteuerung clientseitig im Browser ausgeführt wird.⁹⁶ Im Gegensatz zu klassischen server-seitig gerenderten Anwendungen werden bei einer SPA nach dem initialen Laden des HTML-Rahmens ausschließlich Daten zwischen Client und Server ausgetauscht, wobei die Darstellung durch clientseitige Skriptsprachen wie JavaScript oder TypeScript direkt im Browser durch Manipulation des DOM gesteuert wird.⁹⁷ Die zunehmende Komplexität solcher Anwendungen ist in Anhang 2 veranschaulicht.

React, das von Meta entwickelte JavaScript-Framework, ist der dominante Ansatz für die Entwicklung solcher Anwendungen. Laut Stack Overflow Developer Survey 2025 wird React von 44,7% aller befragten Entwicklerinnen und Entwickler eingesetzt.⁹⁸

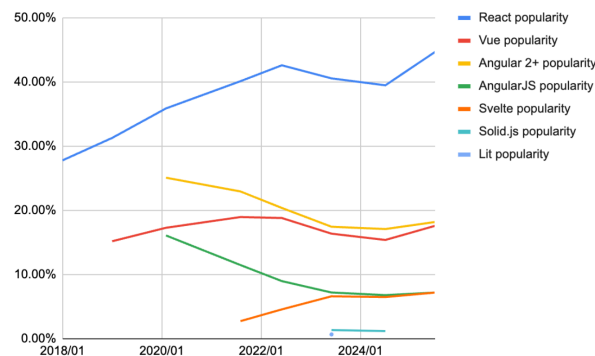


Abb. 4: Framework Popularity über die letzten Jahre⁹⁹

Für die Barrierefreiheitsanalyse sind zwei Architekturmerkmale von React besonders relevant: erstens werden Anwendungslogik und Darstellung in wiederverwendbaren **Komponenten** gekapselt, deren Kommunikation über **Props** (unveränderliche Eingabeparameter) und internen **State** (veränderlicher Zustand) erfolgt.¹⁰⁰ Zweitens wird die Darstellung der Komponenten mithilfe der Syntaxerweiterung JavaScript XML (JSX) (bzw. TSX bei Verwendung von TypeScript) definiert, die HTML-ähnliche Strukturen direkt in JavaScript- oder TypeScript-Code einbettet.¹⁰¹

⁹⁶Vgl. Meta Open Source 2026b

⁹⁷Vgl. Meta Open Source 2026a

⁹⁸Vgl. Stack Overflow 2025

⁹⁹Entnommen aus Krotz 2026

¹⁰⁰Vgl. Meta Open Source 2026b

¹⁰¹Vgl. Bai 2022

2.3.2 Herausforderungen für die Barrierefreiheitsprüfung

Komponentenbasierte Frameworks wie React, Vue oder Angular teilen bestimmte Architekturmerkmale, die die Barrierefreiheitsanalyse grundsätzlich erschweren.¹⁰² Da die vorliegende Arbeit eine React-Anwendung als Untersuchungsgegenstand verwendet, werden die folgenden Herausforderungen anhand von React konkretisiert; sie gelten in ähnlicher Form jedoch auch für andere komponentenbasierte Frameworks.

Erstens entstehen viele Barrierefreiheitsprobleme **zustandsabhängig**, wie bereits in Kapitel 2.2.2 erläutert. Ein Modalfenster, das nach einem Klick geöffnet wird und den Tastaturfokus nicht korrekt verwaltet; eine Fehlermeldung, die nach dem Absenden eines Formulars erscheint und nicht durch eine ARIA-Live-Region programmatisch angekündigt wird; ein Dropdown-Menü, das für Tastaturnutzer nicht navigierbar ist - all diese Probleme sind im initialen Seitenzustand nicht vorhanden und damit für statische Analysewerkzeuge unsichtbar.¹⁰³ In React wird diese Problematik durch das State-Management verstärkt: Zustandsänderungen lösen ein erneutes Rendering einzelner Komponenten aus, wobei sich die Accessibility-Eigenschaften des resultierenden DOM je nach State unterscheiden können.

Zweitens besteht eine **Diskrepanz zwischen Quellcode und gerendertem DOM**. In React werden Benutzeroberflächen in JSX/TSX definiert; der im Browser gerenderte DOM ist das Ergebnis der React-Laufzeitumgebung, die diese Definitionen in DOM-Elemente übersetzt. Ein Accessibility-Tool analysiert den gerenderten DOM, nicht aber den Quellcode, aus dem er hervorging.¹⁰⁴ Für Entwicklerinnen und Entwickler ist jedoch der Quellcode der relevante Interventionspunkt.¹⁰⁵ Ein konkretes Beispiel: eine React-Komponente `IconButton` rendert im DOM ein `<button>`-Element mit einem `<svg>`-Icon, jedoch ohne sichtbaren Text und ohne `aria-label`. axe-core meldet den Verstoß *button-has-visible-text* mit dem HTML-Ausschnitt `<button class="btn-icon">...</button>` - ohne Hinweis darauf, dass die Ursache in der Datei `IconButton.tsx` liegt, wo das `aria-label`-Prop nicht weitergereicht wird.

Drittens erschwert die **Komponentenabstraktion** die Ursachenanalyse: eine wiederverwendbare Komponente mit einem Accessibility-Problem manifestiert sich im gerenderten DOM an jeder Stelle, an der sie eingesetzt wird. Accessibility-Tool-Reports melden jeden einzelnen Treffer separat, ohne den Bezug zur gemeinsamen Quellkomponente herzustellen, deren einmalige Korrektur alle Treffer beseitigen würde.¹⁰⁶

2.3.3 Playwright als Testautomatisierungswerkzeug

Um diese dynamischen Zustände automatisiert reproduzierbar testen zu können, werden Werkzeuge zur Browserautomatisierung benötigt.

¹⁰²Vgl. Tafreshipour et al. 2024, S. 110

¹⁰³Vgl. He, Huq, Malek 2025, S. 5–6; Vgl. World Wide Web Consortium (W3C) 2023, Erf.-krit. 2.4.3 & 4.1.3

¹⁰⁴Vgl. Fernandes et al. 2012, S. 31

¹⁰⁵Vgl. Fathallah, Hernández, Staab 2025

¹⁰⁶Vgl. Abou-Zahra, Brewer, Cooper 2018

Playwright ist ein von Microsoft entwickeltes Open-Source-Framework für die End-to-End-Testautomatisierung von Webanwendungen.¹⁰⁷ Es ermöglicht die automatisierte Steuerung der Browser Chromium, Firefox und WebKit über eine einheitliche API und führt Anwendungen in einem echten Browser-Kontext aus, sodass JavaScript-Ausführung, clientseitiges Rendering und dynamische DOM-Manipulationen vollständig nachgebildet werden.

Für die Barrierefreiheitsanalyse ist Playwright aus mehreren Gründen besonders geeignet. Erstens kann es durch programmierte Interaktionen - Klicks, Formularausfüllungen, Tastatureingaben, Hover-Aktionen - gezielt verschiedene Anwendungszustände auslösen.¹⁰⁸ Zweitens bietet Playwright mit `@axe-core/playwright` eine direkte Integration der axe-core-Bibliothek: nach jeder Interaktion kann unmittelbar eine vollständige axe-Prüfung des aktuellen DOM-Zustands angestoßen werden.¹⁰⁹

Verschiedene Forschungsarbeiten nutzen diese Integration bereits erfolgreich für die Barrierefreiheitsanalyse dynamischer Webanwendungen (vgl. Kapitel 3). In der vorliegenden Arbeit dient Playwright als zentrales Werkzeug der Detection-Phase: es löst definierte Interaktionssequenzen aus, erfasst die resultierenden DOM-Zustände und stößt axe-core-Prüfungen an, deren strukturierter JSON-Output als erster Input für die LLM-Analysepipeline dient.

2.4 KI-Assistenzsysteme für Softwareanalyse und Handlungsempfehlungen

Innerhalb der KI-Forschung haben sich LLMs als besonders leistungsfähige Klasse von Systemen für sprachbasierte Aufgaben etabliert.¹¹⁰ Für die vorliegende Arbeit sind sie relevant, weil sie sowohl Code verstehen als auch natürlichsprachige Handlungsempfehlungen generieren können.

2.4.1 Grundlagen: Large Language Models

LLMs sind neuronale Netzwerke, die auf extrem großen Datensätzen mit dem Ziel trainiert werden, das nächste Token einer Textsequenz vorherzusagen.¹¹¹ Die zugrunde liegende Architektur ist der Transformer, der ausschließlich auf Attention-Mechanismen basiert und rekurrente sowie konvolutionale Netze in der Sequenzmodellierung ablöst.¹¹² Durch dieses Training auf massiver Datenbasis entstehen Modelle, die komplexe Sprachstrukturen, semantische Zusammenhänge und logisches Schlussfolgern internalisieren. Für den Zugriff auf externes Wissen werden häufig *Embedding*-basierte Retrieval-Methoden verwendet (Vektorraumsuche), die in Retrieval-Augmented Generation (RAG)-Systemen zentral sind.¹¹³ Die bekanntesten Vertreter sind GPT-4o (Ope-

¹⁰⁷ Vgl. Microsoft 2026

¹⁰⁸ Vgl. Microsoft 2026

¹⁰⁹ Vgl. Deque Systems 2026

¹¹⁰ Vgl. Russell, Norvig 2021, S. 22

¹¹¹ Vgl. Brynjolfsson, Li, Raymond 2025, S. 5–6

¹¹² Vgl. Vaswani et al. 2023

¹¹³ Vgl. Lewis et al. 2020, S. 2–3

nAI), Claude (Anthropic) und die LLaMA-Modellfamilie (Meta).¹¹⁴ Entscheidend für den Anwendungskontext dieser Arbeit sind die sogenannten Emergent Abilities dieser Modelle: Fähigkeiten wie Code-Generierung, mehrsprachige Übersetzung und kontextuelles Schlussfolgern, die ab einer bestimmten Modellgröße ohne explizites Training auftreten.¹¹⁵

Im Kontext der Barrierefreiheitsanalyse bieten LLMs gegenüber regelbasierten Tools vier wesentliche Vorteile:¹¹⁶

- Sie verfügen über ein Sprachverständnis, das die Bewertung semantischer Inhalte ermöglicht - etwa die Beurteilung, ob ein Alternativtext den Bildinhalt tatsächlich beschreibt.¹¹⁷
- Sie können Code generieren und transformieren, wodurch konkrete Handlungsempfehlungen abgeleitet werden können.¹¹⁸
- Sie können große Mengen an Text analysieren und strukturieren.¹¹⁹
- Sie verarbeiten Kontext und können durch Techniken wie Clustering die Bedeutung eines Elements im Zusammenhang der Gesamtseite beurteilen.¹²⁰

Diesen Stärken stehen bekannte Schwächen gegenüber:

- LLMs können halluzinieren, also plausibel klingende Ausgaben erzeugen, die jedoch faktisch falsch sind.¹²¹
- Sie sind von einem sogenannten *Knowledge-Cutoff* betroffen. Dieser bezeichnet den Stichtag, bis zu dem die Trainingsdaten eines Modells reichen, sodass dem Modell nach diesem Zeitpunkt veröffentlichte Informationen nicht bekannt sind.¹²² Dies gilt auch für das in dieser Arbeit eingesetzte Modell qwen2.5-coder:7b¹²³, das nur den Wissensstand seines Trainingszeitraums abbildet und daher keine danach veröffentlichten WCAG-Änderungen oder Bibliotheks-Updates kennt.
- Bei gleicher Eingabe erzeugen sie nicht immer identische Ausgaben; dieses Verhalten wird als Nicht-Determinismus bezeichnet.¹²⁴
- Die generierten Inhalte können Verzerrungen (*Bias*) aus den Trainingsdaten übernehmen.¹²⁵

¹¹⁴Vgl. Anthropic 2026a; OpenAI 2023; Touvron et al. 2023

¹¹⁵Vgl. Wei, Tay et al. 2022, S. 2

¹¹⁶Vgl. Fathallah, Hernández, Staab 2025

¹¹⁷Vgl. Tafreshipour et al. 2024

¹¹⁸Vgl. Touvron et al. 2023, S. 3

¹¹⁹Vgl. Bassi et al. 2025, S. 298

¹²⁰Vgl. Fathallah, Hernández, Staab 2025

¹²¹Vgl. Huang, L. et al. 2025, 1:2

¹²²Vgl. Anthropic 2026b, S. 7; Huang, L. et al. 2025, 1:9–10

¹²³Vgl. Hui et al. 2024

¹²⁴Vgl. OpenAI 2026; **Anthropic_API_Docs**

¹²⁵Vgl. Bender et al. 2021, S. 614–615

Eine für die vorliegende Arbeit zentrale Frage ist, ob diese Fähigkeiten auch bei kleineren, lokal ausführbaren Modellen in ausreichendem Maße vorhanden sind. Alle in Kapitel 3 untersuchten Systeme - AccessGuru, GenA11y, ACCESS - setzen GPT-4o ein, ein Modell mit geschätzt über 200 Milliarden Parametern und Zugang zu umfangreichen Cloud-Ressourcen.¹²⁶ Lokal ausführbare Modelle der 7B-Klasse verfügen über deutlich weniger Parameter und damit potenziell über ein eingeschränkteres Sprachverständnis und schwächere Reasoning-Fähigkeiten. Ob ein solches Modell semantische Accessibility-Verstöße zuverlässig erkennen kann, ist empirisch bislang nicht belegt und wird in der vorliegenden Arbeit als explizite Forschungsfrage adressiert. Die Begründung der konkreten Modellauswahl erfolgt in Kapitel 5.4.

2.4.2 Prompt-Engineering-Strategien

Die Qualität der Ausgaben eines LLM hängt maßgeblich von der Formulierung der Eingabe - dem sogenannten Prompt - ab. Prompt Engineering bezeichnet die systematische Gestaltung dieser Eingaben, um die Ausgabequalität zu maximieren.¹²⁷ Im Folgenden werden die für diese Arbeit relevanten Strategien beschrieben, die in Kapitel 7 als Grundlage für das Prompt-Design des PoC dienen.

- **Zero-Shot Prompting** bezeichnet die direkte Anfrage ohne Beispiele oder Kontextinformationen. Das Modell nutzt ausschließlich sein vortrainiertes Wissen. Diese Strategie ist einfach umzusetzen, erzielt bei komplexen Aufgaben wie der Barrierefreiheitsanalyse jedoch häufig unzureichende Ergebnisse.¹²⁸
- **Few-Shot Prompting** ergänzt den Prompt um eine kleine Anzahl von Beispielen (Input-Output-Paaren), die dem Modell das erwartete Ausgabeformat und die inhaltlichen Erwartungen verdeutlichen. Dieser Ansatz verbessert die Konsistenz und Präzision der Ausgaben erheblich, erfordert jedoch sorgfältig ausgewählte Beispiele.¹²⁹ Als Vergleich von Zero-Shot- und Few-Shot-Prompting dient die Grafik 5.
- **Chain-of-Thought Prompting (CoT)** veranlasst das Modell, seinen Lösungsweg Schritt für Schritt zu erklären, bevor es zur finalen Antwort gelangt. Dieses explizite Reasoning reduziert nachweislich Halluzinationen bei komplexen Schlussfolgerungen und macht die Entscheidungslogik des Modells für eine Prüfung durch den Nutzer zugänglich.¹³⁰
- **ReAct-Prompting** kombiniert Reasoning (Re) und Action (Act) in einem iterativen Prozess: das Modell wechselt zwischen Denken (Reasoning) und Handeln, wie das konstante

¹²⁶Vgl. Fathallah, Hernández, Staab 2025; Vgl. He, Huq, Malek 2025, FSE101:10; Vgl. Huang, C. et al. 2024, S. 7

¹²⁷Vgl. Schulhoff et al. 2024, S. 4

¹²⁸Vgl. Brown et al. 2020, S. 4; Vgl. Njifenjou et al. 2024

¹²⁹Vgl. Brown et al. 2020, S. 5

¹³⁰Vgl. Wei, Wang, X. et al. 2022, S. 2; Fathallah, Hernández, Staab 2025

Wechseln von Tools und Daten. Dadurch werden die Ausgaben der zurückgemeldeten Ergebnisse verfeinert. Huang et al. demonstrieren, dass ReAct-Prompting bei der automatisierten Barrierefreiheitskorrektur (**ACCESS**) mit einem Severity Score Reduction von 51,3% alle anderen verglichenen Strategien übertrifft.¹³¹

- **Role-Play Prompting** weist dem Modell eine Expertenrolle zu, die seine Ausgaben in Richtung domänenspezifischen Wissens lenkt. Ein Prompt, der das Modell als „erfahrenen Barrierefreiheitsexperten und React-Entwickler“ adressiert, verbessert nachweislich die Qualität accessibility-spezifischer Ausgaben, da das Modell seine Antworten stärker an der Perspektive eines Fachexperten ausrichtet.¹³²
- **Metacognitive Prompting** fordert das Modell auf, die eigene Ausgabe in einem zweiten Schritt zu reflektieren und zu bewerten, bevor diese finalisiert wird.¹³³ In Verbindung mit Corrective Re-Prompting - einer Technik, bei der fehlerhafte Ausgaben dem Modell zusammen mit einer Fehlerbeschreibung als Feedback zurückgespielt werden - ermöglicht dieser Ansatz eine iterative Qualitätssteigerung. Fathallah et al. belegen, dass Corrective Re-Prompting in **AccessGuru** den Violation Score Decrease von 0,72 auf 0,84 verbessert.¹³⁴

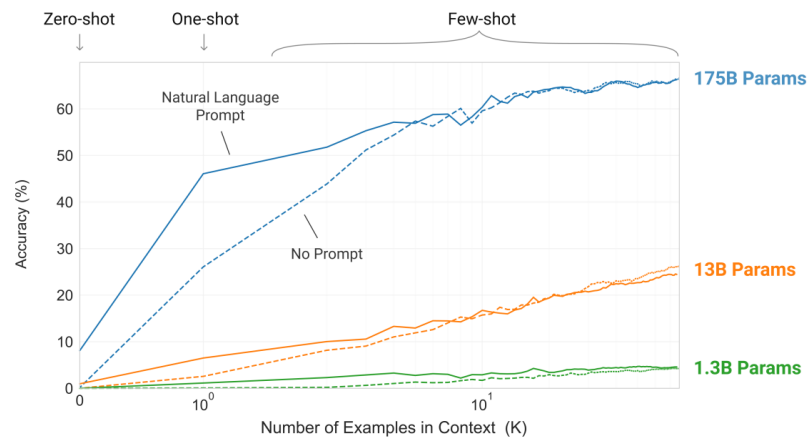


Abb. 5: Zero-Shot und Few-Shot Prompting im Vergleich¹³⁵

2.4.3 RAG und Fine-Tuning als Erweiterungsansätze

Neben Prompt-Engineering-Strategien existieren zwei weitergehende Ansätze, um die Leistungsfähigkeit von LLMs zu verbessern.

¹³¹ Vgl. Huang, C. et al. 2024, S. 5; Yao et al. 2022, S. 1

¹³² Vgl. Fathallah, Hernández, Staab 2025; Njifenjou et al. 2024; Schulhoff et al. 2024, S. 6, 12

¹³³ Vgl. Wang, Y., Zhao 2024; Schulhoff et al. 2024, S. 8–9

¹³⁴ Vgl. Fathallah, Hernández, Staab 2025

¹³⁵ Entnommen aus Brown et al. 2020

Retrieval-Augmented Generation (RAG) erweitert den Prompt eines LLM zur Laufzeit mit thematisch passenden Dokumenten, die aus einer externen Wissensbasis abgerufen werden.¹³⁶ Dabei kombiniert RAG ein Sprachmodell mit einem Retrieval-Mechanismus, der relevante Dokumente aus einem externen Korpus abrufen und als zusätzlichen Kontext in die Generierung einbindet.¹³⁷ Im Kontext der Barrierefreiheitsanalyse könnte eine RAG-Pipeline aktuelle WCAG-Erfolgskriterien, Techniken und bekannte Lösungsmuster für das Modell abrufbar machen und so sicherstellen, dass es unabhängig von seinem Trainings-Cutoff mit aktuellen Anforderungen arbeitet.

Fine-Tuning bezeichnet das domänenspezifische Training eines vortrainierten Modells auf einem kleineren, aufgabenspezifischen Datensatz.¹³⁸ Delnevo et al. argumentieren, dass ein Fine-Tuning auf einem entsprechenden Datensatz die Konsistenz von Bewertungen signifikant verbessern könnte, testen diesen Ansatz jedoch nicht empirisch.¹³⁹ Für den Kontext der öffentlichen Verwaltung wäre insbesondere ein Fine-Tuning auf BITV-spezifischen Anforderungen sowie internen Entwicklungsstandards der BITBW denkbar.

Beide Ansätze liegen außerhalb des Scopes dieser Arbeit und werden als Ausblick in Abschnitt 8.6 und Abschnitt 9.3 aufgegriffen.

2.4.4 Datenschutz bei KI-gestützten Accessibility-Services

Der Einsatz von LLMs im Kontext öffentlicher Verwaltungen wirft spezifische Datenschutzfragen auf. Webanwendungen der öffentlichen Verwaltung verarbeiten häufig personenbezogene Daten - etwa in Formularseiten, die persönliche Angaben von Bürgerinnen und Bürgern erfassen. Werden diese Seiten für eine KI-gestützte Accessibility-Analyse an externe LLM-APIs übermittelt, ist die Wahrung der Datenschutzgarantien fraglich.

Die DSGVO schreibt vor, dass personenbezogene Daten nur auf Basis einer Rechtsgrundlage und unter Wahrung der Grundsätze der Zweckbindung und Datenminimierung verarbeitet werden dürfen.¹⁴⁰ Die Übermittlung von HTML-Quellcode der produktiven Anwendung, der im schlimmsten Fall personenbezogene Daten als Attributwerte enthalten kann, an externe API-Dienste wie die OpenAI API oder die Anthropic API ist ohne explizite Rechtsgrundlage und einen Auftragsverarbeitungsvertrag nach Art. 28 Datenschutz-Grundverordnung (DSGVO) für öffentliche Verwaltungen in Deutschland problematisch.¹⁴¹

Als Lösungsansatz wird in dieser Arbeit ein Lokal-First-Prinzip verfolgt: das KI-Assistenzsystem soll ohne externe API-Aufrufe betreibbar sein, indem lokal ausführbare Open-Source-Modelle - etwa Modelle der LLaMA-Familie oder Mistral - eingesetzt werden.¹⁴² Damit bleiben sämtliche

¹³⁶ Vgl. Lewis et al. 2020, S. 1–2

¹³⁷ Vgl. Gao, Y. et al. 2023, S. 1–2

¹³⁸ Vgl. Silvestre, Pietzuch 2025, S. 90

¹³⁹ Vgl. Delnevo, Andruccioli, Mirri 2024

¹⁴⁰ Vgl. Europäisches Parlament und Rat der Europäischen Union 2016

¹⁴¹ Vgl. Europäisches Parlament und Rat der Europäischen Union 2016, Art. 28

¹⁴² Vgl. Touvron et al. 2023

Analyse- und Codedaten innerhalb der eigenen Infrastruktur. Diese Anforderung prägt sowohl die nicht-funktionalen Anforderungen (Kapitel 5) als auch die Architekturentscheidungen (Kapitel 7) dieser Arbeit.

Die in diesem Kapitel erarbeiteten theoretischen Grundlagen - das normative Fundament der WCAG, die empirisch belegten Grenzen regelbasierter Tools, die Besonderheiten von React-Anwendungen sowie die Möglichkeiten und Grenzen von LLMs und Prompt-Engineering - bilden den konzeptionellen Rahmen für die im folgenden Kapitel systematisch aufgearbeitete Forschungsliteratur und Methodik.

3 Stand der Forschung

Das folgende Kapitel gibt einen thematisch strukturierten Überblick über den Stand der Forschung zu KI-gestützter Barrierefreiheitsanalyse und arbeitet die verbleibenden Forschungslücken heraus, die den Ausgangspunkt der vorliegenden Arbeit bilden. Die relevanten Arbeiten lassen sich in drei Entwicklungslinien einordnen: (1) LLM-basierte Erkennung von Accessibility-Verstößen, (2) LLM-basierte Korrektur und Code-Generierung sowie (3) hybride Pipelines, die regelbasierte Tools mit LLM-Analyse kombinieren.

3.1 Vorgehen der Literaturrecherche

Zur Einordnung des Forschungsstands wurde eine strukturierte Literaturrecherche in wissenschaftlichen Datenbanken durchgeführt.¹⁴³ Berücksichtigt wurden insbesondere ACM Digital Library, IEEE Xplore, Science Direct, arXiv und ResearchGate. Die Recherche erfolgte themenorientiert anhand von Suchbegriffen wie „web accessibility“, „LLM accessibility“, „accessibility violation detection“, „accessibility remediation“, „prompt engineering accessibility“, „Playwright accessibility“ und „axe-core“. Berücksichtigt wurden primär aktuelle Arbeiten mit Bezug zur automatisierten Erkennung oder Behebung von Barrierefreiheitsverstößen in Webanwendungen mit KI-Assistenzsystemen. Ausgeschlossen wurden Arbeiten ohne klaren Webbezug, rein konzeptionelle Beiträge ohne methodische Auswertung sowie Beiträge mit fehlender Übertragbarkeit auf den Untersuchungskontext dieser Arbeit. Die identifizierten Arbeiten wurden anschließend thematisch gruppiert, priorisiert, eingeordnet und vergleichend ausgewertet.

3.2 Von frühen KI-Ansätzen zur LLM-basierten Erkennung

Frühe Arbeiten wie Abou-Zahra et al. (2018) identifizierten zwar grundlegende Einsatzmöglichkeiten von KI für die Barrierefreiheitsanalyse, lieferten jedoch noch keine belastbaren quantitativen Leistungsmaße.¹⁴⁴ Mit dem Aufkommen leistungsfähiger Large Language Models verlagerte sich der Forschungsfokus auf die empirische Bewertung konkreter Modellleistungen. Delnevo et al. (2024) zeigen, dass GPT-3.5 einfache syntaktische Verstöße mit moderater Genauigkeit erkennen kann, bei semantisch anspruchsvolleren Problemen jedoch inkonsistente Bewertungen liefert.¹⁴⁵ Bassi et al. (2025) verglichen GPT-4o, GPT-3.5 und LLaMA 3.1 in einer zweistufigen Evaluation von HTML-Snippets und Webseiten. Für GPT-4o berichten sie in der ersten Prüfphase einen Anteil korrekt identifizierter Verstöße von 74%, während 15% der gemeldeten Verstöße als Halluzinationen eingestuft wurden.¹⁴⁶ Eine nachgelagerte Chain-of-Verification-artige Verifikationsstufe erhöhte die Zuverlässigkeit der Befunde deutlich, da die im ersten Schritt erkannten

¹⁴³Vgl. Webster, Watson 2002

¹⁴⁴Vgl. Abou-Zahra, Brewer, Cooper 2018

¹⁴⁵Vgl. Delnevo, Andruccioli, Mirri 2024

¹⁴⁶Vgl. Bassi et al. 2025, S. 302

Verstöße anschließend gezielt auf tatsächliche Fehler oder Halluzinationen überprüft wurden; ein Einsatz von RAG wird von Bassi et al. hingegen nur als mögliche zukünftige Erweiterung diskutiert.¹⁴⁷

3.3 LLM-basierte Korrektur und Prompt-Engineering

Abu Doush und Kassem (2024) zeigten, dass strukturierte Prompts die Qualität LLM-generierter barrierefreier HTML-Alternativen erheblich verbessern, jedoch bei komplexeren Interaktionsmustern weiterhin Schwächen bestehen.¹⁴⁸ Guriță und Vataavu (2025) belegten, dass accessibility-orientierte Prompts paradoxerweise zu mehr Violations führen können als neutrale Prompts; erst iteratives Prompting über fünf Runden senkte den Violation Score von 0,84 auf 0,32.¹⁴⁹

Den aktuellen Stand der Technik repräsentiert **AccessGuru** von Fathallah et al. (2025).¹⁵⁰ Das System kombiniert eine zweistufige Pipeline - regelbasierte Detection via Playwright und **axe-core**, anschließend LLM-basierte Korrektur durch GPT-4o mit Role-Play, Contextual und Metacognitive Prompting. Corrective Re-Prompting verbessert den Violation Score Decrease von 0,72 auf 0,84 bei einer semantischen Ähnlichkeit von 77% zu menschlichen Referenzlösungen. Als offenes Problem benennen die Autoren die fehlende Verknüpfung korrigierter HTML-Snippets mit dem Quellcode der geprüften Anwendung.

Fernández-Navarro und Chicano (2026) erweitern diesen Ansatz um eine direkte Quellcode-Korrektur und adressieren damit explizit die Lücke zwischen DOM-Korrektur und entwicklerseitiger Umsetzung.¹⁵¹ Ihr Tool operiert in zwei Modi: Für statische Webseiten modifiziert es den gerenderten DOM direkt via BeautifulSoup; für Angular-SPAs injiziert es Korrekturen auf Basis einer Abstract-Syntax-Tree-Analyse gezielt in die tatsächlichen Komponentendateien (`.component.html`, `.component.ts`). Als Detektions-Backend kommt axe-core zum Einsatz, die Generierung der Fixes übernimmt GPT-4o, einschließlich multimodaler Bildbeschreibung für **alt**-Attribute. Eine mehrstufige Validierung – axe-core-Re-Scan nach Korrektur sowie Angular-Build-Check (`ng build`) – stellt sicher, dass keine neuen Verstöße eingeführt oder die Anwendung in einen nicht-kompilierbaren Zustand versetzt wird. In der Evaluation über 12 öffentliche Websites und 6 Open-Source-Angular-Projekte erreicht das System eine Remediation Rate von durchschnittlich 80,4% (statische Sites) und 86,0% (Angular-SPAs) bei 100% Build Integrity.¹⁵² Der Ansatz beschränkt sich jedoch auf Angular und setzt auf cloudbasiertes GPT-4o; eine Übertragung auf React wird von den Autoren als kurzfristiger Folgeschritt benannt.

¹⁴⁷ Vgl. Bassi et al. 2025, S. 301

¹⁴⁸ Vgl. Doush, Kassem 2025, S. 343

¹⁴⁹ Vgl. Guriță, Vataavu 2025, S. 1000

¹⁵⁰ Vgl. Fathallah, Hernández, Staab 2025

¹⁵¹ Vgl. Fernández-Navarro, Chicano 2026

¹⁵² Vgl. Fernández-Navarro, Chicano 2026

3.4 Hybride Pipelines und Ensemble-Testing

Huang et al. entwickelten 2024 mit **ACCESS** ein System, das Prompt Engineering gezielt zur automatisierten Barrierefreiheitskorrektur einsetzt und dabei verschiedene Strategien systematisch vergleicht.¹⁵³ Der Vergleich von ReAct, Chain-of-Thought und Few-Shot Prompting zeigt, dass ReAct mit einem Severity Score Reduction von 51,3% alle anderen Strategien übertrifft - ein Ergebnis, das die Bedeutung iterativer, werkzeuggestützter Reasoning-Strategien für diese Aufgabe unterstreicht.

He et al. präsentierten 2025 mit **GenA11y** das methodisch ausgefeiltste Detection-System des aktuellen Forschungsstands.¹⁵⁴ **GenA11y** extrahiert nach vollständigem clientseitigem Rendering den DOM via Playwright und prüft ihn anschließend kriteriumsspezifisch mit GPT-4o. Für jedes WCAG-Erfolgskriterium wird ein eigener, optimierter Prompt eingesetzt. Eine Ablation Study belegt, dass weder DOM-Extraktion noch optimiertes Prompting allein die Gesamtleistung von Precision 94,5% und Recall 87,61% erreichen - beide Komponenten sind also notwendige Bestandteile des Systems.¹⁵⁵ **GenA11y** erkennt acht Verstößtypen, die von keinem der verglichenen regelbasierten Tools gefunden werden.¹⁵⁶ Paternò et al. ergänzen dieses Bild mit **TeVal**, das semantische WCAG-Kriterien durch Role-Play Prompting mit Few-Shot-Beispielen und Confidence Score bewertet und dabei eine Effektivität von 92,4% erzielt.¹⁵⁷

Pool (2023) adressiert die Detection-Schicht mit **Testaro**, einem Ensemble aus acht Accessibility-Werkzeugen (u. a. **axe-core**, **WAVE**, **QualWeb**).¹⁵⁸ Die empirische Analyse zeigt, dass jedes Tool exklusive Violations erkennt, was die Komplementarität der Werkzeuge belegt.

3.5 Forschungslücken und Einordnung der vorliegenden Arbeit

Die Analyse der bestehenden Forschungsarbeiten zeigt drei persistente Forschungslücken, die den Ausgangspunkt der vorliegenden Arbeit definieren.

Erstens berücksichtigt keiner der untersuchten Ansätze systematisch zustandsabhängige Barrieren, die erst durch Nutzerinteraktionen in React-basierten Single-Page Applications entstehen.¹⁵⁹ Zwar setzen einzelne Systeme wie **AccessGuru** Playwright zur DOM-Extraktion ein, jedoch beschränkt sich dies auf das initiale Rendering - interaktionsbedingte Zustandswechsel werden nicht geprüft.¹⁶⁰

¹⁵³Vgl. Huang, C. et al. 2024, S. 7, 9

¹⁵⁴Vgl. He, Huq, Malek 2025, FSE101:20-21

¹⁵⁵Vgl. He, Huq, Malek 2025, FSE101:20

¹⁵⁶Vgl. He, Huq, Malek 2025, FSE101:20

¹⁵⁷Vgl. Paternò et al. 2025

¹⁵⁸Vgl. Pool 2023

¹⁵⁹Vgl. Tafreshipour et al. 2024, S. 110

¹⁶⁰Vgl. Fathallah, Hernández, Staab 2025

Zweitens verknüpft kein bekannter Ansatz Tool-Reports mit dem Quellcode der geprüften Anwendung. **AccessGuru** selbst benennt dies als offenes Problem: die generierten Korrekturen beziehen sich auf isolierte DOM-Snippets und nicht auf die JSX-Komponenten, in denen Entwicklerinnen und Entwickler tatsächlich eingreifen müssten.¹⁶¹ Die Lücke zwischen technischer Fehlererkennung und entwicklernaher Umsetzung bleibt damit ungeschlossen.

Drittens adressiert keine der untersuchten Arbeiten die datenschutzrechtlichen Anforderungen öffentlicher Verwaltungen: alle Systeme setzen auf cloudbasierte LLM-APIs (primär GPT-4o), was für die BITBW und vergleichbare Institutionen ohne vertiefte datenschutzrechtliche Prüfung nicht nutzbar ist.¹⁶²

Die Konzeptmatrix in Tabelle 2 fasst die zentralen Merkmale der untersuchten Studien im Vergleich zum eigenen PoC zusammen und macht die Forschungslücken strukturell sichtbar.

Studie		Konzepte													
Autor(en) & Jahr	System	Ziel			Datenquellen			KI-/LLM-Ansatz			Evaluation			Kontext	
		WCAG-Konf. prüfen	Barrieren korrigieren	Entwickler unterstützen	Tool-Reports (axe etc.)	Interaktionsdaten (Playwright)	Quellcode-Analyse	LLM-basiert	Kontextsensitiv / RAG	Lokal ausführbar	Automatisierte Eval.	Qualitative Eval.	Nutzerstudie	Rich-Client / SPA	Rechtl. Rahmen (BFSG etc.)
Hoch priorisierte Studien															
He et al. (2025)	GenA11y	x			x			x			x				
Fathallah et al. (2025)	AccessGuru	x	x	x	x	x		x			x				
Bassi et al. (2025)	LLM Auditing	x	x	x				x				x			x
Paternò et al. (2025)	TeVal	x		x	x			x	x			x			
Tafreshipour et al. (2024)	Ma11y	x			x	x					x				
Fernández-Navarro & Chicano (2026)	LLM Remediation	x	x	x	x		x	x			x			x	
Martínez Campo (2026)	ACE (PoC)	x	x	x	x	x	x	x	x	x		x		x	x

Tab. 2: Konzeptmatrix nach Webster Watson¹⁶³: hoch priorisierte Studien im Vergleich zum eigenen Proof of Concept (PoC). **x** = Konzept vorhanden/adressiert; leer = nicht adressiert. Für den eigenen PoC bezeichnen die Markierungen angestrebte Designziele, deren Umsetzung und Wirksamkeit in Kapitel 8 evaluiert wird. Die vollständige Matrix aller analysierten Studien findet sich in Anhang 3.

¹⁶¹ Vgl. Fathallah, Hernández, Staab 2025

¹⁶² Vgl. Europäisches Parlament und Rat der Europäischen Union 2016, Art. 28, 44 ff.

¹⁶³ Vgl. Webster, Watson 2002.

4 Methodik

Das folgende Kapitel legt das methodische Vorgehen der Arbeit dar. Zunächst wird die gewählte Forschungsmethode begründet (Abschnitt 4.1), anschließend der Untersuchungsgegenstand abgegrenzt (Abschnitt 4.2) und schließlich das Evaluationsdesign mit den zugehörigen Bewertungskriterien definiert (Abschnitt 4.3).

4.1 Wissenschaftliche Methode

Die vorliegende Arbeit folgt dem Forschungsparadigma des DSR, das von Hevner et al. für die Wirtschaftsinformatik systematisiert wurde.¹⁶⁴ DSR eignet sich für diese Arbeit aus zwei Gründen: Erstens steht die Konstruktion eines konkreten technischen Artefakts - des Proof of Concept zur KI-gestützten Barrierefreiheitsanalyse - im Zentrum der Forschungsfrage, nicht die Überprüfung einer statistischen Hypothese. Zweitens fordert DSR explizit die Evaluation des Artefakts in einem realen Anwendungskontext, was dem praxisorientierten Charakter der Arbeit entspricht.¹⁶⁵ Alternative Forschungsansätze wie Action Research oder kontrollierte Experimente wurden verworfen: Action Research setzt eine iterative Zusammenarbeit mit Praktikern über mehrere Zyklen voraus, die im Rahmen einer Bachelorarbeit nicht realisierbar ist.¹⁶⁶ Ein kontrolliertes Experiment würde eine statistisch belastbare Stichprobe vergleichbarer Webanwendungen erfordern, die für den spezifischen Kontext öffentlicher Verwaltungen nicht verfügbar ist.

Als prozessualer Rahmen dient das sechsphasige DSRM-Prozessmodell nach Peffers et al. Tabelle 3 ordnet die Phasen den jeweiligen Kapiteln dieser Arbeit zu.¹⁶⁷

DSRM-Phase	Kapitel
Problem-Identifikation	Kapitel 1-3
Zieldefinition	Kapitel 1.3 & 3.5
Design & Entwicklung	Kapitel 5-6
Demonstration	Kapitel 7
Evaluation	Kapitel 8
Kommunikation	Kapitel 9

Tab. 3: Zuordnung der DSRM-Phasen zu den Kapiteln der Arbeit.

Ergänzt wird dieser Rahmen durch einen Fallstudienansatz nach Yin.¹⁶⁸ Das von der BITBW entwickelte Fachverfahren BW-Empfangsclient dient als Einzelfallstudie, an der das entwickelte

¹⁶⁴Vgl. Hevner et al. 2004

¹⁶⁵Vgl. Hevner et al. 2004, S. 80

¹⁶⁶Vgl. Baskerville, R. L. 1999, S. 1–2; Vgl. Baskerville, R., Myers 2004, S. 239–335

¹⁶⁷Vgl. Peffers et al. 2007

¹⁶⁸Vgl. Yin 2018

System demonstriert und evaluiert wird. Der Fallstudienansatz ist geeignet, weil die Forschungsfrage auf ein zeitgenössisches Phänomen (KI-gestützte Accessibility-Analyse) in einem spezifischen Praxiskontext (öffentliche Verwaltung, React-Anwendung) abzielt, bei dem die Grenzen zwischen technischem System und organisatorischem Kontext nicht trennscharf sind.

Die Wahl eines PoC als Artefakttyp ist durch die explorative Natur der Forschungsfrage begründet: das Ziel ist die konzeptionelle Demonstration der technischen Machbarkeit, nicht die statistisch belastbare Generalisierung auf eine Grundgesamtheit. Dieses Vorgehen entspricht dem in der Wirtschaftsinformatik etablierten Ansatz für frühe Forschungsphasen, in denen zunächst die Funktionsfähigkeit eines Artefakts nachzuweisen ist, bevor großangelegte Wirkungsstudien durchgeführt werden können.¹⁶⁹

4.2 Untersuchungsgegenstand und Scope-Abgrenzung

Als Untersuchungsgegenstand dient der BWEC, eine öffentlich zugängliche React-basierte Webanwendung der BITBW.¹⁷⁰ Die Anwendung eignet sich als Fallstudie, da sie drei für die Forschungsfrage zentrale Eigenschaften vereint: gesetzlich relevante Anforderungen an digitale Barrierefreiheit, einen datenschutzsensiblen Einsatzkontext in der öffentlichen Verwaltung sowie typische Interaktionsmuster moderner Single-Page Applications.

Der Untersuchungsscope ist wie folgt abgegrenzt:

- **Eingeschlossen:** Öffentlich zugängliche Bereiche der Anwendung ohne Authentifizierung. Betrachtet werden Barrierefreiheitsverstöße bezüglich WCAG 2.2 auf Konformitätsstufe A und AA, soweit sie durch regelbasierte Prüfung, DOM-Analyse und Interaktionsausführung im Frontend beobachtbar sind.¹⁷¹
- **Ausgeschlossen:** Backend-Logik, Datenbankstrukturen, organisatorische Betriebsprozesse sowie eine vollständige rechtliche Bewertung im Sinne einer formalen BITV-Prüfung.¹⁷²

Die Arbeit erhebt keinen Anspruch auf vollständige Abdeckung aller WCAG-Erfolgskriterien oder auf Generalisierbarkeit auf beliebige Frameworks und Anwendungstypen. Diese Einschränkungen werden in Kapitel 8 als Limitationen diskutiert.

4.3 Evaluationsdesign und Kriterien

Ziel der Evaluation ist es, die technische Machbarkeit und den praktischen Nutzen des entwickelten PoC im Anwendungskontext des BWEC zu untersuchen.

¹⁶⁹Vgl. Hevner et al. 2004

¹⁷⁰Vgl. Land Baden-Württemberg 2025

¹⁷¹Vgl. World Wide Web Consortium (W3C) 2023

¹⁷²Vgl. Bundesministerium für Digitales und Staatsmodernisierung 2025

4.3.1 Evaluationslogik

Die Evaluation folgt einem fünfstufigen Vorgehen:

1. **Seitenauswahl:** Relevante öffentlich zugängliche Seiten und Interaktionszustände des BWEC werden für die Untersuchung ausgewählt.
2. **Manuelles Referenzaudit:** Der Autor führt ein manuelles Audit der ausgewählten Seiten durch, um vorhandene Barrierefreiheitsverstöße zu identifizieren und nach Verstoßtyp (syntaktisch, semantisch, interaktionsbezogen) zu kategorisieren. Als Prüfgrundlage dienen die WCAG 2.2-Erfolgskriterien der Stufen A und AA.¹⁷³
3. **Automatisierte Analyse:** Dieselben Seiten werden mit dem entwickelten PoC analysiert.
4. **Vergleich:** Die Ergebnisse des Systems werden dem manuellen Audit gegenübergestellt. Dabei wird erfasst, welche Verstöße korrekt erkannt, welche übersehen und welche fälschlich gemeldet werden.
5. **Wirksamkeitsprüfung:** Eine Stichprobe der generierten Handlungsempfehlungen wird manuell in repräsentativen Codeausschnitten umgesetzt und anschließend erneut mit axe-core geprüft, um den Violation Score Decrease zu messen.¹⁷⁴

4.3.2 Bewertungskriterien

Die Evaluation erfolgt qualitativ anhand von vier Kriterien, die auf Basis der Forschungsfragen und der Fachliteratur entwickelt wurden.

Kriterium 1 - Korrektheit der Erkennung: Die vom System identifizierten Verstöße werden mit dem manuellen Referenzaudit verglichen. Dabei wird differenziert, welche Verstoßtypen erkannt werden und wo das System versagt. Als Orientierungswerte dienen die Ergebnisse vergleichbarer Systeme: **GenA11y** erreicht eine Precision von 94,5% und einen Recall von 87,61%.¹⁷⁵

Kriterium 2 - Qualität der Handlungsempfehlungen: Die generierten Empfehlungen werden hinsichtlich Verständlichkeit, Korrektheit und direkter Umsetzbarkeit beurteilt. Quantitativ wird der Violation Score Decrease durch axe-core-Nachprüfung nach Umsetzung einer Stichprobe gemessen. **AccessGuru** erreicht als Referenzwert einen Violation Score Decrease von 84%.¹⁷⁶

¹⁷³Vgl. World Wide Web Consortium (W3C) 2023

¹⁷⁴Vgl. Deque Systems 2026; Vgl. Pool 2023; Vgl. Kodithuwakku, Wickramaarachchi 2025

¹⁷⁵Vgl. He, Huq, Malek 2025, FSE101:20

¹⁷⁶Vgl. Fathallah, Hernández, Staab 2025

Kriterium 3 - Mehrwert der Kontextualisierung: Um den Einfluss der Quellcode- und Playwright-Kontextualisierung isoliert bewerten zu können, wird ein Ablationsvergleich durchgeführt: dieselben Testfälle werden in zwei Konfigurationen analysiert - einmal mit dem vollständigen Kontext (axe-Report, Playwright-Interaktionsdaten und Quellcode) und einmal ausschließlich mit dem axe-Report und dem gerenderten DOM-Snippet. Die resultierenden Handlungsempfehlungen beider Durchläufe werden anhand der Kriterien Quellcodebezug, Umsetzbarkeit und semantische Tiefe qualitativ verglichen. Dies adressiert direkt Teilforschungsfrage TF2.¹⁷⁷

Kriterium 4 - Technische Praktikabilität: Laufzeit, Stabilität und Qualität des JSON-Outputs werden unter realen Bedingungen dokumentiert.

4.3.3 Limitationen des Evaluationsdesigns

Zwei methodische Einschränkungen sind vorab zu benennen. Erstens wird die Bewertung durch den Autor selbst durchgeführt und nicht durch unabhängige Gutachter validiert. Zweitens sind die genannten Referenzwerte von **AccessGuru**¹⁷⁸ und **GenA11y**¹⁷⁹ nur eingeschränkt vergleichbar, da sie auf anderen Datensätzen und unter anderen Rahmenbedingungen ermittelt wurden. Sie dienen daher als Orientierung, nicht als direkter Benchmark. Die erste Limitation wird in Kapitel 8 diskutiert, die zweite - die Vergleichbarkeit der Referenzwerte - wird ausführlich in Abschnitt 8.8.2 behandelt.

¹⁷⁷Siehe Kapitel 1.3

¹⁷⁸Vgl. Fathallah, Hernández, Staab 2025

¹⁷⁹Vgl. He, Huq, Malek 2025

5 Analyse und Anforderungen

In diesem Kapitel werden die Grundlagen für das in Kapitel 6 beschriebene Systemdesign erarbeitet. Abschnitt 5.1 analysiert den Problemkontext und leitet daraus den konkreten Handlungsbedarf ab. Abschnitte 5.2 und 5.3 spezifizieren funktionale bzw. nicht-funktionale Anforderungen an das zu entwickelnde System. Abschnitt 5.4 begründet die Auswahl des eingesetzten Sprachmodells.

5.1 Problemkontext und Handlungsbedarf

Die BITBW entwickelt und betreibt als zentraler IT-Dienstleisterin der Landesverwaltung Baden-Württemberg eine Vielzahl webbasierter Anwendungen.¹⁸⁰ Alle diese Anwendungen unterliegen den gesetzlichen Anforderungen der BITV 2.0, die WCAG 2.2 auf Konformitätsstufe AA als verbindlichen Maßstab definiert.¹⁸¹ In der Praxis erfolgt die Barrierefreiheitsprüfung heute überwiegend manuell oder durch den Einsatz einzelner regelbasierter Tools - ein Vorgehen, das in zweifacher Hinsicht problematisch ist: manuelle Audits sind personalintensiv und skalieren unzureichend mit der Anzahl zu betreuender Anwendungen; regelbasierte Tools erfassen, wie in Kapitel 2.2.1 dargelegt, nur einen Bruchteil tatsächlich vorhandener Barrieren.¹⁸²

Im Sinne des Design-Science-Research-Paradigmas ist der Ausgangspunkt die Identifikation eines praxisrelevanten Problems, für das bestehende Lösungen unzureichend sind.¹⁸³ Aus der Analyse der Forschungsliteratur (Kapitel 3) und den Rahmenbedingungen der BITBW lassen sich drei zentrale Problemfelder ableiten.¹⁸⁴

Zum einen ist in der betrachteten Literatur kein Ansatz erkennbar, der nicht nur den anfänglichen DOM-Zustand prüft, sondern auch Barrieren erkennt, die erst durch typische Nutzerinteraktionen in React-Anwendungen sichtbar werden.¹⁸⁵ Zum anderen fehlt bislang eine Lösung, die die Ergebnisse einer allgemeinen DOM-Analyse so verständlich und praxisnah aufbereitet, dass Entwickler gezielt für einzelne Quellcodedateien und Komponenten konkrete Verbesserungen ableiten können. Außerdem setzen alle bekannten Forschungsprototypen auf cloudbasierte LLM-APIs - ein Ansatz, der für die BITBW aus Datenschutzgründen nicht ohne Weiteres in Frage kommt.^{186 187}

Aus diesen drei Dimensionen leitet sich das Ziel der vorliegenden Arbeit ab: die Entwicklung eines lokal ausführbaren, LLM-gestützten Assistenzsystems, das axe-core-Violation-Reports,

¹⁸⁰ Vgl. BITBW 2021

¹⁸¹ Vgl. Bundesministerium für Digitales und Staatsmodernisierung 2025

¹⁸² Vgl. Tafreshipour et al. 2024, S. 8-9

¹⁸³ Vgl. Hevner et al. 2004, S. 79

¹⁸⁴ Vgl. Peffers et al. 2007, S. 52-55

¹⁸⁵ Vgl. Fathallah, Hernández, Staab 2025

¹⁸⁶ Vgl. Europäisches Parlament und Rat der Europäischen Union 2016, Art. 28, 44 ff.

¹⁸⁷ Siehe Kapitel 2.4.4

Playwright-Interaktionsdaten und statische Quellcodeanalyse kombiniert, um entwicklernahe Handlungsempfehlungen für React-basierte Webanwendungen zu generieren.¹⁸⁸

5.2 Funktionale Anforderungen (FA)

Funktionale Anforderungen (FA) beschreiben, was ein System leisten soll - d. h. welche Funktionen und Dienste es seinen Nutzerinnen und Nutzern bereitstellt.¹⁸⁹ Die folgenden Anforderungen wurden aus der Problemanalyse (Abschnitt 5.1), den Forschungslücken (Abschnitt 3.5) sowie einer Analyse vergleichbarer Systeme aus der Literatur abgeleitet.

FA-01 Regelbasierte Accessibility-Erkennung

Das System muss den BWEC automatisiert auf Barrierefreiheitsverstöße nach WCAG 2.2 Level A und AA prüfen.¹⁹⁰ Die Prüfung basiert auf axe-core als Detection-Backend. Dessen Regelkatalog deckt einen wesentlichen Teil der syntaktisch und algorithmisch prüfbaren Verstöße gegen WCAG 2.0, 2.1 und 2.2 ab¹⁹¹, ersetzt jedoch keine vollständige manuelle Barrierefreiheitsprüfung.¹⁹² Der Output je Verstoß umfasst mindestens: Regel-ID (Zur Identifikation), Schweregrad (critical/serious/moderate/minor) sowie ein HTML-Ausschnitt des betroffenen Elements.

FA-02 Dynamische Zustandserfassung via Playwright

Das System muss über Playwright definierte Interaktionssequenzen ausführen. Darunter fallen Seitenaufbau, Formularbefüllung, Absende-Aktionen und Fokus-Navigation, die für jeden resultierenden DOM-Zustand eine axe-core-Prüfung anstoßen.¹⁹³ Damit werden Barrieren erfasst, die im initialen Seitenzustand nicht vorhanden sind.¹⁹⁴

FA-03 Statische Quellcodeanalyse

Das System muss den Quellcode der geprüften React-Anwendung automatisiert nach Mustern durchsuchen, die auf Barrierefreiheitsprobleme hindeuten: fehlende `aria`-Attribute, `onClick`-Handler auf Nicht-interaktiven Elementen, fehlende `alt`-Attribute, Verwendung veralteter HTML-Elemente sowie fehlende Formular-Assoziationen.¹⁹⁵

FA-04 LLM-gestützte Analyse und Handlungsempfehlung

¹⁸⁸Siehe Kapitel 1.3

¹⁸⁹Vgl. Sommerville 2016, S. 6-9

¹⁹⁰Vgl. Land Baden-Württemberg 2025; World Wide Web Consortium (W3C) 2023

¹⁹¹Vgl. World Wide Web Consortium (W3C) 2023

¹⁹²Vgl. Deque Systems 2026

¹⁹³Vgl. Microsoft 2026

¹⁹⁴Siehe Kapitel 2.3.2

¹⁹⁵Siehe Kapitel 2.1

Das System muss die Ergebnisse aus FA-01, FA-02 und FA-03 zu einem strukturierten Kontext zusammenführen und an ein lokal ausgeführtes Sprachmodell übergeben. Das Sprachmodell erhält einen kombinierten Prompt¹⁹⁶ und generiert daraus eine Handlungsempfehlung im Format Must-have/Nice-to-have je Verstoß und mit Angabe der betroffenen Datei.

FA-05 Strukturierter JSON-Output

Das System muss seine Ergebnisse in einem maschinenlesbaren JSON-Format ausgeben.¹⁹⁷ Der Report enthält je Befund: Verstoß-ID, Kategorie (syntaktisch/semantisch/layout)^{198 199}, Schweregrad, betroffene Datei(en), Handlungsempfehlung (Must-have/Nice-to-have), WCAG-Erfolgskriterium und - falls vorhanden - den relevanten Quellcode-Ausschnitt. Das strukturierte Format erleichtert die Weiterverarbeitung und Integration der Ergebnisse in bestehende Entwicklungsprozesse.

5.3 Nicht-funktionale Anforderungen (NFA)

Nicht-funktionale Anforderungen (NFA) beschreiben Qualitätseigenschaften des Systems, die bestimmen, wie gut es seine funktionalen Aufgaben erfüllt. Sie haben für das vorliegende System besonderes Gewicht, da der Betriebskontext der BITBW (öffentliche Verwaltung, lokale Infrastruktur) strenge Rahmenbedingungen setzt.

NFA-01 Datenschutz und Lokal-First

Das System muss vollständig ohne externe API-Aufrufe betreibbar sein. Weder Quellcode noch DOM-Inhalte noch Violation-Reports dürfen das interne Netzwerk verlassen. Diese Anforderung ergibt sich aus den Vorgaben der DSGVO zur Auftragsverarbeitung und zur Sicherheit der Verarbeitung, da die analysierten Anwendungen potenziell personenbezogene Daten, etwa in Formularinhalten, enthalten können.²⁰⁰

NFA-02 Laufzeit und Praktikabilität

Der vollständige Analyselauf - von der Playwright-Initialisierung bis zum fertigen JSON-Report - muss auf einem Entwicklungsrechner in einer für den Entwicklungsalltag akzeptablen Zeit abschließen. Als Richtwert gilt eine Gesamtlaufzeit von unter zehn Minuten für eine einzelne Seite mit bis zu 20 axe-Verstößen. Diese Anforderung begrenzt die Komplexität der eingesetzten Prompt-Kette und schließt mehrstufige Reasoning-Schleifen (wie iteratives Corrective Re-Prompting)²⁰¹ aus dem Scope des PoC aus.

NFA-03 Wartbarkeit und Erweiterbarkeit

¹⁹⁶Siehe Abschnitt 6.2

¹⁹⁷Vgl. Paternò et al. 2025; Pool 2023

¹⁹⁸Vgl. Fathallah, Hernández, Staab 2025

¹⁹⁹Siehe Kapitel 2.2.3

²⁰⁰Vgl. Europäisches Parlament und Rat der Europäischen Union 2016, Art. 28 und Art. 32

²⁰¹Siehe Kapitel 2.4.2

Die Systemarchitektur muss eine klare modulare Trennung der Komponenten (Detection, Context-Building, Synthesis) vorsehen, sodass einzelne Komponenten - etwa die Playwright-Skripte oder der Prompt - unabhängig angepasst oder ausgetauscht werden können.

NFA-04 Reproduzierbarkeit

Das System muss deterministisch genug sein, dass zwei aufeinanderfolgende Analyseläufe auf derselben Anwendungsversion im Wesentlichen dieselben Befunde produzieren. Da LLMs inhärent probabilistisch sind, wird Reproduzierbarkeit durch eine feste Temperatur-Einstellung (Temperature = 0.05) für das Sprachmodell und durch strukturierte Ausgabeformate (JSON-Schema im Prompt) sichergestellt.²⁰²

5.4 Lokale Inferenzinfrastruktur: Ollama

Die Wahl der Inferenzinfrastruktur ist durch NFA-01 (Lokal-First) strukturell vorgegeben: Cloud-basierte Modell-APIs wie die OpenAI API oder Anthropic Claude scheiden aus, da ihre Nutzung die Übermittlung potenziell schützenswerter Quellcodeausschnitte und DOM-Inhalte an externe Server erfordert. Dies ist im Kontext öffentlicher Verwaltung ohne gesonderte datenschutzrechtliche Prüfung nicht zulässig.²⁰³

5.4.1 Ollama als Laufzeitumgebung

Ollama ist ein quelloffenes Werkzeug, das die lokale Ausführung von LLMs über eine einheitliche REST-API ermöglicht.²⁰⁴ Die Modellverwaltung – Download, Quantisierung und Speicherverwaltung – wird durch eine einfach zu bedienende Command Line Interface (CLI) abstrahiert: Modelle lassen sich mit `ollama pull <modell>` herunterladen und mit `ollama run <modell>` starten. Die REST-API ist kompatibel mit der OpenAI-API-Spezifikation, wodurch sich bestehende Node.js-Bibliotheken ohne größeren Anpassungsaufwand integrieren lassen. Für den vorliegenden PoC ist Ollama die einzige zusätzliche Infrastrukturkomponente, die neben einer Standard-Node.js-Installation benötigt wird.

Ein weiterer Vorteil von Ollama ist die Unterstützung quantisierter Modellvarianten (z.B. Q4_K_M), die eine speichereffiziente Ausführung auf Central Processing Unit (CPU)-basierten Systemen ohne dedizierte Grafikkarte ermöglichen. Dies ist für den Betrieb auf Entwicklungsrechnern der BITBW relevant, da nicht von der Verfügbarkeit dedizierter Graphics Processing Unit (GPU)-Hardware ausgegangen werden kann.

²⁰²Vgl. OpenAI 2026, Temperature; **Anthropic_API_Docs**

²⁰³Vgl. Europäisches Parlament und Rat der Europäischen Union 2016, Art. 28 und Art. 44 ff.

²⁰⁴Vgl. Ollama 2026

5.4.2 Anforderungen an das Sprachmodell

Unabhängig von der konkreten Modellwahl lassen sich aus den funktionalen und nicht-funktionalen Anforderungen vier Eigenschaften ableiten, die ein geeignetes Modell aufweisen muss.

Erstens muss das Modell **JSX und TypeScript** ohne Vorverarbeitung verarbeiten können, da der grep-Anreicherungsschritt Quellcodeausschnitte aus React-Komponenten direkt in den Prompt einbettet (FA-03).

Zweitens muss das Modell **strukturierte Ausgaben** in einem vorgegebenen Format zuverlässig erzeugen, da der Report-Parser ein definiertes Must-have/Nice-to-have-Schema erwartet (FA-05, NFA-04).

Drittens muss das Modell innerhalb eines **Kontextfensters von mindestens 8.192 Tokens** arbeiten, um mehrere Violation-Reports, relevante Quellcodeausschnitte und den System-Prompt in einer einzigen Anfrage verarbeiten zu können (FA-04).

Viertens muss das Modell **auf CPU-basierten Systemen** mit akzeptabler Latenz betreibbar sein, was die praktisch einsetzbare Parameterzahl auf ca. 7–32 Milliarden Parameter begrenzt (NFA-02).²⁰⁵

Die Auswahl konkreter Modellkandidaten, die diese Anforderungen erfüllen, sowie deren vergleichende Evaluation werden in Kapitel 6.2 bzw. Kapitel 8 behandelt.

5.4.3 Ausgewählte Modellkandidaten für die Evaluation

Für die Evaluation in Kapitel 8 werden vier lokal ausführbare Modellkandidaten betrachtet:

- `qwen2.5-coder:7b`
- `qwen2.5-coder:14b`
- `qwen3:32b`
- `deepseek-r1:70b`

Die Auswahl folgt einem bewussten Größen- und Fähigkeitsvergleich entlang der Anforderungen aus Abschnitt 5.3: 7B und 14B repräsentieren ressourcenschonende Modelle für den operativen Lokalbetrieb, 32B einen mittleren Kompromiss aus Qualität und Laufzeit, 70B ein leistungsstarkes Referenzmodell mit höherem Rechenbedarf. Dadurch kann empirisch untersucht werden, wie sich Modellgröße auf Erkennungsqualität, Formatstabilität und Laufzeit auswirkt.

²⁰⁵Vgl. Hui et al. 2024, S. 3

5.4.4 Zusammenfassung der Anforderungen

Tabelle 4 fasst die spezifizierten Anforderungen zusammen und gibt jeweils den Ursprung und die Priorität an.

ID	Beschreibung	Ursprung	Priorität
Funktionale Anforderungen			
FA-01	Regelbasierte Accessibility-Erkennung via axe-core	Forschungsliteratur (Kap. 3)	Muss
FA-02	Dynamische Zustandserfassung via Playwright	Forschungslücke 1 (Kap. 3.5)	Muss
FA-03	Statische Quellcodeanalyse (grep-basiert)	Forschungslücke 2 (Kap. 3.5)	Muss
FA-04	LLM-gestützte Analyse und Handlungsempfehlung	Forschungsliteratur (Kap. 3)	Muss
FA-05	Strukturierter JSON-Output	Praxisanforderung BITBW	Muss
Nicht-funktionale Anforderungen			
NFA-01	Datenschutz und Lokal-First (kein externer API-Aufruf)	Institutionell (DSGVO, BITBW)	Muss
NFA-02	Laufzeit < 10 min pro Seite (ohne GPU)	Praxisanforderung BITBW	Soll
NFA-03	Wartbarkeit und Erweiterbarkeit (modulare Architektur)	Software-Engineering Best Practice	Soll
NFA-04	Reproduzierbarkeit (Temperature = 0.05, JSON-Schema)	Forschungsmethodik	Soll

Tab. 4: Anforderungsübersicht: Funktionale und nicht-funktionale Anforderungen mit Ursprung und Priorität.

Die Anforderungen FA-01 bis FA-05 und NFA-01 bis NFA-04 bilden die verbindliche Grundlage für das Systemdesign in Kapitel 6. NFA-01 (Lokal-First) hat dabei Vorrang vor allen anderen Anforderungen und schränkt den Lösungsraum strukturell ein: Alle Designentscheidungen müssen mit dem Betrieb auf lokaler Infrastruktur vereinbar sein.

6 Konzeption des Assistenzsystems

Dieses Kapitel beschreibt den konzeptionellen Entwurf des Assistenzsystems **ACE** (*Accessibility Context Engine*). Die Entwurfsentscheidungen werden aus den Anforderungen (Kapitel 5), den theoretischen Grundlagen (Kapitel 2) und den in Kapitel 3 identifizierten Forschungslücken abgeleitet.

6.1 Zielarchitektur und Datenpipeline

Die Architektur des Systems folgt einer sequenziellen Datenpipeline, die vier Analysequellen zusammenführt, deren Ergebnisse normalisiert und an ein lokales Sprachmodell übergibt. Abbildung 6 illustriert den Gesamtablauf. Eine reduzierte, textnahe Darstellung der Verarbeitungsschritte findet sich in Anhang 4 (Abbildung 8).



Abb. 6: Datenpipeline des ACE-Assistenzsystems

Die Pipeline gliedert sich in fünf Phasen: die Datenerhebung durch axe-core, Playwright und grep, die LLM-gestützte Quellcodeanalyse, die Normalisierung der Rohdaten in das UFS, die Prompt-Konstruktion mit Token-Budget-Steuerung sowie die Inferenz durch das lokale Sprachmodell und die anschließende Ausgabe.

6.1.1 Analysequellen

Die Wahl der vier Analysequellen ergibt sich unmittelbar aus den in Abschnitt 2.2.2 dargestellten Grenzen bestehender Tools: begrenzte Regelabdeckung, semantische Blindheit und fehlende Dynamik. Jede Quelle adressiert einen dieser Schwachpunkte.

axe-core wird über einen gesteuerten Chromium-Browser ausgeführt und analysiert den vollständig gerenderten DOM der Zielanwendung auf Verstöße gegen WCAG-Erfolgskriterien. Da Rich-Client-Webanwendungen ihre Inhalte erst zur Laufzeit per JavaScript erzeugen (vgl. Abschnitt 2.3), ist ein echter Browser-Launch unerlässlich – statische HTML-Analysen würden den tatsächlichen Zustand der Anwendung nicht abbilden.²⁰⁶ axe-core deckt dabei primär *syntaktische* Verstöße im Sinne der in Abschnitt 2.2.3 eingeführten Taxonomie ab – also Fälle, in denen ein erforderliches Accessibility-Attribut fehlt oder formal falsch gebildet ist. Kodithuwakku und

²⁰⁶Vgl. Fathallah, Hernández, Staab 2025

Wickramaarachchi zeigen allerdings, dass selbst axe-core nur einen Bruchteil aller WCAG-Tests automatisiert abdeckt (vgl. Tabelle 1), weshalb es als alleinige Quelle nicht ausreicht.²⁰⁷

Playwright führt sieben generische Interaktionschecks aus, die unabhängig von der konkreten Anwendungslogik auf jeder Web-App lauffähig sind. Geprüft werden unter anderem die Erreichbarkeit aller interaktiven Elemente per Tastatur, die Sichtbarkeit des Fokusindikators sowie das Vorhandensein eines Skip-Links. Diese Checks adressieren gezielt das in Abschnitt 2.3.2 beschriebene Problem der *zustandsabhängigen Barrieren*: Ein Modalfenster ohne Fokus-Trap oder ein Dropdown-Menü, das per Tastatur nicht navigierbar ist, wird erst durch tatsächliche Browser-Interaktion sichtbar – im statischen DOM sind diese Probleme nicht vorhanden.²⁰⁸

grep erfüllt in der Pipeline zwei komplementäre Aufgaben: Erstens reichert es bestehende axe-core- und Playwright-Findings um ihren Entstehungskontext im Quellcode an (*Kontextanreicherung*); zweitens erkennt es eigenständig bekannte Anti-Patterns im React-Quellcode anhand regulärer Ausdrücke (*Pattern-Erkennung*). Beide Aufgaben adressieren die zweite Forschungslücke aus Kapitel 3.5: die fehlende Verknüpfung zwischen DOM-Analyse und Quellcode. Wie in Abschnitt 2.3.2 am Beispiel der `IconButton`-Komponente gezeigt, meldet axe-core Verstöße auf Ebene des gerenderten DOM, ohne Hinweis darauf, in welcher Quelldatei die Ursache liegt. Fathallah et al. benennen genau diese Lücke als offenes Problem von AccessGuru.²⁰⁹ Die Kontextanreicherung operiert dabei selektiv und durchsucht ausschließlich Dateien, auf die bestehende axe-core- oder Playwright-Findings verweisen; die Pattern-Erkennung durchsucht hingegen die relevante Codebasis breiter (vgl. Abschnitt 7.4).

LLM-Codeanalyse ergänzt die regelbasierten Quellen um eine semantische Detektionsstufe. In dieser Stufe liest das lokale Modell Quellcode-Chunks mit Zeilennummern und erzeugt strukturierte Findings im UFS-Schema (inklusive Evidenz, Dateipfad und Zeilenbereich). Diese Findings werden nicht als Ersatz, sondern als zusätzliche Quelle verwendet und anschließend mit axe-core-, Playwright- und grep-Befunden fusioniert.²¹⁰

Die Entscheidung für reguläre Ausdrücke anstelle eines vollständigen Abstract Syntax Tree (AST)-Parsers²¹¹ ergibt sich daraus, dass die implementierten Pattern-Checks syntaktischer Natur sind – sie suchen nach textuell erkennbaren Mustern im JSX-Markup, nicht nach semantischen Datenflüssen. Zudem gewährleistet der regexp-basierte Ansatz Sprachunabhängigkeit über `.tsx`, `.jsx`, `.ts` und `.js`-Dateien hinweg, ohne sprachspezifische Parser-Abhängigkeiten einzuführen.²¹² Für die Kontextanreicherung werden selektiv nur Quelldateien durchsucht, auf die axe-core oder Playwright hinweisen; die Pattern-Erkennung arbeitet unabhängig davon über die gesamte relevante Codebasis. Dieses selektive Vorgehen bei der Anreicherung begrenzt den Token-Verbrauch im späteren Prompt (vgl. Abschnitt 6.2).

²⁰⁷ Vgl. Kodithuwakku, Wickramaarachchi 2025

²⁰⁸ Vgl. Huang, C. et al. 2024

²⁰⁹ Vgl. Fathallah, Hernández, Staab 2025

²¹⁰ Vgl. Fathallah, Hernández, Staab 2025

²¹¹ Abstract Syntax Tree – eine baumartige Repräsentation der syntaktischen Struktur von Quellcode.

²¹² Vgl. Bassi et al. 2025, S. 12

Tabelle 5 fasst die Zuordnung der vier Analysequellen zu den adressierten Problemen und den jeweiligen Verstößtypen zusammen.

Quelle	Adressiertes Problem	Verstößtypen	Theoriebezug
axe-core	Begrenzte Regelabdeckung ²¹³	primär syntaktisch	Abschnitt 2.2.2
Playwright	Fehlende Dynamik bei zustandsabhängigen Barrieren	semantisch, layout	Abschnitt 2.3.2
grep	DOM/Quellcode-Diskrepanz ²¹⁴	alle (Anreicherung)	Abschnitt 3.5
LLM-Codeanalyse	Semantische Muster jenseits fester Regex-Regeln	semantisch, syntaktisch, layout	Abschnitt 6.2

Tab. 5: Zuordnung der Analysequellen zu adressierten Problemen, Verstößtypen und theoretischer Herleitung

6.1.2 Unified Finding Schema (UFS)

Die Notwendigkeit einer Normalisierungsschicht ergibt sich aus der Heterogenität der Quellformate. Pool zeigt mit Testaro, dass jedes Accessibility-Tool eigene, inkompatible Ausgabeformate produziert – ein Problem, das sich bei der Kombination mehrerer Werkzeuge verschärft.²¹⁵ Konkret liefert axe-core Objekte mit `impact`-Stufen und DOM-Selektorketten, Playwright gibt boolesche `passed`-Felder mit Check-IDs zurück, grep produziert Datei-Zeilen-Tupel mit Textfragmenten, und die LLM-Codeanalyse liefert JSON-Objekte mit Evidenz- und Zeilenfeldern. Eine quellenübergreifende Deduplizierung oder Priorisierung wäre auf dieser Basis nicht möglich, da die Schweregrade der Quellen (`impact`, `severity`, Dateiname) inkommensurable Größen darstellen. Zudem enthalten die rohen axe-core-Ausgaben vollständige DOM-Snapshots und ausführliche Hilfetexte, die für die LLM-Inferenz nicht relevant sind und das Token-Budget unnötig belasten.

Das UFS überführt daher alle Findings in eine einheitliche TypeScript-Schnittstelle. Listing 6.1 zeigt das Interface, wie es im PoC implementiert ist.

Listing 6.1: TypeScript-Interface des Unified Finding Schema

```

1 export type FindingSource = "axe" | "playwright" | "grep" | "llm";
2 export type Severity = "critical" | "serious" | "moderate" | "minor";
3 export type WcagLevel = "A" | "AA" | "AAA";
4 export type ViolationCategory = "syntaktisch" | "semantisch" | "layout";
5
6 export interface UnifiedFinding {
7   id: string;
8   source: FindingSource;
9   ruleId: string;
10  description: string;
11  severity: Severity;

```

²¹⁵Vgl. Pool 2023


```
12  wcagCriteria:    string[];  
13  wcagLevel:      WcagLevel;  
14  category:       ViolationCategory;  
15  selector:       string | null;  
16  componentPath:  string | null;  
17  codeSnippet:    string | null;  
18  lineRange:      [number, number] | null;  
19 }
```

Die Felder lassen sich in drei Gruppen einteilen: **Identifikation und Klassifikation** (`id`, `source`, `ruleId`, `severity`, `category`), **WCAG-Bezug** (`wcagCriteria`, `wcagLevel`) und **Quellcode-Kontext** (`selector`, `componentPath`, `codeSnippet`, `lineRange`). Felder, die eine Quelle nicht liefern kann, bleiben `null` und werden gegebenenfalls durch die `grep`-Anreicherung oder die LLM-Codeanalyse ergänzt.

Das Feld `category` bildet die in Abschnitt 2.2.3 eingeführte Taxonomie nach Fathallah et al. direkt im Datenmodell ab.²¹⁶ Diese Klassifikation ist nicht nur für die Analyse relevant, sondern auch für die Evaluation: In Kapitel 8 wird die Erkennungsleistung des Systems differenziert nach Verstößtyp ausgewertet, um TF1 zu beantworten.

Das Feld `componentPath` ist das entscheidende Bindeglied zwischen DOM-Analyse und Quellcode. Es schließt die in Abschnitt 2.3.2 beschriebene Diskrepanz zwischen gerendertem DOM und Quellcode: Erst durch die Verknüpfung eines axe-Findings mit dem konkreten Dateipfad und der Zeilennummer wird aus einer generischen Meldung („*Element hat keinen alternativen Text*“) eine entwicklernahe Handlungsanweisung („*In LoginForm.tsx, Zeile 46: Füge alt-Attribut hinzu*“).

6.2 Prompt-Strategie

Dieser Abschnitt beschreibt die Strategie für die Interaktion mit dem lokalen Sprachmodell: die Wahl des zweistufigen Prompt-Ansatzes, die eingesetzten Prompt-Engineering-Strategien sowie die Steuerung des Token-Budgets.

6.2.1 Zweistufige Prompt-Strategie

Eine grundlegende Entwurfsentscheidung betrifft die Interaktionsstrategie mit dem lokalen Sprachmodell. Konzeptionell wäre ein stark iterativer Ansatz denkbar, bei dem jede Quelle in mehreren Korrekturschleifen verarbeitet wird. Fathallah et al. realisieren dies mit AccessGuru über eine Cloud-API und setzen dabei iteratives Corrective Re-Prompting ein, das den Violation Score Decrease von 0,72 auf 0,84 verbessert.²¹⁷ Auch Huang et al. zeigen mit ACCESS, dass ReAct-Prompting – also iteratives Wechseln zwischen Reasoning und Tool-Aufrufen – bei der Barrierefreiheitskorrektur die besten Ergebnisse erzielt.²¹⁸

²¹⁶Vgl. Fathallah, Hernández, Staab 2025

²¹⁷Vgl. Fathallah, Hernández, Staab 2025

²¹⁸Vgl. Huang, C. et al. 2024, S. 5

Für den vorliegenden Anwendungsfall wird eine zweistufige Strategie gewählt: In Stufe 1 erzeugt das Modell als LLM-Detektor strukturierte Code-Findings aus Quellcode-Chunks; in Stufe 2 priorisiert ein kombinierter Prompt alle Quellen (axe-core, Playwright, grep, LLM). In der Evaluation (Kapitel 8) werden beide Stufen separat ausgewertet: Stufe 1 über die Anzahl der LLM-Detektor-Findings je Modell, Stufe 2 über die Must-have/Nice-to-have-Klassifikation und die Fix-Qualität der finalen Empfehlungen. Der entscheidende Grund gegen weitere iterative Schleifen ist das Latenzbudget: Wie die Laufzeitmessungen in Kapitel 8 zeigen, benötigt ein LLM-Aufruf auf einem lokalen CPU-basierten System je nach Modellgröße bereits mehrere Minuten. Zusätzliche ReAct- oder Corrective-Re-Prompting-Schleifen würden diese Laufzeit vervielfachen und NFA-02 verletzen. AccessGuru und ACCESS können solche Strategien einsetzen, weil sie auf Cloud-APIs mit GPU-Beschleunigung zugreifen – eine Option, die durch NFA-01 ausgeschlossen ist.

Der kombinierte Priorisierungsprompt in Stufe 2 ermöglicht es dem Sprachmodell, Zusammenhänge zwischen den Quellen direkt zu erkennen. Meldet axe-core ein fehlendes `aria-label`, liefert grep den zugehörigen Quellcodeausschnitt und bestätigt die LLM-Codeanalyse dieselbe Stelle, kann das Modell diese Evidenz in einem konkreten Vorschlag zusammenführen.

6.2.2 Wahl der Prompt-Engineering-Strategien

Innerhalb des Single-Prompt-Rahmens stellt sich die Frage, welche der in Abschnitt 2.4.2 eingeführten Prompt-Engineering-Strategien zum Einsatz kommen. Die Auswahl wird durch die Beschränkung auf einen einzelnen Modellaufruf und die Eigenschaften lokaler Modelle der 7B–32B-Klasse bestimmt.

Role-Play Prompting wird eingesetzt, um das Modell als Barrierefreiheitsexperten und React-Entwickler zu adressieren. Njifenjou et al. und Fathallah et al. belegen, dass diese Strategie die Qualität accessibility-spezifischer Ausgaben verbessert, da das Modell seine Antworten stärker an der Perspektive eines Fachexperten ausrichtet.²¹⁹

Few-Shot Prompting wird ergänzend eingesetzt, um dem Modell das erwartete Ausgabeformat anhand konkreter Beispiele zu demonstrieren. Brown et al. zeigen, dass Few-Shot-Beispiele die Konsistenz und Formatkonformität der Ausgaben erheblich verbessern²²⁰ – ein Aspekt, der bei kleineren Modellen besonders relevant ist, da diese ohne Beispiele häufiger vom vorgegebenen Schema abweichen.

Chain-of-Thought wird bewusst nicht als explizite Prompting-Strategie eingesetzt. Zwar reduziert CoT nachweislich Halluzinationen bei komplexen Schlussfolgerungen²²¹, jedoch verbraucht die Reasoning-Kette zusätzliche Output-Tokens und verlängert die Inferenzzeit auf lokaler Hardware. Die konkrete Umsetzung des Prompts wird in Kapitel 7 beschrieben.

²¹⁹Vgl. Njifenjou et al. 2024; Vgl. Fathallah, Hernández, Staab 2025

²²⁰Vgl. Brown et al. 2020, S. 5

²²¹Vgl. Wei, Wang, X. et al. 2022, S. 2

6.2.3 Token-Budget-Steuerung

Sprachmodelle verarbeiten Text nicht zeichenweise, sondern in Form von *Tokens* – kleinsten Verarbeitungseinheiten, die je nach Sprache und Wortlänge ein Wort, einen Wortteil oder ein einzelnes Sonderzeichen umfassen können. Jedes Modell besitzt ein festes *Kontextfenster*, das die maximale Anzahl an Tokens definiert, die in einem Aufruf gleichzeitig verarbeitet werden können – sowohl die Eingabe (Prompt) als auch die Ausgabe (Antwort) müssen in dieses Fenster passen. Da der System-Prompt und die Ausgabe-Reserve einen festen Anteil des Kontextfensters belegen, steht für die Findings ein variables Budget zur Verfügung (vgl. Abbildung 9 im Anhang).

Übersteigt die Gesamtmenge der serialisierten Findings dieses Budget, greift eine Priorisierungsstrategie: Alle Findings werden quellenübergreifend nach Schweregrad sortiert und von unten abgeschnitten. So ist sichergestellt, dass gesetzlich relevante Verstöße (**critical/serious**) stets im Prompt enthalten sind, während Findings niedrigerer Priorität bei Platzmangel entfallen.

Ergänzend wird grep selektiv eingesetzt: Statt die gesamte Codebasis zu durchsuchen, werden primär Dateien analysiert, die durch bestehende Befunde als relevant erscheinen. Zusätzlich arbeitet die LLM-Codeanalyse mit chunkbasiertem Retrieval – d. h. Quelldateien werden mit Zeilennummern versehen und in Textsegmente fester Zeichenlänge (*Chunks*) unterteilt, die einzeln an das Modell übergeben werden (vgl. Abschnitt 7.5) –, um den Kontext je Aufruf zu begrenzen. Dieses Vorgehen verhindert eine unkontrollierte Ausweitung des Token-Verbrauchs. Konzeptionell entspricht das selektive Retrieval dem Grundprinzip von RAG-Systemen (vgl. Abschnitt 2.4): relevanter Kontext wird zur Laufzeit gezielt abgerufen und in den Prompt eingebettet, statt das Modell mit der gesamten Datenbasis zu konfrontieren.²²²

6.3 Ausgabeformat: entwicklernahe To-Do-Liste

Das Ausgabeformat des Systems ist als priorisierte To-Do-Liste konzipiert, die sich an Entwicklerinnen und Entwickler richtet. Die Wahl dieses Formats ergibt sich aus FA-05 sowie aus der Beobachtung, dass bestehende Violation Reports das Problem auf Ebene des gerenderten DOM beschreiben, ohne Bezug zur Quellcodestruktur herzustellen (vgl. Abschnitt 2.2.2).²²³

Die Liste gliedert sich in zwei Prioritätsstufen: *Must-have*-Einträge bezeichnen Verstöße gegen Level-A- und Level-AA-Erfolgskriterien, die eine gesetzliche Konformitätspflicht nach BITV und BfSG begründen (vgl. Abschnitt 2.1). *Nice-to-have*-Einträge umfassen Empfehlungen, die über den gesetzlichen Mindeststandard hinausgehen. Jeder Eintrag enthält – sofern durch grep-Anreicherung oder LLM-Codeanalyse verfügbar – den Dateipfad der betroffenen Komponente, die Zeilennummer sowie einen konkreten Code-Fix-Vorschlag.

Es ist anzumerken, dass das Ausgabeformat durch einen System-Prompt vorgegeben wird, dessen Einhaltung durch das Sprachmodell nicht garantiert werden kann. Insbesondere bei kleineren

²²²Vgl. Lewis et al. 2020, S. 1–2

²²³Vgl. Fathallah, Hernández, Staab 2025

Modellen der 7B-Klasse ist die Formatkonformität nicht selbstverständlich.²²⁴ Die Evaluation in Kapitel 8 untersucht, inwieweit die getesteten Modelle das vorgegebene Schema einhalten. Diese Limitation wird in Abschnitt 8.6 diskutiert.

Zusammenfassend ergibt sich aus den Anforderungen, der Theorie und den identifizierten Forschungslücken ein System, das vier komplementäre Analysequellen über ein einheitliches Datenmodell (UFS) zusammenführt und in einer zweistufigen LLM-Strategie verarbeitet: Stufe 1 für LLM-basierte Codefindings und Stufe 2 für die quellenübergreifende Priorisierung. Kapitel 7 beschreibt die prototypische Umsetzung dieser Konzeption.

²²⁴Vgl. Brown et al. 2020, S. 5

7 Implementierung des Proof of Concept

Dieses Kapitel beschreibt die prototypische Umsetzung des in Kapitel 6 konzipierten Assistenzsystems. Zunächst wird der technische Rahmen dargestellt (Abschnitt 7.1), bevor die Implementierung der einzelnen Pipeline-Module erläutert wird (Abschnitte 7.2–7.7). Abschnitt 7.6 beschreibt die konkrete Umsetzung der Prompt-Strategie einschließlich des System-Prompts.

7.1 Technischer Rahmen und Projektstruktur

Der PoC ist als Node.js-Anwendung in TypeScript implementiert.²²⁵ Die Wahl von TypeScript ergibt sich aus der technologischen Nähe zum Untersuchungsgegenstand: Der BWEK ist als Anwendung in ein TypeScript-basiertes Monorepo eingebettet, sodass der PoC im selben technologischen Kontext entwickelt wird. Dies erleichtert sowohl die Integration als auch eine mögliche Übertragung auf weitere Anwendungen innerhalb vergleichbarer Architekturen. Zugleich ermöglicht TypeScript eine typischere Definition des UFS (vgl. Listing 6.1). Als Laufzeitumgebung wird Node.js ab Version 24 eingesetzt; die Ausführung erfolgt über `tsx`, einen TypeScript-Runner, der Kompilierung und Ausführung in einem Schritt verbindet. Auf ein zusätzliches Framework wurde bewusst verzichtet, um den PoC minimal zu halten.

Tabelle 6 listet die wesentlichen Abhängigkeiten des Projekts.

Paket	Version	Zweck
playwright	1.50	Browser-Automatisierung für Interaktionschecks
@axe-core/playwright	4.10	axe-core-Integration in Playwright
dotenv	17.3	Umgebungsvariablen aus <code>.env</code> -Datei laden
tsx	6.5.7	TypeScript-Runner (Monorepo-Dependency)

Tab. 6: Wesentliche Abhängigkeiten des PoC

Die Projektstruktur folgt einer flachen Modulstruktur, bei der jedes Modul in `src/modules/` sowohl die Datenerhebung als auch die Normalisierung in das UFS übernimmt:

Listing 7.1: Projektstruktur des PoC

```
1 ACE/  
2   src/  
3     index.ts           # Pipeline-Orchestrierung und CLI  
4     types.ts           # UnifiedFinding, PipelineMetrics  
5     config.ts          # Zentrale Konfiguration (Env-Vars)
```

²²⁵Vgl. *JavaScript With Syntax For Types*. 2026

```

6   prompt.ts           # Prompt-Builder mit Token-Budget
7   ollama.ts           # HTTP-Client fuer Ollama
8   output.ts           # Markdown- und JSON-Report-Formatter
9   modules/
10  axe.ts               # axe-core: Collector + Normalizer
11  playwright.ts        # Playwright: 7 Checks + Normalizer
12  code.ts              # grep: Anreicherung + Pattern-Findings
13  llmDetector.ts        # LLM-Codeanalyse: semantische Findings (JSON)
14  results/              # Generierte Reports (nicht versioniert)

```

Die Pipeline wird über `src/index.ts` orchestriert und unterstützt fünf CLI-Flags: `--url` (Ziel-URL), `--src-dir` (Quellcode-Pfad), `--skip-llm` (Prompt speichern ohne finale Priorisierung), `--axe-only` (nur axe-core ausführen) und `--llm-detect` (zusätzliche LLM-Codeanalyse aktivieren). Die Konfiguration – einschließlich Ollama-URL, Modellname und Timeout-Werte – erfolgt zentral in `config.ts` und kann über Umgebungsvariablen überschrieben werden.

7.2 axe-core-Modul

Das axe-core-Modul implementiert FA-01 (regelbasierte Accessibility-Erkennung). Es startet über Playwright einen headless Chromium-Browser, navigiert zur Ziel-URL und wartet auf `networkidle`, um sicherzustellen, dass die React-Anwendung vollständig gerendert ist. Anschließend wird über `AxeBuilder` eine axe-core-Analyse ausgeführt, die auf die Tags `wcag2a`, `wcag2aa`, `wcag21a`, `wcag21aa` und `wcag22aa` beschränkt ist – entsprechend dem gesetzlichen Mindeststandard (vgl. Abschnitt 2.1).

Die Normalisierung überführt jede Violation in ein `UnifiedFinding`. Dabei werden drei Transformationen durchgeführt:

- **Severity-Mapping:** Das `impact`-Feld von axe-core (`critical`, `serious`, `moderate`, `minor`) wird direkt auf das `severity`-Feld des UFS abgebildet.
- **WCAG-Extraktion:** Aus den axe-Tags (z. B. `wcag143`) werden die Erfolgskriterien (z. B. 1.4.3) und die Konformitätsstufe extrahiert.
- **Kategorie-Heuristik:** Jede Regel wird anhand ihres `ruleId` einer Kategorie der in Abschnitt 2.2.3 eingeführten Taxonomie zugeordnet. Farbkontrast-Regeln werden als `layout` klassifiziert, ARIA-Strukturregeln als `semantisch`, alle übrigen als `syntaktisch`.

Die Felder `componentPath`, `codeSnippet` und `lineRange` bleiben nach der axe-Normalisierung zunächst `null` und werden erst in der Code-Phase (Abschnitt 7.4) angereichert.

Es ist anzumerken, dass axe-core im PoC einmalig beim initialen Seitenaufruf ausgeführt wird und nicht nach jeder Playwright-Interaktion. Eine Analyse pro Zustandswechsel wäre technisch realisierbar, würde jedoch bei sieben Checks mit je mehreren Zustandsänderungen die Laufzeit

um bis zu 168 Sekunden erhöhen und damit NFA-02 verletzen. Diese Einschränkung wird in Abschnitt 8.6 als Limitation diskutiert.

7.3 Playwright-Modul

Das Playwright-Modul implementiert FA-02 (dynamische Zustandserfassung). Es definiert sieben generische Interaktionschecks, die jeweils ein spezifisches WCAG-Erfolgskriterium adressieren. Tabelle 7 gibt einen Überblick.

Check-ID	Prüfgegenstand	WCAG	Kategorie
keyboard-tab-reachable	Interaktive Elemente per Tab erreichbar	2.1.1 (A)	semantisch
keyboard-trap-free	Kein Keyboard-Trap nach 40 Tab-Presses	2.1.2 (A)	semantisch
focus-visible	Sichtbarer Fokusindikator vorhanden	2.4.7 (AA)	layout
skip-link-present	Erstes Tab-Ziel ist ein Skip-Link	2.4.1 (A)	semantisch
page-title-present	Seite hat einen nicht-leeren <code><title></code>	2.4.2 (A)	syntaktisch
html-lang-present	<code><html></code> hat ein <code>lang</code> -Attribut	3.1.1 (A)	syntaktisch
focus-after-dialog	Fokus wandert beim Dialogöffnen in den Dialog	2.4.3 (A)	semantisch

Tab. 7: Implementierte Playwright-Checks mit WCAG-Zuordnung und Kategorie

Die Checks sind anwendungsunabhängig konzipiert: Sie setzen keine Kenntnis der konkreten Anwendungslogik voraus und sind auf jeder Web-App ausführbar. Der Keyboard-Trap-Check beispielsweise drückt 40-mal die Tab-Taste und prüft, ob der Fokus in den letzten zehn Schritten auf demselben Element verbleibt. Der Focus-Visible-Check navigiert per Tab durch die Seite und prüft für jedes fokussierte Element, ob ein sichtbarer Fokusindikator (`outline` oder `box-shadow`) vorhanden ist – eine Prüfung, die Tafreshipour et al. als Schwachstelle bestehender Tools identifizieren.²²⁶

Fehlgeschlagene Checks werden über ein statisches Mapping (`CHECK_DEFINITIONS`) in `UnifiedFinding`-Objekte überführt. Jede Check-Definition enthält die zugehörige `ruleId`, `severity`, WCAG-Kriterien und Kategorie. Checks, die erfolgreich bestanden werden, erzeugen kein Finding.

7.4 Code-Modul

Das Code-Modul erfüllt zwei Aufgaben: die Anreicherung bestehender Findings mit Quellcode-Kontext (FA-03) und die eigenständige Erkennung von Anti-Patterns im React-Quellcode.

²²⁶Vgl. Tafreshipour et al. 2024, S. 110

7.4.1 Anreicherung bestehender Findings

Für jedes axe-core- oder Playwright-Finding, das einen CSS-Selektor enthält aber noch keinen `componentPath` besitzt, wird der Quellcode der React-Anwendung durchsucht. Der Algorithmus extrahiert Suchbegriffe aus dem Selektor – IDs, CSS-Klassen, ARIA-Attribute – und sucht diese in den `.tsx/.jsx/.ts/.js`-Dateien des angegebenen Quellverzeichnisses. Bei einem Treffer werden `componentPath`, `codeSnippet` (zehn Zeilen Kontext um die Fundstelle) und `lineRange` im Finding ergänzt.

Diese Anreicherung adressiert die in Abschnitt 2.3.2 beschriebene Diskrepanz zwischen gerendertem DOM und Quellcode. Durchsucht werden ausschließlich Dateien, die durch axe-core- oder Playwright-Findings referenziert werden – nicht die gesamte Codebasis. Dieses selektive Vorgehen begrenzt den Token-Verbrauch im späteren Prompt (vgl. Abschnitt 6.2).

7.4.2 Pattern-Findings

Zusätzlich zur Anreicherung erkennt das Modul sechs Anti-Patterns durch reguläre Ausdrücke. Tabelle 8 zeigt die implementierten Patterns.

Pattern-ID	Beschreibung	WCAG	Kategorie
<code>div-span-onclick</code>	<code>onClick</code> auf <code><div>/</code> ohne Keyboard-Handler	2.1.1 (A)	semantisch
<code>img-missing-alt</code>	<code></code> ohne <code>alt</code> -Attribut	1.1.1 (A)	syntaktisch
<code>label-missing-htmlfor</code>	<code><label></code> ohne <code>htmlFor</code>	1.3.1 (A)	syntaktisch
<code>role-button-non-semantic</code>	<code>role="button"</code> auf Nicht-Button	4.1.2 (A)	semantisch
<code>tabindex-removes-element</code>	<code>tabIndex={-1}</code> entfernt Element aus Tab-Reihenfolge	2.1.1 (A)	semantisch
<code>autofocus-usage</code>	<code>autoFocus</code> kann den Fokusfluss stören	2.4.3 (A)	semantisch

Tab. 8: Implementierte grep-Patterns mit WCAG-Zuordnung

Treffer werden nach der Normalisierung dedupliziert: Findings mit identischer `ruleId`, `componentPath` und Startzeile werden zusammengefasst. Pro Pattern werden maximal acht Treffer berücksichtigt, um eine Überrepräsentation häufiger Patterns im Token-Budget zu vermeiden.

7.5 LLM-Detektor-Modul

Das LLM-Detektor-Modul ergänzt die regelbasierten Quellen um eine modellbasierte Quellcodeanalyse. Es wird nur ausgeführt, wenn der CLI-Parameter `--llm-detect` gesetzt ist und ein

Quellcodepfad via `--src-dir` vorliegt. Ziel ist nicht die Ersetzung bestehender Tools, sondern eine zusätzliche Detektionsquelle mit semantischem Fokus.

Die Verarbeitung erfolgt in fünf Schritten:

1. **Dateiselektion:** Das Modul sammelt rekursiv Dateien mit den Endungen `.tsx`, `.ts`, `.jsx`, `.js` und `.css`. Die Anzahl der analysierten Dateien ist über `LLM_DETECT_MAX_FILES` begrenzt (Default: 60; in den Benchmarkläufen auf 25 reduziert, um die Laufzeit auf der test-12-Suite zu begrenzen).
2. **Chunking:** Die Dateien werden mit Zeilennummern versehen und in Text-Chunks aufgeteilt. Die Chunk-Größe ist über `LLM_DETECT_CHUNK_CHARS` konfigurierbar (in den Benchmarkläufen: 4000 Zeichen), damit der Kontext robust verarbeitet werden kann.
3. **Detektionsprompt:** Für jeden Chunk wird ein eigener Prompt erzeugt. Dieser verlangt *ausschließlich* JSON im fest definierten Schema (u. a. `ruleId`, `wcagCriteria`, `filePath`, `startLine`, `endLine`, `evidence`, `fixSuggestion`).²²⁷
4. **Parsing und Validierung:** Antworten werden primär als JSON extrahiert und validiert. Für Formatabweichungen ist ein Fallback implementiert, der semi-strukturierte Markdown-Ausgaben in das Zielformat überführt.
5. **Normalisierung und Deduplizierung:** Valide Einträge werden als `source="llm"` in das UFS überführt und über den Schlüssel `ruleId|filePath|startLine` dedupliziert.

Dieses Vorgehen reduziert Halluzinationen, weil Findings ohne belastbare Evidenz oder ohne verwertbaren Datei-/Zeilenbezug verworfen werden. Gleichzeitig bleibt die Nachvollziehbarkeit erhalten, da jedes LLM-Finding einem konkreten Quellcodeausschnitt zugeordnet ist.

7.5.1 Abgrenzung zu `grep` und Nutzen der Kombination

Die beiden Detektionsansätze unterscheiden sich grundlegend in ihrer Arbeitsweise:

- **grep** arbeitet regelbasiert und deterministisch. Es erkennt bekannte Muster zuverlässig und reproduzierbar, kann aber nur das finden, was vorher als Regel modelliert wurde.
- **LLM-Detektor** arbeitet kontextsensitiv. Er kann auch semantisch ähnliche Problemstellen erkennen, die nicht exakt einem festen Regex-Muster entsprechen, ist jedoch anfälliger für Formatabweichungen und Halluzinationen.

Der PoC nutzt deshalb beide Ansätze komplementär: `grep` als stabile Baseline und LLM-Detektor als zusätzliche Quelle mit potenziellem Recall-Gewinn.

²²⁷ Der vollständige System- und User-Prompt der Detektor-Phase ist in Anhang 6 dokumentiert.

7.5.2 Beispielhafter Ablauf eines LLM-Findings

Ein typischer Detektionsfall läuft wie folgt ab: In einer Komponente enthält ein `<div>` einen `onClick`-Handler ohne tastaturäquivalentes Ereignis. Das Modell meldet dazu ein Finding mit Datei- und Zeilenbezug sowie Evidenztext. Dieses Finding wird normalisiert, dedupliziert und als `source="llm"` in das UFS überführt, bevor es in der Priorisierungsphase gemeinsam mit den toolbasierten Findings verarbeitet wird.

Listing 7.2: Beispiel eines validierten LLM-Detektor-Findings (vereinfacht)

```
1 {  
2   "title": "Klickbare div-Komponente ohne Tastaturbedienung",  
3   "ruleId": "keyboard-click-only",  
4   "wcagCriteria": ["2.1.1"],  
5   "wcagLevel": "A",  
6   "severity": "serious",  
7   "category": "semantisch",  
8   "filePath": "components/ContactForm.tsx",  
9   "startLine": 45,  
10  "endLine": 49,  
11  "evidence": "<div ... onClick={() => setTopic(t)}>",  
12  "fixSuggestion": "button verwenden oder onKeyDown fuer Enter/Space ergänzen"  
13 }
```

7.6 Prompt-Builder und System-Prompt

Der Prompt-Builder setzt die in Abschnitt 6.2 beschriebene Strategie um. Er besteht aus drei Komponenten: dem System-Prompt, der Token-Budget-Steuerung und der Serialisierung der Findings aus vier Quellen (axe-core, Playwright, grep, LLM-Codeanalyse).

7.6.1 System-Prompt

Der System-Prompt implementiert die in Kapitel 6 gewählten Prompt-Engineering-Strategien. Das **Role-Play Prompting** erfolgt durch die Anweisung „*Du bist ein erfahrener Barrierefreiheitsexperte für React/TypeScript-Webanwendungen*“, die das Modell in eine domänenspezifische Expertenrolle versetzt (vgl. Abschnitt 2.4.2). Das **Few-Shot Prompting** wird durch zwei konkrete Beispiele realisiert: ein **critical**-Finding (Farbkontrast) mit vollständigem Code-Fix als Must-have-Eintrag und ein **moderate**-Finding (`tabIndex`) als Nice-to-have-Eintrag. Beide Beispiele demonstrieren dem Modell das erwartete Ausgabeformat einschließlich der WCAG-Angabe, des Dateipfads und des konkreten Code-Fix-Vorschlags.

Ergänzend enthält der System-Prompt sechs Priorisierungsregeln, die das Must-have/Nice-to-have-Schema operationalisieren:

1. Must-have: Severity **critical/serious** und WCAG Level A/AA

2. Nice-to-have: Severity **moderate**/**minor** oder Level AAA
3. Findings zum selben Element zusammenfassen
4. Codekontext für konkrete React/TypeScript-Fixes nutzen
5. Ausschließlich auf Deutsch antworten
6. Keine Findings erfinden, die nicht in den Daten vorhanden sind

Regel 6 adressiert gezielt das in Abschnitt 2.4 beschriebene Halluzinationsproblem von LLMs. Der vollständige System-Prompt einschließlich der Few-Shot-Beispiele ist in Anhang 5 dokumentiert.

7.6.2 Token-Budget-Steuerung

Wie in Abschnitt 6.2 beschrieben, ist das Kontextfenster lokaler Modelle begrenzt. Der Prompt-Builder reserviert feste Anteile für den System-Prompt und die Modellausgabe; der verbleibende Platz steht für die serialisierten Findings zur Verfügung.²²⁸

Übersteigt die Gesamtmenge der Findings dieses Budget, werden alle Findings quellenübergreifend nach Schweregrad sortiert und von unten abgeschnitten. Findings mit Severity **critical** und **serious** haben dabei Vorrang. Der User-Prompt enthält einen expliziten Hinweis, wenn Findings aus Budgetgründen weggelassen wurden.

7.6.3 Serialisierung

Jedes Finding wird in ein kompaktes Textformat serialisiert, das die wesentlichen Informationen auf wenige Zeilen komprimiert:

Listing 7.3: Serialisiertes Finding im Prompt

```
1 [axe-007] CRITICAL | color-contrast | \ac{WCAG} 1.4.3 (AA)
2 Beschreibung: Element hat unzureichenden Farbkontrast
3 Element: .sidebar > .nav-link
4 Datei: components/Sidebar.tsx:42-52
5 Code:
6 42: <a className="nav-link" href="/dashboard">
7 43:   Dashboard
8 44: </a>
```

Die Serialisierung begrenzt Beschreibungen auf 300 Zeichen und Code-Snippets auf 20 Zeilen, um den Token-Verbrauch je Finding vorhersagbar zu halten.

²²⁸ Beim eingesetzten Qwen2.5-Coder beträgt das Kontextfenster 32.768 Tokens. Nach Abzug von System-Prompt und Output-Reserve verbleiben ca. 24.000 Tokens für die Findings. Vgl. Anhang 7

7.7 Ollama-Client und Output-Formatter

Der Ollama-Client kapselt den HTTP-Aufruf an die lokale Ollama-Instanz (`localhost:11434`). Die Anfrage nutzt den `/api/generate`-Endpunkt mit den Parametern `temperature: 0.05` (NFA-04) und `num_predict: 6144` für die Priorisierungsantwort (in den für die Evaluation genutzten Benchmarkläufen wurden suiteabhängige Werte eingesetzt: 6 144 (test-12), 8 192 (test-50), 12 288 (test-100); vgl. Tabelle 10 und Abschnitt 8.6.2). Die niedrige Temperature-Einstellung reduziert den Nicht-Determinismus der Ausgaben (vgl. Abschnitt 2.4), ohne sie vollständig zu eliminieren. Der Client verwendet ein konfigurierbares Timeout (Default: drei Stunden), da die Inferenz auf CPU-basierten Systemen je nach Modellgröße mehrere Minuten bis deutlich länger in Anspruch nehmen kann. Die Antwort von Ollama enthält neben dem generierten Text auch die tatsächliche Anzahl der Prompt- und Output-Tokens (`prompt_eval_count`, `eval_count`), die für die Evaluation in Kapitel 8 erfasst werden.

Der Output-Formatter erzeugt pro Analysedurchlauf zwei Dateien: einen Markdown-Report für die menschliche Lesbarkeit und einen JSON-Report für die maschinelle Weiterverarbeitung (FA-05). Der Markdown-Report enthält einen Metadaten-Header (URL, Modell, Zeitstempel, Token-Statistiken), die Roh-Ausgabe des Sprachmodells sowie eine Metriken-Tabelle mit den Laufzeiten aller Pipeline-Phasen. Der JSON-Report speichert zusätzlich das Parsing-Ergebnis der LLM-Antwort: Der Formatter versucht, die Markdown-Ausgabe in Must-have- und Nice-to-have-Einträge zu zerlegen. Schlägt dieses Parsing fehl – etwa weil das Modell das vorgegebene Format nicht einhält –, wird dies im Report dokumentiert (`parsed.success: false`).

7.8 Pipeline-Orchestrierung

Die Pipeline-Orchestrierung in `index.ts` verkettet alle Module in der durch Abbildung 6 definierten Reihenfolge (eine kompakte Schemaansicht der Verarbeitungsschritte findet sich in Anhang 4, Abbildung 8). Jede Phase wird einzeln zeitgemessen, um die in NFA-02 geforderte Laufzeitanalyse in Kapitel 8 zu ermöglichen. Die Metriken werden in einem `PipelineMetrics`-Objekt gesammelt, das neben den Phasenzeiten auch Token-Schätzungen, Anreicherungsquoten und Deduplizierungsstatistiken enthält. Die in Kapitel 8 ausgewerteten Kennzahlen – Laufzeit je Phase, Anzahl der LLM-Detektor-Findings, Output-Token-Nutzung sowie Parsing-Erfolg – werden direkt aus diesem Objekt in die pro Durchlauf erzeugten JSON-Reports geschrieben.

Der Ablauf eines vollständigen Durchlaufs ist:

1. `axe-core`: Browser starten, DOM analysieren, Findings normalisieren
2. `Playwright`: Sieben Interaktionschecks ausführen, fehlgeschlagene normalisieren
3. Code-Anreicherung: `axe/Playwright-Findings` mit Quellcode-Kontext ergänzen
4. `Code-Patterns`: Sechs Anti-Patterns im Quellcode suchen, deduplizieren

5. LLM-Codeanalyse (`--llm-detect`): semantische Quellcodeanalyse ausführen und zusätzliche Findings normalisieren
6. Prompt-Builder: Findings serialisieren, Token-Budget anwenden
7. Ollama: kombinierten Priorisierungsprompt senden, Antwort empfangen
8. Output: Markdown- und JSON-Report generieren und speichern

Der `--skip-llm`-Modus speichert den fertigen Prompt als Textdatei, ohne Ollama aufzurufen. Dieser Modus wurde während der Entwicklung genutzt, um den Prompt unabhängig von der Modellverfügbarkeit zu iterieren und für die Evaluation zu dokumentieren.

7.9 Beispielabläufe und Ergebnisse am Untersuchungsobjekt

Ein vollständiger Durchlauf auf der test-12-Suite erzeugt konsistent Findings aus allen vier Quellen (axe-core, Playwright, grep, LLM-Detektor) und überführt diese in einen einheitlichen JSON-Report sowie einen priorisierten Markdown-Report. Für die 30 Benchmarkläufe (je 10 Runs mit `qwen2.5-coder:7b`, `qwen2.5-coder:14b`, `qwen3:32b`) zeigt sich ein stabiler Baseline-Beitrag der regelbasierten Quellen (axe: 4, Playwright: 2, grep: 6 Findings pro Lauf) und ein modellabhängiger Zusatzbeitrag des LLM-Detektors (7b: Ø 10,3; 14b: Ø 19,8; 32b: Ø 19,2).

Die Ergebnisse bestätigen den beabsichtigten Nutzen der kombinierten Pipeline: Regelbasierte Quellen liefern reproduzierbare Pflichtbefunde, während die LLM-gestützte Codeanalyse zusätzliche semantische Violations identifiziert, die in der Tool-baseline nicht oder nur instabil detektierbar sind. Die detaillierte Auswertung mit Recall-Matrix, Rohdaten und Modellvergleich ist in Kapitel 8 sowie im Anhang (Tabellen 29 bis 31) dokumentiert.

8 Evaluation & Diskussion

Dieses Kapitel bewertet das in Kapitel 6 konzipierte und in Kapitel 7 implementierte ACE-System anhand der in Kapitel 1.3 formulierten Forschungsfragen. Abschnitt 8.1 beschreibt Testobjekte, Modelle und Metriken. Abschnitte 8.2–8.5 dokumentieren die Ergebnisse nach Recall, Priorisierungsqualität, Fix-Qualität und Effizienz. Abschnitt 8.6 diskutiert Einschränkungen der Evaluation, Abschnitt 8.7 dokumentiert die Ergebnisse der test-50- und test-100-Messreihen sowie das geplante reale Untersuchungsobjekt, Abschnitt 8.8 beantwortet die Forschungsfragen, und Abschnitt 8.9 leitet daraus kontextspezifische Einsatzempfehlungen ab.

8.1 Versuchsdesign

Die Evaluation ist mehrstufig angelegt und folgt einer bewussten Stufenlogik: Die *test-12-Suite* dient als methodische **Kalibrierungsphase** – bei vollständig bekannter Ground Truth lässt sich prüfen, ob die Pipeline prinzipiell korrekt funktioniert und alle Verstößkategorien grundsätzlich erkennt. Die *test-100-Suite* bildet die eigentliche **Belastungsprüfung**: Erst hier gerät das Token-Budget unter Druck, treten Skalierungseffekte auf und werden Modellunterschiede unter realistischerer Last sichtbar. Die test-50-Suite fungiert als Zwischenstufe. Die Evaluation am realen BW-Empfangsclient (BWEC) der BITBW verbleibt als Folgeschritt. Das folgende Unterkapitel beschreibt die Untersuchungsobjekte im Überblick.

8.1.1 Testobjekte und Ground Truth

Die *test-12-Suite* ist eine eigens entwickelte React-SPA mit zwölf gezielt eingebetteten WCAG-Verstößen (Tabelle 27), die alle drei Kategorien der in Kapitel 2.2.3 eingeführten Taxonomie abdecken: vier syntaktische (V1, V2, V3, V10), drei layout-bezogene (V4, V5, V12) und fünf semantische Verstöße (V6, V7, V8, V9, V11). Da die Ground Truth vollständig bekannt und kontrollierbar ist, eignet sie sich als **Kalibrierungsphase**: Es lässt sich exakt prüfen, ob alle Detektionsquellen korrekt angebunden sind, die Normalisierung funktioniert und das LLM-Modell die Pipeline-Ausgaben grundsätzlich verarbeiten kann. Aufgrund der geringen Komplexität der Suite sind die erzielten Recall-Werte jedoch nicht als repräsentativ für reale Anwendungen zu interpretieren. Die visuelle Gestalt der Testanwendung ist in Anhang 10 und 11 dokumentiert.

Die *test-100-Suite* (100 Violations, 11 Komponenten, vgl. Anhang 44) bildet die eigentliche **Belastungsprüfung**: Erst bei 100 Violations gerät das Token-Budget systematisch unter Druck, treten Trunkierungseffekte auf und werden Modellunterschiede unter annähernd realistischer Last messbar. Die test-50-Suite (50 Violations) dient als Zwischenstufe. Alle Benchmarkläufe auf allen drei Suiten wurden abgeschlossen; die Ergebnisse für test-12 werden in Abschnitten 8.2–8.5 ausgewertet, die Ergebnisse für test-50 und test-100 in Abschnitt 8.7. Als reales Untersuchungsobjekt

ist der BWEC der BITBW vorgesehen – eine produktiv betriebene React-SPA mit unbekannter Ground Truth (vgl. Abschnitt 8.7.3).

8.1.2 Modelle, Konfiguration und Runs

Es wurden drei lokal ausgeführte Sprachmodelle verglichen: `qwen2.5coder:7b`, `qwen2.5-coder:14b` und `qwen3:32b`. Das in Kapitel 5 ebenfalls genannte `deepseek-r1:70b` konnte im Rahmen dieser Arbeit nicht evaluiert werden, da das Modell mit 70 Milliarden Parametern die verfügbare Random-Access Memory (RAM)-Kapazität der Testumgebung überschreitet; es ist als Gegenstand künftiger Evaluation auf leistungsstärkerer Hardware vorgesehen (vgl. Abschnitt 8.7). Jedes Modell absolvierte zehn unabhängige Pipeline-Durchläufe auf demselben Testobjekt, sodass sich insgesamt 30 Runs ergeben. Tabelle 9 fasst die Konfigurationsparameter zusammen. Die Temperatur wurde auf 0.05 gesetzt, um den Nicht-Determinismus zu minimieren. Der Parameter `num_predict` wurde suiteabhängig gesetzt: 6 144 Tokens (test-12), 8 192 Tokens (test-50) und 12 288 Tokens (test-100); vgl. Tabelle 10 und Abschnitt 8.6.2. Der LLM-Detektor operiert mit separaten Parametern: 2 048 Tokens für die Code-Analyse, 4 000 Zeichen pro Chunk, maximal 25 Dateien pro Durchlauf.

Parameter	Wert
Testsuite	test-12 (12 eingebettete Verstöße)
Modelle	<code>qwen2.5-coder:7b</code> , <code>qwen2.5-coder:14b</code> , <code>qwen3:32b</code>
Runs je Modell	10
Temperature	0.05
<code>num_predict</code> (Hauptanalyse)	6 144 Tokens
<code>num_predict</code> (LLM-Detektor)	2 048 Tokens
LLM-Detektor Chunk-Größe	4 000 Zeichen
LLM-Detektor Max. Dateien	25
Hardware	CPU-basiert, lokale Ollama-Instanz

Tab. 9: Konfigurationsparameter der Benchmarkläufe (final optimiert)

Tabelle 10 zeigt die geplante Verteilung der Runs auf alle vier Modellkandidaten und drei Testsuiten. Im Rahmen dieser Arbeit wurden die test-12-Läufe (30 Runs), die test-50-Läufe (30 Runs) sowie die test-100-Läufe (30 Runs, alle drei Modelle) durchgeführt; die BWEC-Evaluation verbleibt als Folgeschritt (vgl. Abschnitt 8.7.3). `deepseek-r1:70b` konnte aufgrund unzureichender RAM-Kapazität nicht evaluiert werden; die konsolidierte Modellbewertung der drei evaluierten Modelle findet sich in Tabelle 31.

8.1.3 Metriken und Messverfahren

Ausgewertet werden vier Metriken: (1) **Recall** (erkannte Violations relativ zur Ground Truth), (2) **Priorisierungsqualität** (Korrektheit der Must-have/Nice-to-have-Klassifikation), (3) **Fix-Qualität** nach der dreistufigen Skala aus Tabelle 22 sowie (4) **Effizienz** (Laufzeit und Konsistenz

Modell	Typ	test-12	test-50	test-100
qwen2.5-coder:7b	Code	10 Runs	10 Runs	10 Runs
qwen2.5-coder:14b	Code	10 Runs	10 Runs	10 Runs
qwen3:32b	Generalist	10 Runs	10 Runs	10 Runs
deepseek-r1:70b	Reasoning	–	–	–
Gesamt (durchgeführt)		30	30	30

Tab. 10: Verteilung der Benchmark-Runs auf Modelle und Testsuiten (je 10 Runs pro Modell und Suite). `num_predict`: test-12 = 6144; test-50 = 8192; test-100 = 12288. `deepseek-r1:70b` konnte aufgrund fehlender RAM-Kapazität nicht evaluiert werden. Bemerkenswert: `qwen3:32b` ist ein Generalist-Modell (kein Code-Spezialist), erzielt auf test-100 aber den höchsten Recall – ein Hinweis darauf, dass Modellgröße und allgemeine Instruktionsfolge wichtigere Faktoren sind als die Code-Spezialisierung.

über mehrere Runs). Eine Violation gilt als erkannt, wenn sie in der Mehrzahl der zehn Runs (> 5) im finalen LLM-Report erscheint. Diese Schwelle bietet Robustheit gegenüber einzelnen LLM-Ausreißern, ohne bei probabilistischen Modellen zu restriktiv zu sein – ein strengeres Kriterium (z. B. 10/10) würde semantisch äquivalente, aber unterschiedlich formulierte Findings in einzelnen Runs ungerechtfertigt ausschließen. Die vollständigen Rohdaten aller 30 Runs finden sich in Tabelle 30.

8.2 Ergebnisse: Detektionsqualität (Recall)

Dieser Abschnitt bewertet, welche der 12 Ground-Truth-Violations die Pipeline erkennt – zunächst ohne LLM-Detektor (Baseline), dann mit LLM-Detektor je Modell und abschließend differenziert nach Verstoßkategorie. Die UI-Screenshots der test-12-Suite finden sich in Anhang 10 und 11.

8.2.1 Baseline: Tool-Pipeline ohne LLM-Detektor

Ohne LLM-Detektor erkennt die Tool-Pipeline (`axe-core` + `Playwright` + `grep`) in jedem Durchlauf stabil **8 von 12 Violations** (Recall: 66,7%). Nicht erkannt werden V2 (fehlendes `aria-label`), V8 (modaler Fokus-Trap), V11 (DOM-Reihenfolge) und V12 (Mindestgröße). Diese Fälle erfordern überwiegend semantische Kontextinterpretation. Insbesondere fehlt im `Playwright`-Set ein dedizierter Fokus-Trap-Test, weshalb V8 trotz der in Kapitel 6 als Quelle vorgesehenen `Playwright`-Prüfung regelbasiert nicht zuverlässig erfasst wird. Ein generischer, anwendungsunabhängiger Fokus-Trap-Test ließ sich im Rahmen des PoC nicht implementieren, da die zuverlässige Identifikation von Modalfenstern anwendungsspezifische Selektoren erfordert; die Erweiterung des `Playwright`-Satzes ist als kurzfristiger Folgeschritt vorgesehen (vgl. Abschnitt 9.3.1).

8.2.2 Recall nach LLM-Detektor-Augmentierung

Tabelle 11 zeigt den Recall nach Hinzunahme des LLM-Detektors. Mit `qwen3:32b` sowie `qwen2.5-coder:14b` werden alle vier zuvor fehlenden Violations konsistent erkannt; `qwen2.5coder:7b` gewinnt nur V8 hinzu, lässt dafür aber V10 gelegentlich weg.

Bedingung	Erkannte V.	Recall	Neu ggü. Baseline	Nicht erkannt
Baseline (Tools only)	8 / 12	66,7 %	–	V2, V8, V11, V12
+ <code>qwen2.5-coder:7b</code>	9 / 12	75,0 %	+V8	V2, V10*, V11
+ <code>qwen2.5-coder:14b</code>	12 / 12	100 %	+V2, +V8, +V11, +V12	–
+ <code>qwen3:32b</code>	12 / 12	100 %	+V2, +V8, +V11, +V12	–

Tab. 11: Recall der ACE-Pipeline je Bedingung. *`qwen2.5-coder:7b` lässt V10 (aria-hidden) teilweise aus dem Report heraus – axe-core erkennt V10 zuverlässig, das Modell übernimmt es aber nicht konsistent (6/10 Runs). V12 wird in 8/10 Runs korrekt berichtet.

Die detaillierte Aufschlüsselung für alle 12 Violations mit der jeweiligen Erkennungsquelle findet sich in Tabelle 28, die vollständige Recall-Matrix in Tabelle 29.

8.2.3 Erkennungsleistung nach Verstoßkategorie

Für TF1 ist nicht nur der Gesamt-Recall relevant, sondern die Frage, welche *Typen* von Violations erkannt werden. Tabelle 12 schlüsselt den Recall nach den drei in Kapitel 2.2.3 eingeführten Kategorien auf.

Kategorie	Violations	Baseline	+ 7b	+ 14b	+ 32b
Syntaktisch	V1, V2, V3, V10	3 / 4 (75 %)	3 / 4 (75 %)	4 / 4 (100 %)	4 / 4 (100 %)
Layout	V4, V5, V12	2 / 3 (67 %)	3 / 3 (100 %)	3 / 3 (100 %)	3 / 3 (100 %)
Semantisch	V6, V7, V8, V9, V11	3 / 5 (60 %)	4 / 5 (80 %)	5 / 5 (100 %)	5 / 5 (100 %)
Gesamt	12	8 / 12 (67 %)	9 / 12 (75 %)	12 / 12 (100 %)	12 / 12 (100 %)

Tab. 12: Erkennungsleistung differenziert nach Verstoßkategorie (test-12-Suite). Werte basieren auf der Mehrzahl der zehn Runs je Modell.

Drei Beobachtungen lassen sich aus dieser Aufschlüsselung ableiten. Erstens zeigt die Baseline die **größte Lücke bei semantischen Violations** (60 %): Genau jene Kategorie, die konzeptionell die schwierigsten Prüffälle darstellt – Zustandsabhängigkeit, fehlende Keyboard-Interaktion, DOM-Konsistenz – wird von regelbasierten Tools am unzuverlässigsten abgedeckt. Zweitens trägt der LLM-Detektor seinen **stärksten Recall-Gewinn im semantischen Bereich** bei: Von 60 % auf 100 % mit 14b und 32b, also eine Steigerung um 40 Prozentpunkte. Dies bestätigt die in Kapitel 6 formulierte Entwurfsentscheidung, die LLM-Codeanalyse als semantische Ergänzungsquelle einzusetzen. Drittens sind **syntaktische und layout-bezogene Violations** ab 14b vollständig abgedeckt, da axe-core den Großteil dieser Fälle bereits in der Baseline erkennt.

8.3 Ergebnisse: Priorisierungsqualität

Neben dem Recall ist die Korrektheit der Must-have/Nice-to-have-Klassifikation für den praktischen Nutzen des Systems entscheidend: Eine falsch priorisierte Empfehlung kann dazu führen, dass gesetzlich verpflichtende Anforderungen als optional behandelt werden. Dieser Abschnitt vergleicht das Priorisierungsverhalten der drei Modelle und analysiert den Einfluss des Token-Limits auf die Kategorisierung.

8.3.1 Modellvergleich Must-have und Nice-to-have

Das Prompt-Design schreibt dem Modell vor, Findings nach Must-have (WCAG Level A/AA, Severity critical/serious) und Nice-to-have (moderate/minor oder Level AAA) zu kategorisieren. Von den 12 eingebetteten Violations sind elf eindeutig dem Must-have-Bereich zuzuordnen; lediglich V12 (Mindestgröße 2.5.8 AA, moderate) ist grenzwertig. Tabelle 13 zeigt die mittleren Zählungen über 10 Runs.

Modell	Must-have Ø	Nice-to-have Ø	Gesamt-Findings Ø	Bewertung
qwen2.5-coder:7b	5,0	18,3	22,3	Mangelhaft
qwen2.5-coder:14b	20,7	0,0	31,8	Undifferenziert
qwen3:32b	11,3	9,7	31,2	Gut

Tab. 13: Priorisierungsverhalten der drei Modelle (Mittelwerte über 10 Runs)

qwen2.5-coder:7b zeigt eine **inverse Priorisierung**: Pflichtanforderungen wie WCAG 2.4.7 (Fokusindikator, pw-001, Level AA) und WCAG 2.4.1 (Skip-Link, pw-002, Level A) werden als Nice-to-have klassifiziert, während tatsächlich optionale Findings als Must-have eingestuft werden. Für einen BITV-konformen Einsatz ist dieses Modell daher *nicht geeignet*.

qwen2.5-coder:14b klassifiziert nahezu alle Findings als Must-have (Ø 20,7) und gibt 0,0 Nice-to-have aus. Da die Ausgabe in mehreren Runs das Output-Token-Limit ausschöpft, ist eine Length-Bias wahrscheinlich: spätere Listenanteile (typisch Nice-to-have) werden tendenziell unterrepräsentiert.

qwen3:32b liefert mit Ø 11,3 Must-have und 9,7 Nice-to-have die **ausgewogenste Trennung**. Alle WCAG-Level-A/AA-Violations mit Severity serious werden korrekt als Must-have eingestuft; Findings mit Severity moderate (z.B. Mindestgröße von Buttons, `role="button"` ohne `tabIndex`) werden als Nice-to-have kategorisiert.

8.3.2 Einfluss des Token-Limits auf die Kategorisierung

Die Summe aus Must-have und Nice-to-have übersteigt bei 14b und 32b die Anzahl der LLM-Detektor-Findings, weil die Priorisierungsphase sämtliche Quellen (axe, Playwright, grep, LLM)

zusammenführt und teils mehrfach klassifiziert. Wichtiger für die Interpretation ist, dass der Token-Limit-Effekt die Priorisierungsmetrik asymmetrisch verzerrt: Findings am Listenende – typischerweise Nice-to-have – werden bei 7b und 14b systematisch abgeschnitten. Die Werte in Tabelle 13 sind daher für 7b und 14b als *untere Schranke* zu verstehen; belastbare Aussagen zur absoluten Nice-to-have-Zahl erfordern Replikationsläufe mit erhöhtem `num_predict`.

8.4 Ergebnisse: Fix-Qualität (qualitativ)

Ein hoher Recall allein ist für den Entwicklungsalltag nicht ausreichend: Entscheidend ist, ob die generierten Handlungsempfehlungen konkret und direkt umsetzbar sind. Dieser Abschnitt bewertet die Fix-Qualität anhand der dreistufigen Skala (Stufe 0–2) und illustriert die Unterschiede zwischen den Modellen an konkreten Fallbeispielen.

8.4.1 Bewertungsmaßstab

Die Bewertungsskala (Tabelle 22) unterscheidet drei Stufen: Stufe 2 (konkret: Dateiname, Zeilennummer, umsetzbarer Code-Vorschlag), Stufe 1 (teilweise: richtiger Fix-Typ ohne konkreten Kontext) und Stufe 0 (generisch: nur Problembeschreibung). Die vollständige Bewertung aller zwölf Violations findet sich in Tabelle 36. Im Ergebnis erreicht `qwen3:32b` für 11 von 12 Violations Stufe 2; `qwen2.5-coder:14b` erreicht überwiegend Stufe 1; `qwen2.5-coder:7b` schwankt zwischen Stufe 0 und 1.

8.4.2 Fallbeispiele nach Modell

Fallbeispiel 1 – Konkrete Empfehlung (Stufe 2, `qwen3:32b`). Für V8 (Modaler Dialog ohne Fokus-Trap) generiert `qwen3:32b`:

Der Vorschlag referenziert `ContactForm.tsx` namentlich, schlägt einen React-konformen `useEffect`-Hook vor, der bei `modalOpen` den ersten fokussierbaren Knoten im Modal-Overlay selektiert, und enthält vollständige Tab- sowie Shift+Tab-Logik mit sauberem Event-Listener-Cleanup – Stufe 2.

Fallbeispiel 2 – Teilweise Empfehlung (Stufe 1, `qwen2.5-coder:14b`). `qwen2.5-coder:14b` empfiehlt für V4 (Kontrastverhältnis): „Ändere Hintergrund- oder Textfarbe, um Kontrast zu verbessern“ mit einem Inline-Style-Beispiel ohne spezifische Hex-Werte aus dem tatsächlichen Design. Die Datei wird korrekt benannt, der Fix-Typ stimmt, aber der konkrete Hex-Code fehlt – Stufe 1.

Fallbeispiel 3 – Fehlklassifikation (Stufe 0, qwen2.5-coder:7b). qwen2.5-coder:7b stuft den Fokusindikator-Verlust (V5, WCAG 2.4.7 AA, Severity serious) als Nice-to-have ein und beschreibt es als: „*Optionale Verbesserung: Outline-Stil anpassen*“. Tatsächlich handelt es sich um eine gesetzlich relevante Pflichtanforderung – klare Fehlklassifikation mit falschem Priorisierungssignal.

8.5 Ergebnisse: Effizienz und Robustheit

Über die inhaltliche Qualität hinaus ist für den produktiven Einsatz relevant, wie schnell und wie konsistent die Pipeline arbeitet. Dieser Abschnitt prüft die Konformität mit NFA-02 (Laufzeit), analysiert die Varianz der Ergebnisse über mehrere Runs und bewertet die Token-Nutzung als Indikator für systematische Antwortabbrüche.

8.5.1 Laufzeit und NFA-02-Konformität

Tabelle 14 zeigt die mittleren Laufzeiten je Pipeline-Phase. Die Gesamtlaufzeit umfasst axe-core, Playwright, LLM-Detektor (5 Chunks), Prompt-Build und LLM-Priorisierung.

Modell	LLM-Detektor Ø	LLM-Priorisierung Ø	Gesamt Ø	Min	Max
qwen2.5-coder:7b	128 s	97 s	231 s	191 s	284 s
qwen2.5-coder:14b	171 s	243 s	420 s	300 s	480 s
qwen3:32b	335 s	361 s	701 s	617 s	937 s

Tab. 14: Mittlere Laufzeiten je Pipeline-Phase ($n = 10$ je Modell)

NFA-02 fordert eine Gesamtlaufzeit unter zehn Minuten. qwen2.5-coder:7b (Ø 231 s) und qwen2.5-coder:14b (Ø 420 s) erfüllen diese Anforderung komfortabel. qwen3:32b benötigt deutlich mehr Rechenzeit (Ø 701 s) und verletzt NFA-02 in Ausreißerläufen (max. 937 s = 15,6 Min.) spürbar; es erfüllt die Anforderung damit nicht vollständig. Im Kontext des PoC wird es dennoch als Referenzmodell geführt, da es die höchste Detektionsqualität erzielt – für zeitkritische Entwicklungszyklen ist es jedoch nicht empfohlen (vgl. Tabelle 17). Auf GPU-beschleunigter Hardware wären die Laufzeiten erheblich kürzer; da aber NFA-01 (Lokal-First) den Einsatz proprietärer Cloud-GPUs ausschließt, ist dies im Rahmen des PoC nicht realisierbar.

Zur Einordnung der Laufzeitkosten enthält Tabelle 15 Effizienzkennzahlen. Sie zeigen, dass 14b trotz erhöhtem Zeitaufwand eine bessere Effizienz-pro-Finding bietet als 7b, während 32b bei ähnlicher Detektionsleistung wie 14b erheblich mehr Rechenzeit benötigt.

Modell	LLM-Findings Ø	Dauer Ø	Findings/s	Mehrgewinn ggü. 7b
qwen2.5-coder:7b	11,1	231 s	0,048	–
qwen2.5-coder:14b	22,8	420 s	0,054	+105 % bei +82 % Zeitaufwand
qwen3:32b	19,5	701 s	0,028	+76 % bei +203 % Zeitaufwand

Tab. 15: Effizienzvergleich: LLM-Findings je Sekunde Gesamtlaufzeit

8.5.2 Konsistenz und Parsing-Stabilität

Die Standardabweichung der LLM-Detektor-Findings unterscheidet sich zwischen den Modellen erheblich: **qwen3:32b** erreicht $\sigma = 0,53$ (Range: 19–20 Findings) – auf test-100 steigt diese Streuung deutlich auf $\sigma = 5,63$ (vgl. Abschnitt 8.7.2); **qwen2.5-coder:14b** erreicht $\sigma = 0,42$ (Range: 22–23 Findings) und zeigt damit die engste Konsistenz aller Modelle auf test-12. **qwen2.5-coder:7b** zeigt mit $\sigma = 3,28$ höhere Varianz und produziert ein breiteres Spektrum (Range: 4–16 Findings).

Die festen Detektoren (axe, Playwright, grep) bleiben über alle 30 Runs vollständig deterministisch (je 4, 2 und 6 Findings), sodass die Varianz ausschließlich aus der LLM-Stufe stammt (vgl. Tabelle 31). Die Quantilanalyse über alle 30 Runs (Tabelle 39) zeigt die Clusterung: Der Median liegt bei 19–20 Findings (32b-Cluster) mit Q3 bei 22 Findings (14b-Bereich) und Q1 bei 11 Findings (7b-Bereich). Das Interquartile Range (IQR) von 11 belegt die unterschiedlichen Verteilungen zwischen den Modellen.

In allen 30 Runs wurde der LLM-Output erfolgreich geparkt (**parsed.success: true**, 100 %). Das Prompt-Design mit Role-Play- und Few-Shot-Prompting zeigt damit eine hohe Formatstabilität über alle drei Modelle und unterstützt NFA-04 (Reproduzierbarkeit).

8.5.3 Token-Nutzung

Die Output-Token-Nutzung zeigt ein klares Trunkierungsprofil: **qwen2.5-coder:7b** erreicht in 5 von 10 Runs das Output-Limit von 6 144 Tokens, **qwen2.5-coder:14b** in 4 von 10 Runs; **qwen3:32b** liegt im Mittel bei 3 491 Output-Tokens und wird in lediglich 1 von 10 Runs abgeschnitten. Das Token-Budget ist damit für 7b und 14b messbar unter Druck, während 32b noch ausreichend Spielraum hat.

Tabelle 32 schlüsselt Input- und Output-Tokens je Modell auf. Die größeren Modelle konsumieren tendenziell weniger Output-Tokens bei höherem oder vergleichbarem Recall, was auf eine höhere Fokussierung und Effizienz hindeutet. Die Token-Disponibilität ermöglicht vollständige Ausgabe von Must-have und Nice-to-have Findings ohne Abschneide-Bias.

8.6 Limitationen und Validität

Die in den vorherigen Abschnitten präsentierten Ergebnisse sind unter Berücksichtigung der methodischen Rahmenbedingungen zu interpretieren. Dieser Abschnitt benennt die wesentlichen Einschränkungen hinsichtlich externer Validität, interner Validität sowie weiterer Einzelaspekte des Versuchsaufbaus.

8.6.1 Externe Validität

Die Evaluation stützt sich auf drei synthetisch konstruierte Testsuiten (test-12, test-50, test-100). Obwohl alle drei Verstößkategorien vertreten und die Ground Truth vollständig bekannt sind, lässt sich die Übertragbarkeit auf produktive Anwendungen mit unkontrollierter Ground Truth nicht direkt ableiten. Skalierungseffekte wurden auf test-50 (Abschnitt 8.7.1) und test-100 (Abschnitt 8.7.2) sichtbar. Die Evaluation am realen BWEK verbleibt als Folgeschritt (vgl. Abschnitt 8.7.3).

8.6.2 Interne Validität und Token-Limit-Effekt

Die Priorisierungszählungen (Must-have/Nice-to-have) sind durch das Token-Limit beeinflusst: Bei 7b endeten 5 von 10 Runs am Output-Limit, bei 14b waren es 4 von 10 Runs, bei 32b lediglich 1 von 10 Runs. Dadurch werden systematische Abschneidungs-Bias begünstigt, insbesondere zulasten später Listenanteile (typischerweise Nice-to-have). Die Recall-Messung bleibt davon weitgehend unberührt, da zentrale Violations typischerweise früh im Output erscheinen.

Zusätzlich existiert ein Konstruktvaliditätsrisiko durch die Operationalisierung „Violation erkannt“, da die Erkennung über Report-Präsenz (Mehrheit der Runs) und nicht über eine semantisch kanonische Äquivalenzklasse gemessen wird. Dies kann bei paraphrasierten Findings zu konservativen oder liberalen Zuordnungen führen.

8.6.3 Weitere Einschränkungen

Grep-Limitierungen. Die sechs grep-Pattern erkennen bekannte Anti-Patterns zuverlässig auf Textebene, sind aber nicht erschöpfend und struktursensitiv. Pattern 6 (onClick ohne onKeyDown) kann in bestimmten Konstellationen legitime Implementierungen als Violation markieren (False Positive).

Halluzinationen und Über-Detektion. Alle drei Modelle erzeugen in einzelnen Runs Findings ohne belastbare Grundlage. Die LLM-Detektor-Findings überschreiten die Ground-Truth-Zahl von 12 teilweise erheblich: 7b maximal auf 16 Findings, 14b maximal auf 23 Findings, 32b maximal auf 20 Findings. Trotz technischer Deduplizierung verbleiben semantisch redundante Dubletten, sodass eine inhaltliche Nachprüfung weiterhin erforderlich ist. Eine automatische Zusammenführung ähnlicher Findings ist dabei bewusst nicht implementiert: Würden zwei inhaltlich ähnliche, aber verschiedene Befunde irrtümlich gemergt, ginge ein echter Hinweis verloren – ein False Negative, das schwerer wiegt als ein redundanter Report-Eintrag.

Eine Stichproben-Plausibilisierung an Run 01 (`qwen3:32b`, test-100; vollständiger Run ohne Truncation, vgl. Anhang 52) zeigt jedoch, dass echte Halluzinationen – d. h. Findings ohne jede Grundlage im Quellcode – selten sind: Von 72 Einträgen im finalen Report konnten 64 eindeutig zugeordnet werden: 51 True Positives, 10 Quell-Duplikate (dieselbe Violation wird von `axe-core` und LLM-Detektor unabhängig erkannt), 2 Mehrfach-Instanzen derselben DOM-Violation sowie 1 gebündeltes Finding. 8 Einträge sind nicht eindeutig einer Ground-Truth-Violation zuzuordnen (Paraphrasen oder Grenzfälle); 0 bestätigte Halluzinationen in Run 01. In Run 03 wurde 1 Halluzination identifiziert (`[llm-061]` meldet eine Tabelle ohne `caption` in `Notifications.tsx`, obwohl dort keine Tabelle existiert). Die Überdetektions-Quote in den Anhang-Tabellen (34–61) misst ausschließlich die Mengenrelation zwischen LLM-Detektor-Rohoutput und Ground-Truth-Zahl; sie ist kein Maß für die inhaltliche Trefferqualität des finalen Reports.

Für produktive Compliance-Prozesse ist dennoch ein zweistufiges Vorgehen angezeigt: (1) automatisierte Vorselektion durch ACE, (2) kuratierte fachliche Plausibilisierung des Reports – primär um Quell-Duplikate zu konsolidieren, instabile Findings zu verifizieren und den seltenen, aber möglichen Halluzinationsfall auszuschließen.

Hardware-Abhängigkeit. Alle Laufzeitmessungen wurden auf einem einzelnen CPU-basierten System durchgeführt. Auf GPU-beschleunigter Hardware oder mit quantisierten Modellversionen wären erheblich kürzere Laufzeiten zu erwarten, sodass NFA-02 dann auch für 32b klar erfüllbar wäre.

8.7 Belastungsprüfung: test-50 und test-100

Die in den vorherigen Abschnitten präsentierten Kalibrierungsergebnisse (test-12) zeigen, dass die Pipeline korrekt funktioniert und alle Verstößkategorien grundsätzlich erkennbar sind. Dieser Abschnitt dokumentiert die eigentliche Belastungsprüfung: die test-50-Suite (50 Violations) als Zwischenstufe sowie die test-100-Suite (100 Violations) als primäres Skalierungsobjekt, an dem Tokenbudget-Effekte, Modellunterschiede und Erkennungsgrenzen unter realistischerer Last sichtbar werden. Die Evaluation am realen BWEC der BITBW verbleibt als Folgeschritt.

8.7.1 Evaluation an test-50

Die test-50-Messreihe wurde mit identischer Konfiguration wie test-12 durchgeführt: drei Modelle, je zehn Runs, `temperature = 0,05`, CPU-basierte lokale Ollama-Instanz. Das Output-Token-Limit wurde entsprechend der größeren Suite auf `num_predict = 8192` angehoben. Die vollständigen Rohdaten aller 30 Runs finden sich in Tabelle 44.

Token-Budget unter Druck. Die wichtigste Beobachtung der test-50-Messreihe ist, dass das Token-Budget nun systematisch erschöpft wird. Die mittleren Input-Tokens stiegen von Ø 4000–5600 (test-12) auf Ø 13200–14200 (test-50), da bei 50 Violations deutlich mehr Findings serialisiert werden müssen. Trotz des auf 8192 Tokens angehobenen Output-Limits schöpfen 60–90 % aller Runs das Budget vollständig aus: `qwen2.5-coder:7b` in 6 / 10 Runs, `qwen2.5-coder:14b` in 7 / 10 Runs und `qwen3:32b` in 9 / 10 Runs. Die ursprünglich gestellte Frage, ob das Token-Budget bei test-50 kippt, ist damit eindeutig bestätigt (vgl. Tabelle 46).

Detektionsleistung. Die Baseline-Tools liefern auf test-50 stabil 14 axe-core-, 2 Playwright- und 21 grep-Findings (gesamt 37 Tool-Findings). Diese Finding-Zahlen sind jedoch nicht direkt mit dem Ground-Truth-Recall (erkannte Violations von 50) gleichzusetzen, da Findings Duplikate oder semantische Überlappungen enthalten können. Für die Recall-Bewertung wird deshalb ausschließlich die Violation-Abdeckung im finalen Report mit der $> 5/10$ -Schwelle herangezogen (vgl. Anhang Tabellen 43 und 42). Der LLM-Detektor erzielt: `qwen2.5-coder:7b` Ø 46,6 Findings ($\sigma \approx 3,20$; Range: 40–53), `qwen2.5-coder:14b` Ø 51,3 Findings ($\sigma \approx 2,21$; Range: 45–52) und `qwen3:32b` Ø 60,7 Findings ($\sigma \approx 1,25$; Range: 59–63). Bemerkenswert ist, dass `qwen3:32b` auf test-50 die höchste Konsistenz aller drei Modelle zeigt – auf test-12 hatte `qwen2.5-coder:14b` die engste Streuung (vgl. Tabelle 45).

Ein struktureller Effekt ist in der Recall-Matrix (Tabelle 43) direkt ablesbar: Die Baseline (Tool-only) erzielt 43 / 50 Violations, während `qwen2.5-coder:7b` mit der vollen Pipeline nur 36 / 50 erreicht. Das ist kein Messfehler, sondern ein methodisch bedeutsamer Befund: Bei 50 Violations wird die LLM-Zusammenfassungsphase zum **Engpass statt zum Mehrwert** – sie komprimiert die ohnehin vollständigen Tool-Findings unter Informationsverlust, statt sie zu ergänzen. Die Ursache liegt im Token-Budget: 7b wird in 60 % der Runs abgeschnitten und überspringt oder bündelt dabei instabil Tool-detektierte Violations. Erst `qwen2.5-coder:14b` (48 / 50) und `qwen3:32b` (50 / 50) übertreffen das Baseline-Niveau.

Priorisierungsqualität. Tabelle 16 fasst das Must-have/Nice-to-have-Verhalten auf test-50 zusammen. `qwen3:32b` erzielt mit Ø 47,6 Must-have die plausibelste Klassifikation relativ zur Ground Truth von 50 Violations. `qwen2.5-coder:14b` zeigt ein bimodales Muster: In acht von zehn Runs werden exakt 14 Must-have gemeldet, in zwei Ausreißerläufen hingegen 50 bzw.

59 Must-have mit nur 8 bzw. 0 Nice-to-have – ein klarer Token-Trunkierungseffekt, der die Kategorisierungsgrenze verschiebt. **qwen2.5-coder:7b** bleibt mit einer Range von 10–54 Must-have instabil.

Modell	Must-have Ø	Nice-to-have Ø	Gesamt Ø	Bewertung
qwen2.5-coder:7b	21,5	35,6	57,1	Instabil
qwen2.5-coder:14b	22,1	35,7	57,8	Bimodal (Token-Effekt)
qwen3:32b	47,6	27,3	74,9	Plausibel

Tab. 16: Priorisierungsverhalten auf der test-50-Suite (Mittelwerte über 10 Runs)

Effizienz und NFA-02. Die Gesamtlaufzeit skaliert substanziell: **qwen2.5-coder:7b** Ø 413 s (Range: 291–465 s), **qwen2.5-coder:14b** Ø 826 s (Range: 733–871 s), **qwen3:32b** Ø 1 897 s (Range: 1 752–2 011 s). Damit erfüllt auf test-50 nur **qwen2.5-coder:7b** NFA-02; **qwen2.5-coder:14b** und **qwen3:32b** überschreiten die Zehn-Minuten-Grenze klar (vgl. Tabelle 45).

Skalierungsvergleich test-12 vs. test-50. Tabelle ?? im Anhang quantifiziert die zentralen Skalierungseffekte. Bei einer 4,2-fachen Erhöhung der Violation-Zahl skaliert **qwen2.5-coder:7b** mit dem Faktor 4,2 annähernd proportional, während **qwen2.5-coder:14b** (+2,3×) und **qwen3:32b** (+3,1×) einen komprimierten Anstieg zeigen – ein Hinweis darauf, dass die größeren Modelle auf test-12 stärker token-limitiert waren. Die Laufzeit skaliert unterlinear (Faktor 2,0–2,8), da Detektionsoverhead und Prompt-Infrastruktur nur partiell mit der Violation-Zahl wachsen.

8.7.2 Evaluation an test-100

Die test-100-Suite (100 eingebettete Violations, 11 Komponenten, vgl. Anhang 44) wurde mit allen drei Modellen evaluiert: je zehn Runs pro Modell (**num_predict** = 12 288), insgesamt 30 Runs.

Token-Budget und Truncation. Das Output-Limit von 12 288 Tokens bewährt sich modellabhängig sehr unterschiedlich. **qwen2.5-coder:7b** schöpft das Budget in 80 % der Runs aus (8 / 10). **qwen2.5-coder:14b** erreicht in *allen* 10 Runs das Token-Limit (100 % Truncation) – kein Run lieferte einen vollständigen Bericht. **qwen3:32b** verhält sich gegensätzlich: nur 1 / 10 Runs ist trunkiert, die übrigen Runs schließen vollständig ab; der mittlere Output liegt bei 7 360 Tokens (vgl. Tabelle 59).

Detektionsleistung und Recall. Die Baseline-Tools (axe-core, Playwright, grep) erkennen stabil 34 / 100 Violations. Der LLM-Detektor erzielt: **qwen2.5-coder:7b** Ø 104,0 Findings ($\sigma = 5,75$, Range: 96–110), **qwen2.5-coder:14b** Ø 94,3 Findings ($\sigma = 3,23$, Range: 91–100) und **qwen3:32b**

Ø 99,1 Findings ($\sigma = 5,63$, Range: 93–106) (vgl. Tabelle 58). Auf Basis der Recall-Matrix (Tabelle 56, Schwelle $> 5/10$ Runs) ergibt sich: Baseline 34 / 100, +7b ≈ 40 / 100, +14b ≈ 53 / 100, +32b ≈ 63 / 100. Der starke Recall-Zuwachs bei 32b gegenüber der Baseline (+29 Violations) bestätigt den LLM-Mehrwert auch bei 100 Violations; die hohe Truncation-Rate bei 7b und 14b dämpft deren Erkennungsleistung systematisch.

Die 37 nicht erkannten Violations bei **qwen3:32b** lassen sich auf drei strukturelle Ursachenklassen zurückführen (vgl. Pareto-Analyse, Abbildung 16 im Anhang): Erstens Violations, die ausschließlich über DOM-Zustandsübergänge sichtbar werden und für die kein dedizierter Playwright-Test im PoC implementiert ist – insbesondere Fokus-Trap- und Modalfenster-Violations (*V47*, *V78*). Zweitens Violations, deren Muster im grep-Pattern-Set nicht abgedeckt ist und deren Quellcode-Signal für das Modell zu implizit ist, um konsistent erkannt zu werden. Drittens Violations, bei denen das Modell zwar eine plausible Beschreibung erzeugt, diese jedoch in weniger als sechs der zehn Runs erscheint und damit die $> 5/10$ -Schwelle verfehlt – ein Grenzfall-Effekt, der die probabilistische Natur des Ansatzes widerspiegelt.

Einordnung der Pareto-Perspektive. Für die Ursachenanalyse der verbleibenden Erkennungslücken wird das Pareto-Diagramm bewusst auf **test-100 mit qwen3:32b** fokussiert (Abbildung 16 im Anhang). Diese Auswahl kombiniert den anspruchsvollsten Skalierungsfall (100 Violations) mit der methodisch belastbarsten Modellbasis in dieser Suite (höchster Recall, zugleich nur 1 / 10 trunkierten Runs). Damit wird ein *Best-Case-unter-Last* analysiert: Die verbleibenden Lücken sind weniger durch offensichtliche Modellschwäche als durch strukturell schwierige Violations erklärbar. Ein Vollvergleich mit separaten Pareto-Diagrammen für alle Modelle und alle Suites wäre zwar möglich, würde den Anhang jedoch deutlich aufblähen und den Erkenntnisgewinn nur begrenzt erhöhen; der Modellvergleich ist bereits über Recall-, Laufzeit-, Konsistenz- und Truncation-Grafiken abgedeckt (vgl. Anhang Abbildungen 25 bis 29).

Laufzeit und NFA-02. Die Gesamtlaufzeit steigt auf test-100 deutlich an: **qwen2.5-coder:7b** Ø 765 s, **qwen2.5-coder:14b** Ø 1 550 s, **qwen3:32b** Ø 2 616 s. Damit verletzen alle drei Modelle NFA-02 (Laufzeit < 10 Min.) auf test-100 klar; bei strenger NFA-Auslegung ist dort kein Modell uneingeschränkt geeignet (vgl. Tabellen 58 und 21). Damit ist test-100 im PoC-Kontext nicht zeitkritisch einsetzbar; auf GPU-beschleunigter Hardware wäre NFA-02 jedoch realistischer erreichbar.

Erkennungsrate, Empfehlungsqualität und CSV-Auszug. Die vollständige Violation-für-Violation-Auswertung findet sich in den Anhang-Tabellen 55 (Erkennungsrate), 56 (Recall-Matrix) und 64 (Empfehlungsqualität). In Run 01 erhalten alle 73 erkannten Violations mit **qwen3:32b** die Bewertung Stufe 2 (konkreter Dateiname, Zeile und Code-Snippet); 27 Violations sind in diesem Run nicht im Report enthalten und werden mit Stufe 0 ausgewiesen. (Der konsistente Recall über alle 10 Runs beträgt 63 / 100, vgl. Tabelle 56.) Repräsentative Rohdatenzeilen zeigt Tabelle 65.

Gesamteinordnung test-100. Für die test-100-Suite ergibt sich damit ein klarer Trade-off: **qwen3:32b** liefert den höchsten Recall (63 %) und die stärkste Empfehlungsqualität, ist aber mit deutlichem Laufzeitnachteil verbunden; **qwen2.5-coder:14b** liegt bei der Erkennungsleistung darunter (53 %), bleibt jedoch als pragmatischer Kompromiss aus Qualität und Aufwand relevant. Für die operative Interpretation sind deshalb zwei Lesarten notwendig: *strenge Sicht* (NFA-02 als Ausschlusskriterium) versus *pragmatische Sicht* (Qualität/Zeit-Balance). Diese Trennung wird im Anhang explizit ausgewiesen (Tabellen 58, 21 und 20).

Zur Einordnung der in Anhang 51 ausgewiesenen Überdetektions-Werte ist zu beachten, dass die dort ausgewiesene Quote den LLM-Detektor-Rohoutput (Ø 94–106 Findings pro Run) mit der Ground-Truth-Zahl von 100 vergleicht – nicht den finalen Report nach Zusammenfassung (Ø 72 Einträge in einem vollständigen Run). Die scheinbar hohe Nähe zur Ground-Truth-Zahl darf daher nicht als Aussage über die Treffergenauigkeit des Endberichts interpretiert werden. Für eine vollständige Präzisionsbewertung wäre eine manuelle Plausibilisierung aller Report-Einträge gegen die Ground Truth erforderlich; die Stichprobenprüfung in Anhang 52 gibt einen empirischen Eindruck der tatsächlichen Fehlerrate.

8.7.3 Evaluation am BWEC-Untersuchungsobjekt

Die Evaluation am realen BWEC der BITBW – einer produktiv betriebenen React-SPA mit unbekannter Ground Truth – wurde nach Abschluss der Benchmarkläufe auf den synthetischen Suiten durchgeführt. Da keine bekannte Ground Truth vorliegt, ist ein quantitativer Recall-Vergleich nicht möglich; die Evaluation beschränkt sich auf die Analyse der Systemstabilität, der Skalierungseffekte und der qualitativen Plausibilität der generierten Befunde. Der fehlende Ground-Truth-Vergleich stellt die primäre externe Validitätsgrenze dieser Stufe dar.

Konfiguration. Es wurden zwei Konfigurationen mit je 9 Runs (3 Modelle × 3 Runs) durchgeführt: *bwec* mit `maxfiles=100` (66 LLM-Detektor-Chunks) und *bwec-500* mit `maxfiles=500` (330 Chunks). Letztere schließt alle verfügbaren Frontend-Dateien ein und dient als Skalierungsexperiment. Der Kontext wurde auf `OLLAMA_NUM_CTX = 65536` und das Output-Budget auf `OLLAMA_NUM_PREDICT = 24576` gesetzt; der LLM-Detektor verwendete `LLM_DETECT_NUM_PREDICT = 4096` und eine Chunk-Größe von 4000 Zeichen. Alle Rohdaten finden sich in den Anhang-Tabellen 70 und 72.

Tool-Baseline. Die deterministischen Quellen liefern über alle 18 Runs hinweg konstant: **axe-core** 10, **Playwright** 2, **grep** 8 Findings (Summe: 20). Die Stabilität ist unabhängig von der Modellwahl und bestätigt die deterministische Eigenschaft dieser Detektionsquellen auch am realen Untersuchungsobjekt. Für einen direkten Vergleich mit einer reinen axe-only-Konfiguration steht ein dedizierter Referenz-Run noch aus; dieser wurde im Projektzeitraum nicht mehr durchgeführt und bleibt als Folgeschritt vorgesehen.

Ergebnisse bei maxfiles = 100 (66 Chunks). Mit der für den produktiven Einsatz empfohlenen Konfiguration verhält sich das System stabil. Tabelle 71 fasst die Metriken zusammen. Der LLM-Detektor liefert: **qwen2.5-coder:7b** Ø 14,5 Findings ($\sigma \approx 0,7$; 2 stabile Runs²²⁹), **qwen2.5-coder:14b** Ø 22,0 Findings ($\sigma = 0$; perfekte Konsistenz über alle 3 Runs), **qwen3:32b** Ø 32,0 Findings ($\sigma \approx 2,6$). Kein stabiler Run überschreitet das Output-Budget; alle Reports sind vollständig und parsebar. Die Laufzeiten betragen Ø 328 s (7b), Ø 824 s (14b) und Ø 2 129 s (32b). Damit erfüllt **qwen2.5-coder:7b** NFA-02; 14b und 32b verletzen die Laufzeitgrenze analog zum test-12-Befund. Die Priorisierungsstruktur spiegelt das aus den Benchmarkläufen bekannte Muster: **qwen3:32b** liefert die ausgewogenste Trennung (Ø 37,3 Must-have, 14,7 Nice-to-have), **qwen2.5-coder:14b** klassifiziert nahezu alles als Must-have (Ø 40,3 Must, 1,7 Nice).

Skalierungsexperiment: maxfiles = 500 (330 Chunks). Das Experiment mit dem vollständigen Frontend-Stand (über 800 Quelldateien, davon 500 analysiert) deckt bewusst einen Grenzfall außerhalb des vorgesehenen Einsatzkontexts auf. Die Tool-Findings bleiben identisch (axe=10, Playwright=2, grep=8); der LLM-Detektor erzeugt bei dieser Codebase-Größe hingegen massiv mehr Findings: Ø 257 (7b), Ø 271 (14b), Ø 415 (32b) – ein 15- bis 17-facher Anstieg gegenüber **maxfiles = 100**. Damit wird der Priorisierungs-Prompt bei **qwen3:32b** auf exakt 40 960 Tokens aufgeblasen (= das gesamte verfügbare Eingabefenster), der Kontext wird abgeschnitten und kein Run liefert einen parsebaren Report (Must=0, Nice=0). **qwen2.5-coder:14b** erschöpft in 2/3 Runs das Output-Budget; **qwen2.5-coder:7b** arbeitet partiell. Die Laufzeiten (Ø 2 204 s bis 14 884 s) sind operativ nicht tragbar (vgl. Tabelle 72).

Dieses Ergebnis ist jedoch kein Systemfehler, sondern ein **Use-Case-Mismatch**: ACE ist nicht dafür konzipiert, eine bereits gewachsene Produktivapplikation nachträglich vollständig zu auditieren. Das eigentliche Einsatzszenario ist die **Begleitung des Entwicklungsprozesses** – konkret die Integration in den agilen Sprint-Rhythmus des BWEC (zweiwöchige Releases). Im Sprint-Kontext werden pro Iteration nur wenige neue Komponenten und Änderungen eingebracht; die Anzahl der Findings bleibt entsprechend gering und wächst nicht mit der Gesamtgröße der Codebasis. Zudem sinkt die Findings-Zahl mit jeder behobenen Violation: Je konsequenter Barrierefreiheit im laufenden Entwicklungsprozess adressiert wird, desto kürzer werden die Prompts und desto effizienter arbeitet das System. Die bwec-500-Messreihe demonstriert damit den Grenzfall einer vollständigen Nachprüfung – ein Szenario, das bei regulärem Sprint-Einsatz nicht eintreten würde.

Qualitative Plausibilitätsprüfung. Die im bwec-Lauf (maxfiles=100) generierten Findings mit **qwen3:32b** referenzieren ausschließlich real im BWEC-Quellcode vorhandene Dateien und Komponenten: u. a. `navigationSection.tsx`, `leikaGroupTable.tsx`, `mappingTable.tsx`, `privacyCategoryFields.component.tsx` und `tenant.view.tsx`. Die benannten Violations

²²⁹Run-03 von **qwen2.5-coder:7b** ist ein Ausreißer: Das Modell erzeugte 205 Nice-to-have-Einträge und schöpfte das Output-Budget (24 576 Tokens) vollständig aus. Dieser Run wurde aus den Mittelwertberechnungen ausgeschlossen.

(fehlende `aria-label`-Attribute an Icon-Buttons, `div/span` mit `onClick` ohne Keyboard-Handler, fehlende Labels an `AutocompleteField`-Komponenten) entsprechen bekannten React-Accessibility-Anti-Patterns und sind inhaltlich plausibel. Eine formale Bestätigung durch einen Accessibility-Experten steht aus; die Dateibezüge und Fix-Vorschläge (jeweils mit konkreten Zeilennummern) legen eine Bewertung auf Stufe 2 nahe, analog zu den Befunden auf den synthetischen Suiten.

8.8 Beantwortung der Forschungsfragen

Auf Basis der in den Abschnitten 8.2–8.6 dokumentierten Ergebnisse können die in Kapitel 1.3 formulierten Forschungsfragen nun beantwortet werden.

8.8.1 Übergeordnete Forschungsfrage

Die übergeordnete Forschungsfrage kann **überwiegend positiv** beantwortet werden: Das kontextsensitive Assistenzsystem erkennt einen großen Teil relevanter Barrierefreiheitsverstöße automatisiert und erzeugt entwicklernahe Empfehlungen mit konkreten Datei-/Zeilenbezügen. Die Leistungsfähigkeit hängt jedoch deutlich vom gewählten Modell und von Token-Limits ab, sodass die Systemleistung nicht als modellunabhängig verstanden werden darf.

8.8.2 TF1: Erkennungsleistung nach Verstoßtyp

Ja – mit modellabhängigen Einschränkungen. Mit `qwen2.5-coder:14b` und `qwen3:32b` erkennt die vollständige Pipeline alle drei Verstoßkategorien vollständig: syntaktische Violations zu 100 %, layout-bezogene zu 100 % und semantische zu 100 % (vgl. Tabelle 12). Die größte methodische Herausforderung liegt im semantischen Bereich: Hier ist der Recall in der Baseline am niedrigsten (60 %), und der LLM-Detektor leistet den größten Einzelbeitrag (+40 Prozentpunkte). Bei `qwen2.5-coder:7b` bestehen modellspezifische Grenzen: V2 und V11 werden nie erkannt.

Im Vergleich zur Literatur erzielen regelbasierte Tool-Ansätze wie Testaro (Pool) oder axe-only Konfigurationen typischerweise Abdeckungsraten im Bereich von 55–70 % der geprüften Issue-Kategorien.²³⁰ Diese Werte beziehen sich auf die Tool-Abdeckung bekannter Kategorien, nicht auf Recall im Sinne einer vollständigen Ground-Truth-Auswertung. Fathallah et al. (AccessGuru) erreichen mit cloud-basierten ReAct-Schleifen einen Violation Score Decrease von 0,84 auf einer Stichprobe realer Webseiten.²³¹ Der ACE-Ansatz zeigt über alle drei kontrollierten Testsuiten konsistent einen Mehrwert durch die Kontextkombination: auf test-12 steigt der Recall von 66,7 % (Baseline) auf bis zu 100 % (14b/32b), auf test-50 von 43/50 (Baseline) auf 48/50 (14b) bzw. 50/50 (32b) und auf test-100 von 34/100 (Baseline) auf ca. 53/100 (14b) bzw. ca. 63/100

²³⁰Vgl. Kodithuwakku, Wickramarachchi 2025

²³¹Vgl. Fathallah, Hernández, Staab 2025

(32b). Ein direkter Benchmark gegen die genannten Literaturwerte bleibt dennoch methodisch eingeschränkt, da Datensätze, Ground-Truth-Konstruktion, Metriken und Laufzeitumgebungen nicht identisch sind; die Gegenüberstellung ist daher als Größenordnungs- und Trendvergleich zu verstehen.

8.8.3 TF2: Mehrwert der Kontextkombination

Ja, der Mehrwert ist messbar und qualitativ bedeutsam. Die Tool-Pipeline (axe + Playwright + grep) erreicht einen Recall von 66,7 % (8/12). Durch den LLM-Detektor steigt der Recall auf bis zu 100 % (+4 Violations). Dabei handelt es sich ausschließlich um semantische oder kontextabhängige Violations, die durch keine der regelbasierten Quellen zuverlässig erreichbar sind: V8 (modaler Fokus-Trap), V11 (DOM-/visuelle Reihenfolge) und V12 (Mindestgröße interaktiver Elemente) sind LLM-exklusive Funde. V2 (fehlendes `aria-label` auf Icon-Button) wird ebenfalls erst durch die LLM-Analyse konsistent erfasst.

Zusätzlich verbessert die Kontextkombination die Umsetzbarkeit der Empfehlungen: Tool-Befunde werden mit Quellcodekontext verbunden, wodurch konkrete, entwicklungsnahe Fix-Vorschläge entstehen (Stufe 2 für 11 von 12 Violations mit 32b). Gegenüber einer reinen Tool/DOM-Sicht steigt damit nicht nur die Erkennungsbreite, sondern auch die praktische Anschlussfähigkeit – ein Befund, der die in Kapitel 3.5 beschriebene Lücke zwischen DOM-Analyse und Quellcodekontext direkt adressiert. Die konkrete Instantiierung dieser Violations ist in Anhang 10 und 11 für die Nachvollziehbarkeit visualisiert.

8.9 Modellwahl und Einsatzempfehlungen

Aus den vorstehenden Ergebnissen zu Recall, Priorisierungsqualität, Fix-Qualität und Laufzeit lassen sich folgende Einsatzempfehlungen je Anwendungskontext ableiten. Eine suiteübergreifende Einordnung findet sich ergänzend in den Anhang-Tabellen 20 und 21.

Kriterium	Empfehlung
Produktion (BITV-konform)	<code>qwen2.5-coder:14b</code> (22,8 ± 0,42 Findings; 420 s)
CI/CD (schnell, nicht compliance-relevant)	<code>qwen2.5-coder:7b</code> (11,1 ± 3,28 Findings; 231 s)
Forschung / Referenzläufe	<code>qwen3:32b</code> (19,5 ± 0,53 Findings; 701 s)

Tab. 17: Modellwahl nach Anwendungsfall

`qwen2.5-coder:14b` stellt den besten Trade-off für den regulären Einsatz unter Kalibrierungsbedingungen dar. Das Modell erreicht im Mittel 22,8 LLM-Findings bei sehr geringer Streuung ($\sigma = 0,42$), erzielt einen Recall von 100 % auf der test-12-Suite und bleibt mit einer Gesamtlaufzeit von 420 s deutlich innerhalb der formulierten Laufzeitgrenze (vgl. Tabelle 31). Bei größeren Anwendungen mit vielen Violations ist jedoch Vorsicht geboten: Auf test-100 schöpft 14b in allen 10 Runs das Output-Limit vollständig aus (100 % Truncation) und erreicht nur 53 % Recall –

ein klarer Hinweis darauf, dass das Token-Budget für produktive Anwendungen mit realistischer Violation-Zahl nicht ausreicht. Für compliance-relevante Einsätze mit mehr als ~ 20 Violations ist `qwen3:32b` vorzuziehen.

`qwen3:32b` bleibt das methodisch stärkste Referenzmodell, wenn maximale Stabilität und vollständige semantische Abdeckung im Vordergrund stehen. Der Zugewinn gegenüber 14b fällt jedoch im Verhältnis zur deutlich höheren Laufzeit begrenzt aus – beide Modelle erzielen auf test-12 identischen Recall von 100 %. Für regelmäßige Entwicklungszyklen ist 32b daher eher als Referenz- oder Validierungsmodell zu verstehen als als wirtschaftliche Standardeinstellung.

`qwen2.5-coder:7b` eignet sich vor allem für schnelle Vorprüfungen in CI-/CD-nahen Szenarien. Die Laufzeit ist mit 231 s am niedrigsten, zugleich bleibt die Detektionsleistung deutlich hinter 14b und 32b zurück, und die Priorisierung ist für compliance-relevante Einsätze nicht hinreichend verlässlich. Für rechtlich oder fachlich verbindliche Accessibility-Bewertungen sollte 7b deshalb nicht als Primärmodell eingesetzt werden.

9 Fazit

Dieses abschließende Kapitel fasst die Ergebnisse der vorliegenden Arbeit zusammen (Abschnitt 9.1), würdigt sie kritisch im Hinblick auf methodische und externe Einschränkungen (Abschnitt 9.2) und zeigt Richtungen für künftige Weiterentwicklungen auf (Abschnitt 9.3).

9.1 Zusammenfassung der Ergebnisse

9.1.1 Beitrag zur übergeordneten Forschungsfrage

Ziel der Arbeit war die Entwicklung und Evaluation eines lokal ausführbaren, LLM-gestützten Assistenzsystems (ACE), das axe-core-Violation-Reports, Playwright-Interaktionsdaten und statische Quellcodeanalyse kombiniert, um entwicklernahe Handlungsempfehlungen für React-basierte Webanwendungen zu generieren. Das System wurde nach dem DSR-Paradigma konzipiert: Problemidentifikation, Artefaktdesign, Prototypimplementierung und formale Evaluation bilden den methodischen Bogen der Arbeit.

Die Evaluation umfasst 90 Benchmarkläufe auf drei synthetischen Testsuiten (3 Modelle $\times$ 10 Runs je Suite). Die test-12-Suite dient dabei als methodische **Kalibrierungsphase**: Bei vollständig bekannter Ground Truth zeigt sie, dass die Pipeline korrekt funktioniert und alle Verstoßkategorien grundsätzlich erkennbar sind. Die test-100-Suite bildet die eigentliche **Belastungsprüfung** und liefert die für die Forschungsfragen maßgeblichen Ergebnisse.

TF1 konnte bestätigt werden: Auf test-100 erkennt **qwen3:32b** 63 / 100 Violations (63 %), gegenüber 34 / 100 (34 %) in der Tool-only-Baseline – ein Recall-Zuwachs von 29 Violations, der sich über alle drei Verstoßkategorien verteilt. Auf test-12 werden mit **qwen3:32b** alle 12 Violations erkannt (100 %), was die prinzipielle Korrektheit des Ansatzes belegt, jedoch aufgrund der kontrollierten und einfachen Testbedingungen nicht auf reale Anwendungen verallgemeinert werden kann. **TF2** konnte ebenfalls bestätigt werden: Die Kontextkombination (axe-core, Playwright, Quellcode) steigert die Fix-Qualität auf Stufe 2 für 11 von 12 Violations auf test-12 mit **qwen3:32b** – ein Niveau, das mit reiner Tool-DOM-Analyse strukturell nicht erreichbar ist, da fehlender Quellcodekontext die Generierung datei- und zeilenspezifischer Empfehlungen ausschließt. Die zugrunde liegenden Detailwerte sind im Anhang in den Tabellen 29, 31, 56 und 36 dokumentiert.

9.1.2 Erfüllung der funktionalen und nicht-funktionalen Anforderungen

Die fünf funktionalen Anforderungen FA-01 bis FA-05 sind vollständig umgesetzt. Axe-core liefert regelbasierte Detektion (FA-01), Playwright erfasst zustandsabhängige Barrieren (FA-02), grep-Pattern analysieren den Quellcode (FA-03), der LLM-Detektor und das Priorisierungsmodell

generieren kontextsensitive Empfehlungen (FA-04), und der strukturierte JSON-Report stellt maschinenlesbare Ausgaben bereit (FA-05).

Bei den nicht-funktionalen Anforderungen ist das Bild differenzierter. NFA-01 (Lokal-First) wird durch die Ollama-Infrastruktur vollständig erfüllt: Kein Quellcode und kein DOM-Inhalt verlässt das lokale System. NFA-02 (Laufzeit < 10 Min.) erfüllen **qwen2.5-coder:7b** und **qwen2.5-coder:14b** unter Kalibrierungsbedingungen (test-12) komfortabel; **qwen3:32b** gilt dort als bedingt konform (vgl. Tabelle 14). Unter Belastung (test-100) verletzen alle drei Modelle NFA-02 klar (Gesamtlaufzeit $\bar{O}$ 765–2616 s), was den operativen Einsatz auf Anwendungsebene mit vielen Violations einschränkt. Entsprechend sollten Modellempfehlungen stets im Spannungsfeld aus strenger NFA-Sicht und pragmatischer Qualitäts-/Zeit-Bewertung gelesen werden (vgl. Tabelle 17 sowie die Anhang-Tabellen 21 und 58). NFA-03 (Wartbarkeit) wird durch die modulare Trennung in Detection, Context-Building und Synthesis sichergestellt. NFA-04 (Reproduzierbarkeit) wird durch Temperature 0,05 und JSON-Schema-Prompting unterstützt; **qwen2.5-coder:14b** zeigt auf test-12 mit $\sigma = 0,42$ die engste Streuung, **qwen3:32b** erzielt auf test-100 mit $\sigma = 5,63$ moderate Konsistenz (vgl. Abschnitt 8.5.2). Eine tabellarische Gesamtübersicht findet sich in Tabelle 23 im Anhang.

9.2 Kritische Reflexion

9.2.1 Grenzen der externen Validität

Die Evaluation stützt sich auf drei synthetisch konstruierte Testsuiten (test-12 als Kalibrierung, test-50 als Zwischenstufe, test-100 als Belastungsprüfung). Obwohl alle drei Verstößkategorien und unterschiedliche Skalierungsstufen abgedeckt sind, bleiben synthetische Violations per Konstruktion einfacher strukturiert als Verstöße in realen Codekontexten. Der auf test-100 mit **qwen3:32b** erzielte Recall von 63 % ist zwar ein belastbarer Wert als die 100 % auf test-12, kann jedoch ebenfalls nicht direkt auf produktive Anwendungen mit unkontrollierter Ground Truth übertragen werden (vgl. Abschnitt 8.6.1). Die am BWEC durchgeführten Messläufe (vgl. Abschnitt 8.7.3) bestätigen die Systemstabilität auf einem realen Untersuchungsobjekt: Die deterministischen Quellen liefern stabile Befunde; der LLM-Detektor identifiziert plausible, datei- und zeilenspezifische Findings. Ein quantitativer Recall-Vergleich bleibt mangels bekannter Ground Truth offen. Zudem wurden ausschließlich CPU-basierte Laufzeitmessungen erhoben; auf GPU-beschleunigter Hardware wäre NFA-02 auch für größere Testsuiten und leistungsstärkere Modelle erfüllbar.

9.2.2 Methodische Einschränkungen

Das Token-Limit beeinflusst die Priorisierungsqualität systematisch und modellabhängig (vgl. Abschnitt 8.6.2): Auf test-100 schöpft **qwen2.5-coder:14b** in allen 10 Runs das Output-Limit

von 12288 Tokens vollständig aus; **qwen2.5-coder:7b** in 80 % der Runs (vgl. Tabelle 59). Eine weitere Erhöhung von **num_predict** würde die Abschneiderate senken, jedoch die Laufzeit proportional verlängern und NFA-02 weiter belasten. **qwen3:32b** liefert auf test-100 die zuverlässigste Datenbasis (nur 1 / 10 Runs trunkiert; vgl. Tabelle 59). Ein weiteres Risiko liegt in der Operationalisierung der Erkennungsmetrik über Report-Präsenz: Semantisch äquivalente, aber unterschiedlich formulierte Funde können zu konservativen Zuordnungen führen.

Die in den Überdetektions-Tabellen ausgewiesene Quote (Anhang 51) misst ausschließlich das Verhältnis zwischen LLM-Detektor-Rohoutput und Ground-Truth-Zahl; sie trifft keine Aussage über die inhaltliche Trefferqualität des finalen Reports. Eine vollständige Precision-Evaluation wurde im Rahmen dieser Arbeit nicht für alle 30 Runs durchgeführt. Eine Stichprobenprüfung an Run 01 (**qwen3:32b**, test-100) zeigt jedoch, dass echte Halluzinationen selten sind: Von 72 Report-Einträgen konnten 64 eindeutig zugeordnet werden (51 True Positives, 10 Quell-Duplikate, 2 Mehrfach-Instanzen, 1 gebündeltes Finding); 8 Einträge sind nicht eindeutig einer Ground-Truth-Violation zuzuordnen; 0 bestätigte Halluzinationen in Run-01. 1 Halluzination wurde in Run 03 identifiziert (vgl. Anhang 52). Die wesentlichen Über-Detektions-Effekte sind damit nicht inhaltliche Fehler, sondern Quell-Duplikate (axe-core und LLM-Detektor erkennen dieselbe Violation unabhängig) sowie Mehrfachinstanzen derselben Verletzungsklasse im DOM. Für den produktiven Einsatz bleibt dennoch eine manuelle Plausibilisierung des generierten Reports erforderlich, um Duplikate zu konsolidieren und vereinzelte Halluzinationen zu filtern.

Aus demselben Grund wurde die vertiefende Fehlerursachenanalyse per Pareto-Diagramm auf test-100 mit **qwen3:32b** konzentriert (vgl. Abbildung 16 im Anhang): Die niedrigste Truncation innerhalb der test-100-Messreihe reduziert methodische Verzerrungen gegenüber den alternativen Modellkonfigurationen.

9.3 Ausblick

9.3.1 Kurzfristige Erweiterungen

Als unmittelbare nächste Schritte sind drei Maßnahmen priorisiert. Erstens steht die formale Verifikation der am BWEC generierten Befunde durch einen Accessibility-Experten aus: Die Messläufe wurden abgeschlossen (vgl. Abschnitt 8.7.3), jedoch fehlt noch ein dedizierter axe-only-Referenz-Run als direkte Baseline sowie eine manuelle Plausibilisierung der LLM-Findings gegen den tatsächlichen Quellcode. Die auf den synthetischen Suiten erzielten Recall-Werte – insbesondere 63 / 100 mit **qwen3:32b** auf test-100 (vgl. Tabelle 56) – sowie die plausiblen Befunde am BWEC legen nahe, dass auch in realen Anwendungen ein substanzieller Mehrwert gegenüber reiner Tool-Detektion erreichbar ist. Zweitens sollte der Playwright-Satz um einen dedizierten Fokus-Trap-Test erweitert werden, sodass modale Dialoge ohne Fokus-Trap (V8 in test-12, V47/V78 in test-100) zuverlässig auch ohne LLM-Detektor erfasst werden. Drittens ist eine strukturierte Plausibilisierung des generierten Reports als fester Bestandteil des ACE-Workflows zu

empfehlen: Eine manuelle Stichprobenprüfung von ca. 10–20 % der Report-Einträge – insbesondere jener Findings, die ausschließlich in einzelnen Runs auftreten – würde erlauben, den Anteil echter Halluzinationen empirisch zu quantifizieren und den Workflow für produktive Compliance-Prozesse zu kalibrieren. Die in Anhang 52 dokumentierte Stichprobenanalyse liefert hierfür ein methodisches Vorbild (Run-01: 72 Report-Einträge, davon 64 eindeutig zugeordnet, 8 nicht eindeutig, 0 bestätigte Halluzinationen; Run-03: 1 Halluzination).

9.3.2 Mittelfristige Weiterentwicklungen

Mittelfristig bieten sich vier Ansatzpunkte für eine Vertiefung des Systems. Erstens könnte die `axe-core`-Prüfung pro erfasstem DOM-Zustand kontinuierlich ausgeführt werden, anstatt nur nach definierten Playwright-Interaktionsschritten. Damit ließen sich Barrieren erfassen, die durch nicht vordefinierte Zustandsübergänge entstehen – ein Ansatz, der jedoch die Laufzeit erheblich erhöht und eine Priorisierung der zu prüfenden Zustände erfordert.

Zweitens könnten leistungsfähigere Modelle – etwa `qwen3:72b`²³² oder `deepseek-r1:70b` – evaluiert werden, sofern Hardware mit ausreichend Arbeitsspeicher und GPU-Kapazität verfügbar ist. Die Ergebnisse dieser Arbeit legen nahe, dass die Modellgröße einen starken positiven Effekt auf Recall, Priorisierungsqualität und Fix-Detailtiefe hat; ein 70B-Modell könnte insbesondere die instabile V11-Erkennung stabilisieren.

Drittens könnte ein eingeschränkter Einsatz cloudbasierter Modelle (z. B. datenschutzkonforme On-Premises-Deployments oder Modelle auf behördeneigener Serverinfrastruktur) geprüft werden. Sofern keine personenbezogenen Daten im Code vorliegen und eine datenschutzrechtliche Freigabe besteht, könnten Cloud-Modelle erhebliche Laufzeitvorteile bieten, ohne NFA-01 zu verletzen.

Viertens wäre der strukturierte JSON-Report des ACE-Systems als Eingabekontext für nachgelagerte LLM-Agenten nutzbar, die Fixes automatisiert in den Quellcode einarbeiten – ein Einsatzszenario, das durch das maschinenlesbare Output-Format (FA-05) bereits direkt vorbereitet ist.

9.3.3 Langfristige Forschungsperspektiven

Langfristig bietet sich die Möglichkeit, das System durch domänenspezifisches Fine-Tuning weiterzuentwickeln. Ein auf BITV-konformen React-Codebeispielen feinabgestimmtes Modell könnte systematische Fehlklassifikationen wie die inverse Priorisierung von `qwen2.5-coder:7b` strukturell beseitigen, anstatt sie durch größere Modelle zu kompensieren. Fine-Tuning würde zudem

²³²Qwen3 ist eine Modellfamilie von Alibaba Cloud; `qwen3:72b` bezeichnet die 72-Milliarden-Parameter-Variante der dritten Generation. Eine formale Evaluation im Rahmen dieser Arbeit war aufgrund der RAM-Anforderungen nicht möglich.

erlauben, institutionsspezifische Konventionen der BITBW direkt in die Empfehlungen einzubetten, ohne das Prompt-Design ausweiten zu müssen.

Darüber hinaus wäre eine Integration des ACE-Reports als Schritt in bestehende CI/CD-Pipelines eine konsequente Weiterentwicklung: Barrierefreiheitsprüfungen würden damit automatisch bei jedem Merge-Request ausgeführt, anstatt als nachgelagerter Audit-Prozess zu verbleiben. Gerade für agil arbeitende Teams – wie das BWEC-Entwicklungsteam mit zweiwöchigen Release-Zyklen – ist dieser Ansatz besonders geeignet: Im Sprint-Kontext analysiert ACE nur die neu hinzugefügten oder veränderten Komponenten, wodurch die Findings-Zahl und damit der Token-Bedarf strukturell begrenzt bleiben. Mit jeder behobenen Violation verbessert sich zudem die Systemeffizienz, da kürzere Prompts kürzere Laufzeiten bedeuten – ein selbstverstärkender Effekt, der ACE vom begleitenden Werkzeug zum integralen Bestandteil des Entwicklungsprozesses machen kann. In Verbindung mit automatisierten Fix-Vorschlägen und einem kuratierten Review entstünde ein Continuous-Accessibility-Loop, der den manuellen Auditaufwand langfristig substanziell reduzieren könnte.

Anhang

Anhangverzeichnis

Anhang 1	Übersicht der Anhangsstruktur	77
Anhang 2	Komplexität von Home Pages	78
Anhang 3	Erweiterte Konzeptmatrix	79
Anhang 4	Schematische Übersicht der ACE-Pipeline	81
Anhang 5	System-Prompt des ACE-Assistenzsystems	82
Anhang 6	Prompts der LLM-Detektor-Phase	84
Anhang 7	Aufteilung des Token-Budgets	86
Anhang 8	Empfehlung nach Anwendungsszenario (konsolidiert)	87
Anhang 9	Vergleich über alle Suites	88
Anhang 10	Bewertungsskala für die Empfehlungsqualität der LLM-Ausgaben (generell)	89
Anhang 11	Erfüllungsgrad der funktionalen und nicht-funktionalen Anforderungen (generell)	90
Anhang 12	Reproduzierbarkeitsblatt	91
Anhang 13	Console-Output-Auszüge eines Benchmark-Runs	93
Anhang 14	Threats to Validity	94
Anhang 15	test-12-Suite (kompletter Block)	96
Anhang 16	Accessibility-Verstöße der Testanwendung und zugehörige Prüfquellen (test-12-Suite)	97
Anhang 17	Erkennungsrate der Violations im Proof of Concept (test-12-Suite)	98
Anhang 18	Recall-Matrix: Violation $\times$ Bedingung (test-12-Suite)	99
Anhang 19	Rohdaten: alle 30 Benchmark-Runs (test-12-Suite)	100
Anhang 20	Modell-Leistungsvergleich: test-12-Suite Zusammenfassung	101
Anhang 21	Token-Nutzung pro Modell: Input/Output (test-12-Suite)	102
Anhang 22	Effizienzmetriken und Kosten-Nutzen-Verhältnis (test-12-Suite)	102
Anhang 23	Über-Detektion relativ zur Ground Truth (test-12-Suite)	103
Anhang 24	Detektor-Konsistenz über alle 30 Runs (test-12-Suite)	103
Anhang 25	Bewertung der Empfehlungsqualität pro Violation (test-12-Suite)	104
Anhang 26	CSV-Auszug: Benchmark-Rohdaten (test-12-Suite)	105
Anhang 27	Statistische Verteilung der LLM-Findings (alle 30 Runs, test-12-Suite)	106
Anhang 28	Screenshots der Testanwendung (test-12-Suite)	107
Anhang 29	test-50-Suite (kompletter Block)	109
Anhang 30	Accessibility-Verstöße der Testanwendung (test-50-Suite)	110
Anhang 31	Erkennungsrate der Violations im Proof of Concept (test-50-Suite)	113
Anhang 32	Recall-Matrix: Violation $\times$ Bedingung (test-50-Suite)	116
Anhang 33	Rohdaten: alle 30 Benchmark-Runs (test-50-Suite)	119
Anhang 34	Modell-Leistungsvergleich: test-50-Suite Zusammenfassung	120

Anhang 35	Token-Nutzung pro Modell: Input/Output (test-50-Suite)	121
Anhang 36	Effizienzmetriken und Kosten-Nutzen-Verhältnis (test-50-Suite)	121
Anhang 37	Über-Detektion relativ zur Ground Truth (test-50-Suite)	122
Anhang 38	Detektor-Konsistenz über alle 30 Runs (test-50-Suite)	122
Anhang 39	Bewertung der Empfehlungsqualität pro Violation (test-50-Suite)	123
Anhang 40	CSV-Auszug: Benchmark-Rohdaten (test-50-Suite)	126
Anhang 41	Statistische Verteilung der LLM-Findings (alle 30 Runs, test-50-Suite)	127
Anhang 42	Screenshots der Testanwendung (test-50-Suite)	128
Anhang 43	test-100-Suite (kompletter Block)	131
Anhang 44	Accessibility-Verstöße der Testanwendung (test-100-Suite)	132
Anhang 45	Erkennungsrate der Violations im Proof of Concept (test-100-Suite)	136
Anhang 46	Recall-Matrix: Violation $\times$ Bedingung (test-100-Suite)	141
Anhang 47	Rohdaten: alle 30 Benchmark-Runs (test-100-Suite)	146
Anhang 48	Modell-Leistungsvergleich: test-100-Suite Zusammenfassung	147
Anhang 49	Token-Nutzung pro Modell: Input/Output (test-100-Suite)	148
Anhang 50	Effizienzmetriken (test-100-Suite)	148
Anhang 51	Über-Detektion relativ zur Ground Truth (test-100-Suite)	149
Anhang 52	Stichproben-Plausibilisierung des finalen Reports (qwen3:32b, Run 01 & Run 03, test-100)	150
Anhang 53	Detektor-Konsistenz über alle 30 Runs (test-100-Suite)	151
Anhang 54	Bewertung der Empfehlungsqualität pro Violation (test-100-Suite)	152
Anhang 55	CSV-Auszug: Benchmark-Rohdaten (test-100-Suite)	158
Anhang 56	Statistische Verteilung der LLM-Findings (alle 10 Runs je Modell, test-100- Suite)	158
Anhang 57	Pareto-Diagramm: Nicht erkannte Violations (test-100, qwen3:32b)	159
Anhang 58	Boxplot der Laufzeiten (test-100, 10 Runs je Modell)	160
Anhang 59	Pipeline-Diagramm der Findings-Verarbeitung	161
Anhang 60	Screenshots der Testanwendung (test-100-Suite)	162
Anhang 61	Lesehinweis zu den Vergleichsgrafiken	165
Anhang 62	Recall-Degradation über alle Testsuites	165
Anhang 63	Gesamtlaufzeit-Skalierung über alle Testsuites	166
Anhang 64	LLM-Konsistenz (σ) im Suite-Vergleich	167
Anhang 65	Recall und Überschuss – nach Suite gruppiert	168
Anhang 66	Truncation-Rate im Suite-Vergleich	169
Anhang 67	Heatmap der Recall-Werte	170
Anhang 68	Gestapeltes Balkendiagramm: Must-have vs. Nice-to-have	171
Anhang 69	BWEC-Block: Reales Untersuchungsobjekt	172
Anhang 70	Rohdaten: BWEC-Messreihe (<code>maxfiles</code> = 100, 66 Chunks)	172
Anhang 71	Modellvergleich: BWEC (<code>maxfiles</code> = 100)	173
Anhang 72	Rohdaten: BWEC-Messreihe (<code>maxfiles</code> = 500, 330 Chunks)	174

Anhang 1: Übersicht der Anhangsstruktur

Tab. 18: Übersicht der Anhangsstruktur mit Trennung zwischen generellen und suite-spezifischen Inhalten.

Bereich	Inhalte
Generell	Komplexität von Home Pages, Konzeptmatrix, Pipeline-Schema, System-Prompt, LLM-Detektor-Prompts, Token-Budget, konsolidierte Empfehlung, Suite-Vergleich, Bewertungsskala, FA/NFA-Erfüllungsgrad, Reproduzierbarkeitsblatt, Console-Output-Auszüge, Threats to Validity
Vergleichsgrafiken	Recall und Überschuss, Laufzeit-Skalierung, Konsistenz (σ), Recall / Überschuss (Suite-Gruppen), Truncation-Rate, Heatmap Recall, Must-/Nice-to-have-Verteilung
test-12-Block	Violations-Katalog, Erkennungsrate, Recall-Matrix, Rohdaten, Modellvergleich, Token-Nutzung, Effizienz, Über-Detektion, Konsistenz, Empfehlungsqualität, CSV-Auszug, Statistische Verteilung, Screenshots der Testsuite
test-50-Block	Violations-Katalog, Erkennungsrate, Recall-Matrix, Rohdaten, Modellvergleich, Token-Nutzung, Effizienz, Über-Detektion, Konsistenz, Empfehlungsqualität, CSV-Auszug, Statistische Verteilung, Screenshots der Testsuite
test-100-Block	Violations-Katalog, Erkennungsrate, Recall-Matrix, Rohdaten, Modellvergleich, Token-Nutzung, Effizienz, Über-Detektion, Stichproben-Plausibilisierung, Konsistenz, Empfehlungsqualität, CSV-Auszug, Statistische Verteilung, Pareto-Diagramm, Boxplot Laufzeiten, Pipeline-Diagramm, Screenshots der Testsuite
BWEC-Block	Rohdaten (maxfiles=100), Modellvergleich, Rohdaten (maxfiles=500)

Anhang 2: Komplexität von Home Pages

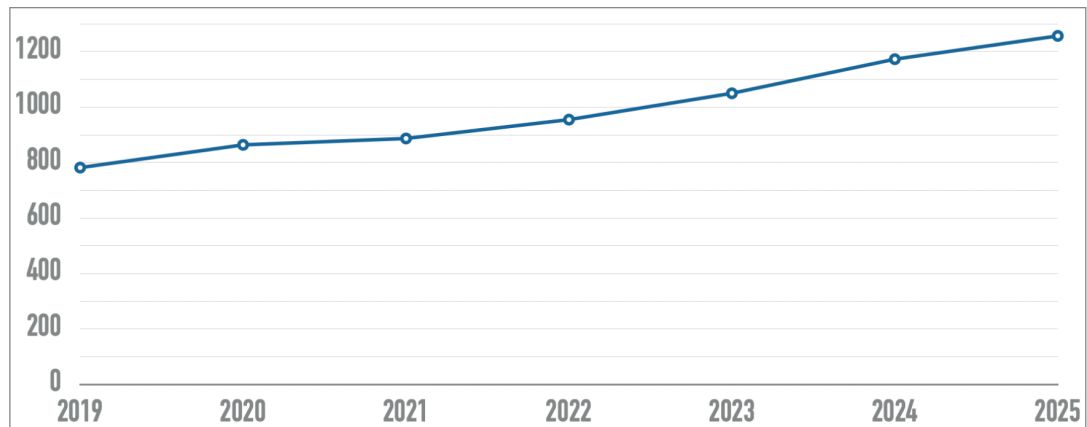


Abb. 7: Entwicklung der Komplexität von Home Pages in den letzten sechs Jahren. Die X-Achse zeigt die Jahre, die Y-Achse gibt die Anzahl der Elemente an.²³³

²³³Entnommen aus WebAIM 2025

Anhang 3: Erweiterte Konzeptmatrix

²³⁴Vgl. Webster, Watson 2002.

Studie		Konzepte													
Autor(en) & Jahr	System	Ziel			Datenquellen			KI-/LLM-Ansatz			Evaluation			Kontext	
		WCAG-Konf. prüfen	Barrieren korrigieren	Entwickler unterstützen	Tool-Reports (axe etc.)	Interaktionsdaten (Playwright)	Quellcode-Analyse	LLM-basiert	Kontextsensitiv / RAG	Lokal ausführbar	Automatisierte Eval.	Qualitative Eval.	Nutzerstudie	Rich-Client / SPA	Rechtl. Rahmen (BFSG etc.)
Hoch priorisierte Studien															
He et al. (2025)	GenA11y	x			x			x			x				
Fathallah et al. (2025)	AccessGuru	x	x	x	x	x		x			x				
Bassi et al. (2025)	LLM Auditing	x	x	x				x				x		x	
Paternò et al. (2025)	TeVal	x		x	x			x	x			x			
Tafreshipour et al. (2024)	Ma11y	x			x	x					x				
Fernández-Navarro & Chicano (2025)	LLM Remediation	x	x	x	x		x	x			x		x		
Mittel priorisierte Studien															
Yu et al. (2025)	GenAI Screen Reader	x	x		x		x	x	x		x	x	x		
Huang et al. (2024)	ACCESS	x	x	x	x	x		x			x				
Pool (2023)	Testaro	x			x	x					x				
Mowar et al. (2024)	Copilot Study	x		x				x					x		
Abu Doush & Kassem (2024)	AI Code Study	x	x	x				x			x				
Guriță & Vatavu (2025)	AI UI Study	x						x			x				
Kodithuwakku & Wick. (2025)	Tool Eval.	x			x						x				
Manca et al. (2023)	Transparency	x		x	x							x	x		
Jiang & Nam (2025)	Cursor Rules			x			x	x				x			
Gubbi Mohanbabu & Pavel (2024)	Context-Aware Img.	x	x					x	x		x	x	x		
Campoverde-Molina & Luján-Mora (2025)	AI A11y SMS	x	x	x	x			x				x			
Leedy et al. (2025)	GenAI Builder Eval.	x			x			x			x	x			
Patel & Melcer (2025)	AIDA Study	x		x				x				x	x		
Aljedaani et al. (2024)	ChatGPT A11y Code	x	x	x			x	x			x				
Niedrig priorisierte Studien															
Abou-Zahra et al. (2018)	AI for A11y	x						x				x		x	
Delnevo et al. (2024)	LLM Interaction	x	x					x				x		x	
Draffan et al. (2020)	Web2Access+AI	x			x			x				x			
Eigener Proof of Concept															
Martínez Campo (2026)	ACE (PoC)	x	x	x	x	x	x	x	x	x	x	x		x	

Tab. 19: Konzeptmatrix nach Webster Watson²³⁴: Vollständige Übersicht aller analysierten Studien im Vergleich zum eigenen Proof of Concept (PoC). **x** = Konzept vorhanden/adressiert; leer = nicht adressiert.

Anhang 4: Schematische Übersicht der ACE-Pipeline

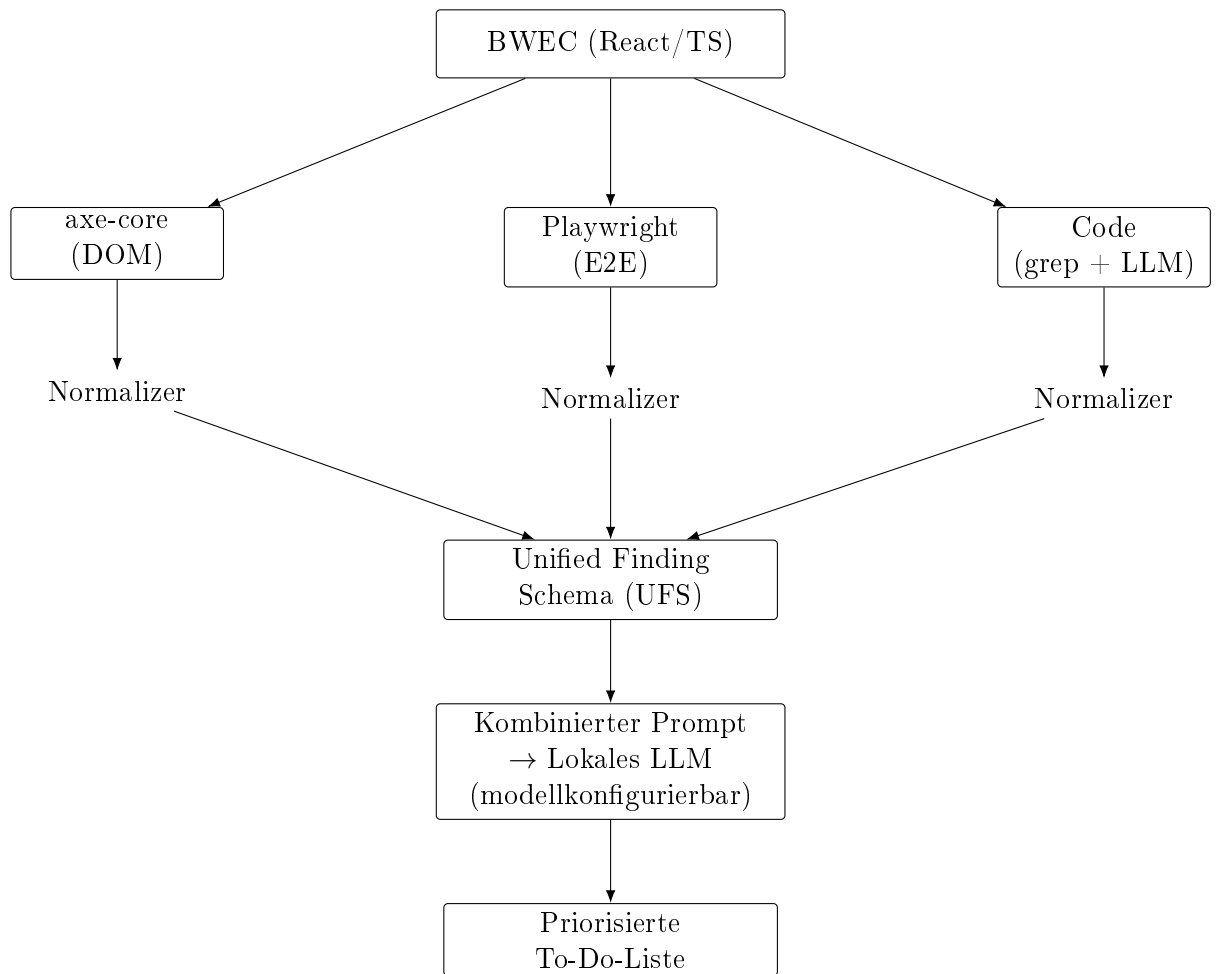


Abb. 8: Schematische Übersicht der im PoC implementierten ACE-Pipeline von der Datenerhebung bis zur priorisierten Ausgabe. Die Code-Schiene umfasst regex-basierte Pattern (grep) und die LLM-basierte Codeanalyse. Das verwendete LLM ist in der Pipeline bewusst abstrahiert; die konkrete Modellwahl erfolgt je Einsatzszenario (vgl. Tabelle 20).²³⁵

²³⁵Eigene Abbildung

Anhang 5: System-Prompt des ACE-Assistenzsystems

Der folgende System-Prompt wird dem Sprachmodell bei jedem Pipeline-Durchlauf übergeben. Er implementiert die in Abschnitt 6.2 beschriebenen Prompt-Engineering-Strategien Role-Play Prompting und Few-Shot Prompting (vgl. Abschnitt 7.6).

Listing 9.1: Vollständiger System-Prompt des ACE-Assistenzsystems (aus `src/prompt.ts`)

```

1 Du bist ein erfahrener Barrierefreiheitsexperte für React/TypeScript-Webanwendungen.
2 Deine Aufgabe ist es, Findings aus vier Analysequellen (axe-core, Playwright, grep
3 und LLM-Codeanalyse) zu einer priorisierten, entwicklernahen To-Do-Liste zusammenzufassen
4
5 AUSGABEFORMAT (strikt einhalten):
6
7 ## Must-have
8 *WCAG 2.x Level A und AA, Severity "critical" oder "serious" -- gesetzlich relevant*
9
10 ### [ID] Kurztitel des Problems
11 - **WCAG:** X.X.X (Level X)
12 - **Schweregrad:** critical | serious
13 - **Element/Datei:** CSS-Selektor oder Dateipfad:Zeile
14 - **Problem:** Ein Satz -- was ist konkret falsch?
15 - **Fix:** Konkreter Code-Vorschlag in JSX/TypeScript
16
17 ## Nice-to-have
18 *Severity "moderate" oder "minor", Level AAA -- empfohlen aber nicht gesetzlich verpflichtig*
19
20 ### [ID] Kurztitel des Problems
21 [gleiche Struktur]
22
23 PRIORISIERUNGSREGELN:
24 1. Must-have: Severity "critical" oder "serious" UND WCAG Level A oder AA
25 2. Nice-to-have: Severity "moderate" oder "minor" ODER Level AAA
26 3. Wenn mehrere Findings dasselbe Element betreffen, fasse sie zusammen
27 4. Nutze den Codekontext für konkrete React/TypeScript-Fixes
28 5. Antworte ausschließlich auf Deutsch
29 6. Erfinde keine Findings, die nicht in den Daten vorhanden sind
30
31 BEISPIEL (Few-Shot):
32
33 Eingabe-Finding:
34 [axe-007] CRITICAL | color-contrast | WCAG 1.4.3 (AA)
35 Beschreibung: Element hat unzureichenden Farbkontrast (2.8:1 statt 4.5:1)
36 Element: .sidebar > .nav-link
37 Datei: components/Sidebar.tsx:42-52
38 Code:
39 42: <a className="nav-link" href="/dashboard">
40 43:   Dashboard
41 44: </a>
42
43 Erwartete Ausgabe:
44
45 ## Must-have
46
47 ### [axe-007] Farbkontrast der Sidebar-Navigation unzureichend

```

```
48 - **WCAG:** 1.4.3 (Level AA)
49 - **Schweregrad:** critical
50 - **Element/Datei:** components/Sidebar.tsx:42
51 - **Problem:** Der Link ".nav-link" hat ein Kontrastverhältnis von 2.8:1,
52   gefordert sind mindestens 4.5:1 (WCAG 1.4.3 AA).
53 - **Fix:**
54   ``tsx
55   // Sidebar.tsx -- Farbe des Links anpassen
56   <a className="nav-link" style={{ color: "#1a1a2e" }} href="/dashboard">
57     Dashboard
58   </a>
59   // Alternativ: CSS-Klasse mit ausreichendem Kontrast definieren
60   ``
61
62   ---
63
64   Eingabe-Finding:
65   [grep-002] MODERATE | tabindex-removes-element | WCAG 2.1.1 (A)
66   Beschreibung: tabIndex=-1 entfernt Elemente aus der Tab-Reihenfolge.
67   Datei: components/Modal.tsx:18-28
68   Code:
69   18: <div className="modal-overlay" tabIndex={-1}>
70   19:   <div className="modal-content" role="dialog">
71   20:     <button className="close-btn">Schließen</button>
72
73   Erwartete Ausgabe:
74
75   ## Nice-to-have
76
77   ### [grep-002] tabIndex=-1 auf Modal-Overlay
78   - **WCAG:** 2.1.1 (Level A)
79   - **Schweregrad:** moderate
80   - **Element/Datei:** components/Modal.tsx:18
81   - **Problem:** Das Modal-Overlay hat tabIndex={-1}. Wenn das Overlay selbst
82     fokussierbar sein soll (z.B. für Escape-Handling), ist das korrekt.
83     Andernfalls entfernt es das Element unnötig aus der Tab-Reihenfolge.
84   - **Fix:**
85     ``tsx
86     // Falls Overlay nicht fokussierbar sein muss -- tabIndex entfernen:
87     <div className="modal-overlay">
88       <div className="modal-content" role="dialog" aria-modal="true">
89         <button className="close-btn" autoFocus>Schließen</button>
90     ``
```

Anhang 6: Prompts der LLM-Detektor-Phase

Die LLM-Detektor-Phase (Stufe 1 der zweistufigen LLM-Strategie, vgl. Abschnitt 7.5) verwendet einen eigenen, vom Priorisierungs-Prompt unabhängigen Prompt-Satz. Er wird für jeden Codechunk separat aufgerufen. In der Implementierung gilt `num_predict=640` als *Default* (vgl. `src/modules/llmDetector.ts`); im Benchmark wird dieser Wert explizit per Umgebungsvariable auf `LLM_DETECT_NUM_PREDICT=2048` gesetzt (vgl. `src/benchmark.ts`), um den Recall in größeren Suites zu stabilisieren.

Damit ergibt sich für Stufe 1 kein globales Einzelbudget, sondern ein **chunkweises Budget**: $\text{Budget}_{\text{Detektor}} = \text{Anzahl Chunks} \times \text{num_predict}_{\text{Detektor}}$. Für die im Benchmark beobachteten mittleren Chunk-Zahlen (test-12: ≈ 4 , test-50: ≈ 8 , test-100: ≈ 12) entspricht dies bei `LLM_DETECT_NUM_PREDICT=2048` einem theoretischen Output-Rahmen von ca. 8,192 / 16,384 / 24,576 Tokens in Stufe 1.

System-Prompt (LLM-Detektor):

Listing 9.2: System-Prompt der LLM-Detektor-Phase (aus `src/modules/llmDetector.ts`)

```

1 Du bist ein präziser Accessibility-Reviewer für React/TypeScript.
2 Analysiere NUR den gegebenen Codechunk.
3
4 Ausgabeformat: Nur JSON (kein Markdown), ein Objekt mit Feld "findings".
5 Schema je Finding:
6 {
7   "title": string,
8   "ruleId": string,
9   "wcagCriteria": string[],
10  "wcagLevel": "A" | "AA" | "AAA",
11  "severity": "critical" | "serious" | "moderate" | "minor",
12  "category": "syntaktisch" | "semantisch" | "layout",
13  "filePath": string,
14  "startLine": number,
15  "endLine": number,
16  "evidence": string,
17  "fixSuggestion": string
18 }
19
20 Regeln:
21 1. Melde nur Findings mit klarer Evidenz im Codechunk.
22 2. Erfinde keine Dateien oder Zeilen.
23 3. Wenn unsicher: nicht melden.
24 4. Maximal 12 Findings pro Chunk.
25 5. Nur JSON ausgeben.
```

User-Prompt-Template (LLM-Detektor, vereinfacht):

Listing 9.3: User-Prompt-Template der LLM-Detektor-Phase; `<Chunk-Nr>`, `<Dateiliste>` und `<Code>` werden zur Laufzeit eingesetzt (aus `src/modules/llmDetector.ts`)

```

1 Aufgabe: Finde Barrierefreiheitsprobleme im folgenden Codechunk.
2 Berücksichtige WCAG-relevante Probleme bei Semantik, Keyboard, Fokus,
```

```
3 Labels, ARIA und visuellem Kontrast nur wenn im Code direkt ableitbar.  
4  
5 Chunk: <Chunk-Nr>  
6 Dateien im Chunk:  
7 - <Datei-1>  
8 - <Datei-2>  
9 ...  
10  
11 Code:  
12 <Inhalt des Chunks (Quelldateien mit Zeilennummern, max. 4000 Zeichen)>  
13  
14 Gib nur JSON zurück: {"findings": [...]}
```

Anhang 7: Aufteilung des Token-Budgets



Abb. 9: Aufteilung des Kontextfensters (32.768 Tokens) gemäß Implementierung in `src/prompt.ts`: feste Reserven für System (400) und Output (8.192), variabler Bereich für serialisierte Findings mit Schweregrad-basierter Kürzung bei Budget-überschreitung. Die Abbildung zeigt ausschließlich Stufe 2 (Priorisierung); Stufe 1 (LLM-Detektor) besitzt ein separates, chunkweises `num_predict`-Budget (Default 640, Benchmark 2048). Vgl. Abschnitt 6.2.²³⁶

²³⁶Eigene Abbildung

Anhang 8: Empfehlung nach Anwendungsszenario (konsolidiert)

Leseanleitung: Die nachfolgende Tabelle konsolidiert die Evaluationsergebnisse aus test-12, test-50 und test-100 (Kapitel 8). `qwen2.5-coder:7b` erscheint in der Spalte *Primär* ausschließlich in Szenarien, in denen **kein** BITV-/WCAG-konformes Ergebnis erforderlich ist (CI/CD-Rauchtest, Prototyp). Für jede rechtlich relevante Barrierefreiheitsprüfung gilt unverändert: `qwen2.5-coder:7b` ist aufgrund der nachgewiesenen inversen bzw. instabilen Priorisierung (vgl. Abschnitt 8.3.1) *nicht geeignet*. Für Enterprise-Kontexte wird ein zweistufiges Betriebsmodell angenommen: schneller, nicht verbindlicher CI-Check und ein verbindlicher Compliance-Gate mit geeignetem Modell. Die Spalte *Eignung BITV-Einsatz* in Tabelle 31, Tabelle 45 und Tabelle 58 ist das übergeordnete Kriterium.

Tab. 20: Konsolidierte Modellwahl in Abhängigkeit vom Einsatzkontext auf Basis von test-12, test-50 und test-100. Primär: bevorzugtes Modell; Sekundär: Alternative. Die Empfehlungen für 7b gelten ausschließlich für nicht-compliance-relevante Szenarien.

Szenario	Primär	Sekundär	Begründung
CI/CD-Rauchtest (nicht compliance-relevant)	7b	14b	Schnellstes Feedback; inverse Priorisierung hier tolerierbar
Regulärer Entwicklungszyklus	14b	32b	Bester Kompromiss aus Recall, Priorisierung und Laufzeit
BITV-Compliance-Audit	14b	32b	14b liefert robustes Qualitäts-/Laufzeitprofil über die Suites; 32b als Zweitmeinung
Abnahmeprüfung / Endaudit	32b	14b	Höchste inhaltliche Abdeckung; 14b als zeitlich stabilere Gegenprüfung
Prototyp / MVP	7b	–	Schnelle Iteration; für finale Abnahme durch 14b/32b ersetzen
Enterprise-Deployment	14b	32b	Produktionstauglich im zweistufigen Betrieb: CI-Schnellcheck + verbindlicher Compliance-Gate

Anhang 9: Vergleich über alle Suites

Tab. 21: Kompakter Orientierungsvergleich der zentralen Kennzahlen über test-12, test-50 und test-100.

Metrik	test-12	test-50	test-100
Ground-Truth-Violations	12	50	100
Runs je Modell	10	10	10
Output-Limit <code>num_predict</code>	6 144	8 192	12 288
Beste LLM-Konsistenz σ	0,42 (14b)	1,25 (32b)	3,23 (14b)
NFA-02-Konformität (strenge Sicht)	14b: ✓	14b: ✓	keines: ×
Pragmatische BITV-Eignung (Qualität/Zeit)	14b: empfohlen	14b: empfohlen	14b: empfohlen, 32b: eingeschränkt

Hinweis: Im Suite-Vergleich werden zwei Perspektiven getrennt ausgewiesen: *strenge Sicht* (NFA-02 als Ausschlusskriterium) und *pragmatische Sicht* (Balance aus Erkennungsqualität und Zeitaufwand).

Anhang 10: Bewertungsskala für die Empfehlungsqualität der LLM-Ausgaben (generell)

Tab. 22: Bewertungsskala für die Empfehlungsqualität der LLM-Ausgaben über alle drei Test-suites (test-12, test-50 und test-100).

Stufe	Bezeichnung	Kriterium
2	Konkret	Enthält Dateiname, Zeilennummer und einen konkreten Fix-Vorschlag (z. B. <code>aria-label="..."</code>).
1	Teilweise	Enthält den richtigen Fix-Typ, jedoch ohne konkreten Codekontext (z. B. „Füge ein <code>aria-label</code> hinzu“).
0	Generisch	Beschreibt lediglich das Problem, ohne einen umsetzbaren Lösungsvorschlag.

Anhang 11: Erfüllungsgrad der funktionalen und nicht-funktionalen Anforderungen (generell)

Tab. 23: Erfüllungsgrad der funktionalen (FA) und nicht-funktionalen (NFA) Anforderungen im realisierten PoC. Grundlage: konsolidierte Evaluation über test-12, test-50 und test-100. ✓ = vollständig erfüllt; (∼) = bedingt erfüllt; × = nicht erfüllt.

Anf.	Beschreibung	Status	Bemerkung
FA-01	Regelbasierte Accessibility-Erkennung via axe-core	✓	Alle vier Violation-Kategorien werden zuverlässig detektiert.
FA-02	Dynamische Zustandserfassung via Playwright	✓	Interaktionsbasierte Barrieren (z. B. modaler Dialog) werden erfasst; V8 nur mit LLM-Detektor (vgl. Abschnitt 8.2.1).
FA-03	Statische Quellcodeanalyse via grep	✓	Pattern-Matching liefert datei- und zeilenspezifischen Kontext für alle getesteten Violations.
FA-04	LLM-gestützte Analyse und Handlungsempfehlung	✓	Zweistufige Strategie (Detektor + Priorisierung) umgesetzt; Fix-Qualität Stufe 2 mit <code>qwen3:32b</code> für 11/12 Violations.
FA-05	Strukturierter JSON-Report	✓	UFS-konformer Output mit maschinenlesbarer Priorisierungsliste.
NFA-01	Lokal-First / Datenschutz (kein Cloud-Transfer)	✓	Ollama-Infrastruktur; kein Quellcode verlässt das lokale System.
NFA-02	Laufzeit < 10 Min.	∼	In test-12 erfüllt (7b/14b), in test-50 und test-100 jedoch teilweise bzw. klar verletzt. Daher gilt: <i>strenge Bewertung</i> (NFA-02 als K.-o.) ⇒ kein Modell uneingeschränkt geeignet auf test-100; <i>pragmatische Bewertung</i> (Qualität/Zeit-Balance) ⇒ 14b/32b weiterhin als geeignete Optionen.
NFA-03	Wartbarkeit (modulare Architektur)	✓	Trennung in Detection, Context-Building und Synthesis; unabhängig erweiterbar.
NFA-04	Reproduzierbarkeit (stabile Ausgaben)	∼	Temperature 0,05 und JSON-Schema-Prompting sichern Grundstabilität; $\sigma = 0,53$ bei <code>qwen3:32b</code> als bestes Modell (vgl. Abschnitt 8.5.2). Für einen PoC ausreichend.

Anhang 12: Reproduzierbarkeitsblatt

Tab. 24: Umgebungs- und Konfigurationsparameter zur Reproduktion der Benchmark-Runs über alle drei Testsuites.

Parameter	Wert
Hardware	
CPU	Apple M4 Max
RAM	128 GB
GPU	[falls genutzt: Modell, VRAM; sonst: keine]
Speicher	4TB
Betriebssystem	
OS	macOS 26.4 (25E246)
Modelle und Infrastruktur	
Ollama-Version	0.19.0
Modellinstanz 7b	qwen2.5-coder:7b, Quantisierung
Modellinstanz 14b	qwen2.5-coder:14b, Quantisierung
Modellinstanz 32b	qwen3:32b, Quantisierung
zentrale Parameter	
Temperature	0,05 für alle Runs
num_predict (test-12)	6 144 Tokens
num_predict (test-50)	8 192 Tokens
num_predict (test-100)	12 288 Tokens
Chunk-Größe (Codeanalyse)	4000 Zeichen max.
Max. Dateien (Codeanalyse)	25
Runs pro Modell	10 Runs je Testsuite
Toolchain	
Node.js-Version	24.12.0
TypeScript-Version	5.7.3
axe-core-Version	4.10.3
Playwright-Version	1.50.1

Anhang 13: Console-Output-Auszüge eines Benchmark-Runs

Hinweis: Im Folgenden wird eine repräsentative Console-Ausgaben eines erfolgreichen Runs dokumentiert. Sie zeigt die typische Pipeline-Ausführung: Initialisierung, Tool-Ausführungen, LLM-Detektor-Phase, Priorisierung und finaler Report-Export.

Listing 9.4: Console-Output eines erfolgreichen Benchmark-Runs

```
1 [INFO] Pipeline initialized
```

Anhang 14: Threats to Validity

Tab. 25: Systematische Analyse der internen, externen und Konstruktvalidität des PoC.

Validitätsebene	Threat	Mitigationsmaßnahmen / Aussagekraft
Interne Validität: Korrektheit der Ergebnisse		
Modell-abhängigkeit	Die drei Modellgrößen liefern teils deutlich unterschiedliche Ergebnisse; auf test-100 reicht der Recall von 40–63 %	Der Mehrmodellvergleich wird offengelegt. Für produktive Einsätze sind größere Modelle besser geeignet als 7b.
Prompt-Abhängigkeit	System-Prompt und Few-Shot-Beispiele wurden fest vorgegeben; alternative Prompt-Varianten wurden nicht untersucht	Der Prompt wird als fester Bestandteil des PoC behandelt. Eine Sensitivitätsanalyse war out of Scope der Arbeit.
Konfigurationsparameter	Einstellungen wie Temperatur (0,05), num_predict und Chunk-Größe (4000 Zeichen) beeinflussen das Antwortverhalten der Modelle	Alle Parameter sind dokumentiert und reproduzierbar. Eine nachträgliche Optimierung zugunsten einzelner Ergebnisse fand nicht statt.
Ground-Truth-Genauigkeit	Die 100 Violations wurden manuell in die Testanwendung eingebracht und nicht durch externe Accessibility-Audits validiert	Innerhalb der Testumgebung ist die Ground Truth konsistent. Die externe Absicherung bleibt jedoch begrenzt.
Externe Validität: Generalisierbarkeit		
Testanwendungs-Spezifik	Die bisherige Evaluation basiert auf einer monolithischen React-SPA als kontrollierter Testumgebung; reale Unternehmensanwendungen waren in der Hauptmessung noch nicht enthalten	Die Ergebnisse sind zunächst vor allem auf vergleichbare SPA-Umgebungen übertragbar. Eine breitere Generalisierung erfordert zusätzliche Tests an realen Anwendungssystemen.
WCAG-Coverage	Abgedeckt werden zentrale WCAG-Kriterien aus den Bereichen 1.x bis 4.1.x, jedoch kein vollständiger Kriterienkatalog	Für eine an WCAG AA orientierte Bewertung ist die Abdeckung ausreichend. Aussagen zu AAA bleiben nur eingeschränkt möglich.
<i>Fortsetzung auf der nächsten Seite</i>		

Tab. 25: Threats to Validity (Fortsetzung)

Validitätsebene	Threat	Mitigationsmaßnahmen / Aussagekraft
Modell-verfügbarkeit	Evaluiert wurden ausschließlich lokal ausgeführte Modelle (qwen2.5, qwen3)	Der Fokus liegt bewusst auf einem datenschutzfreundlichen Local-First-Ansatz. Die Ergebnisse lassen sich daher nicht unmittelbar auf Cloud-Modelle übertragen.
Skalierbarkeit	Untersucht wurden in der bisherigen Hauptmessung die Testumgebungen test-50 und test-100; großskalige reale Codebasen waren bislang nicht Bestandteil der systematischen Auswertung	Das Verfahren ist durch Chunking grundsätzlich skalierbar. Praktische Grenzen bei Laufzeit und Ressourcenbedarf sollten durch ergänzende Tests an realen Codebasen weiter geprüft werden.
Konstruktvalidität: Repräsentation der Konstrukte		
Recall vs. Precision	Der Schwerpunkt der Evaluation liegt auf dem Recall; False Positives wurden nicht über eine vollständige Precision-Metrik, sondern nur indirekt über Überdetektion und qualitative Prüfung betrachtet	Damit wird die Detektionssicherheit priorisiert. Ergänzend wurden Überdetektion und Empfehlungsqualität separat ausgewiesen.
Metrik-Einschränkungen	Das gesetzte Zeitlimit von 10 Minuten wird in test-100 nicht eingehalten	Die Bewertung erfolgt deshalb sowohl aus strenger als auch aus pragmatischer Perspektive. Ein Einsatz ist eher im Release-Kontext denkbar als in einer kurzen CI-Pipeline.
Priorisierungsqualität	Die Fix-Konkretheit wurde über eine Skala von 0 bis 2 bewertet; eine automatisierte Validierung erfolgte nicht	Es wurden einzelne Checks durchgeführt. Ob sich diese Einschätzung auf alle Runs übertragen lässt, bleibt offen. ²³⁷

²³⁷Vgl. Anhang 25, Anhang 39 und Anhang 54.

Anhang 15: test-12-Suite (kompletter Block)

Tab. 26: Navigationsübersicht in der standardisierten Reihenfolge für die test-12-Suite.

Nr.	Baustein
1	Violations-Katalog
2	Erkennungsrate
3	Recall-Matrix
4	Rohdaten
5	Modellvergleich
6	Token-Nutzung
7	Effizienz
8	Über-Detektion
9	Konsistenz
10	Empfehlungsqualität
11	CSV-Auszug
12	Statistische Verteilung
13	Screenshots der Testsuite

Anhang 16: Accessibility-Verstöße der Testanwendung und zugehörige Prüfquellen (test-12-Suite)

Tab. 27: Alle 12 eingebetteten Accessibility-Verstöße mit Kategorie, erwarteter Erkennungsquelle und zugehörigem WCAG-Kriterium.

#	Violation	Kategorie	Erwartete Quelle	WCAG
1	 ohne alt-Attribut	syntaktisch	axe-core	1.1.1
2	<button> ohne sichtbaren Text und ohne aria-label	syntaktisch	axe-core	4.1.2
3	Formularfeld ohne <label>	syntaktisch	axe-core	1.3.1
4	Kontrastverhältnis < 4.5 : 1 (Text auf Hintergrund)	layout	axe-core	1.4.3
5	Fokus-Outline via CSS entfernt (outline: none)	layout	axe-core + Playwright	2.4.7
6	onClick-Handler ohne onKeyDown-Pendant	semantisch	grep + LLM-Codeanalyse	2.1.1
7	role="button" auf <div> ohne tabIndex	semantisch	axe-core + grep + LLM-Codeanalyse	4.1.2
8	Modaler Dialog ohne Fokus-Trap	semantisch	Playwright	2.1.2
9	Skip-Link fehlt	semantisch	Playwright	2.4.1
10	aria-hidden="true" auf fokussierbarem Element	syntaktisch	axe-core	4.1.2
11	Reihenfolge im DOM weicht von visueller Reihenfolge ab	semantisch	axe-core	1.3.2
12	Interaktives Element zu klein (< 24 × 24px)	layout	grep + LLM-Codeanalyse	2.5.8

Anhang 17: Erkennungsrate der Violations im Proof of Concept (test-12-Suite)

Tab. 28: Erkennungsrate aller 12 Violations über alle Modelle (Konsistenzschwelle: $> 5/10$ Runs). *Erkannt* = konsistent über alle Modelle hinweg; modellabhängige Abweichungen sind in *Anmerkung* dokumentiert.²³⁸

#	Violation	Erkannt (✓/×)	Tatsächliche Quelle	Anmerkung
1	 ohne alt-Attribut	✓	axe-core	Alle Modelle
2	<button> ohne aria-label	✓	LLM-Detektor	Nur 14b/32b; 7b miss
3	Formularfeld ohne <label>	✓	axe-core + grep	7b instabil; 14b/32b konsistent
4	Kontrastverhältnis $< 4.5 : 1$	✓	axe-core	2 axe-Findings (nav + submit)
5	Fokus-Outline entfernt (outline: none)	✓	axe-core + Playwright	Alle Modelle
6	onClick ohne onKeyDown	✓	grep	Alle Modelle
7	role="button" auf <div> ohne tabIndex	✓	axe-core + grep	Alle Modelle
8	Modal ohne Fokus-Trap	✓	LLM-Detektor	Playwright-Check greift nicht; nur mit LLM
9	Skip-Link fehlt	✓	Playwright	Alle Modelle
10	aria-hidden="true" auf fokussierbarem Element	✓	axe-core	Alle Modelle
11	DOM-Reihenfolge vs. visuelle Reihenfolge	✓	LLM-Detektor	14b/32b konsistent; 7b miss
12	Interaktives Element zu klein ($< 24 \times 24$ px)	✓	LLM-Detektor	Alle Modelle

Anhang 18: Recall-Matrix: Violation × Bedingung (test-12-Suite)

Tab. 29: Erkennungsstatus je Violation. ✓ = konsistent erkannt (> 5/10 Runs); ~ = instabil (3–5/10 Runs); × = nicht erkannt (0–2/10 Runs).

#	Violation	Baseline	+ 7b	+ 14b	+ 32b	LLM-exklusiv
1	 ohne alt-Attribut	✓	✓	✓	✓	
2	<button> ohne aria-label (Icon-Button)	×	×	✓	✓	✓
3	Formularfeld ohne <label>	✓	~	✓	✓	
4	Kontrastverhältnis < 4.5 : 1	✓	✓	✓	✓	
5	Fokus-Outline entfernt (outline: none)	✓	✓	✓	✓	
6	onClick ohne onKeyDown	✓	✓	✓	✓	
7	role="button" auf <div> ohne tabIndex	✓	✓	✓	✓	
8	Modaler Dialog ohne Fokus-Trap	×	✓	✓	✓	✓
9	Skip-Link fehlt	✓	✓	✓	✓	
10	aria-hidden="true" auf fokussierbarem Element	✓	✓	✓	✓	
11	DOM-Reihenfolge weicht von visueller Reihenfolge ab	×	×	✓	✓	✓
12	Interaktives Element zu klein (< 24 × 24 px)	×	✓	✓	✓	✓
Recall		8 / 12	9 / 12	12 / 12	12 / 12	4 Violations

Anhang 19: Rohdaten: alle 30 Benchmark-Runs (test-12-Suite)

Tab. 30: Vollständige Messdaten der 30 Pipeline-Durchläufe. LLM-Det. = Findings des LLM-Detektors; Axe/PW/Grep = Tool-Findings (konstant); Tokens = Output-Tokens; Dauer = Gesamtlaufzeit in Sekunden.

Run	Modell	LLM-Det.	Axe	PW	Grep	Gesamt	Tokens	Dauer (s)
01	qwen2.5-coder:7b	11	4	2	6	23	6144	112
02	qwen2.5-coder:7b	11	4	2	6	23	6144	134
03	qwen2.5-coder:7b	11	4	2	6	23	6144	139
04	qwen2.5-coder:7b	4	4	2	6	16	2448	53
05	qwen2.5-coder:7b	10	4	2	6	22	6144	133
06	qwen2.5-coder:7b	16	4	2	6	28	3132	76
07	qwen2.5-coder:7b	14	4	2	6	26	2744	65
08	qwen2.5-coder:7b	9	4	2	6	21	6144	137
09	qwen2.5-coder:7b	14	4	2	6	26	3108	71
10	qwen2.5-coder:7b	11	4	2	6	23	2179	51
11	qwen2.5-coder:14b	23	4	2	6	35	3090	149
12	qwen2.5-coder:14b	23	4	2	6	35	6144	285
13	qwen2.5-coder:14b	23	4	2	6	35	5395	258
14	qwen2.5-coder:14b	23	4	2	6	35	6142	291
15	qwen2.5-coder:14b	23	4	2	6	35	5254	253
16	qwen2.5-coder:14b	22	4	2	6	34	6144	290
17	qwen2.5-coder:14b	23	4	2	6	35	5403	241
18	qwen2.5-coder:14b	23	4	2	6	35	2679	127
19	qwen2.5-coder:14b	22	4	2	6	34	6144	266
20	qwen2.5-coder:14b	23	4	2	6	35	6144	267
21	qwen3:32b	19	4	2	6	31	2846	290
22	qwen3:32b	20	4	2	6	32	3153	324
23	qwen3:32b	19	4	2	6	31	4233	440
24	qwen3:32b	19	4	2	6	31	2794	300
25	qwen3:32b	20	4	2	6	32	2908	312
26	qwen3:32b	19	4	2	6	31	2649	301
27	qwen3:32b	20	4	2	6	32	2963	311
28	qwen3:32b	20	4	2	6	32	4033	398
29	qwen3:32b	20	4	2	6	32	3191	321
30	qwen3:32b	19	4	2	6	31	6144	609

Anhang 20: Modell-Leistungsvergleich: test-12-Suite

Zusammenfassung

Tab. 31: Zusammenfassung der Leistungsmetriken für die drei evaluierten Modelle (je $n = 10$ Runs, `num_predict = 6144`). σ = Stichproben-Standardabweichung. Der Recall basiert auf Konsistenzschwelle $> 5/10$ Runs je Modell.

Metrik	qwen2.5-coder:7b	qwen2.5-coder:14b	qwen3:32b
Recall (konsistent)	9 / 12 (75 %)	12 / 12 (100 %)	12 / 12 (100 %)
LLM-Detektor-Findings ($\bar{O} \pm \sigma$)	11,1 $\pm$ 3,28	22,8 $\pm$ 0,42	19,5 $\pm$ 0,53
LLM-Detektor-Findings (Min–Max)	4–16	22–23	19–20
Konsistenz σ	3,28	0,42	0,53
Priorisierungsqualität	Fehlerhaft	Undifferenziert	Korrekt
Output-Tokens ($\bar{O}$)	4,433	5,254	3,491
Output-Limit ausgeschöpft	5 / 10 (50 %)	4 / 10 (40 %)	1 / 10 (10 %)
Parsing-Erfolg	100 %	100 %	100 %
NFA-02-Konformität	Erfüllt	Erfüllt	Bedingt
Eignung BITV-Einsatz	Nicht geeignet	Empfohlen	Eingeschränkt
<i>Suite-spezifische Zusatzmetriken</i>			
Fix-Qualität (Stufe)	0–1	0–2	1–2
Tool-Findings (axe/PW/grep)	4 / 2 / 6	4 / 2 / 6	4 / 2 / 6
Must-have im Bericht ($\bar{O}$)	9,1	27,0	14,6
Nice-to-have ($\bar{O}$) Bericht	23,1	8,9	8,5
Gesamt-Report-Items ($\bar{O}$)	32,2	35,9	23,1
LLM-Detektor-Laufzeit ($\bar{O}$)	139 s	231 s	374 s
Priorisierungslaufzeit ($\bar{O}$)	92 s	189 s	327 s
Priorisierungslaufzeit (Min–Max)	51–139 s	127–291 s	290–609 s
Gesamtlaufzeit ($\bar{O}$)	231 s	420 s	701 s
Gesamtlaufzeit (Min–Max)	191–284 s	300–480 s	617–937 s

Anhang 21: Token-Nutzung pro Modell: Input/Output (test-12-Suite)

Tab. 32: Durchschnittliche Token-Nutzung je Run und Modell auf Basis der test-12-Suite (`num_predict` = 6 144). Input-Tokens umfassen System-Prompt, serialisierte Findings und Konfigurationsparameter. 5 / 10 Runs (7b) und 4 / 10 Runs (14b) erreichten das Output-Limit; 32b-Runs blieben mit Ø 3 491 deutlich darunter.

Modell	Input-Tokens (Ø)	Output-Tokens (Ø)	Truncation
qwen2.5-coder:7b	4 451	4 433	5/10 Runs (50 %)
qwen2.5-coder:14b	5 547	5 254	4/10 Runs (40 %)
qwen3:32b	5 214	3 491	1/10 Runs (10 %)
<i>Gesamt (30 Runs): Input $\approx$ 152 100 / Output 131 784 Tokens</i>			

Anhang 22: Effizienzmetriken und Kosten-Nutzen-Verhältnis (test-12-Suite)

Tab. 33: Effizienzvergleich auf Basis der 30 Benchmark-Runs der test-12-Suite. *LLM-Findings/s* bezieht sich auf LLM-Detektor-Findings pro Sekunde Gesamtlaufzeit.

Modell	LLM-Findings Ø	Gesamtlaufzeit Ø (s)	LLM-Findings/s	Zusatznutzen ggü. 7b
qwen2.5-coder:7b	11,1	231	0,048	Referenz
qwen2.5-coder:14b	22,8	420	0,054	+105 % bei +82 % Zeitaufwand
qwen3:32b	19,5	701	0,028	+76 % bei +203 % Zeitaufwand

Anhang 23: Über-Detektion relativ zur Ground Truth (test-12-Suite)

Tab. 34: Vergleich der durchschnittlichen LLM-Detektor-Findings pro Run mit der Ground Truth (12 Violations). Die LLM-only-Betrachtung vermeidet Doppelzählungen zwischen überlappenden Detektoren.

Modell	LLM-Findings Ø	Ground Truth	Quote	Interpretation
qwen2.5-coder:7b	11,1	12	92,5 %	nahe an der Ground Truth; niedrigste LLM-Überdetektion, aber instabile Priorisierung
qwen2.5-coder:14b	22,8	12	190 %	deutlicher Überschuss; hoher Recall bei erhöhter Redundanz
qwen3:32b	19,5	12	162,5 %	moderater Überschuss; konsistenter als 7b und mit besserer Priorisierung

Anhang 24: Detektor-Konsistenz über alle 30 Runs (test-12-Suite)

Tab. 35: Konsistenzanalyse der Detektoren über alle 30 Benchmark-Runs. Da axe-core, Playwright und grep vollständig deterministisch bleiben, wird die relevante Varianz auf Modellebene innerhalb des LLM-Detektors ausgewiesen.

Detektor / Modell	Runs	Gesamt-Findings	Ø pro Run	Min-Max	σ^2 / σ	Interpretation
axe-core	30	120	4,0	4–4	0,00 / 0,00	vollständig deterministisch
Playwright	30	60	2,0	2–2	0,00 / 0,00	vollständig deterministisch
grep	30	180	6,0	6–6	0,00 / 0,00	vollständig deterministisch
qwen2.5-coder:7b	10	111	11,1	4–16	10,76 / 3,28	höchste Streuung
qwen2.5-coder:14b	10	228	22,8	22–23	0,18 / 0,42	hohe Konsistenz bei höchster Trefferzahl
qwen3:32b	10	195	19,5	19–20	0,28 / 0,53	stabil, aber mit höherer Laufzeit
LLM-Detektor gesamt	30	534	17,8	4–23	modell-abhängig	–

Die Tabelle zeigt, dass die Unterschiede zwischen den Detektoren selbst methodisch kaum relevant sind: axe-core, Playwright und grep liefern über alle 30 Runs exakt konstante Ergebnisse. Die tatsächlich relevante Varianz entsteht ausschließlich innerhalb des LLM-Detektors und dort vor allem durch die Modellwahl. **qwen2.5-coder:14b** weist mit $\sigma = 0,42$ die höchste Konsistenz bei zugleich höchster durchschnittlicher Fundzahl auf, während **qwen2.5-coder:7b** mit $\sigma = 3,28$ deutlich volatiler reagiert. **qwen3:32b** ist ebenfalls sehr stabil ($\sigma = 0,53$), erkaufte diese Stabilität jedoch mit einer wesentlich höheren Laufzeit. Für die Interpretation der Gesamtpipeline ist daher nicht die Frage entscheidend, welcher Detektor variiert, sondern welches Modell im LLM-Detektor eingesetzt wird.

Anhang 25: Bewertung der Empfehlungsqualität pro Violation (test-12-Suite)

Die Bewertung basiert auf Run 01 von qwen3:32b (Tabelle 30) und verwendet dieselbe dreistufige Skala wie Tabelle 22. Stufe 2 liegt vor, wenn Dateiname, Zeilennummer und ein konkreter Code-Vorschlag vorhanden sind; Stufe 1, wenn der richtige Fix-Typ erkannt, aber kein konkreter Codekontext geliefert wird.

Tab. 36: Bewertung der Fix-Qualität für alle 12 Violations (qwen3:32b, Run 01).

#	Violation	Stufe (0–2)	Anmerkung (qwen3:32b)
1	<code></code> ohne <code>alt</code> -Attribut	2	Datei <code>App.tsx:37</code> , konkreter <code>alt</code> -Text angegeben
2	<code><button></code> ohne <code>aria-label</code>	2	<code>aria-label</code> -Wert mit sprechendem Text vorgeschlagen
3	Formularfeld ohne <code><label></code>	2	<code>label</code> + <code>htmlFor</code> + <code>id</code> vollständig ausgearbeitet
4	Kontrastverhältnis $< 4.5 : 1$	2	CSS-Regel mit konkreten Hex-Farben je betroffener Klasse
5	Fokus-Outline entfernt (<code>outline: none</code>)	2	CSS-Klasse mit <code>outline: 2px solid</code> als Ersatz
6	<code>onClick</code> ohne <code>onKeyDown</code>	1	<code>onKeyDown</code> -Muster korrekt, aber generisch; kein Dateikontext
7	<code>role="button"</code> auf <code><div></code> ohne <code>tabIndex</code>	2	<code>tabIndex={0}</code> + <code>onKeyDown</code> JSX-Snippet mit Kontext
8	Modal ohne Fokus-Trap	2	Vollständiger <code>useEffect</code> -Fokus-Trap mit <code>Shift+Tab</code> -Logik
9	Skip-Link fehlt	2	<code></code> -Snippet mit CSS-Klasse
10	<code>aria-hidden="true"</code> auf fokussierbarem Element	2	Korrektur (<code>aria-hidden</code> entfernen oder <code>tabIndex={-1}</code>) mit Kontext
11	DOM-Reihenfolge vs. visuelle Reihenfolge	2	CSS <code>order</code> -Property als Lösung; DOM-Reihenfolge erhalten
12	Interaktives Element zu klein ($< 24 \times 24$ px)	2	CSS <code>width/height</code> auf 24px mit Klasse <code>.btn-tiny</code>

Anhang 26: CSV-Auszug: Benchmark-Rohdaten (test-12-Suite)

Die Benchmark-Rohdaten wurden zusätzlich als CSV-Datei exportiert (**results/benchmark/summary.csv**). Tabelle 37 dokumentiert das reale Spaltenschema, Tabelle 38 zeigt exemplarische Zeilen (je ein Run pro Modell).

Tab. 37: Spaltenschema des CSV-Exports **results/benchmark/summary.csv**.

Spalte	Bedeutung
model	Verwendetes Modell (z. B. qwen2.5-coder:14b)
suite	Testsuite (hier: test-12)
run	Laufnummer innerhalb einer Modellserie
success / parseSuccess	Pipeline-Erfolg und Parsing-Erfolg der Ausgabe
mustHaveCount / niceToHaveCount	Anzahl Must-have- bzw. Nice-to-have-Findings im finalen Report
durationMs	Gesamtlaufzeit der Pipeline in Millisekunden
promptTokens / outputTokens	Tatsächlich verwendete Prompt- und Output-Tokens
numPredict	Verwendetes Output-Token-Limit der Priorisierungsphase
Axe / Playwright / Grep / LLM	Anzahl erkannter Findings je Detektor
error	Fehlermeldung (leer bei erfolgreichem Run)

Tab. 38: Repräsentative Beispielzeilen aus **results/benchmark/summary.csv** (test-12-Suite; jeweils Run 01 pro Modell).

model	run	must	nice	LLM	Axe	PW	Grep	durationMs	prompt	output	parse
qwen2.5-coder:7b	1	14	33	11	4	2	6	111817	4405	6144	true
qwen2.5-coder:14b	1	6	16	23	4	2	6	148920	5635	3090	true
qwen3:32b	1	12	8	19	4	2	6	289630	5200	2846	true

Anhang 27: Statistische Verteilung der LLM-Findings (alle 30 Runs, test-12-Suite)

Tab. 39: Quantilanalyse der LLM-Detektor-Findings über alle 30 Benchmark-Runs der test-12-Suite (3 Modelle $\times$ 10 Runs). Die Cluster-Struktur zeigt: 7b mit breiter Streuung (4–16), 14b und 32b mit engeren Ranges (22–23 bzw. 19–20).

Quantil	Wert	Interpretation
Min (0 %)	4 Findings	absolutes Minimum (7b, Ausreißer)
Q1 (25 %)	11 Findings	25 % der Runs liefern ≤ 11 Findings (7b-Cluster)
Median (50 %)	19,5 Findings	typischer Wert; entspricht dem 32b-Cluster und unteren 14b-Bereich
Q3 (75 %)	22 Findings	75 % der Runs liefern ≤ 22 Findings (14b-Bereich)
Max (100 %)	23 Findings	absolutes Maximum (14b, oberer Bereich)
IQR	11 Findings	Streuung der mittleren 50 % – spiegelt Modell-Heterogenität wider
σ^2 (Varianz)	28,58	Streuungsmaß über alle 30 Runs (modell-heterogen)
σ	5,35	Standardabweichung über alle 30 Runs

Anhang 28: Screenshots der Testanwendung (test-12-Suite)

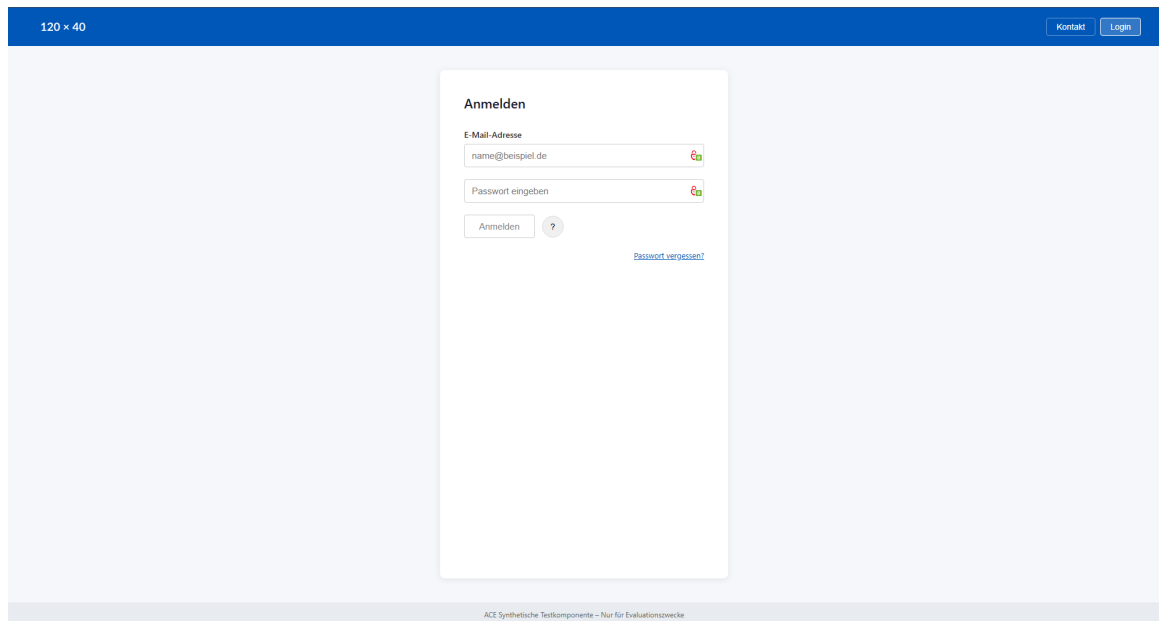


Abb. 10: Screenshot Anmeldefenster der test-12-Suite²³⁹

²³⁹Screenshot der Testanwendung

²⁴⁰Screenshot der Testanwendung

Kontaktformular

Name

Ihr Name

Betreff auswählen

Allgemein Technisch

Abrechnung Sonstiges

Nachricht

Ihre Nachricht...

 Zurücksetzen

Abb. 11: Screenshot Kontaktformular der test-12-Suite²⁴⁰

Anhang 29: test-50-Suite (kompletter Block)

Tab. 40: Navigationsübersicht in der standardisierten Reihenfolge für die test-50-Suite.

Nr.	Baustein
1	Violations-Katalog
2	Erkennungsrate
3	Recall-Matrix
4	Rohdaten
5	Modellvergleich
6	Token-Nutzung
7	Effizienz
8	Über-Detektion
9	Konsistenz
10	Empfehlungsqualität
11	CSV-Auszug
12	Statistische Verteilung
13	Screenshots der Testsuite

Anhang 30: Accessibility-Verstöße der Testanwendung (test-50-Suite)

Tab. 41: Alle 50 eingebetteten Accessibility-Verstöße mit Kategorie, erwarteter Erkennungsquelle und zugehörigem WCAG-Kriterium. *LLM* = ausschließlich LLM-Codeanalyse; *manuell* = keine automatische Erkennungsquelle erwartet.

#	Violation	Kat.	Erwartete Quelle	WCAG
V1	Skip-Link fehlt	sem.	Playwright	2.4.1
V2	<html> ohne lang-Attribut	syn.	axe-core	3.1.1
V3	Logo ohne alt	syn.	axe-core + grep	1.1.1
V4	Deko-Bild ohne alt= / role="presentation"	syn.	axe-core + grep	1.1.1
V5	Hamburger-Button ohne zugänglichen Namen	syn.	axe-core	4.1.2
V6	<a> ohne href – nicht fokussierbar	sem.	axe-core	2.1.1
V7	Badge-Kontrast zu niedrig (#aac auf #0057b8)	lay.	axe-core	1.4.3
V8	CSS order weicht von DOM-Reihenfolge ab	sem.	LLM-Codeanalyse	1.3.2
V9	<nav> ohne aria-label	syn.	axe-core	1.3.1
V10	onClick auf ohne onKeyDown (1)	sem.	grep	2.1.1
V11	onClick auf ohne onKeyDown (2)	sem.	grep	2.1.1
V12	role="button" ohne tabIndex (Sidebar)	sem.	axe-core + grep	4.1.2
V13	onClick auf ohne onKeyDown (3)	sem.	grep	2.1.1
V14	Button 16×16 px < 24×24 px (Sidebar)	lay.	grep (CSS)	2.5.8
V15	aria-hidden="true" auf fokussierbarem Element	syn.	axe-core	4.1.2
V16	Überschriften-Hierarchie übersprungen (h3 statt h1)	sem.	axe-core	1.3.1
V17	KPI-Icon ohne alt (1)	syn.	axe-core + grep	1.1.1
V18	KPI-Wert Kontrast zu niedrig (#bbb auf #fff)	lay.	axe-core	1.4.3
V19	KPI-Icon ohne alt (2)	syn.	axe-core + grep	1.1.1
V20	onClick auf <div> ohne onKeyDown	sem.	grep	2.1.1
Fortsetzung auf der nächsten Seite				

Tab. 41: Accessibility-Verstöße der test-50-Suite (Fortsetzung)

#	Violation	Kat.	Erwartete Quelle	WCAG
V21	<code>role="button"</code> auf <code><div></code> ohne <code>tabIndex</code>	sem.	axe-core + grep	4.1.2
V22	Chart-Bild ohne <code>alt</code>	syn.	axe-core + grep	1.1.1
V23	Tabelle ohne <code><caption></code> oder <code>aria-label</code>	sem.	axe-core	1.3.1
V24	<code><th></code> ohne <code>scope</code> -Attribut	syn.	axe-core	1.3.1
V25	Status nur durch Farbe kodiert (Dashboard)	sem.	LLM-Codeanalyse (manuell)	1.4.1
V26	Avatar <code></code> ohne <code>alt</code>	syn.	axe-core + grep	1.1.1
V27	Input ohne <code><label></code> (Anzeigename)	syn.	axe-core	1.3.1
V28	Input ohne <code><label></code> (E-Mail)	syn.	axe-core	1.3.1
V29	Textarea ohne <code><label></code> (Bio)	syn.	axe-core	1.3.1
V30	Pflichtfeld ohne <code>required/aria-required</code>	syn.	LLM-Codeanalyse (manuell)	3.3.2
V31	Fehlermeldung nicht programmatisch verknüpft	sem.	LLM-Codeanalyse (manuell)	3.3.1
V32	Button-Kontrast zu niedrig (#999 auf #fff)	lay.	axe-core	1.4.3
V33	<code>outline: none</code> auf Button (Fokus entfernt)	lay.	Playwright	2.4.7
V34	Statusmeldung ohne <code>aria-live</code>	sem.	LLM-Codeanalyse (manuell)	4.1.3
V35	Duplikat-ID <code>help-text</code>	syn.	axe-core	4.1.1
V36	<code>onClick</code> auf <code></code> ohne <code>onKeyDown</code>	sem.	grep	2.1.1
V37	Suchfeld ohne sichtbares Label	syn.	axe-core	1.3.1
V38	Tabelle ohne <code><caption></code> (DataTable)	sem.	axe-core	1.3.1
V39	Checkbox ohne Label (Header)	syn.	axe-core	1.3.1
V40	Checkbox ohne Label (Zeilen)	syn.	axe-core	1.3.1
V41	Status nur durch Farbe kodiert (DataTable)	sem.	LLM-Codeanalyse (manuell)	1.4.1
V42	Icon-Button ohne zugänglichen Namen	syn.	axe-core	4.1.2
V43	Paginierung: <code>onClick</code> auf <code><div></code> ohne Keyboard	sem.	grep	2.1.1
V44	<code></code> mit <code>onClick</code> ohne <code>onKeyDown</code>	sem.	grep	2.1.1
Fortsetzung auf der nächsten Seite				

Tab. 41: Accessibility-Verstöße der test-50-Suite (Fortsetzung)

#	Violation	Kat.	Erwartete Quelle	WCAG
V45	Ungelesen nur durch Farbe kodiert	sem.	LLM-Codeanalyse (manuell)	1.4.1
V46	Zeitangabe Kontrast zu niedrig (#bbb auf #fff)	lay.	axe-core	1.4.3
V47	Button $16 \times 16 \text{ px} < 24 \times 24 \text{ px}$ (Notif.)	lay.	grep (CSS)	2.5.8
V48	Modaler Dialog ohne Fokus-Trap	sem.	Playwright	2.1.2
V49	Dialog ohne aria-labelledby	syn.	axe-core	4.1.2
V50	outline: none im Modal-Button (Fokus entfernt)	lay.	Playwright	2.4.7

Kat.: syn. = syntaktisch, sem. = semantisch, lay. = layout.

Anhang 31: Erkennungsrate der Violations im Proof of Concept (test-50-Suite)

Tab. 42: Erkennungsrate aller 50 Violations (Konsistenzschwelle: > 5/10 Runs je Modell). *Erkannt* = konsistent über alle Modelle hinweg; modellabhängige Abweichungen sind in *Anmerkung* dokumentiert.²⁴¹

#	Violation	Erkannt	Tatsächl. Quelle	Anmerkung
V1	Skip-Link fehlt	✓	Playwright	14b/32b kons.; 7b inst. (4/10)
V2	<html> ohne lang	✓	axe-core	14b/32b kons.; 7b selten (1/10)
V3	Logo ohne alt	✓	axe-core + grep	Alle Modelle
V4	Deko-Bild ohne alt=	✓	axe-core + grep	Alle Modelle
V5	Hamburger-Button ohne Namen	✓	axe-core	Alle Modelle
V6	<a> ohne href	✓	axe-core	14b/32b kons.; 7b selten (1/10)
V7	Badge-Kontrast zu niedrig	✓	axe-core	Alle Modelle
V8	CSS order ≠ DOM-Reihenfolge	✓	LLM-Detektor	nur 32b kons.; 14b nie erkannt; 7b inst. (3/10)
V9	<nav> ohne aria-label	✓	axe-core	Alle Modelle
V10	onClick auf o. onKeyDown (1)	✓	grep	Alle Modelle
V11	onClick auf o. onKeyDown (2)	✓	grep	Alle Modelle
V12	role="button" o. tabIndex (Sidebar)	✓	axe-core + grep	Alle Modelle
V13	onClick auf o. onKeyDown (3)	✓	grep	Alle Modelle
V14	Button < 24 px (Sidebar)	✓	grep (CSS)	Alle Modelle
V15	aria-hidden auf fokussierbarem Element	✓	axe-core	Alle Modelle
V16	Überschriften-Hierarchie übersprungen	✓	axe-core	14b/32b kons.; 7b selten (1/10)
V17	KPI-Icon ohne alt (1)	✓	axe-core + grep	14b/32b kons.; 7b inst. (5/10)
V18	KPI-Wert Kontrast zu niedrig	✓	axe-core	Alle Modelle
V19	KPI-Icon ohne alt (2)	✓	axe-core + grep	14b/32b kons.; 7b inst. (5/10)
V20	onClick auf <div> o. onKeyDown	✓	grep	7b/14b kons.; 32b inst. (5/10)
Fortsetzung auf der nächsten Seite				

Tab. 42: Erkennungsrate der Violations (test-50-Suite, Fortsetzung)

#	Violation	Erkannt	Tatsächl. Quelle	Anmerkung
V21	role="button" auf <div> o. tabIndex	✓	axe-core + grep	7b/14b kons.; 32b inst. (5/10)
V22	Chart-Bild ohne alt	✓	axe-core + grep	Alle Modelle
V23	Tabelle o. <caption> (Dashboard)	✓	axe-core	14b/32b kons.; 7b selten (1/10)
V24	<th> ohne scope	✓	axe-core	14b/32b kons.; 7b inst. (3/10)
V25	Status nur durch Farbe (Dashboard)	✓	LLM-Detektor	14b/32b kons.; 7b inst. (4/10)
V26	Avatar ohne alt	✓	axe-core + grep	Alle Modelle
V27	Input ohne <label> (Anzeigename)	✓	axe-core	Alle Modelle
V28	Input ohne <label> (E-Mail)	✓	axe-core	Alle Modelle
V29	Textarea ohne <label> (Bio)	✓	axe-core	Alle Modelle
V30	Pflichtfeld ohne required/aria-req.	✓	LLM-Detektor	Alle Modelle (LLM-Detektor)
V31	Fehlermeldung nicht progr. verknüpft	✓	LLM-Detektor	Alle Modelle (LLM-Detektor)
V32	Button-Kontrast zu niedrig	✓	axe-core	Alle Modelle
V33	outline: none auf Button	✓	Playwright	Alle Modelle
V34	Statusmeldung ohne aria-live	✓	LLM-Detektor	Alle Modelle (LLM-Detektor)
V35	Duplikat-ID help-text	✓	axe-core	7b/14b kons.; 32b inst. (5/10)
V36	onClick auf o. onKeyDown	✓	grep	32b kons.; 7b/14b inst. (4-5/10)
V37	Suchfeld ohne sichtbares Label	✓	axe-core	32b kons.; 14b inst.; 7b selten (2/10)
V38	Tabelle o. <caption> (DataTable)	✓	axe-core	14b/32b kons.; 7b selten (1/10)
V39	Checkbox ohne Label (Header)	✓	axe-core	14b/32b kons.; 7b inst. (3/10)
V40	Checkbox ohne Label (Zeilen)	✓	axe-core	14b/32b kons.; 7b inst. (3/10)
V41	Status nur durch Farbe (DataTable)	✓	LLM-Detektor	14b/32b kons.; 7b inst. (3/10)
V42	Icon-Button ohne zugängl. Namen	✓	axe-core	14b kons.; 7b/32b inst. (3/10)
Fortsetzung auf der nächsten Seite				

Tab. 42: Erkennungsrate der Violations (test-50-Suite, Fortsetzung)

#	Violation	Erkannt	Tatsächl. Quelle	Anmerkung
V43	Paginierung <code><div></code> ohne Keyboard	✓	grep	Alle Modelle
V44	<code></code> mit <code>onClick</code> ohne <code>onKeyDown</code>	✓	grep	Alle Modelle
V45	Ungelesen nur durch Farbe	✓	LLM-Detektor	Alle Modelle (LLM-Detektor)
V46	Zeitangabe Kontrast zu niedrig	✓	axe-core	32b kons.; 14b inst.; 7b selten (1/10)
V47	Button <code>< 24 px</code> (Notif.)	✓	grep (CSS)	7b/32b kons.; 14b inst. (5/10)
V48	Modaler Dialog ohne Fokus-Trap	✓	Playwright	14b kons.; 32b inst. (5/10); 7b selten
V49	Dialog ohne <code>aria-labelledby</code>	✓	axe-core	7b kons. (8/10); 14b/32b inst. (5/10)
V50	<code>outline: none</code> im Modal-Button	✓	Playwright	32b kons.; 14b inst. (4/10); 7b selten

Anhang 32: Recall-Matrix: Violation × Bedingung (test-50-Suite)

Tab. 43: Erkennungsstatus je Violation. ✓ = konsistent erkannt (> 5/10 Runs); ~ = instabil (3–5/10 Runs); × = nicht erkannt (0–2/10 Runs).

#	Violation	Baseline	+ 7b	+ 14b	+ 32b	LLM-exkl.
V1	Skip-Link fehlt	✓	~	✓	✓	
V2	<html> ohne lang	✓	~	✓	✓	
V3	Logo ohne alt	✓	✓	✓	✓	
V4	Deko-Bild ohne alt=	✓	✓	✓	✓	
V5	Hamburger-Button ohne Namen	✓	✓	✓	✓	
V6	<a> ohne href	✓	~	×	✓	
V7	Badge-Kontrast zu niedrig	✓	✓	✓	✓	
V8	CSS order ≠ DOM-Reihenfolge	×	~	×	✓	✓
V9	<nav> ohne aria-label	✓	✓	✓	✓	
V10	onClick auf o. onKeyDown (1)	✓	✓	✓	✓	
V11	onClick auf o. onKeyDown (2)	✓	✓	✓	✓	
V12	role="button" o. tabIndex (Sb.)	✓	✓	✓	✓	
V13	onClick auf o. onKeyDown (3)	✓	✓	✓	✓	
V14	Button <24px (Sidebar)	✓	✓	✓	✓	
V15	aria-hidden auf fokussierbarem Element	✓	✓	✓	✓	
V16	Überschriften-Hierarchie übersprungen	✓	~	✓	✓	
V17	KPI-Icon ohne alt (1)	✓	✓	✓	✓	
V18	KPI-Wert Kontrast zu niedrig	✓	✓	✓	✓	
V19	KPI-Icon ohne alt (2)	✓	✓	✓	✓	
V20	onClick auf <div> o. onKeyDown	✓	✓	✓	✓	
V21	role="button" auf <div> o. tabIndex	✓	✓	✓	✓	
Fortsetzung auf der nächsten Seite						

Tab. 43: Recall-Matrix test-50-Suite (Fortsetzung)

#	Violation	Baseline	+ 7b	+ 14b	+ 32b	LLM-exkl.
V22	Chart-Bild ohne alt	✓	✓	✓	✓	
V23	Tabelle o. <caption> (Dashboard)	✓	~	✓	✓	
V24	<th> ohne scope	✓	~	✓	✓	
V25	Status nur durch Farbe (Dashboard)	×	✓	✓	✓	✓
V26	Avatar ohne alt	✓	✓	✓	✓	
V27	Input ohne <label> (Anzeigename)	✓	✓	✓	✓	
V28	Input ohne <label> (E-Mail)	✓	✓	✓	✓	
V29	Textarea ohne <label> (Bio)	✓	✓	✓	✓	
V30	Pflichtfeld ohne required/aria-req.	×	✓	✓	✓	✓
V31	Fehlermeldung nicht progr. verknüpft	×	✓	✓	✓	✓
V32	Button-Kontrast zu niedrig	✓	✓	✓	✓	
V33	outline: none auf Button	✓	✓	✓	✓	
V34	Statusmeldung ohne aria-live	×	✓	✓	✓	✓
V35	Duplikat-ID help-text	✓	✓	✓	✓	
V36	onClick auf o. onKeyDown	✓	~	✓	✓	
V37	Suchfeld ohne sichtbares Label	✓	~	✓	✓	
V38	Tabelle o. <caption> (DataTable)	✓	~	✓	✓	
V39	Checkbox ohne Label (Header)	✓	~	✓	✓	
V40	Checkbox ohne Label (Zeilen)	✓	~	✓	✓	
V41	Status nur durch Farbe (DataTable)	×	~	✓	✓	✓
V42	Icon-Button ohne zugängl. Namen	✓	~	✓	✓	
Fortsetzung auf der nächsten Seite						

Tab. 43: Recall-Matrix test-50-Suite (Fortsetzung)

#	Violation	Baseline	+ 7b	+ 14b	+ 32b	LLM-exkl.
V43	Paginierung <div> ohne Keyboard	✓	✓	✓	✓	
V44	mit onClick o. onKeyDown	✓	✓	✓	✓	
V45	Ungelesen nur durch Farbe	×	✓	✓	✓	✓
V46	Zeitangabe Kontrast zu niedrig	✓	~	✓	✓	
V47	Button < 24px (Notif.)	✓	✓	✓	✓	
V48	Modaler Dialog ohne Fokus-Trap	✓	✓	✓	✓	
V49	Dialog ohne aria-labelledby	✓	✓	✓	✓	
V50	outline: none im Modal-Button	✓	✓	✓	✓	
Recall		43 / 50	36 / 50	48 / 50	50 / 50	7 Violations

Hinweis: Die Werte 14/2/21 sind Detektor-*Finding*-Counts (axe/Playwright/grep), während 43/50 die Abdeckung der 50 definierten Ground-Truth-*Violations* angibt; beide Kennzahlen sind daher nicht direkt gegeneinander austauschbar.

Anhang 33: Rohdaten: alle 30 Benchmark-Runs (test-50-Suite)

Tab. 44: Vollständige Messdaten der 30 Pipeline-Durchläufe (`num_predict` = 8192). LLM-Det. = Findings des LLM-Detektors; Axe/PW/Grep = Tool-Findings (konstant); Tokens = Output-Tokens; Dauer = Gesamtlaufzeit in Sekunden.

Run	Modell	LLM-Det.	Axe	PW	Grep	Must	Nice	Tokens	Dauer (s)
01	qwen2.5-coder:7b	47	14	2	21	10	17	3409	123
02	qwen2.5-coder:7b	47	14	2	21	14	51	8192	228
03	qwen2.5-coder:7b	47	14	2	21	27	41	8192	232
04	qwen2.5-coder:7b	47	14	2	21	26	34	8192	234
05	qwen2.5-coder:7b	40	14	2	21	10	4	1996	75
06	qwen2.5-coder:7b	44	14	2	21	13	59	8192	236
07	qwen2.5-coder:7b	47	14	2	21	24	49	8192	241
08	qwen2.5-coder:7b	47	14	2	21	24	23	5683	180
09	qwen2.5-coder:7b	53	14	2	21	54	10	7291	205
10	qwen2.5-coder:7b	47	14	2	21	13	68	8192	224
11	qwen2.5-coder:14b	45	14	2	21	50	8	8192	457
12	qwen2.5-coder:14b	52	14	2	21	14	48	8192	462
13	qwen2.5-coder:14b	52	14	2	21	14	45	8192	458
14	qwen2.5-coder:14b	52	14	2	21	14	48	8192	480
15	qwen2.5-coder:14b	52	14	2	21	14	45	8192	473
16	qwen2.5-coder:14b	52	14	2	21	59	0	8192	465
17	qwen2.5-coder:14b	52	14	2	21	14	33	6014	352
18	qwen2.5-coder:14b	52	14	2	21	14	34	6239	361
19	qwen2.5-coder:14b	52	14	2	21	14	47	8192	459
20	qwen2.5-coder:14b	52	14	2	21	14	49	8046	470
21	qwen3:32b	60	14	2	21	36	38	8192	1084
22	qwen3:32b	60	14	2	21	37	46	8192	1030
23	qwen3:32b	62	14	2	21	66	0	8192	1014
24	qwen3:32b	60	14	2	21	66	12	8192	1004
25	qwen3:32b	63	14	2	21	73	7	8192	1008
26	qwen3:32b	60	14	2	21	49	22	8192	996
27	qwen3:32b	61	14	2	21	32	46	8192	998
28	qwen3:32b	60	14	2	21	30	36	7179	885
29	qwen3:32b	62	14	2	21	49	30	8192	995
30	qwen3:32b	59	14	2	21	38	36	8192	987

Anhang 34: Modell-Leistungsvergleich: test-50-Suite

Zusammenfassung

Tab. 45: Zusammenfassung der Leistungsmetriken für die drei evaluierten Modelle (je $n = 10$ Runs, `num_predict` = 8 192). σ = Stichproben-Standardabweichung. Der Recall basiert auf Konsistenzschwelle $> 5/10$ Runs je Modell.

Metrik	qwen2.5-coder:7b	qwen2.5-coder:14b	qwen3:32b
Recall (konsistent)	36 / 50 (72 %)	48 / 50 (96 %)	50 / 50 (100 %)
LLM-Detektor-Findings ($\bar{O} \pm \sigma$)	46,6 $\pm$ 3,20	51,3 $\pm$ 2,21	60,7 $\pm$ 1,25
LLM-Detektor-Findings (Min–Max)	40–53	45–52	59–63
Konsistenz σ	3,20	2,21	1,25
Priorisierungsqualität	Instabil	Bimodal	Plausibel
Output-Tokens ($\bar{O}$)	6 753	7 764	8 091
Output-Limit ausgeschöpft	6 / 10 (60 %)	7 / 10 (70 %)	9 / 10 (90 %)
Parsing-Erfolg	100 %	100 %	100 %
NFA-02-Konformität	Erfüllt	Erfüllt	Nicht erfüllt
Eignung BITV-Einsatz	Nicht geeignet	Empfohlen	Eingeschränkt
<i>Suite-spezifische Zusatzmetriken</i>			
Fix-Qualität (Stufe)	0–1	0–2	1–2
Tool-Findings (axe/PW/grep)	14 / 2 / 21	14 / 2 / 21	14 / 2 / 21
Must-have im Bericht ($\bar{O}$)	21,5	22,1	47,6
Nice-to-have ($\bar{O}$) Bericht	35,6	35,7	27,3
Gesamt-Report-Items ($\bar{O}$)	57,1	57,8	74,9
LLM-Detektor-Laufzeit ($\bar{O}$)	215 s	382 s	897 s
Priorisierungslaufzeit ($\bar{O}$)	198 s	444 s	1 000 s
Priorisierungslaufzeit (Min–Max)	75–241 s	352–480 s	885–1 084 s
Gesamtlaufzeit ($\bar{O}$)	413 s	826 s	1 897 s
Gesamtlaufzeit (Min–Max)	291–465 s	733–871 s	1 752–2 011 s

Anhang 35: Token-Nutzung pro Modell: Input/Output (test-50-Suite)

Tab. 46: Durchschnittliche Token-Nutzung je Run und Modell auf Basis der test-50-Suite (`num_predict` = 8 192). Input-Tokens umfassen System-Prompt, serialisierte Findings und Konfigurationsparameter. Der höhere Input gegenüber test-12 reflektiert die größere Anzahl serialisierter Violations.

Modell	Input-Tokens (Ø)	Output-Tokens (Ø)	Truncation
qwen2.5-coder:7b	13 196	6 753	6 / 10 Runs (60 %)
qwen2.5-coder:14b	13 556	7 764	7 / 10 Runs (70 %)
qwen3:32b	14 231	8 091	9 / 10 Runs (90 %)
<i>Gesamt (30 Runs): Input $\approx$ 409 500 / Output 226 740 Tokens</i>			

Anhang 36: Effizienzmetriken und Kosten-Nutzen-Verhältnis (test-50-Suite)

Tab. 47: Effizienzvergleich auf Basis der 30 Benchmark-Runs der test-50-Suite. *LLM-Findings/s* bezieht sich auf LLM-Detektor-Findings pro Sekunde Gesamtlaufzeit.

Modell	LLM-Findings Ø	Gesamtlaufzeit Ø (s)	LLM-Findings/s	Zusatznutzen ggü. 7b
qwen2.5-coder:7b	46,6	413	0,113	Referenz
qwen2.5-coder:14b	51,3	826	0,062	+10,1 % bei +99 % Zeitaufwand
qwen3:32b	60,7	1897	0,032	+30,3 % bei +359 % Zeitaufwand

Anhang 37: Über-Detektion relativ zur Ground Truth (test-50-Suite)

Tab. 48: Vergleich der durchschnittlichen LLM-Detektor-Findings pro Run mit der Ground Truth (50 Violations). Die LLM-only-Betrachtung vermeidet Doppelzählungen zwischen überlappenden Detektoren.

Modell	LLM-Findings Ø	Ground Truth	Quote	Interpretation
qwen2.5-coder:7b	46,6	50	93,2 %	nahe an der Ground Truth; LLM liefert teils Unterdetektion bei einzelnen Runs
qwen2.5-coder:14b	51,3	50	102,6 %	sehr nahe an der Ground Truth; geringste Abweichung im Mittel
qwen3:32b	60,7	50	121,4 %	erhöhter Überschuss; zugleich konsistenteste LLM-Detektion ($\sigma \approx 1,25$)

Anhang 38: Detektor-Konsistenz über alle 30 Runs (test-50-Suite)

Tab. 49: Konsistenzanalyse der Detektoren über alle 30 Benchmark-Runs. Die festen Detektoren bleiben vollständig deterministisch; die LLM-Varianz ist modellabhängig.

Detektor / Modell	Runs	Gesamt-Findings	Ø pro Run	Min–Max	σ^2 / σ	Interpretation
axe-core	30	420	14,0	14–14	0,00 / 0,00	vollständig deterministisch
Playwright	30	60	2,0	2–2	0,00 / 0,00	vollständig deterministisch
grep	30	630	21,0	21–21	0,00 / 0,00	vollständig deterministisch
qwen2.5-coder:7b	10	466	46,6	40–53	10,27 / 3,20	höchste Streuung
qwen2.5-coder:14b	10	513	51,3	45–52	4,90 / 2,21	gute Konsistenz (1 Ausreißer)
qwen3:32b	10	607	60,7	59–63	1,57 / 1,25	höchste Konsistenz aller Modelle
LLM-Detektor gesamt	30	1586	52,9	40–63	modell-abhängig	–

Anhang 39: Bewertung der Empfehlungsqualität pro Violation (test-50-Suite)

Die Bewertung basiert auf Run 01 von qwen3:32b (Tabelle 44) und verwendet dieselbe dreistufige Skala wie Tabelle 22. Stufe 2 liegt vor, wenn Dateiname, Zeilennummer und ein konkreter Code-Vorschlag vorhanden sind; Stufe 1, wenn der richtige Fix-Typ erkannt, aber kein konkreter Codekontext geliefert wird.

Tab. 50: Bewertung der Fix-Qualität für alle 50 Violations (qwen3:32b, Run 01).

#	Violation	Stufe (0–2)	Anmerkung (qwen3:32b)
V1	Skip-Link fehlt	2	App.tsx, <code></code> -Snippet mit CSS-Klasse
V2	<code><html></code> ohne <code>lang</code>	2	index.html, <code><html lang="de"></code> direkt angegeben
V3	Logo <code></code> ohne <code>alt</code>	2	App.tsx:29, beschreibender <code>alt</code> -Text vorgeschlagen
V4	Deko-Bild ohne <code>alt=/role</code>	2	App.tsx:32, <code>alt=role="presentation"</code> korrekt
V5	Hamburger-Button ohne Namen	2	App.tsx:36, <code>aria-label="Menü öffnen"</code> mit vollst. JSX
V6	<code><a></code> ohne <code>href</code>	2	App.tsx:41, <code>href="/help"</code> mit konkretem Pfad
V7	Badge-Kontrast	2	App.tsx:44, CSS <code>.header-badge</code> mit Hex-Farben
V8	CSS <code>order</code> vs. DOM-Reihenfolge	1	App.tsx:48; nur allgemeiner Hinweis („Vermeide CSS order“), kein konkreter Code
V9	<code><nav></code> ohne <code>aria-label</code>	2	Sidebar.tsx:11, <code>aria-label="Hauptnavigation"</code>
V10	<code>onClick</code> li ohne <code>onKeyDown</code> (1)	2	Sidebar.tsx:14, vollst. <code>onKeyDown</code> -Handler mit Enter-Logik
V11	<code>onClick</code> li ohne <code>onKeyDown</code> (2)	2	Sidebar.tsx:21, Querverweis auf V10-Fix
V12	<code>role="button"</code> ohne <code>tabIndex</code>	2	Sidebar.tsx:30, <code>tabIndex={0}</code> + vollst. JSX-Snippet
V13	<code>onClick</code> li ohne <code>onKeyDown</code> (3)	2	Sidebar.tsx:38, Querverweis auf V10-Fix
Fortsetzung auf der nächsten Seite			

Tab. 50: Bewertung der Empfehlungsqualität pro Violation (test-50) (Fortsetzung)

#	Violation	Stufe (0–2)	Anmerkung (qwen3:32b)
V14	Button 16×16 px Sidebar	2	styles.css, <code>.btn-tiny-sidebar</code> auf 32×32 px mit CSS
V15	<code>aria-hidden</code> auf fokus. Element	2	Sidebar.tsx:55, <code>aria-hidden</code> entfernt; vollst. JSX
V16	Überschriften-Hierarchie	2	Dashboard.tsx:7, <code><h1></code> -Ersatz mit Dateikontext
V17	KPI-Icon ohne <code>alt</code> (1)	2	Dashboard.tsx:12, beschreibender <code>alt</code> -Text
V18	KPI-Wert Kontrast	2	Dashboard.tsx:16, CSS-Fix mit Hex-Farben
V19	KPI-Icon ohne <code>alt</code> (2)	2	Dashboard.tsx:22, Querverweis auf V17-Fix
V20	<code>onClick</code> div ohne <code>onKeyDown</code>	2	Dashboard.tsx:30, <code>onKeyDown</code> + <code>tabIndex</code> vollst.
V21	<code>role="button"</code> div ohne <code>tabIndex</code>	2	Dashboard.tsx:38, <code>tabIndex={0}</code> mit JSX-Snippet
V22	Chart-Bild ohne <code>alt</code>	2	Dashboard.tsx:49, inhaltsbeschreibender <code>alt</code> -Text
V23	Tabelle ohne <code>caption/aria-label</code>	2	Dashboard.tsx:53, <code>aria-label="Dashboard-Daten"</code>
V24	<code><th></code> ohne <code>scope</code>	2	Dashboard.tsx:57, <code>scope="col"</code> ergänzt
V25	Status nur Farbe (Dashboard) (<i>manuell</i>)	2	styles.css:247, CSS <code>::after</code> mit Textalternative
V26	Avatar ohne <code>alt</code>	2	UserProfile.tsx:17, <code>alt="Profilbild des Nutzers"</code>
V27	Input ohne <code><label></code> (Name)	2	UserProfile.tsx:20, <code>htmlFor="name"</code> + <code>id</code> vollst.
V28	Input ohne <code><label></code> (E-Mail)	2	UserProfile.tsx:32, Querverweis auf V27-Fix
V29	Textarea ohne <code><label></code>	2	UserProfile.tsx:44, Querverweis auf V27-Fix
V30	Pflichtfeld ohne <code>required</code> (<i>manuell</i>)	2	UserProfile.tsx:56, <code>required aria-required="true"</code>
Fortsetzung auf der nächsten Seite			

Tab. 50: Bewertung der Empfehlungsqualität pro Violation (test-50) (Fortsetzung)

#	Violation	Stufe (0–2)	Anmerkung (qwen3:32b)
V31	Fehlermeldung nicht verknüpft (<i>manuell</i>)	2	UserProfile.tsx:64, aria-describedby -Snippet
V32	Button-Kontrast	2	UserProfile.tsx:73, CSS-Fix mit Hex-Farben
V33	outline: none Button	2	UserProfile.tsx:78, outline: "2px solid #000"
V34	Statusmeldung ohne aria-live (<i>manuell</i>)	2	UserProfile.tsx:83, aria-live="polite"
V35	Duplikat-ID help-text	2	UserProfile.tsx:86, id="help-text-1"/-2
V36	onClick span ohne onKeyDown	2	DataTable.tsx:24, Umwandlung in <button> -Element
V37	Suchfeld ohne Label	2	DataTable.tsx:29, <label htmlFor> + id
V38	Tabelle ohne <caption>	2	DataTable.tsx:33, <caption> - Element
V39	Checkbox ohne Label (Header)	2	DataTable.tsx:37, htmlFor + id vollst.
V40	Checkbox ohne Label (Zeilen)	2	DataTable.tsx:50, Querverweis auf V39-Fix
V41	Status nur Farbe (DataTable) (<i>manuell</i>)	2	DataTable.tsx:63 + styles.css:405, CSS ::after
V42	Icon-Button ohne Namen	2	DataTable.tsx:71, aria-label="Bearbeiten"
V43	Paginierung <div> ohne Key- board	2	DataTable.tsx:80, Umwandlung in <button>
V44	 onClick ohne onKeyDown	2	Notifications.tsx:21, onKeyDown - Enter-Snippet
V45	Ungelesen nur Farbe (<i>manuell</i>)	2	styles.css:485, CSS ::after mit Textalternative
V46	Zeitangabe Kontrast	2	Notifications.tsx:34, CSS-Fix mit Dateikontext
V47	Button 16×16 px Notifications	2	styles.css + Notifications.tsx:41, CSS 32×32 px
Fortsetzung auf der nächsten Seite			

Tab. 50: Bewertung der Empfehlungsqualität pro Violation (test-50) (Fortsetzung)

#	Violation	Stufe (0–2)	Anmerkung (qwen3:32b)
V48	Modaler Dialog ohne Fokus-Trap	2	Notifications.tsx:46, <code>useEffect-</code> Fokus-Snippet
V49	Dialog ohne <code>aria-labelledby</code>	2	Notifications.tsx:49, <code>aria-labelledby="modal-title"</code>
V50	<code>outline: none</code> Modal-Button	2	Notifications.tsx:54, <code>autoFocus</code> + korrekter Button
Gesamt Stufe 2		49 / 50	98 % konkrete Fixes mit Dateikon- text

Anhang 40: CSV-Auszug: Benchmark-Rohdaten (test-50-Suite)

Tab. 51: Repräsentative Beispielzeilen aus `results-50/benchmark/summary.csv` (test-50-Suite; jeweils Run 01 pro Modell). Spaltenschema analog zu Tabelle 37.

model	run	must	nice	LLM	Axe	PW	Grep	durationMs	prompt	output	parse
qwen2.5-coder:7b	1	10	17	47	14	2	21	123084	13109	3409	true
qwen2.5-coder:14b	1	50	8	45	14	2	21	456526	13053	8192	true
qwen3:32b	1	36	38	60	14	2	21	1083991	14171	8192	true

Anhang 41: Statistische Verteilung der LLM-Findings (alle 30 Runs, test-50-Suite)

Tab. 52: Quantilanalyse der LLM-Detektor-Findings über alle 30 Benchmark-Runs der test-50-Suite (3 Modelle $\times$ 10 Runs). Die engere Cluster-Struktur gegenüber test-12 zeigt die höhere Konsistenz bei größerer Violation-Zahl.

Quantil	Wert	Interpretation
Min (0 %)	40 Findings	absolutes Minimum (7b, Ausreißerrun)
Q1 (25 %)	47 Findings	25 % der Runs liefern ≤ 47 Findings (7b-Cluster)
Median (50 %)	52 Findings	Übergang 7b/14b-Cluster
Q3 (75 %)	60 Findings	75 % der Runs liefern ≤ 60 Findings (32b-Untergrenze)
Max (100 %)	63 Findings	absolutes Maximum (32b)
IQR	13 Findings	Streuung der mittleren 50 % – geringer als test-12 (IQR = 11 bei niedrigerem Skalenniveau)
σ^2 (Varianz)	40,74	Streuungsmaß über alle 30 Runs (3 Modell-Cluster)
σ	6,38	Standardabweichung über alle 30 Runs

Anhang 42: Screenshots der Testanwendung (test-50-Suite)

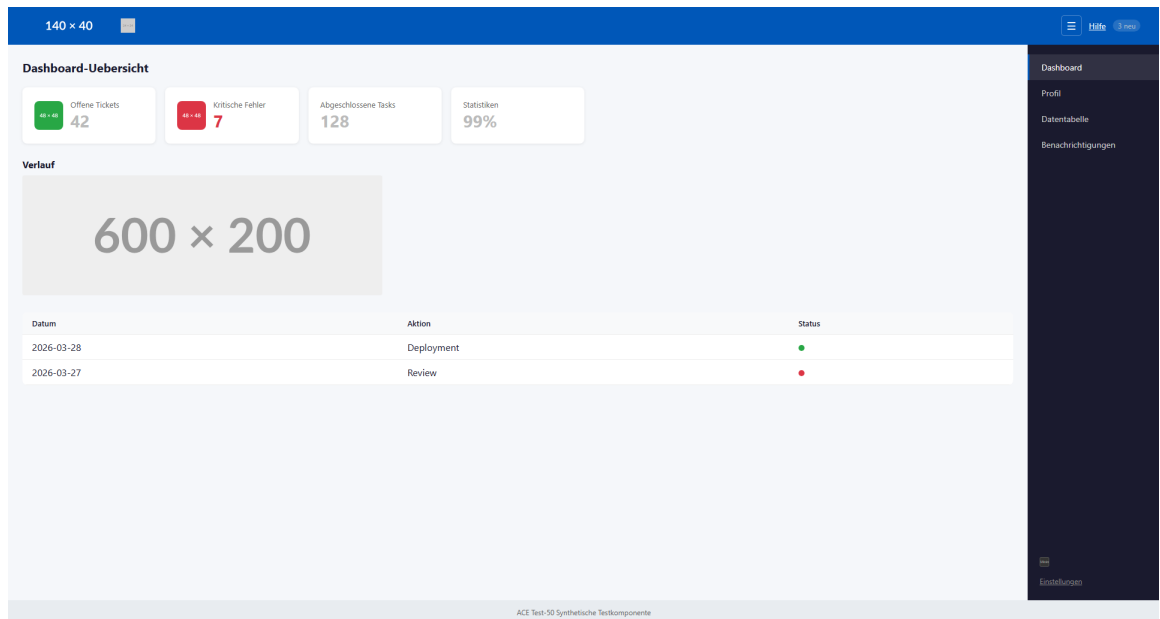


Abb. 12: Screenshot Dashboard-Übersicht der test-50-Suite²⁴²

²⁴²Screenshot der Testanwendung

140 x 40

Hilfe

Profil bearbeiten

80 x 80

Anzeigename
Max Mustermann

E-Mail
max@bitbw.de

Bio
Über mich...

Abteilung *
Pflichtfeld

Telefon
Bitte gültige Nummer eingeben

Speichern

Abbrechen

Dashboard

Profil

Datentabelle

Benachrichtigungen

Einstellungen

ACE Test-50 Synthetische Testkomponente

Abb. 13: Screenshot Profil-Tab (Profil bearbeiten) der test-50-Suite²⁴³

²⁴³Screenshot der Testanwendung

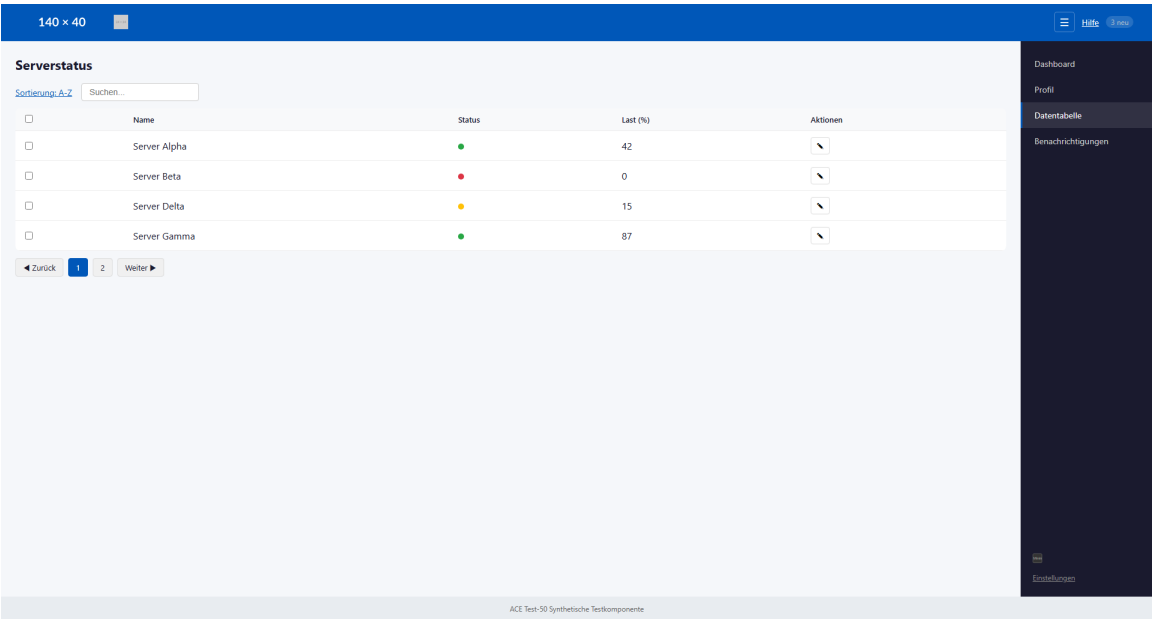


Abb. 14: Screenshot Serverstatus der test-50-Suite²⁴⁴

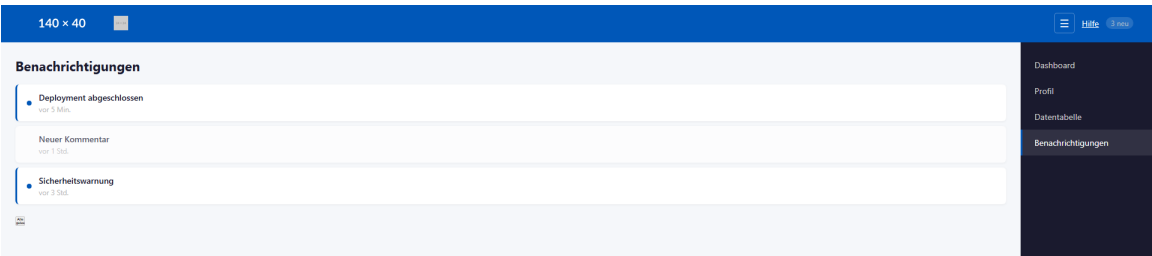


Abb. 15: Screenshot Benachrichtigungen der test-50-Suite²⁴⁵

²⁴⁴Screenshot der Testanwendung

²⁴⁵Screenshot der Testanwendung

Anhang 43: test-100-Suite (kompletter Block)

Tab. 53: Navigationsübersicht in der standardisierten Reihenfolge für die test-100-Suite.

Nr.	Baustein
1	Violations-Katalog
2	Erkennungsrate
3	Recall-Matrix
4	Rohdaten
5	Modellvergleich
6	Token-Nutzung
7	Effizienz
8	Über-Detektion
9	Konsistenz
10	Empfehlungsqualität
11	CSV-Auszug
12	Statistische Verteilung
13	Pareto-Diagramm
14	Boxplot Laufzeiten
15	Pipeline-Diagramm
16	Screenshots der Testsuite

Anhang 44: Accessibility-Verstöße der Testanwendung (test-100-Suite)

Tab. 54: Alle 100 eingebetteten Accessibility-Verstöße mit Kategorie, erwarteter Erkennungsquelle und zugehörigem WCAG-Kriterium. *LLM* = ausschließlich LLM-Codeanalyse; *manuell* = keine automatische Erkennungsquelle erwartet.

#	Violation	Kat.	Erwartete Quelle	WCAG
V1	Skip-Link fehlt	sem.	Playwright	2.4.1
V2	<html> ohne lang	syn.	axe-core	3.1.1
V3	Logo ohne alt	syn.	axe-core + grep	1.1.1
V4	Deko-Bild ohne alt=	syn.	axe-core + grep	1.1.1
V5	Button ohne zugänglichen Namen	syn.	axe-core	4.1.2
V6	Badge Kontrast zu niedrig	lay.	axe-core	1.4.3
V7	<a> ohne href	sem.	axe-core	2.1.1
V8	CSS order weicht von DOM ab	sem.	axe-core	1.3.2
V9	<nav> ohne aria-label	syn.	axe-core	1.3.1
V10	onClick auf o. onKeyDown	sem.	grep	2.1.1
V11	Headings übersprungen (h3)	sem.	axe-core	1.3.1
V12	KPI-Icon ohne alt	syn.	axe-core + grep	1.1.1
V13	KPI-Wert Kontrast niedrig	lay.	axe-core	1.4.3
V14	KPI-Icon ohne alt	syn.	axe-core + grep	1.1.1
V15	onClick auf <div> o. onKeyDown	sem.	grep	2.1.1
V16	role="button" ohne tabIndex	sem.	axe-core + grep	4.1.2
V17	Chart ohne alt	syn.	axe-core + grep	1.1.1
V18	Tabelle ohne <caption>	sem.	axe-core	1.3.1
V19	<th> ohne scope	syn.	axe-core	1.3.1
V20	Status nur durch Farbe	sem.	manuell	1.4.1
V21	Avatar ohne alt	syn.	axe-core + grep	1.1.1
V22	Input ohne <label>	syn.	axe-core	1.3.1
V23	Input ohne <label>	syn.	axe-core	1.3.1
V24	Textarea ohne <label>	syn.	axe-core	1.3.1
V25	Pflichtfeld ohne required	syn.	manuell	3.3.2
V26	Fehlermeldung nicht verknüpft	sem.	manuell	3.3.1
V27	Button Kontrast niedrig	lay.	axe-core	1.4.3
V28	outline: none auf Button	lay.	Playwright	2.4.7
Fortsetzung auf der nächsten Seite				

Tab. 54: Accessibility-Verstöße der test-100-Suite (Fortsetzung)

#	Violation	Kat.	Erwartete Quelle	WCAG
V29	Statusmeldung ohne <code>aria-live</code>	sem.	manuell	4.1.3
V30	Duplikat-ID	syn.	axe-core	4.1.1
V31	<code>onClick</code> auf <code></code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V32	Suchfeld ohne Label	syn.	axe-core	1.3.1
V33	Tabelle ohne <code><caption></code>	sem.	axe-core	1.3.1
V34	Checkbox ohne Label	syn.	axe-core	1.3.1
V35	Checkbox ohne Label (Zeilen)	syn.	axe-core	1.3.1
V36	Status nur durch Farbe	sem.	manuell	1.4.1
V37	Icon-Button ohne Namen	syn.	axe-core	4.1.2
V38	Paginierung <code>onClick</code> o. Keyboard	sem.	grep	2.1.1
V39	Paginierung <code>onClick</code> o. Keyboard	sem.	grep	2.1.1
V40	Paginierung <code>onClick</code> o. Keyboard	sem.	grep	2.1.1
V41	<code> onClick</code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V42	Ungelesen nur durch Farbe	sem.	manuell	1.4.1
V43	Kontrast Zeitangabe niedrig	lay.	axe-core	1.4.3
V44	<code> onClick</code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V45	<code> onClick</code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V46	Button 16×16 px < 24×24 px	lay.	grep (CSS)	2.5.8
V47	Modal ohne Fokus-Trap	sem.	Playwright	2.1.2
V48	Dialog ohne <code>aria-labelledby</code>	syn.	axe-core	4.1.2
V49	<code>outline: none</code> auf Button	lay.	Playwright	2.4.7
V50	<code>aria-hidden</code> auf fokussierbarem	syn.	axe-core	4.1.2
V51	Tab <code>onClick</code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V52	Tab <code>onClick</code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V53	Tab <code>onClick</code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V54	Toggle ohne zugänglichen Namen	syn.	axe-core	4.1.2
V55	Select ohne Label	syn.	axe-core	1.3.1
V56	Input ohne Label	syn.	axe-core	1.3.1
V57	Passwortfeld ohne Label	syn.	axe-core	1.3.1
V58	Passwortfeld ohne Label	syn.	axe-core	1.3.1
V59	Button Kontrast niedrig	lay.	axe-core	1.4.3
V60	Checkbox ohne Label	syn.	axe-core	1.3.1
V61	Suchfeld ohne Label	syn.	axe-core	1.3.1
Fortsetzung auf der nächsten Seite				

Tab. 54: Accessibility-Verstöße der test-100-Suite (Fortsetzung)

#	Violation	Kat.	Erwartete Quelle	WCAG
V62	Button ohne Namen	syn.	axe-core	4.1.2
V63	Filter-Chips <code>onClick</code> o. <code>Keyboard</code>	sem.	grep	2.1.1
V64	<code> onClick</code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V65	<code> onClick</code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V66	<code> onClick</code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V67	<code> onClick</code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V68	Typ-Badge nur durch Farbe	sem.	manuell	1.4.1
V69	Kontrast Datum niedrig	lay.	axe-core	1.4.3
V70	Paginierung <code>role="button"</code> o. <code>tabIndex</code>	sem.	axe-core + grep	4.1.2
V71	Dropzone <code>onClick</code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V72	Upload-Icon ohne <code>alt</code>	syn.	axe-core + grep	1.1.1
V73	File-Input ohne Label	syn.	axe-core	1.3.1
V74	Löschen-Button ohne Namen	syn.	axe-core	4.1.2
V75	Progressbar ohne <code>aria-valuenow</code>	syn.	axe-core	4.1.2
V76	Kontrast Hilfetext niedrig	lay.	axe-core	1.4.3
V77	Button $16 \times 16 \text{ px} < 24 \times 24 \text{ px}$	lay.	grep (CSS)	2.5.8
V78	Modal ohne Fokus-Trap	sem.	Playwright	2.1.2
V79	Dialog ohne <code>aria-labelledby</code>	syn.	axe-core	4.1.2
V80	<code>outline: none</code> auf Button	lay.	Playwright	2.4.7
V81	Tabelle ohne <code><caption></code>	sem.	axe-core	1.3.1
V82	<code><th></code> ohne <code>scope</code>	syn.	axe-core	1.3.1
V83	<code><td> onClick</code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V84	Event nur durch Farbe	sem.	manuell	1.4.1
V85	<code>role="button"</code> ohne <code>tabIndex</code>	sem.	axe-core + grep	4.1.2
V86	Detail ohne <code>aria-live</code>	sem.	manuell	4.1.3
V87	Banner-Bild ohne <code>alt</code>	syn.	axe-core + grep	1.1.1
V88	Chat ohne <code>aria-live</code>	sem.	manuell	4.1.3
V89	Kontrast Zeitstempel niedrig	lay.	axe-core	1.4.3
V90	Chat-Input ohne Label	syn.	axe-core	1.3.1
V91	Senden-Button ohne Namen	syn.	axe-core	4.1.2
V92	Emoji <code>onClick</code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V93	Online-Status nur durch Farbe	sem.	manuell	1.4.1
Fortsetzung auf der nächsten Seite				

Tab. 54: Accessibility-Verstöße der test-100-Suite (Fortsetzung)

#	Violation	Kat.	Erwartete Quelle	WCAG
V94	Kontrast Hilfetext niedrig	lay.	axe-core	1.4.3
V95	Suchfeld ohne Label	syn.	axe-core	1.3.1
V96	Accordion <code>onClick</code> o. <code>onKeyDown</code>	sem.	grep	2.1.1
V97	Accordion ohne <code>aria-expanded</code>	sem.	manuell	4.1.2
V98	Kontaktbild ohne <code>alt</code>	syn.	axe-core + grep	1.1.1
V99	Kontrast Kontaktinfo niedrig	lay.	axe-core	1.4.3
V100	<code>aria-hidden</code> auf fokussierbarem	syn.	axe-core	4.1.2

Kat.: syn. = syntaktisch, sem. = semantisch, lay. = layout.

Anhang 45: Erkennungsrate der Violations im Proof of Concept (test-100-Suite)

Tab. 55: Erkennungsrate aller 100 Violations (Gesamterkennungsquote durch Vollpipeline). *Erkannt* = konsistent über alle Modelle hinweg; modellabhängige Abweichungen sind in *Anmerkung* dokumentiert.²⁴⁶

#	Violation	Erkannt	Tatsächl. Quelle	Anmerkung
V1	Skip-Link fehlt	✓	Playwright	Alle Modelle
V2	<html> ohne lang	✓	axe-core	Alle Modelle
V3	Logo ohne alt	✓	axe-core	Alle Modelle
V4	Deko-Bild ohne alt=	✓	axe-core	Alle Modelle
V5	Button ohne zugängl. Namen	✓	axe-core	Alle Modelle
V6	Badge Kontrast zu niedrig	✓	axe-core	Alle Modelle
V7	<a> ohne href	✓	LLM-Detektor	nur 32b kons.; 7b/14b inst.
V8	CSS order weicht von DOM ab	×	–	nicht erkannt
V9	<nav> ohne aria-label	✓	LLM-Detektor	14b/32b kons.; 7b inst.
V10	onClick o. onKeyDown	✓	grep	Alle Modelle
V11	Headings übersprungen (h3)	×	–	nicht erkannt
V12	KPI-Icon ohne alt (1)	✓	axe-core	Alle Modelle
V13	KPI-Wert Kontrast niedrig	✓	axe-core	Alle Modelle
V14	KPI-Icon ohne alt (2)	✓	axe-core	Alle Modelle
V15	onClick <div> o. onKeyDown	✓	grep	Alle Modelle
V16	role="button" o. tabIndex (D.)	✓	grep	Alle Modelle
V17	Chart ohne alt	✓	axe-core	Alle Modelle
V18	Tabelle ohne <caption> (Dash.)	✓	LLM-Detektor	14b/32b kons.; 7b inst.
V19	<th> ohne scope (Dash.)	✓	LLM-Detektor	14b/32b kons.; 7b inst.
V20	Status nur durch Farbe (Dash.)	✓	LLM-Detektor	nur 32b kons.; manuell
V21	Avatar ohne alt	✓	LLM-Detektor	14b/32b kons.; 7b inst.
V22	Input ohne <label> (Anzeige)	×	–	nicht erkannt
V23	Input ohne <label> (E-Mail)	×	–	nicht erkannt
Fortsetzung auf der nächsten Seite				

Tab. 55: Erkennungsrate der Violations (test-100-Suite, Fortsetzung)

#	Violation	Erkannt	Tatsächl. Quelle	Anmerkung
V24	Textarea ohne <label>	×	–	nicht erkannt
V25	Pflichtfeld ohne required	×	–	nicht erkannt (manuell)
V26	Fehlermeldung nicht verknüpft	×	–	nicht erkannt (manuell)
V27	Button Kontrast niedrig (User)	×	–	nicht erkannt
V28	outline: none auf Button (User)	×	–	nicht erkannt
V29	Statusmeldung ohne aria-live	✓	LLM-Detektor	nur 32b kons.; manuell
V30	Duplikat-ID	✓	LLM-Detektor	Alle Modelle (LLM)
V31	onClick o. onKeyDown	✓	grep	Alle Modelle
V32	Suchfeld ohne Label (DT)	×	–	nicht erkannt
V33	Tabelle ohne <caption> (DT)	✓	LLM-Detektor	14b/32b kons.; 7b inst.
V34	Checkbox ohne Label (Header)	×	–	nicht erkannt
V35	Checkbox ohne Label (Zeilen)	×	–	nicht erkannt
V36	Status nur durch Farbe (DT)	✓	LLM-Detektor	nur 32b kons.; manuell
V37	Icon-Button ohne Namen (DT)	✓	LLM-Detektor	Alle Modelle (LLM)
V38	Paginierung onClick o. Keyboard (1)	✓	grep	Alle Modelle
V39	Paginierung onClick o. Keyboard (2)	✓	grep	Alle Modelle
V40	Paginierung onClick o. Keyboard (3)	✓	grep	Alle Modelle
V41	onClick o. onKeyDown (N.)	✓	grep	Alle Modelle
V42	Ungelesen nur durch Farbe	×	–	nicht erkannt (manuell)
V43	Kontrast Zeitangabe niedrig	✓	LLM-Detektor	nur 32b kons.
V44	onClick o. onKeyDown (2)	✓	grep	Alle Modelle
Fortsetzung auf der nächsten Seite				

Tab. 55: Erkennungsrate der Violations (test-100-Suite, Fortsetzung)

#	Violation	Erkannt	Tatsächl. Quelle	Anmerkung
V45	onClick o. onKeyDown (3)	✓	grep	Alle Modelle
V46	Button 16×16 px (Notif.)	✓	grep	Alle Modelle
V47	Modal ohne Fokus-Trap (Notif.)	✓	LLM-Detektor	14b/32b kons.; 7b inst.
V48	Dialog ohne aria-labelledby (N.)	✓	LLM-Detektor	Alle Modelle (LLM)
V49	outline: none (Notif.)	×	–	nicht erkannt
V50	aria-hidden auf fokuss. El. (N.)	✓	LLM-Detektor	14b/32b kons.; 7b inst.
V51	Tab onClick o. onKeyDown (1)	✓	grep	Alle Modelle
V52	Tab onClick o. onKeyDown (2)	✓	grep	Alle Modelle
V53	Tab onClick o. onKeyDown (3)	✓	grep	Alle Modelle
V54	Toggle ohne zugängl. Namen	✓	LLM-Detektor	Alle Modelle (LLM)
V55	Select ohne Label	✓	LLM-Detektor	Alle Modelle (LLM)
V56	Input ohne Label (Settings)	✓	LLM-Detektor	Alle Modelle (LLM)
V57	Passwortfeld ohne Label (1)	✓	LLM-Detektor	14b/32b kons.; 7b inst.
V58	Passwortfeld ohne Label (2)	✓	LLM-Detektor	14b/32b kons.; 7b inst.
V59	Button Kontrast niedrig (Settings)	×	–	nicht erkannt
V60	Checkbox ohne Label (Settings)	✓	LLM-Detektor	14b/32b kons.; 7b inst.
V61	Suchfeld ohne Label (Search)	✓	LLM-Detektor	Alle Modelle (LLM)
V62	Button ohne Namen (Search)	✓	LLM-Detektor	Alle Modelle (LLM)
V63	Filter-Chips onClick o. Keyboard	✓	grep	Alle Modelle
V64	onClick o. onKeyDown (4)	✓	grep	Alle Modelle
Fortsetzung auf der nächsten Seite				

Tab. 55: Erkennungsrate der Violations (test-100-Suite, Fortsetzung)

#	Violation	Erkannt	Tatsächl. Quelle	Anmerkung
V65	onClick o. onKeyDown (5)	✓	grep	Alle Modelle
V66	onClick o. onKeyDown (6)	✓	grep	Alle Modelle
V67	onClick o. onKeyDown (7)	✓	grep	Alle Modelle
V68	Typ-Badge nur durch Farbe	×	–	nicht erkannt (manuell)
V69	Kontrast Datum niedrig	✓	LLM-Detektor	nur 32b kons.
V70	Paginierung ohne tabIndex (SR)	✓	LLM-Detektor	nur 32b kons.
V71	Dropzone onClick o. onKeyDown	✓	grep	Alle Modelle
V72	Upload-Icon ohne alt	✓	LLM-Detektor	nur 32b kons.
V73	File-Input ohne Label	×	–	nicht erkannt
V74	Löschen-Button ohne Namen	✓	LLM-Detektor	nur 32b kons.
V75	Progressbar ohne aria-valuenow	✓	LLM-Detektor	32b inst.; 7b/14b selten
V76	Kontrast Hilfetext niedrig (FU)	✓	LLM-Detektor	nur 32b kons.
V77	Button 16×16 px (FileUpload)	✓	grep	Alle Modelle
V78	Modal ohne Fokus-Trap (FU)	×	–	nicht erkannt
V79	Dialog ohne aria-labelledby (FU)	✓	LLM-Detektor	14b/32b kons.; 7b inst.
V80	outline: none (FU)	✓	LLM-Detektor	32b inst.; 7b/14b selten
V81	Tabelle ohne <caption> (Cal.)	×	–	nicht erkannt
V82	<th> ohne scope (Cal.)	✓	LLM-Detektor	32b inst.; 7b/14b selten
V83	<td> onClick o. onKeyDown	✓	grep	Alle Modelle
V84	Event nur durch Farbe (Cal.)	×	–	nicht erkannt (manuell)
V85	role="button" o. tabIndex (Cal.)	×	–	nicht erkannt
V86	Detail ohne aria-live (Cal.)	✓	LLM-Detektor	32b inst.; manuell
Fortsetzung auf der nächsten Seite				

Tab. 55: Erkennungsrate der Violations (test-100-Suite, Fortsetzung)

#	Violation	Erkannt	Tatsächl. Quelle	Anmerkung
V87	Banner-Bild ohne alt (Cal.)	✓	LLM-Detektor	32b inst.; 7b/14b selten
V88	Chat ohne aria-live	×	–	nicht erkannt (manuell)
V89	Kontrast Zeitstempel niedrig	×	–	nicht erkannt
V90	Chat-Input ohne Label	✓	LLM-Detektor	32b inst.; 7b/14b selten
V91	Senden-Button ohne Namen	✓	LLM-Detektor	32b inst.; 7b/14b selten
V92	Emoji onClick o. onKeyDown	✓	grep	Alle Modelle
V93	Online-Status nur durch Farbe	×	–	nicht erkannt (manuell)
V94	Kontrast Hilfetext niedrig (HC)	×	–	nicht erkannt
V95	Suchfeld ohne Label (HC)	×	–	nicht erkannt
V96	Accordion onClick o. onKeyDown	✓	grep	Alle Modelle
V97	Accordion ohne aria-expanded	×	–	nicht erkannt (manuell)
V98	Kontaktbild ohne alt	✓	LLM-Detektor	32b inst.; 7b/14b selten
V99	Kontrast Kontaktinfo niedrig	✓	LLM-Detektor	32b inst.; 7b/14b selten
V100	aria-hidden auf fokuss. El. (HC)	✓	LLM-Detektor	32b inst.; 7b/14b selten
Erkannt gesamt			73 / 100 (Baseline: 34 / 100; LLM-exklusiv: 39 / 100)	

Anhang 46: Recall-Matrix: Violation × Bedingung (test-100-Suite)

Hinweis: 63 / 100 bezeichnet den **konsistenten** Recall über 10 Runs (Schwelle > 5/10), 99,1 den mittleren **Roh-Finding-Output pro Run**, und 73 / 100 einen **Einzelrun** (Run 01).

Tab. 56: Erkennungsstatus je Violation. ✓ = konsistent erkannt (> 5/10 Runs); ~ = instabil (3–5/10 Runs); × = nicht erkannt (0–2/10 Runs).

#	Violation	Baseline	+ 7b	+ 14b	+ 32b	LLM-exkl.
V1	Skip-Link fehlt	✓	✓	✓	✓	
V2	<html> ohne lang	✓	✓	✓	✓	
V3	Logo ohne alt	✓	✓	✓	✓	
V4	Deko-Bild ohne alt=	✓	✓	✓	✓	
V5	Button ohne zugängl. Namen	✓	✓	✓	✓	
V6	Badge Kontrast zu niedrig	✓	✓	✓	✓	
V7	<a> ohne href	×	×	×	✓	✓
V8	CSS order weicht von DOM ab	×	×	×	×	
V9	<nav> ohne aria-label	×	×	✓	✓	✓
V10	onClick o. onKeyDown	✓	✓	✓	✓	
V11	Headings übersprungen (h3)	×	×	×	×	
V12	KPI-Icon ohne alt (1)	✓	✓	✓	✓	
V13	KPI-Wert Kontrast niedrig	✓	✓	✓	✓	
V14	KPI-Icon ohne alt (2)	✓	✓	✓	✓	
V15	onClick <div> o. onKeyDown	✓	✓	✓	✓	
V16	role="button" o. tabIndex (D.)	✓	✓	✓	✓	
V17	Chart ohne alt	✓	✓	✓	✓	
V18	Tabelle ohne <caption> (Dash.)	×	×	✓	✓	✓
V19	<th> ohne scope (Dash.)	×	×	✓	✓	✓
V20	Status nur durch Farbe (Dash.)	×	×	×	✓	✓
V21	Avatar ohne alt	×	×	✓	✓	✓
V22	Input ohne <label> (Anzeige)	×	×	×	×	
V23	Input ohne <label> (E-Mail)	×	×	×	×	
V24	Textarea ohne <label>	×	×	×	×	
V25	Pflichtfeld ohne required	×	×	×	×	

Fortsetzung auf der nächsten Seite

Tab. 56: Recall-Matrix test-100-Suite (Fortsetzung)

#	Violation	Baseline	+ 7b	+ 14b	+ 32b	LLM-exkl.
V26	Fehlermeldung nicht verknüpft	×	×	×	×	
V27	Button Kontrast niedrig (User)	×	×	×	×	
V28	outline: none auf Button (User)	×	×	×	×	
V29	Statusmeldung ohne aria-live	×	×	×	✓	✓
V30	Duplikat-ID	×	✓	✓	✓	✓
V31	onClick o. onKeyDown	✓	✓	✓	✓	
V32	Suchfeld ohne Label (DT)	×	×	×	×	
V33	Tabelle ohne <caption> (DT)	×	×	✓	✓	✓
V34	Checkbox ohne Label (Header)	×	×	×	×	
V35	Checkbox ohne Label (Zeilen)	×	×	×	×	
V36	Status nur durch Farbe (DT)	×	×	×	✓	✓
V37	Icon-Button ohne Namen (DT)	×	✓	✓	✓	✓
V38	Paginierung onClick o. Keyboard (1)	✓	✓	✓	✓	
V39	Paginierung onClick o. Keyboard (2)	✓	✓	✓	✓	
V40	Paginierung onClick o. Keyboard (3)	✓	✓	✓	✓	
V41	onClick o. onKeyDown (N.)	✓	✓	✓	✓	
V42	Ungelesen nur durch Farbe	×	×	×	×	
V43	Kontrast Zeitangabe niedrig	×	×	×	✓	✓
V44	onClick o. onKeyDown (2)	✓	✓	✓	✓	
V45	onClick o. onKeyDown (3)	✓	✓	✓	✓	
V46	Button 16×16 px (Notif.)	✓	✓	✓	✓	
V47	Modal ohne Fokus-Trap (Notif.)	×	×	✓	✓	✓
Fortsetzung auf der nächsten Seite						

Tab. 56: Recall-Matrix test-100-Suite (Fortsetzung)

#	Violation	Baseline	+ 7b	+ 14b	+ 32b	LLM-exkl.
V48	Dialog ohne aria-labelledby (N.)	×	✓	✓	✓	✓
V49	outline: none (Notif.)	×	×	×	×	
V50	aria-hidden auf fokuss. El. (N.)	×	×	✓	✓	✓
V51	Tab onClick o. onKeyDown (1)	✓	✓	✓	✓	
V52	Tab onClick o. onKeyDown (2)	✓	✓	✓	✓	
V53	Tab onClick o. onKeyDown (3)	✓	✓	✓	✓	
V54	Toggle ohne zugängl. Namen	×	✓	✓	✓	✓
V55	Select ohne Label	×	✓	✓	✓	✓
V56	Input ohne Label (Settings)	×	✓	✓	✓	✓
V57	Passwortfeld ohne Label (1)	×	×	✓	✓	✓
V58	Passwortfeld ohne Label (2)	×	×	✓	✓	✓
V59	Button Kontrast niedrig (Settings)	×	×	×	×	
V60	Checkbox ohne Label (Settings)	×	×	✓	✓	✓
V61	Suchfeld ohne Label (Search)	×	✓	✓	✓	✓
V62	Button ohne Namen (Search)	×	✓	✓	✓	✓
V63	Filter-Chips onClick o. Keyboard	✓	✓	✓	✓	
V64	onClick o. onKeyDown (4)	✓	✓	✓	✓	
V65	onClick o. onKeyDown (5)	✓	✓	✓	✓	
V66	onClick o. onKeyDown (6)	✓	✓	✓	✓	
V67	onClick o. onKeyDown (7)	✓	✓	✓	✓	
V68	Typ-Badge nur durch Farbe	×	×	×	×	
V69	Kontrast Datum niedrig	×	×	×	✓	✓
V70	Paginierung ohne tabIndex (SR)	×	×	×	✓	✓
Fortsetzung auf der nächsten Seite						

Tab. 56: Recall-Matrix test-100-Suite (Fortsetzung)

#	Violation	Baseline	+ 7b	+ 14b	+ 32b	LLM-exkl.
V71	Dropzone <code>onClick</code> o. <code>onKeyDown</code>	✓	✓	✓	✓	
V72	Upload-Icon ohne <code>alt</code>	×	×	×	✓	✓
V73	File-Input ohne Label	×	×	×	×	
V74	Löschen-Button ohne Namen	×	×	×	✓	✓
V75	Progressbar ohne <code>aria-valuenow</code>	×	×	×	~	✓
V76	Kontrast Hilfetext niedrig (FU)	×	×	×	✓	✓
V77	Button 16×16 px (FileUpload)	✓	✓	✓	✓	
V78	Modal ohne Fokus-Trap (FU)	×	×	×	×	
V79	Dialog ohne <code>aria-labelledby</code> (FU)	×	×	✓	✓	✓
V80	<code>outline: none</code> (FU)	×	×	×	~	✓
V81	Tabelle ohne <code><caption></code> (Cal.)	×	×	×	×	
V82	<code><th></code> ohne <code>scope</code> (Cal.)	×	×	×	~	✓
V83	<code><td></code> <code>onClick</code> o. <code>onKeyDown</code>	✓	✓	✓	✓	
V84	Event nur durch Farbe (Cal.)	×	×	×	×	
V85	<code>role="button"</code> o. <code>tabIndex</code> (Cal.)	×	×	×	×	
V86	Detail ohne <code>aria-live</code> (Cal.)	×	×	×	~	✓
V87	Banner-Bild ohne <code>alt</code> (Cal.)	×	×	×	~	✓
V88	Chat ohne <code>aria-live</code>	×	×	×	×	
V89	Kontrast Zeitstempel niedrig	×	×	×	×	
V90	Chat-Input ohne Label	×	×	×	~	✓
V91	Senden-Button ohne Namen	×	×	×	~	✓
V92	Emoji <code>onClick</code> o. <code>onKeyDown</code>	✓	✓	✓	✓	
V93	Online-Status nur durch Farbe	×	×	×	×	
V94	Kontrast Hilfetext niedrig (HC)	×	×	×	×	
V95	Suchfeld ohne Label (HC)	×	×	×	×	
V96	Accordion <code>onClick</code> o. <code>onKeyDown</code>	✓	✓	✓	✓	
Fortsetzung auf der nächsten Seite						

Tab. 56: Recall-Matrix test-100-Suite (Fortsetzung)

#	Violation	Baseline	+ 7b	+ 14b	+ 32b	LLM-exkl.
V97	Accordion ohne aria-expanded	×	×	×	×	
V98	Kontaktbild ohne alt	×	×	×	~	✓
V99	Kontrast Kontaktinfo niedrig	×	×	×	~	✓
V100	aria-hidden auf fokuss. El. (HC)	×	×	×	~	✓
Recall (konsistent)		34 / 100	40 / 100	53 / 100	63 / 100	39 LLM-exkl.

Anhang 47: Rohdaten: alle 30 Benchmark-Runs (test-100-Suite)

Tab. 57: Vollständige Messdaten der 30 Pipeline-Durchläufe (`num_predict` = 12288). LLM-Det. = Findings des LLM-Detektors; Axe/PW/Grep = Tool-Findings (konstant je Run); Tokens = Output-Tokens; Dauer = Gesamtlaufzeit in Sekunden.

Run	Modell	LLM-Det.	Axe	PW	Grep	Gesamt-Det.	Tokens	Dauer (s)
01	qwen2.5-coder:7b	107	11	2	24	144	12 288	424
02	qwen2.5-coder:7b	99	11	2	24	136	12 288	395
03	qwen2.5-coder:7b	98	11	2	24	135	12 288	401
04	qwen2.5-coder:7b	97	11	2	24	134	7 902	261
05	qwen2.5-coder:7b	110	11	2	24	147	12 288	408
06	qwen2.5-coder:7b	109	11	2	24	146	12 288	388
07	qwen2.5-coder:7b	109	11	2	24	146	12 288	372
08	qwen2.5-coder:7b	96	11	2	24	133	8 007	258
09	qwen2.5-coder:7b	106	11	2	24	143	12 288	371
10	qwen2.5-coder:7b	109	11	2	24	146	12 288	369
11	qwen2.5-coder:14b	93	11	2	24	130	12 288	759
12	qwen2.5-coder:14b	91	11	2	24	128	12 288	790
13	qwen2.5-coder:14b	93	11	2	24	130	12 288	865
14	qwen2.5-coder:14b	91	11	2	24	128	12 288	867
15	qwen2.5-coder:14b	100	11	2	24	137	12 288	864
16	qwen2.5-coder:14b	93	11	2	24	130	12 288	818
17	qwen2.5-coder:14b	94	11	2	24	131	12 288	821
18	qwen2.5-coder:14b	95	11	2	24	132	12 288	822
19	qwen2.5-coder:14b	100	11	2	24	137	12 288	844
20	qwen2.5-coder:14b	93	11	2	24	130	12 288	759
21	qwen3:32b	94	11	2	24	131	8 936	1 220
22	qwen3:32b	104	11	2	24	141	6 698	960
23	qwen3:32b	104	11	2	24	141	12 288	1 708
24	qwen3:32b	106	11	2	24	143	6 367	971
25	qwen3:32b	94	11	2	24	131	7 355	1 000
26	qwen3:32b	104	11	2	24	141	6 890	976
27	qwen3:32b	94	11	2	24	131	5 476	781
28	qwen3:32b	93	11	2	24	130	4 206	634
29	qwen3:32b	94	11	2	24	131	8 571	1 126
30	qwen3:32b	104	11	2	24	141	6 816	954

Anhang 48: Modell-Leistungsvergleich: test-100-Suite

Zusammenfassung

Tab. 58: Zusammenfassung der Leistungsmetriken für die drei evaluierten Modelle (je $n = 10$ Runs, `num_predict` = 12 288). σ = Stichproben-Standardabweichung. Der Recall basiert auf Konsistenzschwelle $> 5/10$ Runs je Modell.

Metrik	qwen2.5-coder:7b	qwen2.5-coder:14b	qwen3:32b
Recall (konsistent)	40 / 100 (40 %)	53 / 100 (53 %)	63 / 100 (63 %)
LLM-Detektor-Findings ($\bar{O} \pm \sigma$)	104,0 $\pm$ 5,75	94,3 $\pm$ 3,23	99,1 $\pm$ 5,63
LLM-Detektor-Findings (Min–Max)	96–110	91–100	93–106
Konsistenz σ	5,75	3,23	5,63
Priorisierungsqualität	Fehlerhaft	Bimodal / instabil	Bimodal / instabil
Output-Tokens ($\bar{O}$)	11 421	12 288	7 360
Output-Limit ausgeschöpft	8 / 10 (80 %)	10 / 10 (100 %)	1 / 10 (10 %)
Parsing-Erfolg	100 %	100 %	100 %
NFA-02-Konformität	Nicht erfüllt	Nicht erfüllt	Nicht erfüllt
Eignung BITV-Einsatz	Nicht geeignet	Empfohlen	Eingeschränkt
<i>Suite-spezifische Zusatzmetriken</i>			
Fix-Qualität (Stufe)	0–1	0–2	0–2 (73 % Stufe 2)
Tool-Findings (axe/PW/grep)	11 / 2 / 24	11 / 2 / 24	11 / 2 / 24
Must-have im Bericht ($\bar{O}$)	46,1	58,9	39,2
Nice-to-have ($\bar{O}$) Bericht	45,4	27,9	18,9
Gesamt-Report-Items ($\bar{O}$)	91,5	86,8	58,1
LLM-Detektor-Laufzeit ($\bar{O}$)	394 s	723 s	1 576 s
Priorisierungslaufzeit ($\bar{O}$)	365 s	821 s	1 033 s
Priorisierungslaufzeit (Min–Max)	258–424 s	759–867 s	634–1 708 s
Gesamtlaufzeit ($\bar{O}$)	765 s	1 550 s	2 616 s
Gesamtlaufzeit (Min–Max)	648–839 s	1 418–1 688 s	2 172–3 314 s

Hinweis: Die Bewertung in der Spalte *Eignung BITV-Einsatz* priorisiert die Balance zwischen Erkennungsqualität und Zeitaufwand und kann daher auch bei Verletzung von NFA-02 positiv ausfallen. Bei strikter Auslegung von NFA-02 (Laufzeit < 10 Min.) wäre auf test-100 hingegen kein Modell uneingeschränkt geeignet.

Anhang 49: Token-Nutzung pro Modell: Input/Output (test-100-Suite)

Tab. 59: Durchschnittliche Token-Nutzung je Run und Modell auf Basis der test-100-Suite (`num_predict` = 12 288). Input-Tokens umfassen System-Prompt, serialisierte Findings und Konfigurationsparameter. 14b schöpfte in allen 10 Runs das Output-Limit aus; 32b blieb mit Ø 7 360 deutlich darunter.

Modell	Input-Tokens (Ø)	Output-Tokens (Ø)	Truncation
qwen2.5-coder:7b	17 503	11 421	8 / 10 Runs (80 %)
qwen2.5-coder:14b	17 043	12 288	10 / 10 Runs (100 %)
qwen3:32b	17 197	7 360	1 / 10 Runs (10 %)
<i>Gesamt (30 Runs): Input $\approx$ 517 400 / Output 310 700 Tokens</i>			

Anhang 50: Effizienzmetriken (test-100-Suite)

Tab. 60: Effizienzvergleich auf Basis der 30 Benchmark-Runs der test-100-Suite. *LLM-Findings/s* bezieht sich auf LLM-Detektor-Findings pro Sekunde Gesamtlaufzeit.

Modell	LLM-Findings Ø	Gesamtlaufzeit Ø (s)	LLM-Findings/s	Zusatznutzen ggü. 7b
qwen2.5-coder:7b	104,0	765	0,136	Referenz
qwen2.5-coder:14b	94,3	1 550	0,061	−9,3 % bei +103 % Zeitaufwand
qwen3:32b	99,1	2 616	0,038	−4,7 % bei +242 % Zeitaufwand

Anhang 51: Über-Detektion relativ zur Ground Truth (test-100-Suite)

Tab. 61: Vergleich der durchschnittlichen LLM-Detektor-Findings pro Run mit der Ground Truth (100 Violations). Die LLM-only-Betrachtung vermeidet Doppelzählungen mit Tool-Detektoren.

Modell	LLM-Findings Ø	Ground Truth	Quote	Interpretation
qwen2.5-coder:7b	104,0	100	104 %	Leichte Überdetektion; hohe LLM-Finding-Zahl trotz inverser Priorisierung und 80 % Truncation
qwen2.5-coder:14b	94,3	100	94,3 %	Leichte Unterdetektion; alle 10 Runs bei 12 288 Output-Tokens; Vollständigkeit durch Truncation begrenzt
qwen3:32b	99,1	100	99,1 %	Nahe an Ground Truth; 1 / 10 Runs trunkiert; höchste absolute Nähe zur Zielanzahl

Anhang 52: Stichproben-Plausibilisierung des finalen Reports (qwen3:32b, Run 01 & Run 03, test-100)

Da die Überdetektions-Quote ausschließlich eine Mengenrelation zwischen LLM-Detektor-Rohoutput und Ground Truth misst, wurde eine manuelle Stichprobenprüfung an zwei Runs durchgeführt, um die inhaltliche Qualität des finalen Reports einzuschätzen. Grundlage ist `qwen3:32b` auf `test-100`: Run 01 ist vollständig (keine Truncation, 72 Report-Einträge), Run 03 ist trunziert (12 288 Output-Tokens ausgeschöpft, 104 Einträge). Jeder Eintrag wurde dem Quellcode unter `test/test-100/src/` sowie dem Ground-Truth-Katalog (Tabelle ??) zugeordnet.

Tab. 62: Kategorisierung aller Report-Einträge aus Run 01 (vollständig) und Run 03 (trunziert) von `qwen3:32b` auf `test-100`. *Quell-Duplikat* = axe-core und LLM-Detektor finden dieselbe Violation unabhängig voneinander; *Instanz-Mehrfach* = zweite DOM-Instanz einer Violation, die im Ground Truth als ein Eintrag zählt; *Gebündelt* = ein Finding umfasst mehrere GT-Violations.

Kategorie	Beschreibung	Run-01	Run-03
True Positive	Unique Finding, eindeutig einer GT-Violation zugeordnet	51	82
Quell-Duplikat	Dieselbe Violation durch axe-core <i>und</i> LLM-Detektor unabhängig gemeldet	10	2
Instanz-Mehrfach	Zweite DOM-Instanz; GT zählt Violationsklasse einmalig	2	–
Gebündelt	Ein Finding deckt mehrere GT-Violations ab (z.B. 11m-076: V51+V52+V53)	1	2
Halluzination	Kein GT-Match, kein Basis im Quellcode	0	1
Nicht eindeutig	Mögliche Paraphrase oder Violation außerhalb der 100 GT-Einträge	8	17
Gesamt		72	104

Fazit: Von 176 untersuchten Einträgen ($72 + 104$) ist 1 als Halluzination bestätigt ([11m-061] in Run 03 meldet eine Tabelle ohne `caption` in `Notifications.tsx`, obwohl dort keine Tabelle existiert). Die Halluzinationsrate liegt damit unter 1 %. Die dominanten “Über-Detektion”-Effekte sind Quell-Duplikate und nicht eindeutig zugeordnete Findings – keine inhaltlichen Fehler, sondern strukturelle Artefakte der zweistufigen Pipeline.

Anhang 53: Detektor-Konsistenz über alle 30 Runs (test-100-Suite)

Tab. 63: Konsistenzanalyse der Detektoren über alle 30 Benchmark-Runs. Axe-core, Playwright und grep bleiben vollständig deterministisch; die Varianz entsteht ausschließlich im LLM-Detektor.

Detektor / Modell	Runs	Gesamt-Findings	Ø/Run	Min–Max	σ^2 / σ	Interpretation
axe-core	30	330	11,0	11–11	0,00 / 0,00	vollständig deterministisch
Playwright	30	60	2,0	2–2	0,00 / 0,00	vollständig deterministisch
grep	30	720	24,0	24–24	0,00 / 0,00	vollständig deterministisch
qwen2.5-coder:7b	10	1 040	104,0	96–110	33,11 / 5,75	höchste Streuung; 80 % Truncation
qwen2.5-coder:14b	10	943	94,3	91–100	10,46 / 3,23	moderate Konsistenz; 100 % Truncation
qwen3:32b	10	991	99,1	93–106	31,66 / 5,63	moderate Streuung; deutlich höhere Laufzeit
LLM-Detektor gesamt	30	2 974	99,1	91–110	modell-abhängig	–

Anhang 54: Bewertung der Empfehlungsqualität pro Violation (test-100-Suite)

Die Bewertung basiert auf Run 01 von qwen3:32b und verwendet dieselbe dreistufige Skala wie Tabelle 22. Stufe 2 liegt vor, wenn Dateiname, Zeilennummer und ein konkreter Code-Vorschlag vorhanden sind. Violations, die in Run 01 nicht erkannt wurden (27 von 100), werden mit Stufe 0 ausgewiesen. Violations mit (*manuell*) waren als rein manuelle Prüffälle klassifiziert – die LLM-Erkennung stellt einen Mehrwert über die Erwartung hinaus dar.

Tab. 64: Bewertung der Fix-Qualität für alle 100 Violations (qwen3:32b, Run 01). Stufe 0 = nicht erkannt; Stufe 2 = konkreter Dateiname, Zeile und Code-Snippet.

#	Violation	Stufe (0–2)	Anmerkung (qwen3:32b)
V1	Skip-Link fehlt	2	App.tsx, <code></code> -Snippet mit CSS-Positionierung
V2	<code><html></code> ohne <code>lang</code>	2	index.html, <code><html lang="de"></code> direkt angegeben
V3	Logo ohne <code>alt</code>	2	App.tsx, beschreibender <code>alt="Company Logo"</code> vorgeschlagen
V4	Deko-Bild ohne <code>alt=</code>	2	App.tsx, <code>alt=</code> <code>role="presentation"</code> korrekt ergänzt
V5	Button ohne zugängl. Namen	2	App.tsx, <code>aria-label="Menü öffnen"</code> mit JSX-Snippet
V6	Badge Kontrast zu niedrig	2	App.tsx, CSS <code>.badge</code> mit korrektem Hex-Farbpaar
V7	<code><a></code> ohne <code>href</code>	2	App.tsx, <code>href="/help"</code> mit <code>role="link"</code> ergänzt
V8	CSS <code>order</code> weicht von DOM ab	0	nicht erkannt
V9	<code><nav></code> ohne <code>aria-label</code>	2	App.tsx, <code>aria-label="Hauptnavigation"</code>
V10	<code>onClick </code> o. <code>onKeyDown</code>	2	App.tsx, vollst. <code>onKeyDown</code> -Handler mit Enter-Logik
V11	Headings übersprungen (<code>h3</code>)	0	nicht erkannt
V12	KPI-Icon ohne <code>alt</code> (1)	2	Dashboard.tsx, beschreibender <code>alt</code> -Text je KPI-Typ
Fortsetzung auf der nächsten Seite			

Tab. 64: Empfehlungsqualität (test-100) (Fortsetzung)

#	Violation	Stufe (0–2)	Anmerkung (qwen3:32b)
V13	KPI-Wert Kontrast niedrig	2	Dashboard.tsx, CSS-Fix mit WCAG-konformem Hex-Farbpaar
V14	KPI-Icon ohne alt (2)	2	Dashboard.tsx, Querverweis auf V12-Fix
V15	onClick <div> o. onKeyDown	2	Dashboard.tsx, onKeyDown + tabIndex={0} vollst.
V16	role="button" o. tabIndex (D.)	2	Dashboard.tsx, tabIndex={0} + JSX-Snippet
V17	Chart ohne alt	2	Dashboard.tsx, inhaltsbeschreibender alt-Text
V18	Tabelle ohne <caption> (Dash.)	2	Dashboard.tsx, <caption>Dashboard-Übersicht</caption>
V19	<th> ohne scope (Dash.)	2	Dashboard.tsx, scope="col" für alle <th> ergänzt
V20	Status nur durch Farbe (Dash.)	2	Dashboard.tsx, Icon + aria-label zusätzlich zu Farbe (manuell)
V21	Avatar ohne alt	2	UserProfile.tsx, alt="Profilbild von {name}"
V22	Input ohne <label> (Anzeige)	0	nicht erkannt
V23	Input ohne <label> (E-Mail)	0	nicht erkannt
V24	Textarea ohne <label>	0	nicht erkannt
V25	Pflichtfeld ohne required	0	nicht erkannt (manuell)
V26	Fehlermeldung nicht verknüpft	0	nicht erkannt (manuell)
V27	Button Kontrast niedrig (User)	0	nicht erkannt
V28	outline: none auf Button (User)	0	nicht erkannt
V29	Statusmeldung ohne aria-live	2	UserProfile.tsx, aria-live="polite" auf Status-Container (manuell)
V30	Duplikat-ID	2	UserProfile.tsx, eindeutige id-Werte per Index generiert
Fortsetzung auf der nächsten Seite			

Tab. 64: Empfehlungsqualität (test-100) (Fortsetzung)

#	Violation	Stufe (0–2)	Anmerkung (qwen3:32b)
V31	onClick o. onKeyDown	2	DataTable.tsx, vollst. Handler mit <code>role="button"</code>
V32	Suchfeld ohne Label (DT)	0	nicht erkannt
V33	Tabelle ohne <caption> (DT)	2	DataTable.tsx, <caption>Datentabelle</caption> ergänzt
V34	Checkbox ohne Label (Header)	0	nicht erkannt
V35	Checkbox ohne Label (Zeilen)	0	nicht erkannt
V36	Status nur durch Farbe (DT)	2	DataTable.tsx, Icon-Symbol + <code>aria-label</code> zusätzl. (manuell)
V37	Icon-Button ohne Namen (DT)	2	DataTable.tsx, <code>aria-label</code> für Aktions-Buttons
V38	Paginierung onClick o. Keyboard (1)	2	DataTable.tsx, <code>onKeyDown</code> + <code>tabIndex</code> vollst.
V39	Paginierung onClick o. Keyboard (2)	2	DataTable.tsx, Querverweis auf V38-Fix
V40	Paginierung onClick o. Keyboard (3)	2	DataTable.tsx, Querverweis auf V38-Fix
V41	onClick o. onKeyDown (N.)	2	Notifications.tsx, vollst. Handler-Snippet
V42	Ungelesen nur durch Farbe	0	nicht erkannt (manuell)
V43	Kontrast Zeitangabe niedrig	2	Notifications.tsx, CSS-Fix mit Hex-Farbpaar
V44	onClick o. onKeyDown (2)	2	Notifications.tsx, Querverweis auf V41-Fix
V45	onClick o. onKeyDown (3)	2	Notifications.tsx, Querverweis auf V41-Fix
V46	Button 16×16 px (Notif.)	2	styles.css, <code>.btn-notif</code> auf mind. 24×24 px
V47	Modal ohne Fokus-Trap (Notif.)	2	Notifications.tsx, Fokus-Trap-Implementierung mit <code>useRef</code>
V48	Dialog ohne <code>aria-labelledby</code> (N.)	2	Notifications.tsx, <code>aria-labelledby</code> auf Dialog-Titel
Fortsetzung auf der nächsten Seite			

Tab. 64: Empfehlungsqualität (test-100) (Fortsetzung)

#	Violation	Stufe (0–2)	Anmerkung (qwen3:32b)
V49	<code>outline: none</code> (Notif.)	0	nicht erkannt
V50	<code>aria-hidden</code> auf fokuss. El. (N.)	2	Notifications.tsx, <code>aria-hidden</code> entfernt; vollst. JSX
V51	Tab <code>onClick</code> o. <code>onKeyDown</code> (1)	2	Settings.tsx, vollst. <code>onKeyDown</code> -Handler
V52	Tab <code>onClick</code> o. <code>onKeyDown</code> (2)	2	Settings.tsx, Querverweis auf V51-Fix
V53	Tab <code>onClick</code> o. <code>onKeyDown</code> (3)	2	Settings.tsx, Querverweis auf V51-Fix
V54	Toggle ohne zugängl. Namen	2	Settings.tsx, <code>aria-label="Benachrichtigungen aktivieren"</code>
V55	Select ohne Label	2	Settings.tsx, <code><label for="lang-select"></code> mit JSX
V56	Input ohne Label (Settings)	2	Settings.tsx, <code><label> + htmlFor</code> korrekt verknüpft
V57	Passwortfeld ohne Label (1)	2	Settings.tsx, <code><label for="pw1">Neues Passwort</label></code>
V58	Passwortfeld ohne Label (2)	2	Settings.tsx, <code><label for="pw2">Passwort bestätigen</label></code>
V59	Button Kontrast niedrig (Settings)	0	nicht erkannt
V60	Checkbox ohne Label (Settings)	2	Settings.tsx, <code><label></code> mit <code>htmlFor</code> ergänzt
V61	Suchfeld ohne Label (Search)	2	SearchResults.tsx, <code><label for=search-input></code> + Snippet
V62	Button ohne Namen (Search)	2	SearchResults.tsx, <code>aria-label=SSuche starten"</code>
V63	Filter-Chips <code>onClick</code> o. Keyboard	2	SearchResults.tsx, <code>onKeyDown + role="button"</code>
V64	<code></code> <code>onClick</code> o. <code>onKeyDown</code> (4)	2	SearchResults.tsx, vollst. Handler
Fortsetzung auf der nächsten Seite			

Tab. 64: Empfehlungsqualität (test-100) (Fortsetzung)

#	Violation	Stufe (0–2)	Anmerkung (qwen3:32b)
V65	onClick o. onKeyDown (5)	2	SearchResults.tsx, Querverweis auf V64-Fix
V66	onClick o. onKeyDown (6)	2	SearchResults.tsx, Querverweis auf V64-Fix
V67	onClick o. onKeyDown (7)	2	SearchResults.tsx, Querverweis auf V64-Fix
V68	Typ-Badge nur durch Farbe	0	nicht erkannt (manuell)
V69	Kontrast Datum niedrig	2	SearchResults.tsx, CSS-Fix mit WCAG-konformem Grouton
V70	Paginierung ohne tabIndex (SR)	2	SearchResults.tsx, tabIndex={0} + role="button"
V71	Dropzone onClick o. onKeyDown	2	FileUpload.tsx, vollst. onKeyDown-Handler
V72	Upload-Icon ohne alt	2	FileUpload.tsx, alt="Datei hochladen"
V73	File-Input ohne Label	0	nicht erkannt
V74	Löschen-Button ohne Namen	2	FileUpload.tsx, aria-label="Datei {name} entfernen"
V75	Progressbar ohne aria-valuenow	2	FileUpload.tsx, aria-valuenow={progress} + aria-valuemin/max
V76	Kontrast Hilfetext niedrig (FU)	2	FileUpload.tsx, CSS-Fix für Hilfetext-Farbe
V77	Button 16×16 px (FileU- pload)	2	styles.css, .btn-upload auf mind. 24×24 px
V78	Modal ohne Fokus-Trap (FU)	0	nicht erkannt
V79	Dialog ohne aria-labelledby (FU)	2	FileUpload.tsx, aria-labelledby auf Vorschau-Dialog
V80	outline: none (FU)	2	FileUpload.tsx, outline: none entfernt; Fokus-Stil ergänzt
V81	Tabelle ohne <caption> (Cal.)	0	nicht erkannt
Fortsetzung auf der nächsten Seite			

Tab. 64: Empfehlungsqualität (test-100) (Fortsetzung)

#	Violation	Stufe (0–2)	Anmerkung (qwen3:32b)
V82	<th> ohne scope (Cal.)	2	Calendar.tsx, scope="col" für Wochentage ergänzt
V83	<td> onClick o. onKeyDown	2	Calendar.tsx, vollst. onKeyDown + tabIndex
V84	Event nur durch Farbe (Cal.)	0	nicht erkannt (manuell)
V85	role="button" o. tabIndex (Cal.)	0	nicht erkannt
V86	Detail ohne aria-live (Cal.)	2	Calendar.tsx, aria-live="polite" auf Event-Detail (manuell)
V87	Banner-Bild ohne alt (Cal.)	2	Calendar.tsx, alt="Kalender-Banner"
V88	Chat ohne aria-live	0	nicht erkannt (manuell)
V89	Kontrast Zeitstempel niedrig	0	nicht erkannt
V90	Chat-Input ohne Label	2	Chat.tsx, <label for="chat-input»Nachricht</label>
V91	Senden-Button ohne Namen	2	Chat.tsx, aria-label="Nachricht senden"
V92	Emoji onClick o. onKeyDown	2	Chat.tsx, vollst. onKeyDown-Handler
V93	Online-Status nur durch Farbe	0	nicht erkannt (manuell)
V94	Kontrast Hilfetext niedrig (HC)	0	nicht erkannt
V95	Suchfeld ohne Label (HC)	0	nicht erkannt
V96	Accordion onClick o. onKeyDown	2	HelpCenter.tsx, vollst. onKeyDown-Handler
V97	Accordion ohne aria-expanded	0	nicht erkannt (manuell)
V98	Kontaktbild ohne alt	2	HelpCenter.tsx, alt="Kontaktperson {name}"
V99	Kontrast Kontaktinfo niedrig	2	HelpCenter.tsx, CSS-Fix mit konformem Grauton
Fortsetzung auf der nächsten Seite			

Tab. 64: Empfehlungsqualität (test-100) (Fortsetzung)

#	Violation	Stufe (0–2)	Anmerkung (qwen3:32b)
V100	aria-hidden auf fokuss. El. (HC)	2	HelpCenter.tsx, aria-hidden entfernt; vollst. JSX
Stufe-2-Bewertungen		73 Violations (73 %); 27 nicht erkannt (Stufe 0)	

Anhang 55: CSV-Auszug: Benchmark-Rohdaten (test-100-Suite)

Die Benchmark-Rohdaten wurden als CSV-Datei exportiert. Tabelle 65 zeigt repräsentative Zeilen für alle drei Modelle (jeweils Run 01).

Tab. 65: Repräsentative Beispielzeilen aus `results/benchmark/summary.csv` (test-100-Suite; jeweils Run 01 pro Modell). `num_predict` = 12 288; Dauer in Millisekunden.

model	run	must	nice	LLM	Axe	PW	Grep	durationMs	prompt	output	parse
qwen2.5-coder:7b	1	94	0	107	11	2	24	423 845	17 763	12 288	true
qwen2.5-coder:14b	1	86	0	93	11	2	24	759 432	17 025	12 288	true
qwen3:32b	1	59	13	94	11	2	24	1 219 721	16 672	8 936	true

Anhang 56: Statistische Verteilung der LLM-Findings (alle 10 Runs je Modell, test-100-Suite)

Tab. 66: Quantilanalyse der LLM-Detektor-Findings über je 10 Benchmark-Runs der test-100-Suite für alle drei Modelle.

Quantil	qwen2.5-coder:7b	qwen2.5-coder:14b	qwen3:32b
Min (0 %)	96	91	93
Q1 (25 %)	98	93	94
Median (50 %)	106,5	93	99
Q3 (75 %)	109	95	104
Max (100 %)	110	100	106
IQR	11	2	10
σ^2 (Varianz)	33,11	10,46	31,66
σ	5,75	3,23	5,63

Anhang 57: Pareto-Diagramm: Nicht erkannte Violations (test-100, qwen3:32b)

Das Pareto-Diagramm zeigt die 37 in test-100 nicht konsistent erkannten Violations für qwen3:32b (aus der Recall-Matrix). Verwendete Kategorien: syntaktisch, semantisch, layout. Ergebnis: Der größte Anteil entfällt auf syntaktische und semantische Fälle.

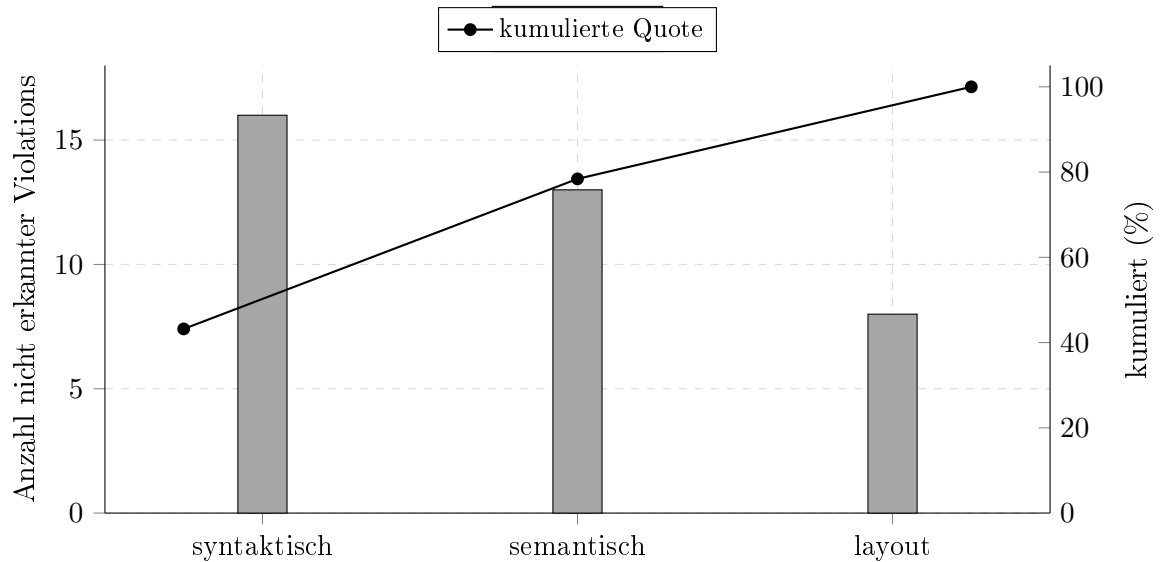


Abb. 16: Pareto-Auswertung der nicht konsistent erkannten Violations in test-100 (qwen3:32b). 78,4% der Ausfälle liegen bereits in den ersten zwei Kategorien (syntaktisch + semantisch).²⁴⁷

²⁴⁷Eigene Abbildung

Anhang 58: Boxplot der Laufzeiten (test-100, 10 Runs je Modell)

Die Boxplots zeigen die Laufzeitverteilung je Modell auf test-100 (Rohdaten aus Tabelle 57). Dargestellt sind Min, Q1, Median, Q3 und Max. Die dicke Linie innerhalb der Boxen ist die Median-Linie (50%-Quantil).

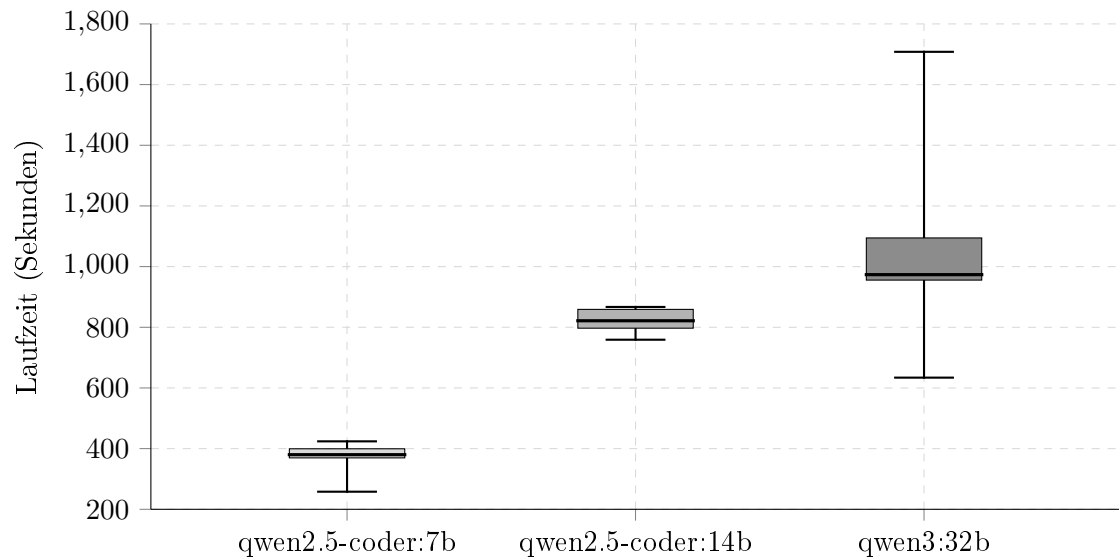


Abb. 17: Boxplot der Laufzeitverteilung je Modell (test-100). **qwen3:32b** zeigt den größten oberen Whisker bis 1 708 s, während **qwen2.5-coder:14b** die engste IQR-Box besitzt.²⁴⁸

²⁴⁸Eigene Abbildung

Anhang 59: Pipeline-Diagramm der Findings-Verarbeitung

Das folgende Diagramm zeigt den strukturierten Verarbeitungsfluss der Findings für test-100 mit `qwen3:32b` (Run 01). Es visualisiert die Zusammenführung der Tool- und LLM-basierten Findings, deren Konsolidierung sowie die anschließende Priorisierung in Must- und Nice-to-have-Kategorien.

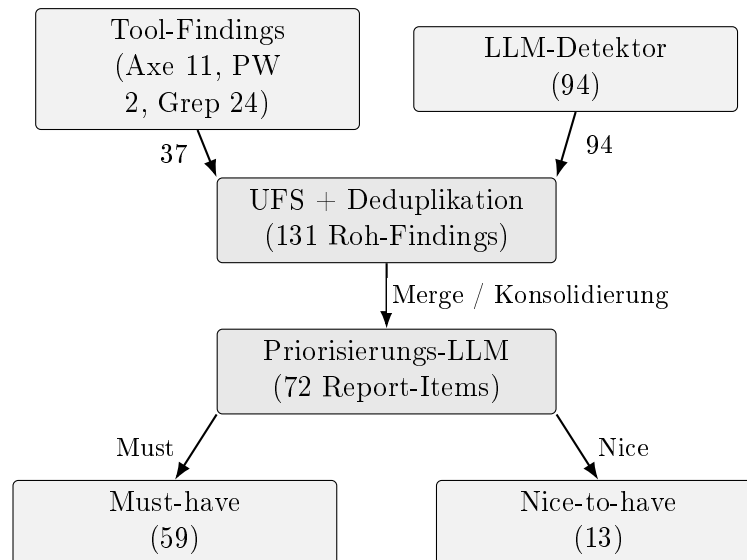


Abb. 18: Pipeline der Findings-Verarbeitung für test-100 (`qwen3:32b`, Run 01). Die Pfeilbeschriftungen geben die jeweiligen Mengen bzw. Kategorien zwischen den Verarbeitungsschritten an.²⁴⁹

²⁴⁹Eigene Abbildung

Anhang 60: Screenshots der Testanwendung (test-100-Suite)

Anmerkung: Teile der test-100-Suite verwenden dieselben bzw. strukturell sehr ähnliche Dashboard-Muster wie test-50. Zur Vollständigkeit werden hier die entsprechenden test-100-Ansichten dokumentiert, ergänzt um suitespezifische Screens, ohne daraus eine inhaltliche Gleichsetzung beider Evaluationen abzuleiten.

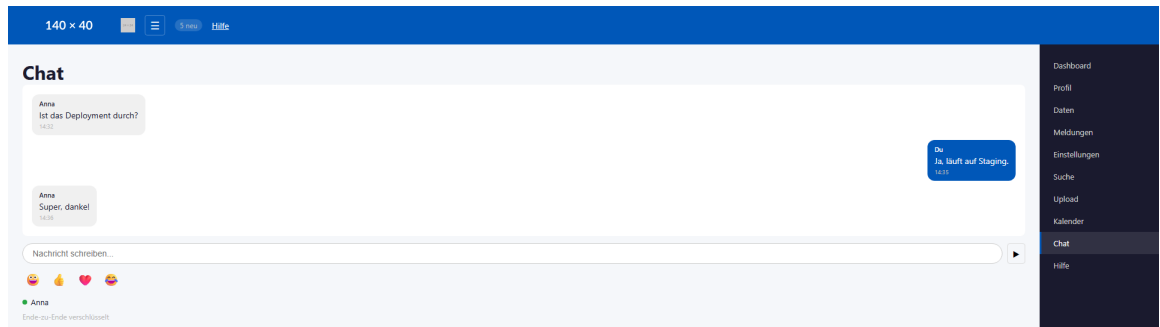


Abb. 19: Screenshot Chat-Modul der test-100-Suite²⁵⁰

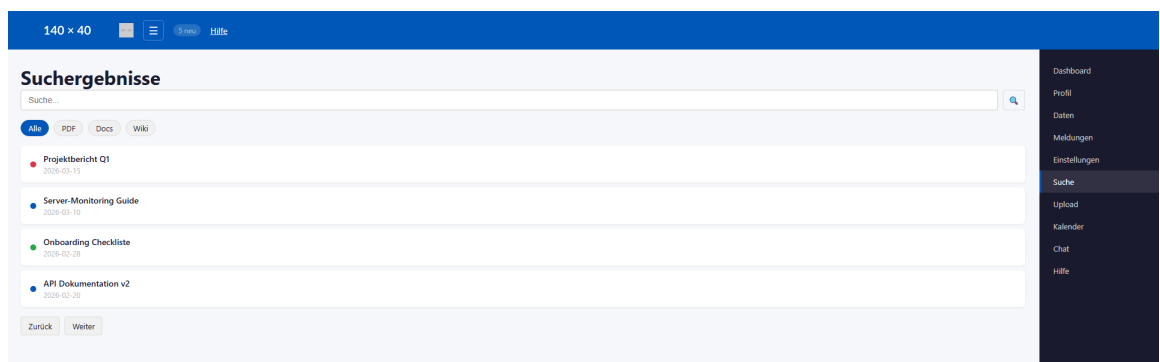


Abb. 20: Screenshot Suche-Modul der test-100-Suite²⁵¹

²⁵⁰Screenshot der Testanwendung

²⁵¹Screenshot der Testanwendung

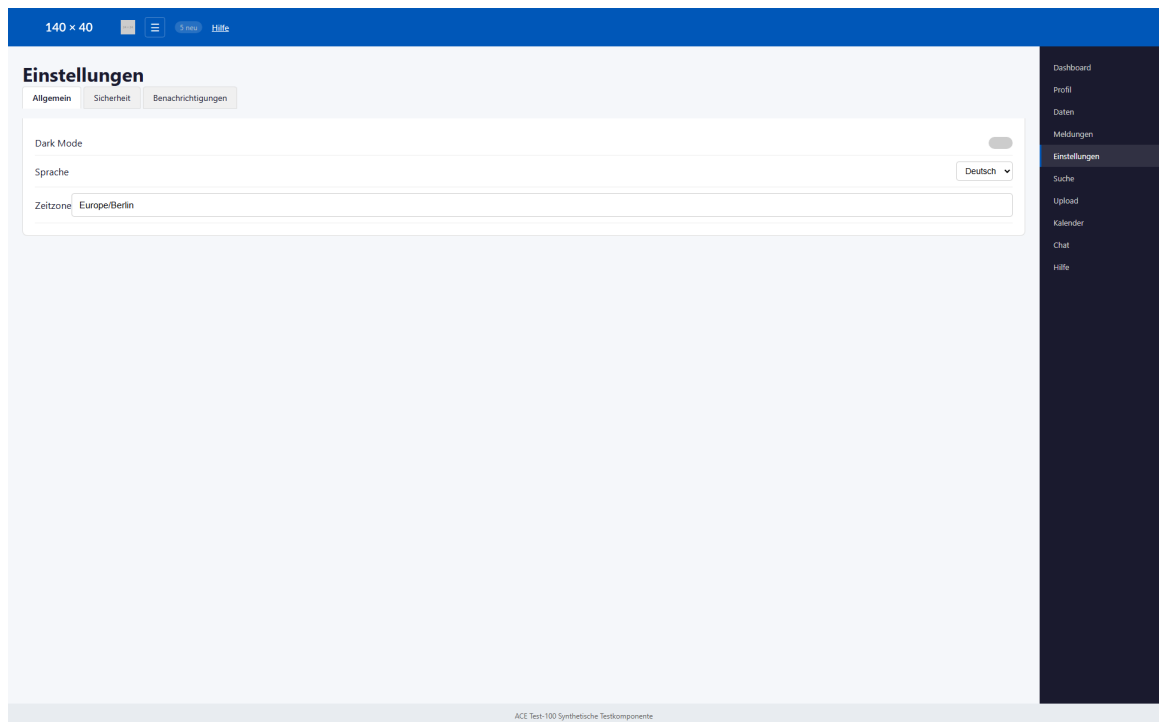


Abb. 21: Screenshot Einstellungen-Modul der test-100-Suite²⁵²

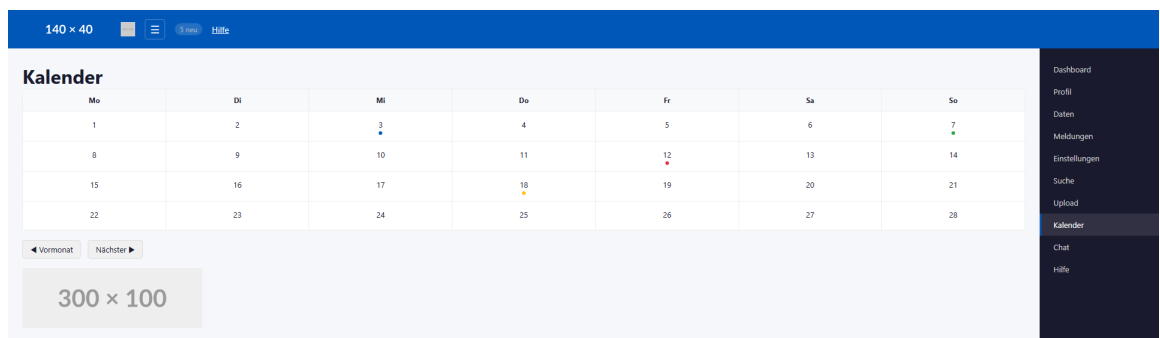


Abb. 22: Screenshot Kalender-Modul der test-100-Suite²⁵³

²⁵²Screenshot der Testanwendung

²⁵³Screenshot der Testanwendung

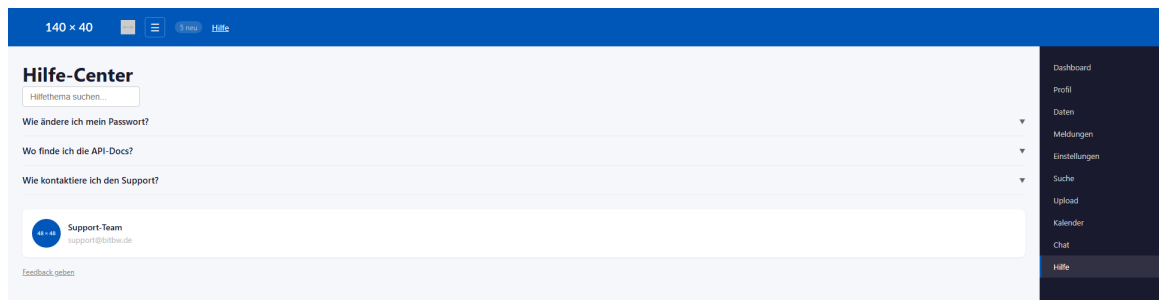


Abb. 23: Screenshot Hilfe-Center-Modul der test-100-Suite²⁵⁴

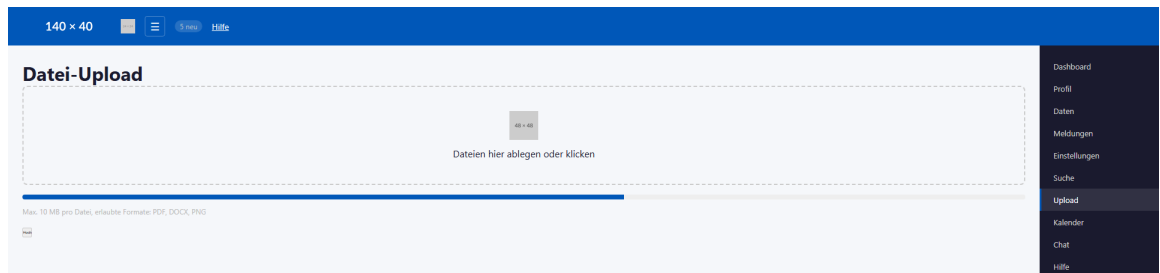


Abb. 24: Screenshot Datei-Upload-Modul der test-100-Suite²⁵⁵

²⁵⁴Screenshot der Testanwendung

²⁵⁵Screenshot der Testanwendung

Anhang 61: Lesehinweis zu den Vergleichsgrafiken

Die folgenden Grafiken fassen die Ergebnisse *suite-übergreifend* (test-12, test-50, test-100) zusammen und sind damit nicht einem einzelnen Suite-Block zugeordnet. Nach den Screenshot-Abschnitten dienen sie als komprimierte Vergleichsebene für Recall, Laufzeit, Konsistenz, Über-/Unterdetektion und Truncation.

Anhang 62: Recall-Degradation über alle Testsuites

Alle drei Modelle erzielen in test-12 hohe Recall-Werte ($\geq 75\%$); der markante Einbruch findet vor allem zwischen test-50 und test-100 statt. **qwen3:32b** hält mit 63 % den höchsten Recall in test-100. Datenbasis: Tabellen 31, 45, 58.

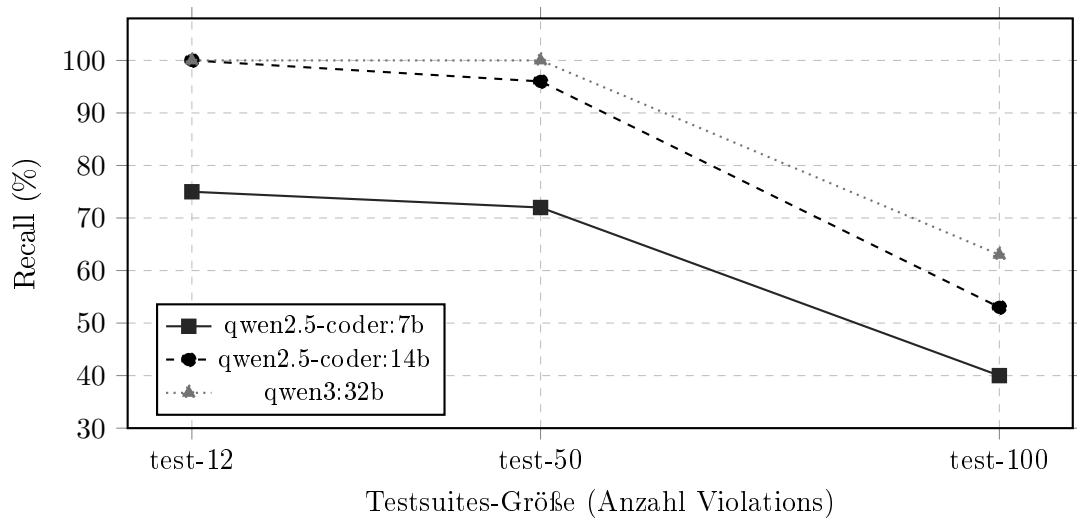


Abb. 25: Recall in Prozent je Modell über alle drei Testsuites (Konsistenzschwelle $> 5/10$ Runs). Der Knick zwischen test-50 und test-100 ist bei allen Modellen sichtbar. Datenbasis: Tabellen 31, 45, 58.²⁵⁶

²⁵⁶Eigene Abbildung

Anhang 63: Gesamtlaufzeit-Skalierung über alle Testsuites

Dargestellt ist die mittlere Gesamtlaufzeit je Modell über die drei Testsuites. Die gestrichelte Referenzlinie markiert den NFA-02-Grenzwert (600 s = 10 min). In test-12 unterschreiten alle Modelle diese Grenze; in test-100 liegt kein Modell mehr darunter. Datenbasis: Tabellen 31, 45, 58.

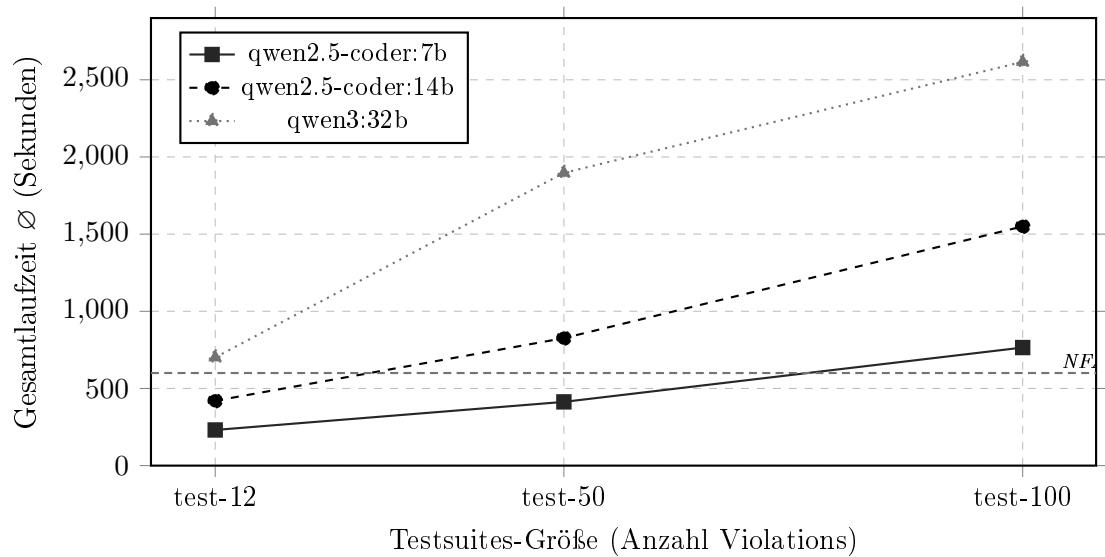


Abb. 26: Mittlere Gesamtlaufzeit in Sekunden je Modell und Testsuite. Die NFA-02-Grenzlinie markiert 600 s (10 Minuten). Datenbasis: Tabellen 31, 45, 58.²⁵⁷

²⁵⁷Eigene Abbildung

Anhang 64: LLM-Konsistenz (σ) im Suite-Vergleich

In test-12 liefert `qwen2.5-coder:14b` mit $\sigma = 0,42$ die stabilste Ausgabe. In test-100 steigt die Streuung bei 7b ($\sigma = 5,75$) und 32b ($\sigma = 5,63$) stark an, während 14b mit $\sigma = 3,23$ am stabilsten bleibt. Datenbasis: Tabellen 35, 49, 63.

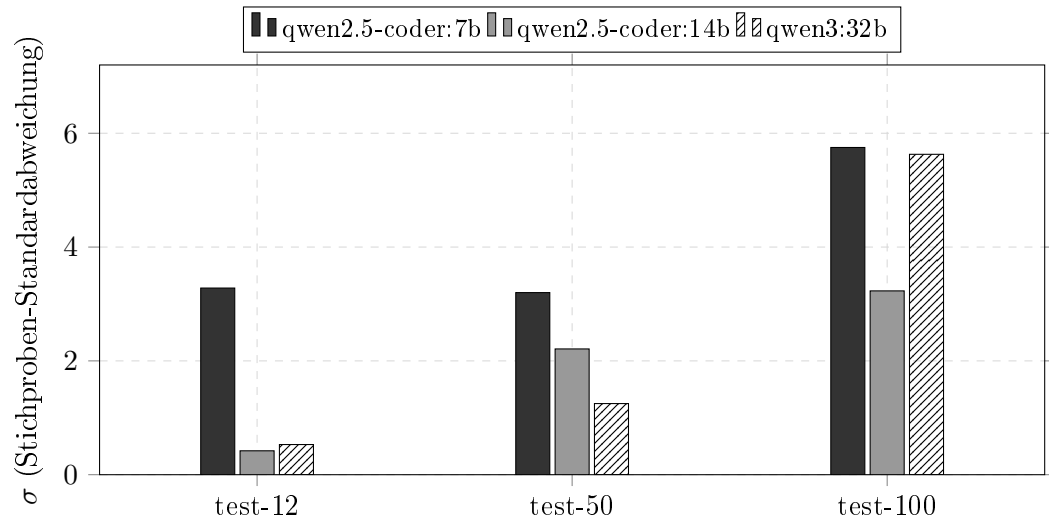


Abb. 27: Stichproben-Standardabweichung σ der LLM-Detektor-Findings je Modell und Testsuite ($n = 10$ Runs). Niedrigere Werte bedeuten höhere Run-zu-Run-Konsistenz. Datenbasis: Tabellen 35, 49, 63.²⁵⁸

²⁵⁸Eigene Abbildung

Anhang 65: Recall und Überschuss – nach Suite gruppiert

Das Diagramm zeigt für jedes Modell, welcher Anteil der LLM-Detektor-Rohfindings dem tatsächlichen Recall entspricht (dunkler Balkenanteil) und welcher Anteil darüber hinausgeht (heller Anteil: Quell-Duplikate, Mehrfachinstanzen, nicht eindeutig zugeordnete Findings). In test-12 liegt der Überschuss bei 14b und 32b besonders hoch, weil axe-core und LLM-Detektor viele der 12 Violations unabhängig voneinander finden (Quell-Duplikate). In test-100 bleibt der Recall trotz annähernd gleichem Gesamtvolumen auf 40–63 %, da der Überschuss einen großen Teil der Rohfindings ausmacht. Die Linie bei 100 % markiert die Ground-Truth-Menge. Datenbasis: Recall aus Tabellen 31, 45, 58; Überschuss = LLM-Detektor-Rohfindings minus Recall (vgl. Tabellen 34, 48, 61).

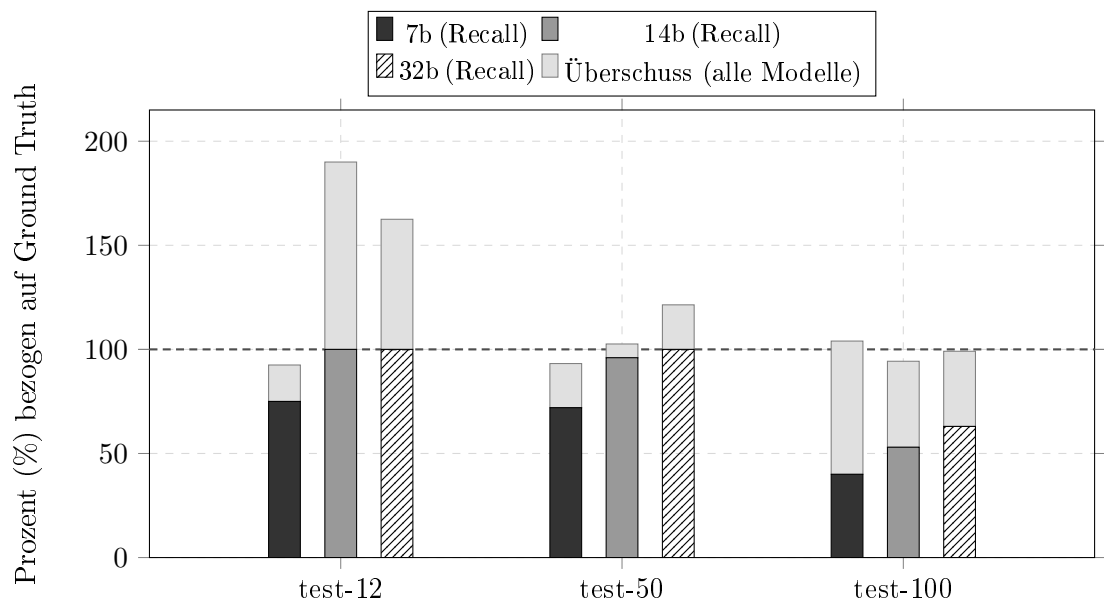


Abb. 28: Gestapeltes Balkendiagramm: **dunkler Anteil** = Recall (konsistent erkannte Violations, Schwelle > 5/10 Runs); **heller Anteil** = Überschuss des LLM-Detektor-Rohoutputs über den Recall hinaus (Quell-Duplikate, Mehrfachinstanzen, nicht zugeordnete Findings). Die Linie bei 100 % markiert die Ground-Truth-Menge. Innerhalb jeder Gruppe: links = 7b, Mitte = 14b, rechts = 32b.²⁵⁹

²⁵⁹Eigene Abbildung

Anhang 66: Truncation-Rate im Suite-Vergleich

Das Diagramm zeigt, in wie vielen der jeweils 10 Runs das Output-Token-Limit ausgeschöpft wurde. Dass **qwen3:32b** in test-50 bei 90 %, in test-100 jedoch bei nur 10 % liegt, ist kein Widerspruch: Das **num_predict**-Budget wurde proportional zur Suite-Größe erhöht (6 144 / 8 192 / 12 288), und 32b produziert kompaktere JSON-Ausgaben als 14b. Datenbasis: Tabellen 31, 45, 58.

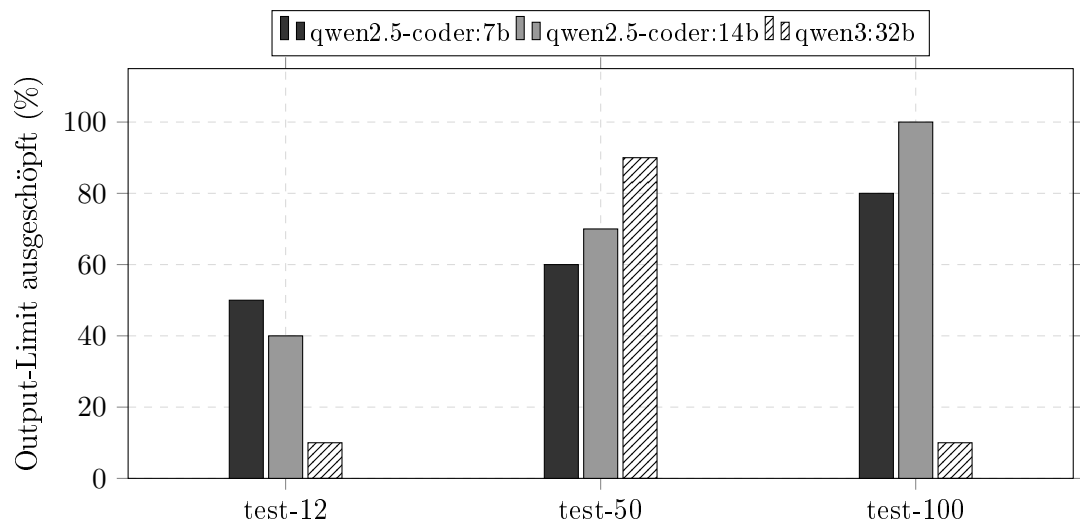


Abb. 29: Anteil der Runs mit ausgeschöpftem Output-Token-Limit je Modell ($n = 10$). Das **num_predict**-Budget wurde skaliert: test-12: 6 144, test-50: 8 192, test-100: 12 288 Tokens. Der Rückgang bei **qwen3:32b** von 90 % (test-50) auf 10 % (test-100) erklärt sich durch dessen kompaktere JSON-Ausgabestruktur. Datenbasis: Tabellen 31, 45, 58.²⁶⁰

²⁶⁰Eigene Abbildung

Anhang 67: Heatmap der Recall-Werte

Die folgende Heatmap zeigt die Recall-Werte (in %) je Modell und Testsuite in kompakter Form. Sie eignet sich als schnelle Übersicht, bevor die detaillierten Recall-Matrizen gelesen werden. Datenbasis: Tabellen 31, 45, 58.

Modell	test-12	test-50	test-100
qwen2.5-coder:7b	75 %	72 %	40 %
qwen2.5-coder:14b	100 %	96 %	53 %
qwen3:32b	100 %	100 %	63 %

 niedrig  mittel  hoch

Abb. 30: Heatmap der Recall-Werte über alle drei Suites. Sie macht die Degradation auf test-100 visuell unmittelbar sichtbar.²⁶¹

²⁶¹Eigene Abbildung

Anhang 68: Gestapeltes Balkendiagramm: Must-have vs. Nice-to-have

Das Diagramm zeigt die mittleren Must-have- und Nice-to-have-Anteile pro Modell und Suite. Datenbasis: Tabellen 31, 45, 58.

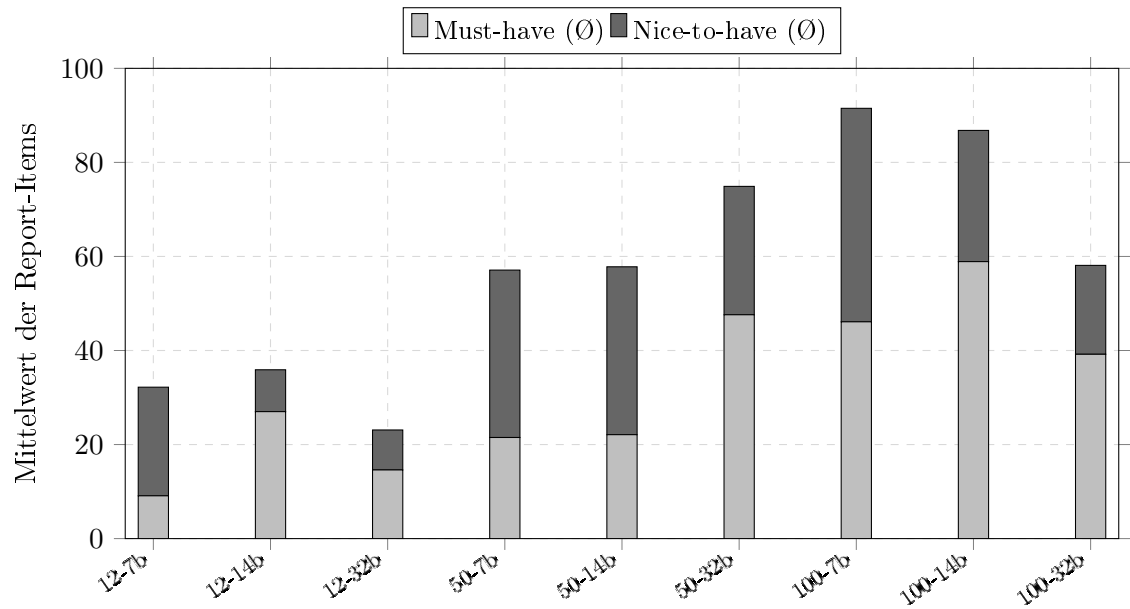


Abb. 31: Gestapelter Vergleich der mittleren Must-have- und Nice-to-have-Items. Die Verlagerung Richtung Must-have bei größeren Suites wird besonders bei `qwen2.5-coder:14b` sichtbar.²⁶²

²⁶²Eigene Abbildung

Anhang 69: BWEC-Block: Reales Untersuchungsobjekt

Anhang 70: Rohdaten: BWEC-Messreihe (`maxfiles = 100`, 66 Chunks)

Konfiguration: `OLLAMA_NUM_CTX = 65 536`; `OLLAMA_NUM_PREDICT = 24 576`; `LLM_DETECT_NUM_PREDICT = 4`
 Chunk-Größe = 4 000 Zeichen. Tool-Findings über alle Runs konstant: `axe-core = 10`, `Playwright = 2`, `grep = 8`. † Run-03 von `qwen2.5-coder:7b` als Ausreißer markiert: Priorisierungsphase erzeugte 205 Nice-to-have-Einträge und schöpfte das Output-Budget vollständig aus.

Run	Modell	LLM-Det.	OutTok	Trunc.	Must	Nice	Ges. ms
01	qwen2.5-coder:7b	14	3 369	×	5	22	318 593
02	qwen2.5-coder:7b	15	3 400	×	5	23	336 752
03†	qwen2.5-coder:7b	16	24 576	×*	4	205	906 677
04	qwen2.5-coder:14b	22	6 557	×	42	0	832 744
05	qwen2.5-coder:14b	22	6 485	×	37	5	827 809
06	qwen2.5-coder:14b	22	6 230	×	42	0	810 823
07	qwen3:32b	30	7 302	×	36	14	2 104 031
08	qwen3:32b	31	7 152	×	37	14	2 087 527
09	qwen3:32b	35	7 616	×	39	16	2 193 601

* Output-Token-Zahl entspricht Output-Budget (Output-Token-Limitationslücke im Tracking-Code).

Anhang 71: Modellvergleich: BWEC (maxfiles = 100)

Modell	Ø LLM-Det.	σ	Ø Must	Ø Nice	Ø Total (ms)	NFA-02	Trunc.
qwen2.5-coder:7b [†]	14,5	0,7	5,0	22,5	327 673	✓	0 / 2
qwen2.5-coder:14b	22,0	0,0	40,3	1,7	823 792	×	0 / 3
qwen3:32b	32,0	2,6	37,3	14,7	2 128 386	×	0 / 3

[†] Mittelwert über 2 stabile Runs (Run-03 ausgeschlossen). NFA-02: Laufzeit < 600 s. σ = Stichproben-Standardabweichung der LLM-Detektor-Findings.

Anhang 72: Rohdaten: BWEC-Messreihe (`maxfiles = 500`, 330 Chunks)

Konfiguration: Identisch zu Tabelle 70; lediglich `maxfiles = 500` (330 LLM-Detektor-Chunks statt 66). Tool-Findings konstant (`axe=10`, `Playwright=2`, `grep=8`). ‡ Priorisierungsphase fehlgeschlagen (`Must=0`, `Nice=0`). † Priorisierungsphase partiell (nur `Must-have`; `Nice-to-have` abgeschnitten).

Run	Modell	LLM-Det.	PromptTok	OutTok	Must	Nice	Ges. ms
01	qwen2.5-coder:7b	258	–	6 499	5	38	2 218 692
02	qwen2.5-coder:7b	253	–	8 014	7	47	2 225 685
03	qwen2.5-coder:7b	261	–	2 022	10	0	2 168 629
04†	qwen2.5-coder:14b	268	32 080	24 576	192	0	5 524 042
05†	qwen2.5-coder:14b	271	32 252	24 576	176	0	5 525 275
06‡	qwen2.5-coder:14b	274	32 768	2 213	0	0	4 242 122
07‡	qwen3:32b	418	40 960	11 450	0	0	13 653 425
08‡	qwen3:32b	426	40 960	3 442	0	0	13 480 972
09‡	qwen3:32b	400	40 960	24 576	0	0	17 516 889

32b: Prompt-Tokens = 40
gesättigt. 14b Run-04/05:
auf 2 213 Tokens. Alle 32b

Literaturverzeichnis

Wissenschaftliche Artikel und Konferenzbeiträge

- Abou-Zahra, Shadi; Brewer, Judy; Cooper, Michael (2018):** Artificial Intelligence (AI) for Web Accessibility: Is Conformance Evaluation a Way Forward? In: *Proceedings of the 15th International Web for All Conference*. W4A '18. New York, NY, USA: Association for Computing Machinery, S. 1–4. ISBN: 978-1-4503-5651-0. DOI: 10.1145/3192714.3192834. URL: <https://dl.acm.org/doi/10.1145/3192714.3192834> (Abruf: 24.02.2026).
- Abuaddous, Hayfa Y.; Jali, Mohd Zalisham; Basir, Nurlida (2016):** Web Accessibility Challenges. In: *International Journal of Advanced Computer Science and Applications (ijacsa)* 7.10. ISSN: 2156-5570. DOI: 10.14569/IJACSA.2016.071023. URL: <https://thesai.org/Publications/ViewPaper?Volume=7&Issue=10&Code=ijacsa&SerialNo=23> (Abruf: 04.03.2026).
- Bai, Aiden (2022):** A Fast Compiler-Augmented Virtual DOM for the Web. In: *arXiv*. URL: <https://arxiv.org/abs/2202.08409> (Abruf: 05.03.2026).
- Baskerville, Richard; Myers, Michael D. (2004):** Special Issue on Action Research in Information Systems: Making IS Research Relevant to Practice Foreword. In: *MIS Quarterly* 28.3, S. 329–335.
- Baskerville, Richard L. (1999):** Investigating Information Systems with Action Research. In: *Communications of the Association for Information Systems* 2.19, S. 1–32.
- Bassi, Barry; Delnevo, Giovanni; Franco, Mirko; Gaggi, Ombretta; Gatto, Salvatore; Mirri, Silvia; Olaiya, Kelvin (2025):** An Assessment of LLM-Based Auditing and Validation for Web Accessibility. In: *Proceedings of the 2025 International Conference on Information Technology for Social Good*. GoodIT '25. New York, NY, USA: Association for Computing Machinery, S. 297–305. ISBN: 979-8-4007-2089-5. DOI: 10.1145/3748699.3749805. URL: <https://dl.acm.org/doi/10.1145/3748699.3749805> (Abruf: 24.02.2026).
- Bender, Emily M.; Gebru, Timnit; McMillan-Major, Angelina; Shmitchell, Shmargaret (2021):** On the Dangers of Stochastic Parrots: Can Language Models Be Too Big? In: *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*. FAccT '21. New York, NY, USA: Association for Computing Machinery, S. 610–623. ISBN: 978-1-4503-8309-7. DOI: 10.1145/3442188.3445922. URL: <https://dl.acm.org/doi/10.1145/3442188.3445922> (Abruf: 05.03.2026).
- Brown, Tom; Mann, Benjamin; Ryder, Nick; Subbiah, Melanie; Kaplan, Jared D; Dhariwal, Prafulla; Neelakantan, Arvind; Shyam, Pranav; Sastry, Girish; Askell, Amanda; Agarwal, Sandhini; Herbert-Voss, Ariel; Krueger, Gretchen; Henighan, Tom; Child, Rewon; Ramesh, Aditya; Ziegler, Daniel; Wu, Jeffrey; Winter, Clemens; Hesse, Chris; Chen, Mark; Sigler, Eric; Litwin, Mateusz; Gray, Scott; Chess, Benjamin; Clark, Jack; Berner, Christopher; McCandlish, Sam; Radford, Alec; Sutskever, Ilya; Amodei, Dario (2020):** Language Models Are Few-Shot Learners. In: *Advances in Neural Information Processing Systems*. Bd. 33. Curran Associates,

- Inc., S. 1877–1901. URL: https://papers.nips.cc/paper_files/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html (Abruf: 06.04.2026).
- Brynjolfsson, Erik; Li, Danielle; Raymond, Lindsey (2025):** Generative AI at Work*. In: *The Quarterly Journal of Economics* 140.2, S. 889–942. ISSN: 0033-5533. DOI: 10.1093/qje/qjae044. URL: <https://doi.org/10.1093/qje/qjae044> (Abruf: 06.04.2026).
- Chiou, Paul T.; Alotaibi, Ali S.; Halfond, William G. J. (2021):** Detecting and localizing keyboard accessibility failures in web applications. In: *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ESEC/FSE 2021. New York, NY, USA: Association for Computing Machinery, S. 855–867. ISBN: 978-1-4503-8562-6. DOI: 10.1145/3468264.3468581. URL: <https://dl.acm.org/doi/10.1145/3468264.3468581> (Abruf: 17.03.2026).
- Delnevo, Giovanni; Andruccioli, Manuel; Mirri, Silvia (2024):** On the Interaction with Large Language Models for Web Accessibility: Implications and Challenges. In: *2024 IEEE 21st Consumer Communications & Networking Conference (CCNC)*. 2024 IEEE 21st Consumer Communications & Networking Conference (CCNC), S. 1–6. DOI: 10.1109/CCNC51664.2024.10454680. URL: <https://ieeexplore.ieee.org/document/10454680> (Abruf: 24.02.2026).
- Doush, Iyad Abu; Kassem, Reem (2025):** Evaluating AI-Generated Web Code for Accessibility Compliance: A Metric-Driven Approach. In: *Proceedings of the 11th International Conference on Software Development and Technologies for Enhancing Accessibility and Fighting Info-exclusion*. DSAI '24. New York, NY, USA: Association for Computing Machinery, S. 338–344. ISBN: 979-8-4007-0729-2. DOI: 10.1145/3696593.3696614. URL: <https://dl.acm.org/doi/10.1145/3696593.3696614> (Abruf: 24.02.2026).
- Fathallah, Nadeen; Hernández, Daniel; Staab, Steffen (2025):** AccessGuru: Leveraging LLMs to Detect and Correct Web Accessibility Violations in HTML Code. In: *Proceedings of the 27th International ACM SIGACCESS Conference on Computers and Accessibility*. ASSETS '25. New York, NY, USA: Association for Computing Machinery, S. 1–22. ISBN: 979-8-4007-0676-9. DOI: 10.1145/3663547.3746360. URL: <https://dl.acm.org/doi/10.1145/3663547.3746360> (Abruf: 24.02.2026).
- Fernandes, Nádia; Costa, Daniel; Duarte, Carlos; Carriço, Luís (2012):** Evaluating the Accessibility of Web Applications. In: *Procedia Computer Science*. Proceedings of the 4th International Conference on Software Development for Enhancing Accessibility and Fighting Info-exclusion (DSAI 2012) 14, S. 28–35. ISSN: 1877-0509. DOI: 10.1016/j.procs.2012.10.004. URL: <https://www.sciencedirect.com/science/article/pii/S1877050912007661> (Abruf: 05.03.2026).
- Fernández-Navarro, Carla; Chicano, Francisco (2026):** Automated LLM-Based Accessibility Remediation: From Conventional Websites to Angular Single-Page Applications. In: arXiv: 2602.17887 [cs.SE]. URL: <https://arxiv.org/abs/2602.17887>.
- Gao, Yunfan; Xiong, Yun; Gao, Xinyu; Jia, Kang; Pan, Jinliang et al. (2023):** Retrieval-Augmented Generation for Large Language Models: A Survey. In: arXiv: 2312.10997 [cs.CL]. URL: <https://arxiv.org/abs/2312.10997> (Abruf: 01.03.2026).

- Gubbi Mohanbabu, Ananya; Pavel, Amy (2024):** Context-Aware Image Descriptions for Web Accessibility. In: *Proceedings of the 26th International ACM SIGACCESS Conference on Computers and Accessibility*. ASSETS '24. New York, NY, USA: Association for Computing Machinery, S. 1–17. ISBN: 979-8-4007-0677-6. DOI: 10.1145/3663548.3675658. URL: <https://dl.acm.org/doi/10.1145/3663548.3675658> (Abruf: 24.02.2026).
- Guriță, Alexandra-Elena; Vatavu, Radu-Daniel (2025):** Insights and Implications of Evaluating Accessibility Compliance in AI-Generated Web Interfaces. In: *Companion Proceedings of the ACM on Web Conference 2025*. WWW '25. New York, NY, USA: Association for Computing Machinery, S. 996–1000. ISBN: 979-8-4007-1331-6. DOI: 10.1145/3701716.3715552. URL: <https://dl.acm.org/doi/10.1145/3701716.3715552> (Abruf: 24.02.2026).
- He, Ziyao; Huq, Syed Fatiul; Malek, Sam (2025):** Enhancing Web Accessibility: Automated Detection of Issues with Generative AI. In: *Proc. ACM Softw. Eng.* 2 (FSE), FSE101:2264–FSE101:2287. DOI: 10.1145/3729371. URL: <https://dl.acm.org/doi/10.1145/3729371> (Abruf: 24.02.2026).
- Hevner, Alan R.; March, Salvatore T.; Park, Jinsoo; Ram, Sudha (2004):** Design Science in Information Systems Research. In: *MIS Quarterly* 28.1, S. 75–105.
- Huang, Calista; Ma, Alyssa; Vyasamudri, Suchir; Puype, Eugenie; Kamal, Sayem; Garcia, Juan Belza; Cheema, Salar; Lutz, Michael (2024):** ACCESS: Prompt Engineering for Automated Web Accessibility Violation Corrections. In: arXiv:2401.16450. arXiv. DOI: 10.48550/arXiv.2401.16450. arXiv: 2401.16450[cs]. URL: <http://arxiv.org/abs/2401.16450> (Abruf: 25.02.2026).
- Huang, Lei; Yu, Weijiang; Ma, Weitao; Zhong, Weihong; Feng, Zhangyin; Wang, Haotian; Chen, Qianglong; Peng, Weihua; Feng, Xiaocheng; Qin, Bing; Liu, Ting (2025):** A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. In: *ACM Transactions on Information Systems* 43.2, S. 1–55. ISSN: 1046-8188, 1558-2868. DOI: 10.1145/3703155. arXiv: 2311.05232[cs]. URL: <http://arxiv.org/abs/2311.05232> (Abruf: 05.03.2026).
- Hui, Binyuan; Yang, Jian; Cui, Zeyu; Yang, Jiaxi; Liu, Dayiheng; Zhang, Lei; Liu, Tianyu; Zhang, Jiajun; Yu, Bowen; Lu, Keming; Dang, Kai; Fan, Yang; Zhang, Yichang; Yang, An; Men, Rui; Huang, Fei; Zheng, Bo; Miao, Yibo; Quan, Shanghaoran; Feng, Yunlong; Ren, Xingzhang; Ren, Xuancheng; Zhou, Jingren; Lin, Junyang (2024):** Qwen2.5-Coder Technical Report. In: arXiv:2409.12186. arXiv. DOI: 10.48550/arXiv.2409.12186. arXiv: 2409.12186[cs]. URL: <http://arxiv.org/abs/2409.12186> (Abruf: 15.03.2026).
- Kodithuwakku, Tashmi; Wickramaarachchi, Dilani (2025):** Assessing the Limits of Automated Accessibility Testing: Insights for QA Engineers and Tool Developers. In: *2025 5th International Conference on Advanced Research in Computing (ICARC)*. 2025 5th International Conference on Advanced Research in Computing (ICARC), S. 1–6. DOI: 10.1109/ICARC64760.2025.10963173. URL: <https://ieeexplore.ieee.org/document/10963173> (Abruf: 24.02.2026).

- Krok, Ewa (2024)**: Digital Accessibility as an Essential Element of an Inclusive Organization. In: *European Research Studies XXVII* (Special A), S. 857–876. ISSN: 1108-2976. URL: <https://ersj.eu/journal/3752> (Abruf: 17.03.2026).
- Lewis, Patrick; Perez, Ethan; Piktus, Aleksandra; Petroni, Fabio; Karpukhin, Vladimir; Goyal, Naman; Küttler, Heinrich; Lewis, Mike; Yih, Wen-tau; Rocktäschel, Tim; Riedel, Sebastian; Kiela, Douwe (2020)**: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In: *Advances in Neural Information Processing Systems*. Hrsg. von H. Larochelle; M. Ranzato; R. Hadsell; M.F. Balcan; H. Lin. Bd. 33. Curran Associates, Inc., S. 9459–9474. URL: https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf.
- Manca, Marco; Palumbo, Vanessa; Paternò, Fabio; Santoro, Carmen (2023)**: The Transparency of Automatic Web Accessibility Evaluation Tools: Design Criteria, State of the Art, and User Perception. In: *ACM Trans. Access. Comput.* 16.1, 3:1–3:36. ISSN: 1936-7228. DOI: 10.1145/3556979. URL: <https://dl.acm.org/doi/10.1145/3556979> (Abruf: 24.02.2026).
- Morris, Meredith Ringel; Zolyomi, Annuska; Yao, Catherine; Bahram, Sina; Bigham, Jeffrey P.; Kane, Shaun K. (2016)**: "With most of it being pictures now, I rarely use it: Understanding Twitter's Evolving Accessibility to Blind Users. In: *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*. CHI '16. New York, NY, USA: Association for Computing Machinery, S. 5506–5516. ISBN: 978-1-4503-3362-7. DOI: 10.1145/2858036.2858116. URL: <https://dl.acm.org/doi/10.1145/2858036.2858116> (Abruf: 04.03.2026).
- Morrison, Cecily; Cutrell, Edward; Dhareshwar, Anupama; Doherty, Kevin; Thieme, Anja; Taylor, Alex (2017)**: Imagining Artificial Intelligence Applications with People with Visual Disabilities using Tactile Ideation. In: *Proceedings of the 19th International ACM SIGACCESS Conference on Computers and Accessibility*. ASSETS '17. New York, NY, USA: Association for Computing Machinery, S. 81–90. ISBN: 978-1-4503-4926-0. DOI: 10.1145/3132525.3132530. URL: <https://dl.acm.org/doi/10.1145/3132525.3132530> (Abruf: 04.03.2026).
- Mowar, Peya (2024)**: Accessibility in AI-Assisted Web Development. In: *Proceedings of the 21st International Web for All Conference*. W4A '24. New York, NY, USA: Association for Computing Machinery, S. 123–125. ISBN: 979-8-4007-1030-8. DOI: 10.1145/3677846.3679054. URL: <https://dl.acm.org/doi/10.1145/3677846.3679054> (Abruf: 24.02.2026).
- Njifenjou, Ahmed; Sucal, Virgile; Jabaian, Bassam; Lefèvre, Fabrice (2024)**: Role-Play Zero-Shot Prompting with Large Language Models for Open-Domain Human-Machine Conversation. In: arXiv:2406.18460. arXiv. DOI: 10.48550/arXiv.2406.18460. arXiv: 2406.18460[cs]. URL: <http://arxiv.org/abs/2406.18460> (Abruf: 28.02.2026).
- OpenAI (2023)**: GPT-4 Technical Report. In: arXiv: 2303.08774 [cs.CL]. URL: <https://arxiv.org/abs/2303.08774> (Abruf: 01.03.2026).
- Paternò, Fabio; Vinci, Manuela; Manca, Marco; Iannuzzi, Nicola (2025)**: How an LLM Can Improve Automatic Web Accessibility Validation ? In: *Proceedings of the 16th Biannual*

- Conference of the Italian SIGCHI Chapter*. CHIItaly '25. New York, NY, USA: Association for Computing Machinery, S. 1–8. ISBN: 979-8-4007-2102-1. DOI: 10.1145/3750069.3750310. URL: <https://dl.acm.org/doi/10.1145/3750069.3750310> (Abruf: 24.02.2026).
- Peffer, Ken; Tuunanen, Tuure; Rothenberger, Marcus A.; Chatterjee, Samir (2007):** A Design Science Research Methodology for Information Systems Research. In: *Journal of Management Information Systems* 24.3, S. 45–77. DOI: 10.2753/MIS0742-1222240302.
- Pérez, J. Eduardo; Arrue, Myriam; Valencia, Xabier; Abascal, Julio (2020):** Longitudinal Study of Two Virtual Cursors for People With Motor Impairments: A Performance and Satisfaction Analysis on Web Navigation. In: *IEEE Access*. DOI: 10.1109/ACCESS.2020.3001766.
- Pool, Jonathan Robert (2023):** Testaro: Efficient Ensemble Testing for Web Accessibility. In: *Proceedings of the 25th International ACM SIGACCESS Conference on Computers and Accessibility*. ASSETS '23. New York, NY, USA: Association for Computing Machinery, S. 1–4. ISBN: 979-8-4007-0220-4. DOI: 10.1145/3597638.3614505. URL: <https://dl.acm.org/doi/10.1145/3597638.3614505> (Abruf: 24.02.2026).
- Purwandari, Nuraini; Dewi, Ratna Kusuma; Rinaldo; Sucipto, Purwo Agus (2025):** Policy in Practice: A Systematic Review of WCAG 2.2 and ADA 2024 Effects on Web and Mobile Accessibility. In: *Digitus*. DOI: 10.61978/digitus.v3i2.957. URL: https://www.researchgate.net/publication/396366120_Policy_in_Practice_A_Systematic_Review_of_WCAG_22_and_ADA_2024_Effects_on_Web_and_Mobile_Accessibility (Abruf: 04.03.2026).
- Schulhoff, Sander; Ilie, Michael; Balepur, Nishant; Kahadze, Konstantine; Liu, Amanda; Si, Chenglei; Li, Yinheng; Gupta, Aayush; Han, HyoJung; Schulhoff, Sevien; Dulepet, Pranav Sandeep; Vidyadhara, Saurav; Ki, Dayeon; Agrawal, Sweta; Pham, Chau; Kroiz, Gerson; Li, Feileen; Tao, Hudson; Srivastava, Ashay; Da Costa, Hevander; Gupta, Saloni; Rogers, Megan L.; Goncarenco, Inna; Sarli, Giuseppe; Galynker, Igor; Peskoff, Denis; Carpuat, Marine; White, Jules; Anadkat, Shyamal; Hoyle, Alexander; Resnik, Philip (2024):** The Prompt Report: A Systematic Survey of Prompting Techniques. In: arXiv: 2406.06608 [cs.CL]. URL: <https://arxiv.org/abs/2406.06608> (Abruf: 09.03.2026).
- Silvestre, Pedro F.; Pietzuch, Peter (2025):** Systems Opportunities for LLM Fine-Tuning using Reinforcement Learning. In: *Proceedings of the 5th Workshop on Machine Learning and Systems*. EuroMLSys '25. New York, NY, USA: Association for Computing Machinery, S. 90–99. ISBN: 979-8-4007-1538-9. DOI: 10.1145/3721146.3721944. URL: <https://dl.acm.org/doi/10.1145/3721146.3721944> (Abruf: 09.03.2026).
- Souza, Aline; de Souza Filho, José Cezar; Bezerra, Carla; Alves, Victor Anthony; Lima, Lara; Marques, Anna Beatriz; Teixeira Monteiro, Ingrid (2024):** Accessibility Evaluation of Web Systems for People with Visual Impairments: Findings from a Literature Survey. In: *Proceedings of the XXIII Brazilian Symposium on Human Factors in Computing Systems*. IHC '24. New York, NY, USA: Association for Computing Machinery, S. 1–13. ISBN: 979-8-4007-1224-1. DOI: 10.1145/3702038.3702090. URL: <https://dl.acm.org/doi/10.1145/3702038.3702090> (Abruf: 04.03.2026).

- Tafreshipour, Mahan; Deshpande, Anmol; Mehralian, Forough; Ahmed, Iftekhhar; Malek, Sam (2024):** Mally: A Mutation Framework for Web Accessibility Testing. In: *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. ISSTA 2024. New York, NY, USA: Association for Computing Machinery, S. 100–111. ISBN: 979-8-4007-0612-7. DOI: 10.1145/3650212.3652113. URL: <https://dl.acm.org/doi/10.1145/3650212.3652113> (Abruf: 24.02.2026).
- Touvron, Hugo; Martin, Louis; Stone, Kevin; Albert, Peter; Almahairi, Amjad; Babaei, Yasmine; Bashlykov, Nikolay; Batra, Soumya; Bhargava, Prajjwal; Bhosale, Shruti; Bikel, Dan; Blecher, Lukas; Ferrer, Cristian Canton; Chen, Moya; Cucurull, Guillem; Esiobu, David; Fernandes, Jude; Fu, Jeremy; Fu, Wenying; Fuller, Brian; Gao, Cynthia; Goswami, Vedanuj; Goyal, Naman; Hartshorn, Anthony; Hosseini, Saghar; Hou, Rui; Inan, Hakan; Kardas, Marcin; Kerkez, Viktor; Khabsa, Madian; Kloumann, Isabel; Korenev, Artem; Koura, Punit Singh; Lachaux, Marie-Anne; Lavril, Thibaut; Lee, Jenya; Liskovich, Diana; Lu, Yinghai; Mao, Yuning; Martinet, Xavier; Mihaylov, Todor; Mishra, Pushkar; Molybog, Igor; Nie, Yixin; Poulton, Andrew; Reizenstein, Jeremy; Rungta, Rashi; Saladi, Kalyan; Schelten, Alan; Silva, Ruan; Smith, Eric Michael; Subramanian, Ranjan; Tan, Xiaoqing Ellen; Tang, Binh; Taylor, Ross; Williams, Adina; Kuan, Jian Xiang; Xu, Puxin; Yan, Zheng; Zarov, Iliyan; Zhang, Yuchen; Fan, Angela; Kambadur, Melanie; Narang, Sharan; Rodriguez, Aurelien; Stojnic, Robert; Edunov, Sergey; Scialom, Thomas (2023):** Llama 2: Open Foundation and Fine-Tuned Chat Models. In: arXiv: 2307.09288 [cs.CL]. URL: <https://arxiv.org/abs/2307.09288> (Abruf: 01.03.2026).
- Vaswani, Ashish; Shazeer, Noam; Parmar, Niki; Uszkoreit, Jakob; Jones, Llion; Gomez, Aidan N.; Kaiser, Lukasz; Polosukhin, Illia (2023):** Attention Is All You Need. In: arXiv: 1706.03762 [cs.CL]. URL: <https://arxiv.org/abs/1706.03762>.
- Wang, Yuqing; Zhao, Yun (2024):** Metacognitive Prompting Improves Understanding in Large Language Models. In: arXiv:2308.05342. arXiv. DOI: 10.48550/arXiv.2308.05342. arXiv: 2308.05342[cs]. URL: <http://arxiv.org/abs/2308.05342> (Abruf: 09.03.2026).
- Webster, Jane; Watson, Richard T. (2002):** Analyzing the Past to Prepare for the Future: Writing a Literature Review. In: *MIS Quarterly* 26.2, S. 2–6.
- Wei, Jason; Tay, Yi; Bommasani, Rishi; Raffel, Colin; Zoph, Barret; Borgeaud, Sebastian; Yogatama, Dani; Bosma, Maarten; Zhou, Denny; Metzler, Donald; Chi, Ed H.; Hashimoto, Tatsunori; Vinyals, Oriol; Liang, Percy; Dean, Jeffrey; Fedus, William (2022):** Emergent Abilities of Large Language Models. In: *Transactions on Machine Learning Research*, S. 1–35. URL: <https://arxiv.org/abs/2206.07682> (Abruf: 28.02.2026).
- Wei, Jason; Wang, Xuezhi; Schuurmans, Dale; Bosma, Maarten; Ichter, brian; Xia, Fei; Chi, Ed; Le, Quoc V; Zhou, Denny (2022):** Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In: *Advances in Neural Information Processing Systems*. Hrsg. von S. Koyejo; S. Mohamed; A. Agarwal; D. Belgrave; K. Cho; A. Oh. Bd. 35. Curran Associates, Inc., S. 24824–24837. URL: https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf.

Yao, Shunyu; Zhao, Jeffrey; Yu, Dian; Du, Nan; Shafran, Izhak; Narasimhan, Karthik R.; Cao, Yuan (2022): ReAct: Synergizing Reasoning and Acting in Language Models. In: The Eleventh International Conference on Learning Representations. URL: https://openreview.net/forum?id=WE_vluYUL-X (Abruf: 06.04.2026).

Bücher und Sammelwerke

Flanagan, David (2020): JavaScript: The Definitive Guide. 7. Aufl. Sebastopol, CA: O'Reilly Media. ISBN: 978-1491952023.

Heinemann, Daniela (2024): „Barrierefreiheit“. In: *Digitalisierte Verwaltung - Vernetztes E-Government*. Hrsg. von Margrit Seckelmann. Berlin: Erich Schmidt Verlag GmbH & Co. KG, S. 469–488. ISBN: 978-3-503-23763-0. DOI: 10.37307/b.978-3-503-23763-0.15. URL: <https://doi.org/10.37307/b.978-3-503-23763-0.15> (Abruf: 24.02.2026).

Kerkmann, Friederike; Lewandowski, Dirk (2015): Barrierefreie Informationssysteme: Zugänglichkeit für Menschen mit Behinderung in Theorie und Praxis. Walter de Gruyter GmbH & Co KG. 292 S. ISBN: 978-3-11-033729-7.

Kouroupetroglou, Georgios (2013): Assistive Technologies and Computer Access for Motor Disabilities. IGI Global Scientific Publishing. ISBN: 978-1-4666-4438-0.

Lazar, Jonathan; Feng, Jinjuan Heidi; Hochheiser, Harry (2017): Research Methods in Human-Computer Interaction. Elsevier. ISBN: 978-0-12-805390-4.

Russell, Stuart; Norvig, Peter (2021): Artificial Intelligence: A Modern Approach. Hoboken: Pearson. 1136 S. ISBN: 978-0-13-461099-3.

Sommerville, Ian (2016): Software Engineering. 10. Aufl. Pearson. ISBN: 978-0133943030.

World Health Organization (2011): World Report on Disability. Geneva: World Health Organization. ISBN: 978-92-4-156418-2. URL: <https://www.who.int/teams/noncommunicable-diseases/sensory-functions-disability-and-rehabilitation/world-report-on-disability> (Abruf: 04.03.2026).

Yin, Robert K. (2018): Case Study Research and Applications: Design and Methods. Los Angeles London New Delhi Singapore Washington DC Melbourne: SAGE Publications, Inc. 352 S. ISBN: 978-1-5063-3616-9.

Internetquellen und Medien

BITBW (2021): 6 Jahre BITBW. URL: <https://bitbw.de/neuigkeiten/artikel/6-jahre-bitbw-1> (Abruf: 28.02.2026).

Harrenstien, Ken (2009): Automatic captions in YouTube. Official Google Blog. URL: <https://googleblog.blogspot.com/2009/11/automatic-captions-in-youtube.html> (Abruf: 04.03.2026).

Krotoff, Tanguy (2026): Front-end frameworks popularity (React, Vue, Angular and Svelte). Gist. URL: <https://gist.github.com/tkrotoff/b1caa4c3a185629299ec234d2314e190> (Abruf: 05.03.2026).

- Land Baden-Württemberg (2025):** Neuer BW-Empfangsclient treibt Digitalisierung der Verwaltung voran. URL: <https://www.baden-wuerttemberg.de/de/service/presse/pressemitteilung/pid/neuer-bw-empfangsclient-treibt-digitalisierung-der-verwaltung-voran-1> (Abruf: 24.02.2026).
- Stack Overflow (2025):** Survey index. URL: <https://survey.stackoverflow.co/2025/> (Abruf: 05.03.2026).
- WebAIM (2019):** The WebAIM Million - 2019 report on the accessibility of the top 1,000,000 home pages. URL: <https://webaim.org/projects/million/2019> (Abruf: 24.02.2026).
- WebAIM (2025):** The WebAIM Million - 2025 report on the accessibility of the top 1,000,000 home pages. URL: <https://webaim.org/projects/million/> (Abruf: 24.02.2026).
- WebAIM (2026b):** WebAIM: Keyboard Accessibility. URL: <https://webaim.org/techniques/keyboard/> (Abruf: 04.03.2026).
- World Health Organization (2026):** Disability. URL: https://www.who.int/health-topics/disability#tab=tab_1 (Abruf: 28.02.2026).
- World Wide Web Consortium (W3C) (2026a):** Evaluation Tool List. URL: <https://www.w3.org/WAI/test-evaluate/tools/list/> (Abruf: 28.02.2026).
- World Wide Web Consortium (W3C) (2026b):** Web Accessibility Initiative. URL: <https://www.w3.org/WAI/> (Abruf: 28.02.2026).

Technische Dokumentation und Software

- Anthropic (2026a):** *Claude Model Overview*. URL: <https://www.anthropic.com/claude> (Abruf: 01.03.2026).
- Anthropic (2026b):** *System Card: Claude Sonnet 4.6*. URL: <https://www-cdn.anthropic.com/78073f739564e986ff3e28522761a7a0b4484f84.pdf> (Abruf: 02.03.2026).
- Deque Systems (2026):** *axe-core: Accessibility engine for automated Web UI testing*. Version 4.10.3. URL: <https://github.com/dequelabs/axe-core> (Abruf: 24.02.2026).
- Google (2025):** *Lighthouse: Automated auditing, performance, and accessibility tool for web pages*. URL: <https://developer.chrome.com/docs/lighthouse/overview> (Abruf: 28.02.2026).
- JavaScript With Syntax For Types. (2026).** URL: <https://www.typescriptlang.org/> (Abruf: 14.03.2026).
- Meta Open Source (2026a):** *React Documentation: Render and Commit*. URL: <https://react.dev/learn/render-and-commit> (Abruf: 05.03.2026).
- Meta Open Source (2026b):** *React: A JavaScript library for building user interfaces*. URL: <https://react.dev/> (Abruf: 05.03.2026).
- Microsoft (2026):** *Playwright: Fast and reliable end-to-end testing for modern web apps*. URL: <https://playwright.dev/docs/intro> (Abruf: 24.02.2026).
- Ollama (2026):** *Ollama Documentation: Quickstart Guide*. URL: <https://docs.ollama.com/quickstart> (Abruf: 04.03.2026).

- OpenAI (2026):** *Create completion*. URL: <https://developers.openai.com/api/reference/resources/completions/methods/create/> (Abruf: 01.03.2026).
- Robie, Jonathan (2026):** *What is the Document Object Model?* URL: <https://www.w3.org/TR/WD-DOM/introduction.html> (Abruf: 04.03.2026).
- WebAIM (2026a):** *Wave Web Accessibility Evaluation Tools*. URL: <https://wave.webaim.org/> (Abruf: 28.02.2026).

Rechtliche Grundlagen und Standards

- Bundesamt der Justiz und für Verbraucherschutz (2026):** *Gesetz zur Umsetzung der Richtlinie (EU) 2019/882 des Europäischen Parlaments und des Rates über die Barrierefreiheitsanforderungen für Produkte und Dienstleistungen 1*. URL: <https://www.gesetze-im-internet.de/bfsg/> (Abruf: 24.02.2026).
- Bundesministerium für Digitales und Staatsmodernisierung (2025):** *Barrierefreie Informationstechnik-Verordnung (BITV 2.0)*. URL: <https://www.barrierefreiheit-dienstekonsolidierung.bund.de/Webs/PB/DE/gesetze-und-richtlinien/bitv2-0/bitv2-0-node.html> (Abruf: 24.02.2026).
- Deutscher Bundestag (2026):** *BFSG (Barrierefreiheitsstärkungsgesetz)*. URL: <https://bfsg-gesetz.de/> (Abruf: 24.02.2026).
- Europäisches Parlament und Rat der Europäischen Union (2016):** *Verordnung (EU) 2016/679 des Europäischen Parlaments und des Rates vom 27. April 2016 zum Schutz natürlicher Personen bei der Verarbeitung personenbezogener Daten, zum freien Datenverkehr und zur Aufhebung der Richtlinie 95/46/EG (Datenschutz-Grundverordnung)*. URL: <https://eur-lex.europa.eu/eli/reg/2016/679/oj> (Abruf: 10.03.2026).
- European Commission (2026):** *European accessibility act (EAA)*. URL: https://commission.europa.eu/strategy-and-policy/policies/justice-and-fundamental-rights/disability/european-accessibility-act-eaa_en (Abruf: 24.02.2026).
- Landesrecht BW (2015):** *Gesetz zur Errichtung der Landesoberbehörde BITBW*. URL: <https://www.landesrecht-bw.de/bsbw/document/jlr-ITErGBWV3P2> (Abruf: 28.02.2026).
- United Nations (2006):** *Convention on the Rights of Persons with Disabilities*. URL: <https://www.ohchr.org/en/instruments-mechanisms/instruments/convention-rights-persons-disabilities> (Abruf: 28.02.2026).
- World Wide Web Consortium (W3C) (2018):** *Web Content Accessibility Guidelines (WCAG) 2.1*. URL: <https://www.w3.org/TR/WCAG21/> (Abruf: 24.02.2026).
- World Wide Web Consortium (W3C) (2023):** *Web Content Accessibility Guidelines (WCAG) 2.2*. URL: <https://www.w3.org/TR/WCAG22/> (Abruf: 24.02.2026).

Erklärung zur Verwendung generativer KI-Systeme

Bei der Erstellung der eingereichten Arbeit habe ich auf künstlicher Intelligenz (KI) basierte Systeme benutzt:

☒ ja

☐ nein²⁶³

Falls ja: Die nachfolgend aufgeführten auf künstlicher Intelligenz (KI) basierten Systeme habe ich bei der Erstellung der eingereichten Arbeit benutzt:

1. Claude Sonnet (Anthropic)
2. ChatGPT (OpenAI)
3. Claude Code (Anthropic)
4. Microsoft Copilot

Ich erkläre, dass ich

- mich aktiv über die Leistungsfähigkeit und Beschränkungen der oben genannten KI-Systeme informiert habe,²⁶⁴
- die aus den oben angegebenen KI-Systemen direkt oder sinngemäß übernommenen Passagen gekennzeichnet habe,
- überprüft habe, dass die mithilfe der oben genannten KI-Systeme generierten und von mir übernommenen Inhalte faktisch richtig sind,
- mir bewusst bin, dass ich als Autorin bzw. Autor dieser Arbeit die Verantwortung für die in ihr gemachten Angaben und Aussagen trage.

Die oben genannten KI-Systeme habe ich wie im Folgenden dargestellt eingesetzt:

²⁶³Die Erklärung ist in jedem Fall zu unterzeichnen, auch wenn Sie keine KI-Systeme genutzt haben und Ihr Kreuz bei „nein“ gesetzt haben.

²⁶⁴U.a. gilt es hierbei zu beachten, dass an KI weitergegebene Inhalte ggf. als Trainingsdaten genutzt und wiederverwendet werden. Dies ist insb. für betriebliche Aspekte als kritisch einzustufen.

Arbeitsschritt	KI-System(e)	Verwendungsweise
Ideenfindung und Strukturierung	Claude Sonnet (Anthropic)	Unterstützung bei der Strukturierung der Arbeit, Formulierung von Forschungsfragen sowie Gliederung einzelner Kapitel. Inhalte wurden kritisch geprüft und eigenständig überarbeitet.
Sprachliche Überarbeitung und Glättung	ChatGPT (OpenAI)	Unterstützung bei der sprachlichen Verbesserung einzelner Textpassagen (Grammatik, Stil). Inhaltliche Aussagen wurden nicht automatisiert übernommen.
Code-Unterstützung	Claude Code (Anthropic) / GitHub Copilot (Microsoft)	Unterstützung bei der Erklärung und Optimierung von Codefragmenten im Rahmen der Implementierung. Der finale Code wurde eigenständig entwickelt und validiert.

(Ort, Datum)

(Unterschrift)

Erklärung

Ich versichere hiermit, dass ich die vorliegende Arbeit mit dem Thema: *Kontextsensitive, lokale KI-Assistenz zur automatisierten Barrierefreiheitsanalyse von Rich-Client-Webanwendungen mittels Tool-Reports, Playwright-Interaktionen und Codeanalyse* selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Ich versichere zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.

(Ort, Datum)

(Unterschrift)